

The Backend Engineer's Library

Operating Systems and Linux

Volume 2

Contents

The Process Model: Processes, Threads, and Address Spaces
CPU Scheduling: CFS, Real-Time, and cgroup CPU Control
Virtual Memory and Paging
Memory in Practice: Allocators, Page Cache, Huge Pages, OOM
System Calls and the Kernel Boundary
File Systems and the VFS
Linux I/O Models: Blocking, epoll, and io_uring
Signals, Pipes, and IPC
Namespaces, cgroups, and Container Internals
The Linux Network Stack
Performance Analysis: perf, ftrace, and eBPF
Boot, init, and systemd

Chapter 1 — The Process Model: Processes, Threads, and Address Spaces

What this chapter covers. This is the opening chapter of a volume about the operating system as it actually behaves underneath a high-throughput backend service. Everything else in this volume — scheduling, virtual memory, system calls, I/O models, namespaces — builds on one foundation: the kernel's model of a *running program*. This chapter builds that foundation for Linux specifically. You run processes and threads every day. You fork workers, you size thread pools, you debug a container that leaks zombies, you argue about threads versus processes versus async. This chapter gives you the kernel-level model those daily decisions actually rest on: what a process *is* to the kernel, how Linux represents every schedulable thing as a *task*, how `fork/exec/wait` and `clone` really work, why "a thread is just a task that shares memory" is the single most useful sentence you can memorize about Linux concurrency, and what a context switch costs you.

We are deliberately precise about Linux mechanics, because the abstractions leak in ways that matter at scale. The difference between a thread and a process on Linux is a bitmask of sharing flags, not two different kinds of object — and that fact explains everything from why `fork` is cheap to why a PID namespace's `init` must reap zombies.

Learning goals — after this chapter you should be able to:

- Define a Linux process as the tuple of an *address space*, one or more *threads of execution*, a set of *resources* (file descriptors, etc.), and *metadata* (PID, credentials, namespaces), and explain why the process is the unit of resource ownership and isolation.
- Describe the kernel's actual representation — the `task_struct` per schedulable entity — and the distinction between a kernel task PID and the userspace PID/TID (`getpid` vs `gettid`, `pid` vs `tgid`).
- Explain the `fork/exec/wait` lifecycle precisely: copy-on-write, why Unix splits process creation from program loading, what a zombie is, and why some process must reap.

- State the unifying insight: `fork` and `pthread_create` are both `clone` with different sharing flags, and place "process" and "thread" as two points on a spectrum of sharing.
- Reason about the threads-vs-processes-vs-async decision for a backend service, and about context-switch cost (including TLB effects) as a real performance factor.
- Read a process's virtual-memory map and its credentials, and connect PID 1 / zombie reaping to a concrete container gotcha.

What a process is

Ask ten engineers what a process is and you get "a running program." That is true and useless. To the kernel, a process is a *container for resources* — the noun the kernel hangs everything else on. Concretely, a Linux process is the bundle of four things:

- **An address space.** A private virtual address space: the set of virtual-to-physical mappings that define what memory the program can see and touch. This is where the code, the globals, the heap, and the stacks live. The address space is the strongest isolation boundary the hardware and kernel jointly provide: process A cannot read or corrupt process B's memory, because the page tables simply do not map it. Virtual memory is the subject of Chapter 3; here we treat the address space as the first-class thing a process owns.
- **One or more threads of execution.** A thread is an execution context: a program counter, a register set, and a stack, scheduled onto a CPU. A process always has at least one thread. Multiple threads in a process share the one address space. On Linux, as we will see, each thread is itself a scheduler-visible task.
- **Resources.** Open file descriptors (which point at open files, sockets, pipes, epoll instances, `eventfds`, `timerfds` — nearly everything in Linux is a file), the current and root directory, the umask, memory mappings, timers, and so on. File descriptors are the most important: they are small integers indexing a per-process table of open file descriptions, and they are the handles through which a backend service talks to the outside world.
- **Metadata.** The process identifier (PID), the parent PID, credentials (user and group IDs, capabilities), scheduling parameters, resource

limits (`rlimit`), signal state, and namespace membership. This is the bookkeeping that makes the process a governable unit — the thing you can `kill`, `nice`, `cgroup`, `setuid`, or place in a namespace.

The reason to hold all four in your head at once is that *the process is the unit of both resource ownership and isolation*. When we discussed isolation as a security boundary in Book 1, Chapter 10, the address space plus credentials plus namespaces are exactly what that boundary is made of at the OS level. When a container "runs a process," it is this tuple — address space, threads, FDs, metadata — that the container runtime configures.

```
flowchart TB
    subgraph P["Process (PID 4217)"]
        direction TB
        AS["Address space  
(private virtual memory:  
text, data, heap, stacks, mmap)"]
        subgraph TH["Threads of execution"]
            T1["Thread (TID 4217)  
regs + stack + PC"]
            T2["Thread (TID 4219)  
regs + stack + PC"]
            T3["Thread (TID 4221)  
regs + stack + PC"]
        end
        subgraph RES["Resources"]
            FD["File descriptor table  
0,1,2 ... sockets, epoll, files"]
            CWD["cwd / root / umask"]
        end
        subgraph META["Metadata"]
            IDS["PID / PPID / PGID / SID"]
            CRED["Credentials: UID/GID, capabilities"]
            NS["Namespace membership"]
        end
    end
    T1 --> AS
    T2 --> AS
    T3 --> AS
    T1 --> FD
    T2 --> FD
```

The threads share the address space and the FD table; each has its own stack and registers. Hold that picture — the rest of the chapter is essentially an elaboration of it.

The kernel's view: everything is a task

Here is the first place Linux differs from the textbook. Classic OS courses describe a "process control block" (PCB) per process and a separate "thread control block" per thread. Linux does not work that way. **Linux has exactly one kind of schedulable object: the task.** Each task is represented by a `struct task_struct`, an enormous structure defined in `include/linux/sched.h`. There is no separate process object. A "process" is an emergent concept — a *group of tasks that happen to share the right things* — not a distinct kernel structure.

Every schedulable entity — what userspace calls a thread — is one `task_struct`. A single-threaded process is one task. A process with eight threads is eight `task_struct`s that share an address space and other resources. The scheduler (Chapter 2) schedules tasks, full stop; it does not know or care whether a task is "a process" or "a thread."

PID versus TID: `pid` and `tgid`

This design creates a naming problem that trips up almost everyone the first time. Inside the kernel, each `task_struct` has:

- a `pid` field — a unique per-task identifier, and
- a `tgid` field — the *thread group ID*.

The thread group is the set of tasks that constitute one userspace process. The first task created — the thread group leader — has `pid == tgid`. Every additional thread created in that process gets its own unique `pid` but inherits the leader's `tgid`.

Now the mapping to what userspace calls things:

- `getpid()` returns the `tgid` — the process ID that is stable across all threads. All threads in a process see the same `getpid()`.
- `gettid()` returns the `pid` — the unique per-thread kernel task ID, what tools and `/proc` call the *thread ID* (TID).

So the kernel's `pid` is the userspace TID, and the kernel's `tgid` is the userspace PID. The naming is historically backwards, but the model is clean: a process is a thread group, identified by the leader's TID, and every thread has its own TID. You can see this directly: `/proc/<pid>/task/` lists one directory per thread, each named by its TID.

```
$ ps -T -p 4217
  PID  SPID  TTY          TIME CMD
  4217  4217  pts/3      00:00:01 server      # thread group leader:
pid==tgid
  4217  4219  pts/3      00:00:00 server      # worker thread, own TID
  4217  4221  pts/3      00:00:00 server      # worker thread, own TID
```

The process tree, PID 1, and reparenting

Processes form a tree by parentage. Every process except the first records a parent PID (PPID); `fork` creates a child whose parent is the caller. At the root of the tree sits **PID 1**, the `init` process, started by the kernel at boot (Chapter 12 covers `systemd` as the modern PID 1). PID 1 is special in several ways we will return to.

When a process exits, its children do not die with it — they become **orphans**. An orphan must still have a parent (the tree must stay connected and someone must eventually reap it), so the kernel *reparents* orphans. By default they are reparented to PID 1. Since Linux 3.4, a process can call `prctl(PR_SET_CHILD_SUBREAPER)` to volunteer as a "subreaper," so that its orphaned descendants reparent to *it* rather than all the way up to PID 1 — this is how `systemd` user sessions and some container supervisors keep process subtrees under their own management. The reparenting mechanism is exactly why a container's `init` matters, as we will see under zombies.

Process groups and sessions

Two more groupings sit alongside the parent/child tree, and both matter for signals and job control (Chapter 8):

- A **process group** is a set of related processes, identified by a process group ID (PGID). It is the unit of *signal delivery for job control*: sending a signal to `-PGID` delivers it to every member. When your shell runs `cmd1 | cmd2 | cmd3`, all three land in one process group so that Ctrl-C hits the whole pipeline.
- A **session** is a collection of process groups, identified by a session ID (SID), created with `setsid()`. A session may have a *controlling terminal*; one process group in the session is the *foreground* group (receives terminal-generated signals), the rest are background. Daemons call `setsid()` precisely to escape a controlling terminal so they are not killed when it closes.

These groupings are why "who receives this signal" is more subtle than "the process I named." We defer the full treatment to Chapter 8, but note them here because PGID and SID are part of the process metadata the kernel tracks per task.

The process lifecycle: fork, exec, wait

Unix creates processes with a design that surprises engineers coming from other systems: process *creation* and program *loading* are two separate operations. `fork()` duplicates the calling process; `execve()` replaces the current program image with a new one. You almost always do them together — fork, then exec — but the split is deliberate and, it turns out, deeply useful.

fork() and copy-on-write

`fork()` creates a new process that is a near-exact duplicate of the caller. The child gets a copy of the parent's address space, a copy of its file descriptor table (the descriptors are duplicated, so parent and child share the *open file descriptions* and thus file offsets), its credentials, and so on. `fork` returns twice: it returns the child's PID in the parent, and `0` in the child. That is how each side knows which it is.

Copying an entire address space eagerly would make `fork` catastrophically expensive — a multi-gigabyte process would take forever to duplicate, and most of that copy would be pointless because the child usually calls `exec` immediately and throws the whole image away. Linux avoids this with **copy-on-write (COW)**. At `fork`:

1. The kernel copies the *page tables*, not the *pages*. Parent and child page tables point at the same physical frames.
2. Every writable page in both processes is marked **read-only** in the page tables.
3. Both processes run, reading shared physical pages happily.
4. The instant either process *writes* a shared page, the CPU raises a page fault. The kernel's fault handler sees a write to a COW page, allocates a fresh physical frame, copies the page's contents into it, remaps the faulting process's page to the private copy (now writable), and resumes. Only the pages actually written are ever duplicated.

This makes `fork` cheap-ish: its cost is proportional to the size of the page tables, not the size of the address space. A large process still pays to copy its page-table hierarchy (and to take those first write faults), which is one reason `fork` in a huge multi-gigabyte JVM or a database can be noticeably slow and why the `fork()`-based multiprocessing start method in Python has to be used carefully. But it is vastly cheaper than a full copy, and it is the reason the fork-then-exec idiom is affordable.

COW has one more famous consequence worth internalizing: `fork` in a *multithreaded* process is a minefield. `fork` duplicates only the calling thread; the child has a single thread but inherits the full memory image, including any mutexes that other threads held at the instant of the fork. Those locks can never be released in the child. This is why POSIX restricts a child between `fork` and `exec` to *async-signal-safe* functions only, and why fork-without-exec in a threaded program is a known source of deadlocks.

exec(): replacing the program

`execve()` (and its `libc` front-ends `execl`, `execvp`, etc.) does the opposite of `fork`: it keeps the process — same PID, same PPID, same open file descriptors (unless marked close-on-exec, `O_CLOEXEC`), same credentials subject to `setuid` rules — but *discards the entire address space* and loads a new program in its place. The old text, data, heap, and stack are gone; the kernel maps in the new executable's segments, sets up a fresh stack with the new `argv / envp`, and jumps to the new entry point. `exec` does not return on success — there is nothing to return to.

The fork-then-exec pattern, then, is: `fork` to get a new process that is a copy of the shell/server, then `exec` in the child to become the target program, while the parent keeps running. The child inherits the parent's file descriptors, which is precisely how a shell wires up pipelines and redirections: it `forks`, rearranges FDs 0/1/2 with `dup2` in the child *before* `exec`, and the new program starts with `stdin/stdout/stderr` already pointing where the shell wants them.

Why separate the two operations at all? Because the gap between `fork` and `exec` is where the child — running as a copy of the parent, with the parent's privileges — can *set up the environment for the new program*: redirect file descriptors, change the working directory, drop privileges (`setuid / setgid`), adjust signal dispositions, join namespaces, set resource limits. A combined "spawn" call would need every one of those

as a parameter. Unix instead says: `fork` gives you a fully-configurable copy of yourself; do whatever setup you like with ordinary syscalls; then `exec`. That is why Unix never needed a baroque `CreateProcess` with dozens of arguments. The cost is the expense and multithreading hazards of `fork`, which is why `posix_spawn()` (often implemented over `clone/vfork`) exists as a fast path for the common "fork+exec with a little setup" case.

wait(), zombies, and reaping

When a process exits (via `exit()`, returning from `main`, or a fatal signal), it does not vanish. The kernel tears down its address space, closes its file descriptors, and releases most of its resources — but it *retains* a small husk of the `task_struct` holding the exit status (the return code or terminating signal). The process is now in state **EXIT_ZOMBIE** (shown as `Z` and `<defunct>` in `ps`). It exists only so its parent can retrieve the exit status.

The parent retrieves it by calling `wait()` / `waitpid()` / `waitid()`. This is called **reaping**: the parent reads the child's exit status, and *then* the kernel finally frees the last of the child's `task_struct`. A reaped child is gone. An unreaped terminated child stays a zombie forever.

Zombies consume almost no memory, but they hold a **PID**, and PIDs are a finite resource (`/proc/sys/kernel/pid_max`). A process that forks children and never `wait`s leaks PIDs; a busy server doing this exhausts the PID space and then *nothing on the machine can fork* — `fork` returns `EAGAIN`, and the box is effectively dead until the leak is cleared. Zombie leaks are a classic production incident.

Two cases determine who reaps:

- **Normal case.** The parent outlives the child and calls `wait`. Clean.
- **Orphan case.** The parent dies first. The child (alive or already a zombie) is reparented to PID 1 (or the nearest subreaper). Well-behaved inits **reap in a loop** — they call `wait` repeatedly to clear any child that reparents to them. This is a *core responsibility* of PID 1.

```

stateDiagram-v2
    [*] --> Running: fork() (COW copy)
    Running --> Running: exec() replaces image
    Running --> Zombie: exit() / fatal signal
    (address space freed,
    exit status retained)
    Zombie --> Reaped: parent wait()/waitpid()
    reads status, frees task_struct
    Reaped --> [*]
    Running --> Orphaned: parent dies first
    Orphaned --> Zombie: child exits later
    note right of Orphaned
        reparented to PID 1
        (or nearest subreaper)
    end note
    note right of Zombie
        if parent never wait(s),
        zombie persists and
        leaks a PID
    end note

```

The container gotcha. This is not academic trivia — it is one of the most common container bugs. In a container, the process the runtime launches becomes **PID 1 inside the container's PID namespace** (Chapter 9). If your entrypoint is an application that was never written to be an init — say, a shell script that execs your server, or a server that spawns subprocesses — then *it* is PID 1, and it inherits PID 1's reaping duty. Most applications never call `wait` for children they did not directly create, so any process that reparents to them becomes an immortal zombie. Over time the container accumulates zombies and can exhaust PIDs. PID 1 also has *special signal semantics*: the kernel does not apply default signal dispositions to PID 1, so a naive PID 1 that installs no handler will ignore `SIGTERM` entirely — which is why `docker stop` on such a container hangs for ten seconds and then `SIGKILL`s it. The fix is a tiny real init as PID 1: `tini` (shipped as Docker's `--init`), `dumb-init`, or `s6`. They do two jobs: reap orphaned zombies in a `wait` loop, and forward signals to the real application. We revisit this in Book 6 on cloud-native security and in Chapter 9; for now, remember: **a container's PID 1 must reap.**

clone(): the primitive under fork and threads

We have described `fork` (new process) and, shortly, `pthread_create` (new thread) as if they were different mechanisms. On Linux they are the same mechanism. Both are thin wrappers over one syscall: `clone()`. This is the unifying insight of the whole chapter.

`clone()` creates a new task. What makes the new task a "child process" versus a "thread" is entirely a set of **sharing flags** you pass. Each flag says "share this resource with the caller instead of copying it":

clone flag	Meaning when set
<code>CLONE_VM</code>	Share the address space (same page tables) — the defining flag of a thread
<code>CLONE_FILES</code>	Share the file-descriptor table
<code>CLONE_FS</code>	Share filesystem info: cwd, root directory, umask
<code>CLONE_SIGHAND</code>	Share the table of signal handlers
<code>CLONE_THREAD</code>	Same thread group (same <code>tgid</code>) — makes it a thread, not a child process
<code>CLONE_SYSVSEM</code>	Share System V semaphore adjustments
<code>CLONE_SETTLS</code>	Set up a new thread-local-storage area for the new task
<code>CLONE_PARENT_SETTID</code> / <code>CLONE_CHILD_CLEARTID</code>	Machinery for <code>pthread_join</code> via a <code>futex</code>
<code>CLONE_NEWNS</code> , <code>CLONE_NEWPID</code> , <code>CLONE_NEWNET</code> , ...	Create <i>new</i> namespaces (the container primitives, Chapter 9)

Now the two familiar operations are just two points in this flag space:

- `fork()` $\approx$ `clone` with *none* of the sharing flags (only `SIGCHLD` so the parent is notified on exit). Nothing is shared: the child gets its own COW copy of the address space, its own FD table, its own signal handlers. Two independent processes.
- `pthread_create()` $\approx$ `clone` with `CLONE_VM` | `CLONE_FS` | `CLONE_FILES` | `CLONE_SIGHAND` | `CLONE_THREAD` | `CLONE_SYSVSEM` | `CLONE_SETTLS` |

`CLONE_PARENT_SETTID` | `CLONE_CHILD_CLEARTID`. Everything is shared: same address space, same FDs, same signal handlers, same thread group. A new thread.

```
flowchart LR
    subgraph SPEC["clone() sharing spectrum"]
        direction LR
        A["fork()
share nothing
(new address space,
own FDs)"]
        B["clone(CLONE_VM
| CLONE_FILES)
share memory + FDs,
separate signals"]
        C["pthread_create()
CLONE_VM|CLONE_FILES
|CLONE_THREAD|...
share everything"]
    end
    A -->|"more sharing ->"| B
    B --> C
    note1["'Process' and 'thread' are not
two kinds of object – they are
two settings of the same dial."]
```

Between the extremes lies a continuum most engineers never touch but which the kernel fully supports: you can share memory but not FDs, share FDs but not memory, and so on. `vfork()` is another point — it shares the address space and *suspends the parent* until the child execs or exits, an optimization for the fork-immediately-exec case that predates COW. The takeaway is conceptual and permanent: **on Linux, "process" and "thread" are not two kinds of object. They are two configurations of one object — a task — distinguished by how much they share.**

Threads on Linux

With `clone` understood, Linux threading is almost anticlimactic. A **thread is a task that shares an address space** (and FDs, signal handlers, and thread-group identity) with its siblings. There is no special "thread" kernel object. This is different from some other kernels that maintain a heavyweight process object with lightweight threads dangling off it; on Linux the thread is a first-class scheduler task, and the process is the group.

1:1 threading (NPTL)

Modern Linux uses the **1:1 threading model**: every userspace thread maps to exactly one kernel task. This is implemented by **NPTL**, the Native POSIX Thread Library, which replaced the older, badly-broken LinuxThreads around Linux 2.6 (2003) and is part of `glibc`. Under 1:1, when you `pthread_create`, the library calls `clone` and the kernel gets a new schedulable task; the kernel scheduler multiplexes those tasks across CPUs directly.

The historical alternative was **M:N threading**: map M userspace threads onto N kernel threads via a userspace scheduler. Linux tried it (the NGPT project) and abandoned it: M:N interacts terribly with blocking syscalls, signals, priorities, and the kernel scheduler, and was not worth the complexity once kernel tasks were made cheap enough. So Linux is firmly 1:1. (Language runtimes that want cheap "threads" — Go goroutines, Java virtual threads — build an *M:N-like* userspace scheduler on *top of* 1:1 kernel threads, to get the cheap-userspace-switch benefit without the kernel-level problems. That is a runtime concern, Volume 4; the kernel still sees one task per OS thread.)

What is shared, what is per-thread

Because a thread is a task sharing selected resources, "what's shared vs. per-thread" is just a reading of the `clone` flags plus a few architectural facts:

Resource	Shared across threads?	Notes
Address space (code, globals, heap, mmaps)	Shared	<code>CLONE_VM</code> — one set of page tables
Open file descriptors	Shared	<code>CLONE_FILES</code> — one FD table; closing in one affects all
Signal handlers (dispositions)	Shared	<code>CLONE_SIGHAND</code> — one handler table
cwd, root dir, umask	Shared	<code>CLONE_FS</code>

Resource	Shared across threads?	Notes
PID (<code>tgid</code>)	Shared	All threads report the same <code>getpid()</code>
Credentials (UID/GID)	Shared by <i>POSIX</i>	Kernel <code>cred</code> is per-task; <code>glibc</code> broadcasts changes (see below)
Stack	Per-thread	Each thread has its own stack (main thread's on the classic stack; others <code>mmap</code> ed)
Registers / instruction pointer	Per-thread	The execution context itself
Thread ID (<code>gettid</code>)	Per-thread	Unique kernel <code>pid</code>
Thread-local storage (TLS)	Per-thread	<code>CLONE_SETTLS</code> ; <code>errno</code> lives here
Signal mask	Per-thread	Each thread can block signals independently (<code>pthread_sigmask</code>)
Scheduling priority / affinity	Per-thread	Set per task
<code>errno</code>	Per-thread	It is a TLS macro, not a global, precisely for this reason

Two subtleties worth carrying:

- **`errno` is per-thread.** In a threaded program `errno` is not a global variable; it expands to a per-thread location in TLS. If it were shared, one thread's failed syscall would clobber another's error code. This is invisible in your C but essential to correctness.
- **Credentials are per-task in the kernel, per-process in POSIX.** The kernel stores credentials in each `task_struct`, and the raw `setuid` syscall changes only the *calling* thread. But POSIX mandates that `setuid()` affect the whole process. `glibc` bridges the gap: its `setuid()` wrapper uses an internal real-time signal to make every thread perform the credential change, so the process-wide semantics

hold. It is a good example of the Linux "threads are tasks" model poking through an abstraction that pretends otherwise.

Threads vs. processes vs. async for backend concurrency

Now the decision you actually make. A backend service that handles concurrent requests must choose a concurrency substrate, and the process model gives you three fundamental options, often combined. This is one of the most consequential architectural choices a service makes, so it deserves a clear-eyed comparison.

Dimension	Threads (one process, many threads)	Processes (many processes)	Async / event loop (one thread, many connections)
Communication	Shared memory — cheap, direct	IPC required: pipes, sockets, shared mem (Ch 8)	Shared memory within the thread; no cross-thread needed
Isolation / fault domain	Weak — one thread's memory bug or crash takes down the whole process	Strong — a worker crash is contained; the rest survive	Weak — a crash or a blocking call stalls all connections on that loop
Memory footprint	Low — one address space, shared code/heap	Higher — N address spaces (COW mitigates shared pages)	Lowest — one stack, per-connection state is small
Synchronization complexity	High — locks, races, memory model (Vol 4)	Low within a worker — no shared mutable state	Low — cooperative, single-threaded logic
CPU parallelism	Yes — threads run on multiple cores	Yes — processes run on multiple cores	No — one loop uses one core (run one loop per core)

Dimension	Threads (one process, many threads)	Processes (many processes)	Async / event loop (one thread, many connections)
Scaling limit	Context-switch cost and lock contention at high thread counts	Memory and fork cost per worker	Excellent connection scaling; blocked by any blocking call
Blast radius of a leak/corruption	Whole process	One worker	Whole process

A few real architectures make the trade-offs concrete:

- **Thread-per-request** (classic Apache `worker` / `event` MPM, many Java servers): a thread handles a connection start to finish. Simple to reason about, but each thread needs a stack (often megabytes of reserved address space) and each blocking I/O parks a whole thread. This is the model that hit a wall at the **C10K problem** (Dan Kegel's famous articulation): ten thousand concurrent connections meant ten thousand threads, and the context-switch and memory overhead made that untenable. C10K is the historical pressure that drove the industry toward event loops and thread pools — we take it up fully in Chapter 7.
- **Multi-process for isolation and to dodge shared state.** `nginx` runs a master process plus a small number of **worker processes** (typically one per core), each running an event loop. Processes, not threads, so that a worker crash cannot corrupt siblings and so each worker is a clean fault domain. **PostgreSQL** forks **one backend process per connection**: strong isolation between sessions, and it long predates good threading — the process model is baked into its architecture. **Python** services frequently reach for the `multiprocessing` module rather than threads for CPU-bound work, because CPython's **Global Interpreter Lock (GIL)** serializes bytecode execution across threads in one interpreter; to use multiple cores for Python-level compute you need multiple *processes*, each with its own interpreter and GIL. (The GIL is a CPython implementation detail — a runtime topic for Volume 4 — but it is a first-order reason real Python services are multi-process.)

- **The hybrid: multi-process *and* multi-threaded (and async).** The dominant high-scale pattern combines them. A supervisor forks **one worker process per core** (fault isolation, no cross-core lock contention, no NUMA-hostile shared heap — see Volume 1, Chapter 6 on NUMA), and *within* each worker runs either a small thread pool or an event loop to multiplex many connections onto that one core's worth of CPU. Apache's `event` MPM, gunicorn with multiple workers each running an async worker class, and Envoy's worker-thread-per-core model are all variations on this theme. You get the isolation of processes at the granularity where it matters (a core) and the efficiency of shared-memory concurrency inside each unit.

The connecting idea — the distributed-systems lens on all of this — is that **multi-process isolation at the node is the same idea as fault isolation across the distributed system, one level down.** A distributed system partitions work across nodes so that one node's failure does not take down the service; a multi-process server partitions work across worker processes so that one worker's segfault does not take down the node. `nginx` workers, PostgreSQL backends, and Python multiprocessing pools are node-level bulkheads, mirroring the bulkheads you draw between services. The threads-vs-processes-vs-async choice is not a micro-optimization; it sets the fault-domain granularity and the scaling ceiling of every node your distributed system runs on.

Context switching and its cost

The scheduler (Chapter 2) multiplexes many tasks over few CPUs by *switching* between them. A **context switch** saves the currently-running task's execution state and restores another's. Understanding what it costs is essential, because "too many runnable threads" is a real and common performance failure, and the cost is the reason.

What actually happens on a context switch from task A to task B:

1. **Enter the kernel.** A switch happens in kernel mode — triggered by A blocking (a syscall that must wait), A being preempted (its timeslice expired, or a higher-priority task woke), or an interrupt.
2. **Save A's register state** into A's `task_struct` (kernel stack): general registers, program counter, stack pointer, and the FPU/SIMD state.
3. **Switch address space if B is in a different process.** If A and B belong to different processes, the kernel loads B's page-table base

into the MMU — on x86-64, writing the CR3 register. **This is the expensive part.** A naive CR3 reload *flushes the TLB* (the translation lookaside buffer, Volume 1, Chapter 3 / Chapter 10), because the cached virtual-to-physical translations belonged to A's address space and are now wrong. After the switch, B runs into a cold TLB and pays page-walk costs on its early memory accesses.

4. **Restore B's register state** and return to userspace, resuming B where it left off.

```
sequenceDiagram
    participant A as Task A (userspace)
    participant K as Kernel (scheduler)
    participant B as Task B (userspace)
    A->>K: block / preempt / interrupt (enter kernel)
    Note over K: save A registers + FPU into A.task_struct
    Note over K: pick B (scheduler, Ch 2)
    alt B in different address space
        Note over K: load B page tables (CR3)
    → TLB flush (unless PCID/ASID)
    cold TLB for B afterwards
    else B is a sibling thread (same address space)
        Note over K: no CR3 reload, TLB stays warm
    end
    Note over K: restore B registers + FPU
    K->>B: return to userspace, resume B
```

The cost has two components:

- **Direct cost:** the cycles to save/restore registers, run the scheduler, and reload page tables. On modern x86 this is on the order of a microsecond or a few, dominated by the register/FPU save-restore and the address-space switch.
- **Indirect cost — usually larger:** cache and TLB pollution. When B runs, the L1/L2 caches and the TLB are full of A's data. B suffers a burst of cache and TLB misses warming up its working set, and A pays again when it is switched back in. This *indirect* cost does not show up in a simple "time to switch" microbenchmark but dominates in practice, especially when many tasks with large working sets rotate through a core. This is the mechanical reason a machine can be "busy" with context switches and yet get little useful work done — the caches never stay warm.

Two crucial refinements:

- **Thread-to-thread within a process is cheaper than process-to-process.** Sibling threads share page tables, so step 3 is skipped — no CR3 reload, no TLB flush. The TLB stays valid across the switch. This is one concrete efficiency advantage of threads over processes, and it is why intra-process concurrency has a lower per-switch tax.
- **PCID/ASID soften the flush.** Modern x86-64 CPUs support PCID (Process Context Identifiers; ARM calls the analogous feature ASID), which tags TLB entries with an address-space identifier. With PCID, switching address spaces need *not* flush the whole TLB — the CPU keeps A's and B's entries side by side, distinguished by tag, so a later switch back to A finds warm entries. The Linux kernel uses PCID, and it materially reduced context-switch cost — a benefit that became conspicuous when the Meltdown mitigation (KPTI, kernel page-table isolation) forced extra address-space switches on every syscall and PCID was what kept that overhead survivable.

The kernel distinguishes **voluntary** context switches (a task blocks on I/O, a lock, or `sleep` — it *gives up* the CPU) from **involuntary** ones (the scheduler *preempts* a runnable task, e.g. its timeslice expired). You can read both counts per process in `/proc/<pid>/status` as `voluntary_ctxt_switches` and `nonvoluntary_ctxt_switches`. High involuntary counts mean CPU contention — more runnable tasks than cores, everyone getting preempted; high voluntary counts mean lots of blocking, often on I/O or locks. This distinction is a first diagnostic when a service is slow.

Do not confuse a context switch with a **user-kernel transition** (a mode switch). A syscall (Chapter 5) crosses from user mode to kernel mode on the *same* task — no page-table switch, no scheduler, no register hand-off to a different task. It is far cheaper than a context switch (tens of nanoseconds to a few hundred, versus a microsecond-plus), though KPTI made even mode switches pricier. Every context switch involves kernel entry, but most kernel entries are *not* context switches.

The distributed-systems consequence closes the loop with the concurrency-model section: **context-switch cost is why thread-per-request does not scale to C10K, and why the industry moved to event loops and bounded thread pools.** If every one of ten thousand connections owns a thread, and thousands are runnable, the core spends

its time switching and polluting caches instead of serving requests. An event loop (Chapter 7) inverts this: one thread stays resident, keeps its caches warm, and multiplexes thousands of connections with *no* context switch between them — you switch stack frames and state machines in userspace, not tasks in the kernel. Sizing thread pools to roughly the core count, rather than the connection count, is the same insight applied to threaded servers. The process model is what makes the cost real; the I/O model (Chapter 7) is how you dodge it.

The process memory layout

A process's address space is not an undifferentiated blob; it is a set of **regions** (the kernel calls them virtual memory areas, VMAs), each with its own permissions and purpose. Knowing the classic layout lets you read a crash address, a memory map, or a `/proc/<pid>/maps` dump and immediately know *what* was at fault. Chapter 3 covers how these mappings are backed by physical memory and paging; here is the layout itself, for a 64-bit Linux process, from low addresses to high:

```
flowchart TB
    K["Kernel space  
(top of address space; not accessible from user mode)"]
    STK["Stack  
(grows DOWN ↓)  
call frames, locals, return adrs"]
    GAP1["... unmapped gap ...  
(ASLR randomizes region bases)"]
    MMAP["mmap / shared-library region  
(libc.so, anonymous mmaps, thread stacks)  
(typically grows DOWN ↓)"]
    GAP2["... unmapped gap ..."]
    HEAP["Heap  
(grows UP ↑; brk / sbrk; malloc arenas)"]
    BSS["BSS  
(uninitialized globals, zero-filled)"]
    DATA["Data  
(initialized globals/statics)"]
    RODATA["Rodata  
(string literals, const)"]
    TEXT["Text / code  
(the program instructions, read-only + executable)"]
    K --- STK --- GAP1 --- MMAP --- GAP2 --- HEAP --- BSS --- DATA --- RODATA --- TEXT
```

Region by region:

- **Text (code).** The machine instructions, mapped read-only and executable from the binary. Read-only so the code cannot be rewritten, and so the same physical pages are shared across every process running the same binary — run a hundred copies of a server and they share one physical copy of the text.
- **Rodata / Data / BSS.** Read-only constants (`rodata`); initialized global and static variables (`data` , loaded from the binary); and **BSS**, the zero-initialized globals, which take *no space in the file* (the loader just maps zero-filled pages) but do take address space and memory once touched.
- **Heap.** Dynamically allocated memory, classically grown *upward* by moving the "program break" with `brk / sbrk`. In practice `malloc` (Chapter 4) also uses `mmap` for large allocations and per-thread arenas, so the neat "heap grows up" picture is only part of the story — but the `brk`-managed heap still exists.
- **mmap region.** Where shared libraries (`libc.so` and friends), large `malloc` chunks, file mappings, and **the stacks of non-main threads** live. When you `pthread_create`, the new thread's stack is `mmap` ed here, not carved from the main stack. This region typically grows downward from below the stack.
- **Stack.** The main thread's call stack, growing *downward*. Each function call pushes a frame (return address, saved registers, locals); it unwinds on return. It grows on demand up to a limit (`RLIMIT_STACK` , commonly 8 MB); overrun that and you get the segfault every engineer has met via infinite recursion — or, historically, a stack-clash attack.
- **Kernel space.** The top of the address space is reserved for the kernel and simply not accessible from user mode; a user access there faults. It is mapped into every process so that syscalls and interrupts can run without a full address-space switch (a design that Meltdown/KPTI complicated, as noted above).

ASLR (Address Space Layout Randomization) randomizes the base of the stack, the `mmap` region, the heap, and — for position-independent executables — the text, so that an attacker cannot predict where anything lives. This is why the addresses in two runs of the same program differ, and why the gaps in the diagram exist.

Note the per-thread multiplicity: there is *one* text, *one* data/BSS, *one* heap — but *one stack per thread*. Each thread's stack is a separate region; the shared heap is where threads communicate. That single fact is the memory-layout expression of "threads share the address space but each has its own stack," and it is exactly why passing a pointer to a stack-local variable to another thread is a bug waiting to happen: that thread is reaching into a region that belongs to a different execution context.

You can read all of this live:

```
$ cat /proc/self/maps
55e3c1a00000-55e3c1a01000
r--p 00000000 fe:01 ... /usr/bin/cat      # text (r--/r-xp)
55e3c1a01000-55e3c1a05000 r-xp 00001000 fe:01 ... /usr/bin/cat
...
55e3c2b1f000-55e3c2b40000 rw-p 00000000 00:00 0      [heap]
7f9a2c000000-7f9a2c028000 r-xp ...          /usr/lib/libc.so.6
# mmap'd library
7ffe1b2c0000-7ffe1b2e1000 rw-p 00000000 00:00 0      [stack]
fffffffff6000000-... --xp ...              [vsyscall]
```

Each line is a VMA: address range, permissions (r/w/x, private or shared), and what backs it. This is the single most useful file for understanding a process's memory at runtime.

Credentials, capabilities, and namespaces

The last piece of the process tuple is the metadata that governs *what a process is allowed to do* — the subject of least-privilege design (Book 1, Chapter 10; Book 6). Briefly, because these thread through the rest of the volume:

- **User and group IDs.** Each task carries a real, effective, and saved UID and GID, plus supplementary groups. The *effective* UID is what permission checks use; the *real* UID is who you are; the *saved* UID lets a program temporarily drop and later regain privilege. UID 0 is `root`. The fork-then-exec window is exactly where a service *drops privileges* — a server may bind port 443 as `root`, then `setuid` to an unprivileged account before serving requests, so a compromise cannot exploit `root`.
- **Capabilities.** Linux breaks the monolithic power of `root` into ~40 discrete **capabilities** — `CAP_NET_BIND_SERVICE` (bind ports below

1024), `CAP_NET_ADMIN`, `CAP_SYS_ADMIN` (dangerously broad), `CAP_CHOWN`, and so on. A process can hold a subset, so you can grant a service exactly the privilege it needs and nothing more. This is the least-privilege lever containers pull: a container runtime drops all capabilities and adds back only the few required, so a "root" process inside a container is far weaker than real root. `prctl(PR_SET_NO_NEW_PRIVS)` further guarantees that no `exec` can gain privilege, closing `setuid`-binary escalation.

- **Namespaces.** Each task points (via `nsproxy`) at a set of **namespaces** — PID, mount, network, UTS, IPC, user, cgroup, time — that virtualize what the process can see of the system. Two processes in different PID namespaces have different views of the process tree (and each namespace has its own PID 1, connecting straight back to the reaping discussion). Namespaces are created with the `CLONE_NEW*` flags to `clone` — the very same syscall that makes threads and processes. This is not a coincidence: **a container is a process (or process tree) created by `clone` with new-namespace flags, running with dropped capabilities and a cgroup limit.** Chapter 9 builds the full container out of exactly these primitives. The process model is the container model; containers are not a separate technology bolted on, but a particular configuration of the task/clone machinery this chapter describes.

Distributed-systems lens

Pull the threads together (so to speak). The process/thread model is the **per-node concurrency substrate** on which every backend service runs, and its details propagate all the way up to system architecture:

- **The concurrency-model choice sets the node's scaling ceiling and fault granularity.** Threads vs. processes vs. async (fleshed out in Chapter 7) is not an implementation detail; it decides how many connections a node serves, how a crash is contained, and how much synchronization complexity your code carries. Every service in your fleet made this choice, and it shows up in their tail latency and their failure modes.
- **Context-switch cost drove an industry-wide architectural shift.** C10K was, mechanically, a context-switch and per-thread-memory wall. The move to event loops (nginx, Node, async runtimes) and

bounded thread pools is the process model's cost structure expressed as architecture. When you choose an async framework, you are choosing to *not pay* the context-switch and TLB-pollution tax this chapter quantified.

- **Multi-process isolation is node-level bulkheading.** nginx workers, PostgreSQL backends, and Python multiprocessing pools isolate faults at the process boundary the same way you isolate faults at the service boundary in a distributed system. The address space is a bulkhead; the fleet is bulkheads all the way up.
- **PID 1 / zombie reaping is a real container operational hazard.** The kernel's reaping contract — orphans reparent to PID 1, PID 1 must `wait` — becomes your problem the moment your app is a container's PID 1. The `tini/dumb-init` pattern exists solely to satisfy a process-model invariant. Miss it and you leak PIDs or ignore `SIGTERM`; Book 6 and Chapter 9 return to this.
- **Containers are the process model, configured.** Namespaces, capabilities, and cgroups are attributes of tasks. The isolation your distributed system leans on at the node — one tenant's container cannot see another's — is the address space, credentials, and namespace membership of ordinary Linux tasks. There is no magic layer; there is `clone` with the right flags.

The rest of this volume zooms into each facet: how the scheduler picks among tasks (Chapter 2), how the address space maps to physical memory (Chapter 3), how the user-kernel boundary is crossed (Chapter 5), how threads and processes actually talk (Chapter 8), and how namespaces and cgroups turn tasks into containers (Chapter 9). All of it stands on the model in this chapter: a process is an address space plus threads plus resources plus metadata, and on Linux, every one of those threads is a task, and every task — thread, process, or container init — was born from `clone`.

Key takeaways

- **A Linux process is a tuple:** a private *address space*, one or more *threads of execution*, a set of *resources* (file descriptors, cwd, timers), and *metadata* (PID, credentials, namespaces). It is the unit of both resource ownership and isolation.

- **Linux has one schedulable object: the task (`task_struct`).** A "process" is a *thread group* — tasks sharing an address space and identity. The kernel's `pid` is the userspace **TID** (`gettid`); the kernel's `tgid` is the userspace **PID** (`getpid`).
- **`fork` is cheap-ish because of copy-on-write:** it copies page tables, marks pages read-only, and duplicates a page only when it is first written. **`exec` replaces the address space** with a new program. Unix splits them so the child can configure the environment (redirect FDs, drop privilege, join namespaces) in the gap before `exec` .
- **A terminated-but-unreaped child is a zombie**, holding a PID until its parent `wait` s. Orphans reparent to PID 1 (or a subreaper), which is why **a container's PID 1 must reap** — the `tini / dumb-init` pattern — and must forward signals.
- **`fork` and `pthread_create` are both `clone` with different sharing flags.** A thread is a task that shares the address space (`CLONE_VM`), FDs, signal handlers, and thread-group identity; a process shares nothing. "Process" and "thread" are two settings of one dial.
- **Linux threading is 1:1 (NPTL):** one kernel task per userspace thread, `glibc`-backed, having replaced M:N. Threads share the address space, FDs, and signal handlers; each has its own stack, registers, TID, TLS (`errno` lives there), and signal mask.
- **Choose threads vs. processes vs. async deliberately:** threads give cheap shared-memory communication at the cost of a shared fault domain and synchronization complexity; processes give isolation at the cost of IPC; async gives connection scaling on one core. Real high-scale servers are hybrids — one process per core, a loop or pool inside.
- **Context switches are expensive** — direct register/page-table cost plus larger *indirect* cache/TLB pollution. Cross-process switches reload page tables (`CR3` , TLB flush unless PCID); sibling-thread switches do not. Too many runnable threads is a real performance failure and the mechanical root of C10K.
- **The address space has structured regions** — text, rodata, data, BSS, heap, mmap (libraries + thread stacks), stack, kernel — randomized by ASLR and readable in `/proc/<pid>/maps` . One heap, one text, but one stack per thread.

- **Credentials, capabilities, and namespaces are task attributes.**

Containers are not a separate mechanism: a container is a task created by `clone` with new-namespace flags, reduced capabilities, and a cgroup limit. The process model is the container model.

Further reading

- **The Linux Programming Interface**, Michael Kerrisk, No Starch Press, 2010. The definitive practitioner's reference for the Linux/Unix process model — `fork`, `exec`, `wait`, `clone`, threads, credentials, process groups and sessions. Chapters 24–28 and 33–35 cover this chapter's material in exhaustive, accurate detail.
- **Linux man pages**, section 2/3: `fork(2)`, `execve(2)`, `wait(2)`, `clone(2)`, `pthread(7)`, `credentials(7)`, `capabilities(7)`, `pid_namespaces(7)`, and `proc(5)` (for `/proc/<pid>/maps` and `/proc/<pid>/status`). The `clone(2)` and `pthread(7)` pages in particular document the exact sharing flags and the 1:1 NPTL model.
- **Understanding the Linux Kernel**, 3rd ed., Bovet and Cesati, O'Reilly, 2005, and **Professional Linux Kernel Architecture**, Wolfgang Mauerer, Wrox, 2008 — for the `task_struct`, process descriptor, and scheduler internals at the source level. Dated on specifics but sound on the model.
- **Linux Kernel Development**, 3rd ed., Robert Love, Addison-Wesley, 2010. Chapters 3 (process management) and 4 (process scheduling) are a concise, accurate tour of tasks, `clone`, and the thread-group model.
- Dan Kegel, "**The C10K Problem**" (kegel.com/c10k.html). The historical statement of the connection-scaling wall that thread-per-request hit, and the survey of I/O models that followed — essential background for Chapter 7.
- Ulrich Drepper, "**The Native POSIX Thread Library for Linux**" (2003) and "**Thread-Local Storage Dwarves**" — primary sources on NPTL's 1:1 design and TLS, from one of its authors.
- "**Copy-on-write**" and **PCID/ASID** treatments in the Linux kernel documentation (`Documentation/`), and Ulrich Drepper, "**What Every Programmer Should Know About Memory**" (2007) for TLB and address-space-switch costs underpinning the context-switch discussion.

- The `tini` and `dumb-init` project READMEs (github.com/krallin/tini, github.com/Yelp/dumb-init) for the container PID-1 reaping and signal-forwarding problem, and Docker's `--init` documentation.

Chapter 2 — CPU Scheduling: CFS, Real-Time, and cgroup CPU Control

What this chapter covers. Chapter 1 gave us the runnable thing — the *task*. This chapter is about the decision the kernel makes millions of times a second: *given many runnable tasks and few CPUs, which task runs next, on which CPU, for how long?* That decision is the CPU scheduler, and for a backend engineer it is not academic. It determines your service's tail latency, how fairly a shared node divides its cores among tenants, and whether a container that is nowhere near its CPU budget still stalls for tens of milliseconds. The single most common cause of "mysterious p99 latency" in a containerized fleet — CFS quota throttling — is a scheduling phenomenon, and by the end of this chapter you will understand its mechanism well enough to diagnose and fix it.

We stay precise about Linux specifically: the scheduling-class hierarchy, the Completely Fair Scheduler and its `vruntime`/weight machinery, its EEVDF successor (merged in 6.6), how the scheduler behaves across many CPUs and NUMA nodes, the kernel preemption models, and — the practical heart of the chapter — how cgroup CPU controls (`cpu.weight`, `cpu.max`, `cpuset`) and their Kubernetes surface (`requests/limits`) actually work underneath.

Learning goals — after this chapter you should be able to:

- State the scheduling problem and its tensions (throughput vs. latency vs. fairness vs. power), and explain preemption, time slices, and the per-CPU run queue.
- Place Linux's scheduling policies in their strict priority hierarchy — `SCHED_DEADLINE` > real-time (`SCHED_FIFO` / `SCHED_RR`) > normal (`SCHED_OTHER` / CFS) > `SCHED_BATCH` / `SCHED_IDLE` — and know when each is appropriate and how RT policies can starve a system.
- Explain CFS from first principles: `vruntime`, nice-as-weight, the red-black tree ordered by `vruntime`, and how targeted latency plus

minimum granularity replace fixed time slices; and describe accurately what EEVDF changes.

- Reason about multiprocessor scheduling: per-CPU run queues, load balancing, migration cost, affinity/ `cpuset` , NUMA-aware placement, and the locality-vs-balance tension.
- Explain the CFS bandwidth (quota) throttling mechanism, why it produces latency spikes on a node with idle CPU, how Kubernetes `requests / limits` map to `shares / quota` , and why setting CPU limits can *hurt* latency.
- Observe scheduling with `/proc` , `schedstat` , `perf sched` , run-queue-latency tooling, and PSI, and diagnose scheduling-induced latency.

The scheduling problem

A modern server has a few dozen to a few hundred logical CPUs and, at any instant, potentially thousands of runnable threads: your service's worker pool, the sidecars, the runtime's GC and JIT threads, kernel worker threads, the neighbours' pods. The scheduler *multiplexes* those runnable tasks onto the physical CPUs, giving each a turn. It runs constantly and it must be cheap — a scheduler that spends microseconds deciding is a scheduler that has stolen those microseconds from real work.

The goals are in genuine tension, and no single policy optimizes all of them:

- **Throughput** — total useful work per unit time. Favoured by long time slices (fewer context switches, warmer caches) and by never preempting a task that is making progress.
- **Latency / responsiveness** — how quickly a newly-runnable task gets a CPU. Favoured by short slices and aggressive preemption, which cost throughput. This is the one backend tail latency lives and dies on: a request-handler thread that becomes runnable when its I/O completes wants a CPU *now*, not in 12 ms.
- **Fairness** — every task (or every tenant) gets its due share; nobody starves. On a multi-tenant node this is *isolation*: one tenant's runaway thread must not consume the node.

- **Power / efficiency** — on laptops and increasingly in data centres, keeping CPUs in deep idle states and packing work onto few cores saves energy, which fights the latency goal of spreading work out.

Linux resolves these with **preemptive scheduling**: the kernel can take a CPU away from a running task involuntarily — at a timer interrupt, at a wakeup that makes a higher-priority task runnable, or (depending on the preemption model) at almost any kernel point. This is unlike cooperative scheduling, where a task keeps the CPU until it voluntarily yields. Preemption is what bounds how long any one task can monopolize a core and is therefore the foundation of both fairness and latency.

The unit of "a turn" is the **time slice** (quantum): the interval a task is allowed to run before the scheduler reconsiders. Classic schedulers used a fixed slice; CFS, as we will see, computes it dynamically. When a task's slice expires, or a higher-priority task wakes, the kernel performs a **context switch** (Chapter 1) — save the outgoing task's registers, load the incoming task's, possibly switch the address space — and the new task runs.

Runnable tasks live on a **run queue**, and the first structural fact to internalize is that Linux keeps **one run queue per CPU** (`struct rq`), not one global queue. Each CPU schedules primarily from its own queue, which is what makes the scheduler scale to hundreds of cores without a single contended lock. The cost of that design — keeping the per-CPU queues balanced — is *load balancing*, discussed later. A task is on exactly one CPU's run queue at a time; moving it to another is a *migration*, and migrations are not free.

Scheduling classes and policies

Linux does not have one scheduler. It has a stack of **scheduling classes**, consulted in a strict, fixed priority order. When a CPU needs to pick the next task, it asks each class in turn, highest first, "do you have a runnable task?" The first class that says yes wins. Only when every higher class is empty does a lower class get to run at all.

```

flowchart TB
    STOP["stop_sched_class  
(internal: CPU stop, migration)"]
    DL["dl_sched_class  
SCHED_DEADLINE (EDF + CBS)"]
    RT["rt_sched_class  
SCHED_FIFO / SCHED_RR (prio 1-99)"]
    FAIR["fair_sched_class  
SCHED_OTHER/NORMAL (CFS/EEVDF)  
+ SCHED_BATCH"]
    IDLE["idle_sched_class + SCHED_IDLE  
(only when nothing else runnable)"]
    STOP --> DL
    DL --> RT
    RT --> FAIR
    FAIR --> IDLE
    classDef top fill:#b71c1c,color:#fff
    classDef mid fill:#e65100,color:#fff
    classDef low fill:#1b5e20,color:#fff
    class STOP,DL top
    class RT mid
    class FAIR,IDLE low

```

The consequence is absolute precedence: a single busy `SCHED_FIFO` task at priority 50 will completely lock out every normal (CFS) task on its CPU for as long as it runs, because the RT class is always consulted before the fair class. This is a feature for latency-critical code and a foot-gun for everything else.

The policies, from a task's point of view (set with `sched_setscheduler(2)` / `sched_setattr(2)`, or `chrt`):

Policy	Class	Selection rule	Time slice	Typical use
<code>SCHED_DEADLINE</code>	deadline	Earliest virtual deadline (EDF) with bandwidth reservation (CBS)	Per-task runtime / period	Hard-ish real-time: media, control loops with explicit CPU budgets

Policy	Class	Selection rule	Time slice	Typical use
<code>SCHED_FIFO</code>	real-time	Highest fixed priority (1-99); runs until it blocks/yields/is preempted by higher prio	None (run to completion)	Latency-critical threads that must not be interrupted by normal work
<code>SCHED_RR</code>	real-time	Like FIFO but round-robins equal-priority tasks	<code>sched_rr_timeslice_ms</code> (default 100 ms)	RT work that should share among equal-priority peers
<code>SCHED_OTHER</code> (a.k.a. <code>SCHED_NORMAL</code>)	fair	Fair share by weight (CFS/EEVDF)	Dynamic	The default — essentially all application threads
<code>SCHED_BATCH</code>	fair	Like <code>SCHED_OTHER</code> but assumed CPU-bound; no wakeup preemption	Dynamic	Throughput batch jobs that should not preempt interactive tasks
<code>SCHED_IDLE</code>	fair (very low weight)	Runs only when nothing else wants the CPU	Dynamic	Best-effort background work; the lowest of the low

Note the two ends. `SCHED_DEADLINE` sits *above* the RT priorities and uses Earliest Deadline First with a Constant Bandwidth Server: each task declares a `(runtime, deadline, period)` triple — "I need `runtime` microseconds of CPU every `period`, done by `deadline`" — and the kernel

runs **admission control**, refusing `sched_setattr` if the total reservation would exceed capacity. That admission control is what makes it the only Linux policy with a real schedulability guarantee. At the other end, `SCHED_IDLE` tasks (weight 3, versus 1024 for a nice-0 task) get a CPU only when literally nothing else is runnable.

Real-time policies and the danger of starvation

`SCHED_FIFO` and `SCHED_RR` implement classic fixed-priority real-time scheduling over 99 priority levels (higher number = higher priority). A `SCHED_FIFO` task, once running, keeps the CPU until it voluntarily blocks (I/O, lock, `sched_yield`) or a *higher*-priority RT task preempts it. It is never preempted by an equal- or lower-priority task and never by a normal task. `SCHED_RR` adds a round-robin quantum so that equal-priority RT tasks take turns.

This is exactly what a low-jitter thread wants — a packet-processing thread, an audio callback, a market-data handler. It is also a loaded gun pointed at the whole system. A `SCHED_FIFO` task that spins in a loop and never blocks will starve *everything* of lower priority on its CPU indefinitely, including kernel threads the system needs to make progress. A classic way to hang a box is a buggy real-time thread in an infinite loop.

Linux ships a safety valve: **RT throttling**. By default the RT classes are collectively capped at `sched_rt_runtime_us` out of `sched_rt_period_us` — 950,000 of 1,000,000 microseconds, i.e. RT tasks may consume at most 95% of each CPU-second, leaving 5% for normal tasks so the system can recover and you can log in to kill the runaway. You can see and tune these in `/proc/sys/kernel/`. Do not disable the throttle unless you have engineered the RT workload to be provably bounded.

```
# Inspect the RT bandwidth safety valve
$ cat /proc/sys/kernel/sched_rt_period_us      # 1000000
$ cat /proc/sys/kernel/sched_rt_runtime_us     # 950000 (95% cap for
all RT tasks)

# Run a latency-critical thread at FIFO priority 80
$ chrt -f 80 ./packet_pump
# Move an existing thread to SCHED_IDLE (lowest)
$ chrt -i 0 -p <pid>
```

CFS: the Completely Fair Scheduler

Almost everything you run is `SCHED_OTHER`, handled by the fair class. From 2007 (kernel 2.6.23) until 6.6, that class *was* the Completely Fair Scheduler. CFS is worth understanding in its own right both because it still describes the mechanism accurately for most deployed kernels and because EEVDF is an evolution of it, not a replacement of its core ideas.

The core idea: ideal fairness via virtual runtime

CFS models an idealized, infinitely-fine multitasking CPU: if there are N equally-important runnable tasks, each should get exactly $1/N$ of the CPU, all running "simultaneously" at $1/N$ speed. Real hardware runs one task at a time, so CFS approximates that ideal by always running the task that is *furthest behind* its fair share.

To measure "furthest behind," CFS gives every task a **virtual runtime** (`vruntime`): a running total of CPU time the task has consumed, *weighted by priority*. For a default-priority (nice 0) task, `vruntime` advances at exactly wall-clock rate while it runs. The scheduling rule is then almost trivially simple:

Always run the runnable task with the smallest `vruntime`.

A task that has run a lot has a large `vruntime` and drifts to the back; a task that just woke up, or has been starved, has a small `vruntime` and gets picked. Over time everyone's `vruntime` advances at roughly the same rate, which *is* fairness. The kernel also tracks `min_vruntime` per run queue — a monotonically-advancing floor — and places newly-woken tasks near it so a task that slept for an hour cannot come back with `vruntime = 0` and monopolize the CPU to "catch up."

Nice as a weight, not a slice

The subtlety that trips people up: **nice values do not directly set time slices**. They set a *weight*, and the weight controls the *rate* at which `vruntime` accumulates. Each task has a weight from the kernel's `sched_prio_to_weight[]` table; nice 0 maps to weight 1024 (`NICE_0_LOAD`), and each nice level changes the weight by roughly 1.25×, chosen so that one nice level corresponds to about a 10% change in CPU share. The `vruntime` update is:

```
delta_vruntime = delta_exec * (NICE_0_LOAD / task_weight)
```

So a high-priority task (low nice, large weight) accumulates `vruntime` *slowly* — for the same real CPU time, its virtual clock ticks less — so it stays near the front of the queue and is picked more often. A niced-down task (high nice, small weight) accumulates `vruntime` fast and falls behind. Two nice-0 tasks split a CPU 50/50; a nice-0 against a nice-5 splits it roughly 75/25. Priority is expressed as *how fast your fairness clock runs*, which is a far more elegant mechanism than handing out bigger quanta.

```
flowchart LR
    subgraph RB["Per-CPU run queue: red-black tree keyed by vruntime"]
        direction TB
        N1["vruntime = 105  
(nice 0)"]
        N2["vruntime = 140  
(nice 0)"]
        N3["vruntime = 210  
(nice +5, ticks fast)"]
        N4["vruntime = 300"]
        N1 --> N2
        N1 --> N3
        N3 --> N4
    end
    LEFT["Leftmost node  
= smallest vruntime  
= run NEXT (cached, 0(1))"]
    LEFT -.-> N1
```

CFS stores the runnable tasks in a **red-black tree keyed by `vruntime`**. The leftmost node is the smallest `vruntime` — the next task to run — and the kernel caches a pointer to it, so picking is O(1); inserting a woken or re-queued task is O(log N). When the current task's slice ends, its updated `vruntime` is reinserted and the new leftmost is chosen.

No fixed slices: targeted latency and minimum granularity

How long does a task run before CFS reconsiders? Not a constant. CFS aims to give every runnable task a turn within a **targeted scheduling latency** (`sched_latency_ns`, on the order of 6–24 ms, scaled up with CPU count). It divides that window among the runnable tasks in proportion to their weights: with 4 equal tasks and a 24 ms target, each gets ~6 ms; with 8 tasks, ~3 ms each. This adapts the slice to load automatically —

light load means long slices and good throughput; heavy load means short slices and good responsiveness.

There is a floor. If you divide the target among too many tasks, the slice shrinks until context-switch overhead dominates. So CFS enforces a **minimum granularity** (`sched_min_granularity_ns`, around 0.75–3 ms): no task runs for less than that. Below a threshold task count the effective latency window simply grows. A related knob, `sched_wakeup_granularity_ns`, controls how much better a waking task's `vruntime` must be before it preempts the current task — hysteresis that prevents wakeup thrashing. These live under `/proc/sys/kernel/` (or `/sys/kernel/debug/sched/` on newer kernels).

EEVDF: the successor merged in 6.6

Since kernel 6.6 (October 2023), the default fair-class scheduler is **EEVDF** — Earliest Eligible Virtual Deadline First — replacing the classic CFS pick rule. Treat what follows as an accurate but deliberately high-level account; EEVDF is still maturing and details shift release to release, and the policy name userspace sees is unchanged (`SCHED_OTHER` / `SCHED_NORMAL`, in `kernel/sched/fair.c`).

EEVDF keeps the fair-share core — weights, a virtual clock, a (now augmented) red-black tree — but replaces "always pick the smallest `vruntime`" with a formulation that gives better *latency* guarantees. Two concepts:

- **Lag** — the difference between the CPU time a task *should* have received (its fair share) and what it *actually* got. A task with positive lag is owed time; with negative lag it is ahead. A task is **eligible** only when its lag is non-negative — i.e. it has not yet consumed more than its fair share. This is a cleaner fairness invariant than CFS's `min_vruntime` heuristics.
- **Virtual deadline** — from a task's requested slice length and its virtual clock, EEVDF computes a virtual deadline and, among *eligible* tasks, runs the one with the **earliest deadline**. Shorter requested slices produce nearer deadlines, so latency-sensitive tasks that ask for small slices get scheduled promptly without needing a higher priority.

The practical payoff is that **latency becomes a first-class, per-task knob** distinct from CPU share. CFS had only nice, which coupled "how

much CPU" and, indirectly, "how promptly." EEVDF introduces **latency-nice** (`sched_setattr` with `SCHED_FLAG_LATENCY_NICE`): a task can ask to be scheduled more eagerly (shorter slices, earlier deadlines) *without* claiming a larger share, or accept higher latency in exchange for longer, throughput-friendly slices. For a backend service this is the knob you have always wanted — "this request-handler pool is latency-critical, this batch pool is not" — expressed independently of relative CPU weight. For most operational purposes CFS-era intuition still holds: fair share by weight, priority via nice, plus a better handle on tail latency.

Scheduling on many CPUs

Everything so far described one CPU and one run queue. Real servers have many, and the second half of the scheduler's job is deciding *which* CPU a task runs on. The design is per-CPU run queues (for scalability) plus **load balancing** to keep them roughly even — and the whole thing is a negotiation between two opposing goods: **balance** (keep all CPUs busy) and **locality** (keep a task where its data is warm).

Migration is not free — and the reason is Volume 1

When a task migrates from CPU A to CPU B it leaves behind everything CPU A had cached on its behalf: L1/L2 lines, branch predictor state, and — if B is on a different socket — the L3 and, crucially, *NUMA locality*. As Volume 1, Chapter 6 (NUMA and Cache Coherence) explains, a task's memory is physically resident in some NUMA node's DRAM; a thread pinned near that memory reaches it in tens of nanoseconds, while a thread on a remote socket pays a cross-interconnect penalty on every miss. Migrating a task across sockets can silently double its effective memory latency until (if ever) its pages follow it. So the scheduler treats migration as expensive and balances *reluctantly*, especially across sockets.

Scheduling domains model the topology

Linux represents the machine's topology as a hierarchy of **scheduling domains** (`sched_domain`): SMT siblings (two hardware threads on one core) at the bottom, then cores sharing an L2/L3, then a socket/NUMA node, then the whole machine. Balancing is *cheap and aggressive* within low domains (moving between SMT siblings loses almost nothing) and *expensive and reluctant* across high domains (moving across NUMA

nodes loses locality). The scheduler runs periodic load-balancing passes at each domain level, and on the fast paths — a task waking up, a CPU going idle — it makes placement decisions using heuristics like `wake_affine` (put a woken task near the task that woke it, since they likely share data) balanced against spreading load.

```
flowchart TB
    MC["Machine (NUMA domain)"]
    S0["Socket 0 / NUMA node 0  
(shared L3, local DRAM)"]
    S1["Socket 1 / NUMA node 1  
(shared L3, local DRAM)"]
    C0["Core 0 (L2)"]
    C1["Core 1 (L2)"]
    T0["CPU0 (SMT)"]
    T1["CPU1 (SMT)"]
    MC --- S0
    MC --- S1
    S0 --- C0
    S0 --- C1
    C0 --- T0
    C0 --- T1
    MC -. "balance rarely  
(migration = lost locality)" .-> S1
    C0 -. "balance freely  
(cheap)" .-> C1
```

Affinity, cpuset, and NUMA-aware placement

You can override the scheduler's placement:

- **CPU affinity** — `sched_setaffinity(2)` or `taskset` pins a thread to a set of CPUs. The scheduler will not migrate it off that set. Pinning a latency-critical thread to a dedicated core keeps its cache warm and removes it from load-balancing churn — the price is you have hand-bound one core's worth of policy the scheduler can no longer optimize.
- **cpuset** — the cgroup way (below) to confine a whole group of tasks to a set of CPUs and NUMA nodes.
- **Automatic NUMA balancing** (`kernel.numa_balancing`) — the kernel samples page faults to learn which node a task actually touches and migrates *pages toward the task* and *tasks toward their pages*, converging a workload onto local memory. It helps unpinned NUMA

workloads but adds fault overhead; latency-sensitive services often prefer explicit pinning to leave nothing to chance.

```
# Pin a thread pool to cores 8-15 (e.g. one NUMA node)
$ taskset -c 8-15 ./service
# Inspect where a running thread is allowed to run
$ taskset -cp <tid>
# Confirm memory/CPU locality
$ numactl --hardware          # node distances
$ numastat -p <pid>          # per-node memory of a process
```

The locality-vs-balance tension is a genuine engineering choice, not a solved problem. Let the scheduler balance freely and you get high utilization but jittery latency as threads migrate; pin aggressively and you get predictable latency but risk idle cores while other cores queue. Latency- critical services lean toward pinning and isolation; throughput batch fleets lean toward letting the balancer pack work tightly.

Preemption and latency

When the kernel is allowed to preempt determines your worst-case scheduling delay, which translates directly into tail latency. Linux is built with one of several **preemption models**:

Model	Kernel preemptible?	Latency	Throughput	Use
PREEMPT_NONE	Only at syscall return / explicit points	Worst	Best	Throughput servers, batch
PREEMPT_VOLUNTARY	Adds explicit reschedule points in long kernel paths	Better	Slightly less	General/ desktop-ish
PREEMPT (full)	Almost anywhere except critical sections	Good	Lower	Low-latency desktop/ interactive

Model	Kernel preemptible?	Latency	Throughput	Use
<code>PREEMPT_RT</code>	Nearly everything preemptible (sleeping locks, threaded IRQs)	Best/bounded	Lowest	Hard real-time, determinism

With `PREEMPT_NONE`, a task executing a long operation *in the kernel* cannot be preempted until it returns to user space or hits a voluntary point; a higher-priority task that wakes must wait, adding to its scheduling delay. That is fine for throughput but bad for tails. Full `PREEMPT` shortens the non-preemptible windows at some throughput cost.

`PREEMPT_RT` is the real-time variant, long maintained out-of-tree and now largely mainlined (the bulk landed by 2024, around kernel 6.12). It converts almost all kernel spinlocks into sleeping mutexes, threads interrupt handlers so they can be scheduled and preempted, and makes critical sections as short as possible. The result is a bounded, small worst-case scheduling latency — what hard real-time needs — at a real throughput cost. Most backend fleets do *not* run `PREEMPT_RT`; they run a general model and lean on `SCHED_FIFO` / `DEADLINE` for the few threads that need determinism.

Modern kernels also support **dynamic preemption** (`PREEMPT_DYNAMIC`): the model can be selected at boot (`preempt=none|voluntary|full`) or switched at runtime, so a single kernel build can be tuned per node without recompiling.

The tick. The scheduler is driven partly by a periodic timer interrupt, the **tick**, at `CONFIG_HZ` (commonly 250 or 1000 Hz). Each tick is a chance to update `vruntime` accounting and decide whether to preempt. But a tick on every CPU every millisecond is overhead and jitter. Two mitigations:

- **NOHZ_IDLE** (tickless idle, standard everywhere) — stop the periodic tick on a CPU that is idle, so it can stay in a deep C-state and save power, waking only for real work.
- **NOHZ_FULL** (full tickless) — on a CPU running exactly one runnable task, stop the tick *even while busy*. With nothing to preempt to, the

tick is pure noise. Combined with `isolcpus` / `cpuset` isolation and IRQ steering, `NOHZ_FULL` gives a nearly-uninterrupted core for a latency-critical pinned thread — the standard recipe for jitter-sensitive workloads (packet processing, HFT, HPC).

cgroup CPU control: the multi-tenant knob

Everything above schedules *tasks*. On a shared node — which is every Kubernetes node — you also need to allocate CPU among *tenants*: pods, containers, services owned by different teams. That is the job of the **cgroup CPU controllers**, and this is the material most likely to bite you in production. (cgroups themselves — the hierarchy, controllers, and delegation — are Chapter 9; here we cover the CPU controllers' scheduling behaviour.) We describe cgroup **v2**, the unified hierarchy that is now the default on modern distros and Kubernetes; the v1 names are noted for translation.

There are two fundamentally different CPU controls, and conflating them is the root of most confusion:

Proportional share: `cpu.weight` (v1: `cpu.shares`)

`cpu.weight` (range 1-10000, default 100; v1 `cpu.shares`, default 1024) sets a group's *relative* claim on CPU. It is **soft and only matters under contention**. If two sibling groups have weights 100 and 300 and both are CPU-hungry on a contended CPU, they get 25% and 75% respectively. If one is idle, the other may use the *entire* CPU — weight imposes **no cap**. This maps cleanly onto the CFS/EEVDF weight machinery: a cgroup is essentially a scheduling entity with a weight, and the fair scheduler divides time among groups exactly as it divides time among tasks. Weight gives you *fairness and isolation without waste*: a well-behaved tenant is never throttled below its share, but is free to soak up idle CPU that no one else wants.

Hard cap: `cpu.max` (v1: `cpu.cfs_quota_us` / `cpu.cfs_period_us`)

`cpu.max` sets an absolute ceiling: "`<quota> <period>`" in microseconds, e.g. "`50000 100000`" means "at most 50,000 μ s of CPU time per 100,000 μ s period" = 0.5 CPU-equivalent, *no matter how idle the node is*. This is **CFS bandwidth control** (a.k.a. CFS quota/throttling). The default period is 100 ms. Once a group has consumed its quota within a period, every

task in the group is **throttled** — removed from the run queues — until the next period boundary, even if there are free CPUs sitting idle right next to it. That "even if idle" is the entire problem, and we devote the next section to it.

cpuset : pinning groups to CPUs and NUMA nodes

The third control is `cpuset` (`cpuset.cpus` , `cpuset.mems`): confine a whole cgroup to a specific set of CPUs and NUMA nodes. Unlike weight and quota, which are *time*-based, `cpuset` is *space*-based — it partitions cores rather than time. It is how you give a latency-critical service exclusive cores (no noisy neighbours, warm caches, local memory) and how Kubernetes' CPU Manager static policy pins Guaranteed pods. `cpuset` is the strongest isolation of the three and, notably, is the *only* one that does not cause quota throttling — a pinned pod on dedicated cores has no aggregate time budget to exhaust.

Control	v2 name	Semantics	Cap under idle node?	Best for
Proportional share	<code>cpu.weight</code>	Relative CPU share when contended	No — can burst to full	Fair multi-tenancy without waste
Hard bandwidth	<code>cpu.max</code>	Absolute quota per 100 ms period	Yes — throttled even if idle	Billing caps, hard isolation (with a cost)
CPU pinning	<code>cpuset.cpus</code> / <code>cpuset.mems</code>	Confine group to specific CPUs/NUMA nodes	N/A (space, not time)	Latency-critical exclusive cores

```

flowchart TB
    subgraph W["cpu.weight (proportional)"]
        direction TB
        A["Group A weight 100"]
        B["Group B weight 300"]
        CPUw["Contented CPU:"]
        A --> CPUw
        B --> CPUw
    end
    subgraph Q["cpu.max (hard cap)"]
        direction TB
        G["Group quota 50ms / 100ms period"]
        CPUq["Uses 50ms, then THROTTLED"]
        G --> CPUq
    end

```

CFS quota throttling: the fleet-wide p99 killer

This is the single most valuable operational idea in the chapter. CFS bandwidth throttling is a top cause of unexplained tail latency in containerized backends, and the mechanism is subtle enough that teams chase it for weeks.

The mechanism

Quota is granted **per period** (default 100 ms) as an amount of *aggregate CPU time* summed across *all threads* in the group. Consider a container with `cpu.max = "40000 100000"` — 40 ms of CPU per 100 ms, i.e. "0.4 CPUs" on paper. Now suppose the container is a typical multi-threaded service (a JVM, a Go binary, a Node process with a worker pool) with, say, 8 runnable threads and it receives a burst of requests. Those 8 threads run *in parallel* on 8 different cores. They burn 40 ms of aggregate CPU time in as little as **5 ms of wall-clock time** (8 threads × 5 ms = 40 ms). Quota exhausted. For the **remaining 95 ms of the period, every thread in the container is throttled** — off-CPU, not runnable — *even though the node has dozens of idle cores*.

```
sequenceDiagram
    participant App as Container (8 threads, quota 40ms/100ms)
    participant CFS as CFS bandwidth controller
    participant Node as Node (many idle CPUs)
    Note over App,Node: Period boundary – quota refilled to 40ms
    App->>CFS: 8 threads run in parallel, burn CPU fast
    Note over App: 40ms aggregate consumed in ~5ms wall-clock
    CFS->>App: Quota exhausted – THROTTLE all threads
    Note over App,Node: 95ms of wall-clock: threads off-CPU
    Note over Node: CPUs sit IDLE – nothing may use the quota
    Note over App: In-flight requests stall ~95ms → p99 spike
    Note over App,Node: Next period boundary – refill, repeat
```

The result, seen from a client, is a request that was flying along suddenly stalling for the tail of a period — up to ~95 ms — because the handler thread was throttled mid-request. Averaged over many requests it barely dents mean latency (the container really is using only its 0.4 CPUs on average), but it wrecks p99/p999, which is exactly what SLOs are written against. The **more threads, the worse the amplification**: parallelism drains the aggregate budget faster in wall- clock terms, so highly-threaded runtimes are hit hardest, and giving a container *more* cores can make throttling *worse* if a low limit stays fixed.

The historical accounting bug (pre-5.4)

For years the problem was compounded by a genuine kernel bug. CFS bandwidth distributed quota to per-CPU "slices" (default 5 ms) and *expired* unused slices at period boundaries. On a many-CPU box, a multi-threaded app would get small slices scattered across CPUs, fail to use them fully before they expired, and get throttled while *below* its configured quota — throttling that should not have happened at all. This was fixed in **kernel 5.4** (2019) by Dave Chiluk, commit `de53fd7aedb1` "sched/fair: Fix low cpu usage with high throttling by removing expiration of slice" — unused runtime within a period is no longer expired. If you are on a pre-5.4 kernel and see throttling far below the limit, that bug is likely why; upgrade. Note the distinction: 5.4 fixed *spurious* throttling below quota, but the *inherent* throttling of a bursty, parallel app that genuinely hits its quota is not a bug — it is the quota doing exactly what it says.

Later kernels (5.14+) added **CFS burst** (`cpu.max.burst`): a group may accumulate a bounded amount of unused quota from quiet periods and spend it to absorb a spike, smoothing the bursty- workload case without

raising the average cap. It mitigates but does not eliminate the fundamental tension.

How Kubernetes maps onto this — and the "don't set CPU limits" debate

Kubernetes exposes exactly these two knobs as pod `requests` and `limits`, and the mapping is direct:

Kubernetes field	cgroup control (v2)	Meaning	Effect
CPU request (e.g. 500m)	cpu.weight (from cpu.shares)	Relative share; 1000m ≈ 1024 shares	Scheduling weight + node bin-packing; guarantees a <i>floor</i> under contention, no cap
CPU limit (e.g. 500m)	cpu.max quota (limit/1000 × 100ms)	Absolute ceiling per 100 ms period	Hard cap → CFS throttling when exhausted

A `request` sets `cpu.weight` (via shares: 1 core ≈ 1024 shares) *and* is what the kube-scheduler uses to bin-pack pods onto nodes — it reserves a proportional floor without limiting upside. A `limit` sets `cpu.max` quota: a 500m limit becomes quota 50000 per 100000 µs period. The moment a pod's threads collectively burn 50 ms of CPU inside a 100 ms window, they are throttled for the rest of it.

This produces the well-known guidance, argued forcefully by Kubernetes maintainers (Tim Hockin among them) and echoed across SRE incident write-ups: **be very cautious with CPU limits, and often omit them.** The reasoning:

- **Limits cause throttling that hurts latency even when the node is idle.** You pay tail-latency cost to enforce a ceiling that, on an under-utilized node, buys you nothing.
- **Requests already provide isolation.** `cpu.weight` guarantees each pod its proportional floor under contention *and* lets it use idle CPU that would otherwise be wasted. That is usually the behaviour you actually want from "fair sharing."

- **Correctly-set requests plus no limits** gives every pod at least its share when the node is busy and full burst when it is not — better p99 than the same pods throttled to their limits.

The counter-arguments are real and you should not cargo-cult "never set limits":

- Limits give **predictability and hard isolation** — a mandatory ceiling so one tenant cannot starve co-tenants during a bug or attack, and reproducible capacity planning.
- Some environments require them for **billing, chargeback, or multi-tenant guarantees**.
- For a **Guaranteed** QoS pod with *integer* CPU requests, the **CPU Manager static policy** side-steps the whole problem: it gives the pod *exclusive, pinned* cores via `cpuset` instead of a time quota, so there is no aggregate budget to exhaust and no throttling — the right tool for a latency-critical service. (Kubernetes QoS classes: **Guaranteed** = requests equal limits; **Burstable** = requests below limits; **BestEffort** = neither.)

The pragmatic synthesis most teams converge on: **set CPU requests carefully** (they drive both fairness and bin-packing), **avoid CPU limits on latency-sensitive services** unless a specific isolation or billing requirement forces them, and for the truly latency-critical, **pin with cpuset** (CPU Manager static policy) rather than cap with quota. Always **monitor throttling** (next section) so the decision is data-driven.

Observing scheduling

You cannot fix what you cannot see, and scheduling problems hide well — the CPU graphs look fine, the app just occasionally stalls. The tools, roughly from cheapest to deepest:

cgroup throttling counters — check these first for any container.

`cpu.stat` exposes exactly whether and how much a group is throttled:

```
$ cat /sys/fs/cgroup/<path>/cpu.stat
usage_usec 1423000000
nr_periods 84210      # bandwidth-control periods elapsed
nr_throttled 5130     # periods in which the group WAS throttled
throttled_usec 41230000 # total time spent throttled
```

A non-trivial `nr_throttled / nr_periods` ratio, or growing `throttled_usec`, is a smoking gun. In a fleet this is surfaced by cAdvisor as `container_cpu_cfs_throttled_periods_total` and `container_cpu_cfs_throttled_seconds_total` in Prometheus — alert on the ratio. Many p99 investigations end here.

Pressure Stall Information (PSI) — the modern, low-overhead answer to "is *anything* stalling for lack of CPU?" `/proc/pressure/cpu` (and per-group `cpu.pressure`) reports the share of time tasks were runnable but *waiting for a CPU*:

```
$ cat /proc/pressure/cpu
some avg10=7.35 avg60=4.12 avg300=1.90 total=98234123
```

`some` = at least one task was stalled on CPU. Rising `some` pressure means real CPU contention — runnable work that could not get a core — distinct from raw utilization, and a far better signal for saturation and for autoscaling than load average.

Run-queue latency — the time a task spends *runnable but not yet running* (waiting in the run queue) is the most direct measure of scheduling delay and a primary tail-latency contributor. `runqlat` (a bcc/BPF tool, Chapter 11) prints it as a histogram:

```
$ runqlat 10 1
  usecs      : count    distribution
  16 -> 31   : 4821     |*****
|
  32 -> 63   : 9210     |*****
|
  ...
  4096 -> 8191 : 137     |*
| <- tail: 4-8ms in queue
```

Those multi-millisecond outliers are threads that became runnable and then *waited* for a CPU — pure scheduling-induced latency, invisible in utilization metrics.

perf sched — records every scheduling event and reconstructs the timeline:

```
$ perf sched record -- sleep 5
$ perf sched latency          # per-task avg/max scheduling delay
$ perf sched timehist         # per-event timeline: wakeup -> run,
delays
```

`perf sched latency` gives per-task average and maximum run-queue delay; `timehist` shows the blow-by-blow, letting you attribute a stall to a specific preemption or migration.

`/proc and schedstat` — `/proc/<pid>/schedstat` (three numbers: time on CPU, time waiting on run queue, timeslices run) and `/proc/<pid>/sched` expose per-task scheduling statistics; `/proc/ schedstat` gives per-CPU/per-domain balancing stats (requires `CONFIG_SCHEDSTATS`). Lower-level than the above but useful for scripted per-thread accounting.

A workable diagnosis flow: PSI or run-queue-latency says "scheduling delay is real" → `cpu.stat` says "this cgroup is throttled" → the fix is a limits/requests/cpuset change; or `cpu.stat` is clean but `perf sched / runqlat` shows migration- or preemption-driven delay → the fix is affinity, `NOHZ_FULL`, or a preemption/priority adjustment.

The distributed-systems lens

Zoom out and the CPU scheduler is a **per-node multi-tenancy engine**, and its concerns recur at every layer of a fleet.

The node scheduler and the cluster scheduler are the same problem at two scales. Linux packs runnable tasks onto CPUs subject to fairness and locality; Kubernetes packs pods onto nodes subject to requests and topology. Bin-packing tasks-to-CPU $\approx$ bin-packing pods-to-nodes; load balancing across per-CPU run queues $\approx$ the cluster autoscaler and descheduler rebalancing pods across nodes; CPU affinity/cpuset $\approx$ node affinity and taints/tolerations. The `request/limit` knobs you set in a pod spec are literally the cgroup `cpu.weight/cpu.max` values the node's scheduler enforces — the cluster policy and the node policy are one continuous mechanism (Volume 12, on cloud and Kubernetes, and Book 6, on cloud-native platforms).

CFS quota throttling is a fleet-wide gotcha, not a one-off. It is *the* canonical example of a per-node scheduling detail causing fleet-scale symptoms: a default template that sets CPU limits, copied across hundreds of services, produces p99 latency spikes on thousands of pods,

on nodes that are *under-utilized*. The failure is counter-intuitive precisely because CPU dashboards look healthy — the diagnostic signal is throttled counters and run-queue latency, not utilization. This is well-documented across the industry's incident write-ups and conference talks (Zalando/Omio-style "CPU limits caused our latency" postmortems, and the long-running Kubernetes issues on CFS throttling). Treat "are we setting CPU limits, and are pods throttled?" as a standing question in any latency investigation.

Requests and limits are the fleet's isolation knobs — and they encode a philosophy. Choosing weight-only (requests, no limits) versus hard caps (limits) is choosing between *work-conserving fairness* (never waste a core; a tenant may burst) and *strict partitioning* (predictable ceilings; some cores may idle). This is the multi-tenancy trade-off of Volume 12 in miniature, decided per CPU controller. For the services that must not be perturbed by noisy neighbours, the answer is neither knob but *space* partitioning — `cpuset` pinning (CPU Manager static policy), the OS-level echo of dedicating hardware, and the same isolation instinct as Volume 1, Chapter 6 (NUMA) and the isolation discussion of Book 1, Chapter 10.

Scheduling delay is a first-class contributor to tail latency at scale. Volume 11 (on latency) treats p99/p999 as the metric that matters, and a large slice of unexplained tail latency is scheduling: a handler thread that woke on I/O completion and then waited in a run queue; a thread throttled mid-request; a task migrated off its warm cache; a low-priority task holding a CPU under `PREEMPT_NONE`. None of these show up as high CPU utilization. Across a fleet of many services and high request rates, these millisecond-scale scheduling stalls, multiplied by fan-out, are a dominant source of the long tail — which is why the observability signals above (PSI, run-queue latency, throttling counters) belong in your standard dashboards, not just your debugging toolkit.

Key takeaways

- **The scheduler multiplexes many runnable tasks onto few CPUs**, trading throughput against latency, fairness, and power. Linux is **preemptive**, uses **per-CPU run queues**, and computes slices dynamically rather than fixing a quantum.

- **Scheduling classes form a strict hierarchy:** `SCHED_DEADLINE` (EDF + admission control) > real-time `SCHED_FIFO` / `SCHED_RR` (fixed priority 1-99) > normal `SCHED_OTHER` (CFS/EEVDF) > `SCHED_BATCH` / `SCHED_IDLE`. A higher class fully preempts a lower one — an unbounded `SCHED_FIFO` loop can hang the box, which is why **RT throttling** caps RT at 95% by default.
- **CFS approximates ideal fairness via `vruntime`:** run the task with the smallest weighted virtual runtime, stored in a **red-black tree** (leftmost = next, O(1) pick). **Nice is a weight, not a slice** — it changes how fast `vruntime` accrues. Slices come from a **targeted latency** divided by weight, floored by **minimum granularity**.
- **EEVDF (default since 6.6)** keeps fair-share-by-weight but replaces "pick min `vruntime`" with **eligibility (non-negative lag) + earliest virtual deadline**, and adds **latency-nice** so responsiveness is tunable independently of CPU share. Details still evolving; CFS-era intuition mostly carries over.
- **Multiprocessor scheduling balances per-CPU run queues against locality.** Migration loses cache and NUMA warmth (Volume 1, Chapter 6); **scheduling domains** make balancing cheap within cores and reluctant across NUMA nodes. Pin with `taskset` / `sched_setaffinity` / `cpuset` for latency-critical threads.
- **Preemption models set worst-case scheduling delay:** `PREEMPT_NONE` (throughput) → `PREEMPT_RT` (bounded latency, mainlined ~6.12). `NOHZ_FULL` removes the tick from a single-task isolated core for near-zero jitter.
- **Two cgroup CPU controls, do not conflate them:** `cpu.weight` (proportional, soft, no cap, work-conserving) vs. `cpu.max` (absolute quota per 100 ms period, **hard cap that throttles even on an idle node**). `cpuset` partitions *space* (CPUs/NUMA), the strongest isolation, and does not throttle.
- **CFS quota throttling is a top cause of container p99 latency.** Quota is aggregate CPU per period; a multi-threaded app burns it in a fraction of the period's wall-clock time, then stalls for the remainder despite idle cores. The pre-5.4 slice-expiry bug made it worse (fixed by commit `de53fd7aedb1`); `cpu.max.burst` (5.14+) helps the bursty case.
- **Kubernetes:** `request` → `cpu.weight` /shares (**floor + bin-packing**), `limit` → `cpu.max` **quota (hard cap → throttling)**.

Setting CPU limits can *hurt* latency; prefer well-set requests, avoid limits on latency-sensitive services unless isolation/billing demands them, and use CPU Manager static policy (`cpuset` `pinning`) for the truly latency-critical. Always monitor `cpu.stat` `throttling`.

- **Observe scheduling with `cpu.stat` (`throttling`), PSI (`/proc/pressure/cpu`), run-queue latency (`runqlat`), `perf sched` , and `schedstat` .** Scheduling-induced latency is invisible in utilization graphs and is a dominant, fan-out-amplified contributor to fleet tail latency.

Further reading

- **Linux kernel documentation**, `Documentation/scheduler/`: `sched-design-CFS.rst` , `sched-bwc.rst` (CFS bandwidth control / quota — the authoritative description of the throttling mechanism), `sched-rt-group.rst` , `sched-deadline.rst` , and `sched-domains.rst` . Primary sources, version-accurate. `Documentation/admin-guide/cgroup-v2.rst` for `cpu.weight` , `cpu.max` , `cpu.stat` , and `cpuset` .
- **Peter Zijlstra et al., the EEVDF patch series and LWN coverage** — "An EEVDF CPU scheduler for Linux" (lwn.net, 2023) and follow-ups. The clearest accessible account of what changed in 6.6 and why, including lag, eligibility, virtual deadlines, and latency-nice.
- **Robert Love, *Linux Kernel Development*, 3rd ed. (2010), Chapter 4**; and **Bovet & Cesati, *Understanding the Linux Kernel*, 3rd ed. (2005)** — dated on EEVDF but sound on run queues, `vruntime` /weights, priorities, and load balancing at the source level.
- **Dave Chiluk, "sched/fair: Fix low cpu usage with high throttling by removing expiration of slice"** (kernel commit `de53fd7aedb1` , merged 5.4, 2019), and the associated `kubernetes/kubernetes` issues on CFS quota throttling (notably the long-running issues `#67577` and `#51135`). The primary record of the throttling bug and the community's response.
- **The `man` pages**: `sched(7)` (policies overview), `sched_setscheduler(2)` , `sched_setattr(2)` (deadline params and latency-nice), `sched_setaffinity(2)` , `chrt(1)` , `taskset(1)` , `cpuset(7)` . Exact syscall and flag semantics.

- **Brendan Gregg, *Systems Performance*, 2nd ed. (2020), Chapter 6 (CPUs)**, and his `runqlat / runqlen / cpudist` BCC tools — the practical guide to measuring run-queue latency and scheduler behaviour with `perf` and eBPF (see also Chapter 11 of this volume).
- **PSI (Pressure Stall Information)** — `Documentation/accounting/psi.rst` and Facebook/Meta's original PSI write-ups. The modern saturation signal for CPU, memory, and I/O.
- **Kubernetes documentation:** "Resource Management for Pods and Containers", "CPU Management Policies on the Node" (CPU Manager static policy), and "Configure Quality of Service for Pods". The authoritative mapping of `requests / limits / QoS` onto cgroup controls, and the "for the truly latency-critical, pin" guidance.

Chapter 3 — Virtual Memory and Paging

What this chapter covers. Chapter 1 defined a process as, first and foremost, an *address space*: a private, contiguous-looking range of virtual addresses that only that process can see. This chapter explains the machinery that makes that abstraction real. Every memory access your backend service makes — dereferencing a pointer, pushing a stack frame, reading a `mmap`'d index file — passes through hardware address translation before it reaches a byte of DRAM. That translation layer is *virtual memory*, and it is not a performance detail you can ignore. It is the mechanism behind process isolation, behind the cheap `fork` Redis uses to snapshot, behind the mysterious multi-millisecond stalls that wreck your p99, behind the container that keeps getting `OOMKilled`, and behind the difference between the `VSZ` your monitoring reports and the RSS that actually costs you money.

We are Linux- and x86-64-specific where it matters, because the abstraction leaks in concrete, generation-specific ways. An engineer who can reason about page tables, the TLB, minor versus major faults, copy-on-write, `mmap`, reclaim, overcommit, and the OOM killer can debug an entire class of production incidents that otherwise look like black magic.

Learning goals — after this chapter you should be able to:

- Explain **why** virtual memory exists and what it buys: per-process isolation, the illusion of a large contiguous address space, and the features (demand paging, COW, `mmap`, swap) it enables.
- Describe **paging mechanics** on x86-64: pages and frames, the 4-level (and 5-level) page table hierarchy, the hardware page-table walk, and the meaning of the key PTE bits.
- Explain the **TLB** as the cache that makes translation affordable, why a TLB miss triggers a page walk, why context switches historically flush it, and how **PCID/ASID** avoid that.
- Distinguish **minor from major page faults** and explain why a major fault in a hot path is a tail-latency catastrophe, plus the difference between anonymous and file-backed pages.
- Explain **copy-on-write**, why it makes `fork` cheap, and the transient memory blow-up it can cause (the Redis-snapshot gotcha).
- Reason about `mmap` — file-backed vs anonymous, shared vs private — and articulate the case for and against `mmap` as a database's storage manager.
- Explain **swap, page reclaim, and the OOM killer**, why latency-sensitive fleets disable swap and `mlock` hot data, and how Linux **overcommit** sets up the OOM failure mode.
- Read the memory metrics that actually matter — **RSS vs VSZ**, page cache, `PSI` — and stop being misled by the ones that don't.

Why virtual memory exists

Give a program raw physical addresses and two problems appear immediately. First, **isolation** is gone: any process can read or scribble on any other's memory, or the kernel's, because addresses are physical and shared. Second, **placement** becomes the programmer's problem: two programs both want to load at address `0x400000`, physical memory is fragmented, and nothing can assume a contiguous region large enough for its heap. Early systems lived with both. Modern systems solve both with one idea.

Virtual memory interposes a translation layer between the addresses a program uses (*virtual* addresses) and the addresses DRAM responds to (*physical* addresses). Each process gets its own private mapping. The

hardware unit that performs the translation on every access is the **MMU** (Memory Management Unit), and the mapping it consults is the process's **page table**, which the kernel builds and the CPU's `CR3` register points at. Switch processes, switch `CR3`, switch the entire meaning of every address.

That single indirection delivers a stack of properties:

- **Isolation.** Process A cannot touch process B's memory because A's page table simply does not contain a mapping for B's physical frames. The absence of a translation is the security boundary — the same boundary Volume 1, Chapter 10 (*Hardware Support for Virtualization and Isolation*) described from the silicon side, and the one Chapter 1 named as the core of what a process is. There is no software check to bypass; an unmapped access simply faults.
- **The illusion of contiguity.** A process sees a flat linear address space (the layout from Chapter 1: text, data, heap, `mmap` region, stacks). The physical frames backing it can be scattered anywhere in DRAM, or not present at all. The program neither knows nor cares.
- **The illusion of abundance.** A process can map far more address space than the machine has RAM. Most of it is never touched, and what is touched can be paged in on demand and paged out under pressure.
- **A substrate for features.** Because every access is mediated by a per-page table entry with permission bits, the kernel can lazily populate memory (**demand paging**), share pages until someone writes (**copy-on-write**), back a region with a file (`mmap`), or evict pages to disk and fault them back (**swap**). All of these are the *same mechanism* — page faults on absent or protected pages — used for different ends.

The cost is that translation is now on the critical path of every load and store. The rest of this chapter is, in one sense, the story of how hardware and kernel keep that cost low enough to be worth paying — and what happens at scale when they can't.

Paging mechanics

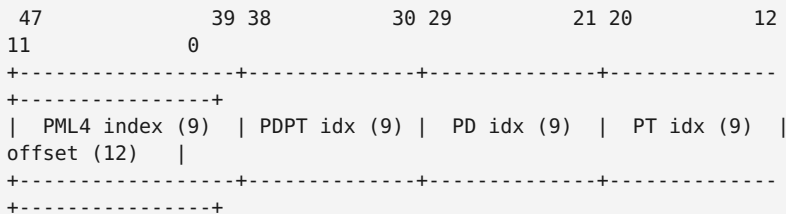
Pages and frames

Physical memory is divided into fixed-size **page frames**; the virtual address space is divided into equal-size **pages**. On x86-64 the base page is **4 KiB**. Translation is done at page granularity: the low 12 bits of an address are the **offset** within a page and pass through untranslated; the high bits are the **virtual page number**, which the MMU maps to a **physical frame number**. Paging (fixed-size units) rather than segmentation (variable-size units) is what makes physical memory allocation tractable — any free frame can back any page, so external fragmentation of physical memory disappears.

Why the page table is a tree

A flat page table — one entry per virtual page — is a non-starter on 64-bit machines. The x86-64 virtual address space is 48 bits wide in the common configuration (256 TiB). At 4 KiB pages that is 2^{36} pages; at 8 bytes per entry, a flat table would be **512 GiB per process**, almost all of it describing unmapped address space. The address space is enormously **sparse**: a real process maps a few megabytes of text, some heap, some stacks, some libraries, and leaves terabytes of the range untouched.

The fix is a **radix tree** — a multi-level page table. On x86-64 with 4 KiB pages the tree has **four levels**, each indexed by 9 bits of the virtual address, with the final 12 bits as the page offset:



Intel calls the levels PML4 → PDPT → PD → PT; Linux's architecture-neutral names are **PGD** → **PUD** → **PMD** → **PTE** (with an extra **P4D** level slotted in for 5-level paging). Each table is itself one 4 KiB page holding 512 eight-byte entries. The beauty of the tree is that **unmapped subtrees cost nothing**: if a process maps no memory in some 512 GiB region, the corresponding PML4 entry is simply absent and no lower

tables exist. A typical process's full page-table hierarchy is a few tens of kilobytes, not half a terabyte.

Modern server CPUs (Intel Ice Lake and later, recent AMD) support **5-level paging**, adding a PML5 (Linux **P4D**) level to widen the virtual address space to 57 bits (128 PiB) and the physical range accordingly. Linux enables it when the hardware supports it and it is configured; the vast majority of deployments still run 4-level. The mechanism is identical — one more 9-bit index, one more memory reference on a walk.

The page-table walk

When the MMU needs a translation it does not have cached, it performs a **page-table walk**: starting from the physical address in CR3, it uses the PML4 index to select an entry, reads that entry to get the physical address of the next table, uses the PDPT index there, and so on down four levels until it reaches a **PTE** holding the physical frame number. On x86 this walk is done **in hardware** by the MMU's page-miss handler (unlike software-managed TLBs on some RISC architectures). Crucially, each level is a **memory access**: a 4-level walk is up to four dependent DRAM reads before the actual access can proceed. That is why the walk is expensive, and why the TLB (next section) exists. CPUs also keep **page-walk caches** that memoize the upper levels, so most walks that reach memory only touch the last level or two.

Page table entry bits

A PTE is 64 bits: mostly the physical frame address plus a set of control and status flags the MMU and kernel both use. The ones a backend engineer should recognize:

Bit	Name	Meaning
P	Present	If clear, the page is not mapped in RAM; any access faults. The kernel uses "not present" for unmapped pages, demand-zero pages, and swapped-out pages, stashing swap location in the other bits.
R/ W	Read/Write	If clear, writes fault. This is how COW and read-only text are enforced.
U/ S	User/ Supervisor	If clear, user-mode access faults. Separates kernel pages from user pages.

Bit	Name	Meaning
A	Accessed	Set by hardware on any access. The reclaim code reads and clears it to approximate LRU.
D	Dirty	Set by hardware on write. Tells reclaim whether a page must be written back before it can be dropped.
PS	Page Size	At the PD/PMD or PDPT/PUD level, marks a 2 MiB or 1 GiB huge page (Chapter 4) — the walk stops early.
G	Global	The translation survives a TLB flush on CR3 change (used for kernel mappings).
NX	No-Execute	Bit 63. If set, instruction fetches from the page fault — the hardware W^X primitive.

The **A** and **D** bits are the hardware's small contribution to memory management: they let the kernel observe usage and modification cheaply without trapping every access.

```

flowchart TB
    V["Virtual address  
from load/store"] --> TLB["TLB lookup"]
    TLB -->|"hit (~1 cycle)"| PA["Physical address  
access proceeds"]
    TLB -->|"miss"| W["MMU page-table walk"]
    subgraph WALK ["Hardware page walk (up to 4 memory refs)"]
        direction TB
        CR3["CR3 -> PML4"] --> L2["PML4 entry -> PDPT"]
        L2 --> L3["PDPT entry -> PD"]
        L3 --> L4["PD entry -> PT"]
        L4 --> PTE["PT entry: frame + flags"]
    end
    end
    W --> WALK
    PTE -->|"present"| FILL["Fill TLB, then access"]
    PTE -->|"not present or protection"| FAULT["Page fault -> kernel  
handler"]
    FILL --> PA

```

The TLB, deepened

Volume 1, Chapter 3 (*The Memory Hierarchy and Caches*) introduced the **Translation Lookaside Buffer** as "the other cache." Here is why it is the linchpin of usable virtual memory. If every memory access required a full page-table walk, every access would cost up to four extra dependent

DRAM reads — virtual memory would be unaffordable. The TLB is a small, fast, fully-or-set-associative cache of recent virtual-page → physical-frame translations. On a **hit**, translation costs roughly a cycle and the access proceeds. On a **miss**, the MMU walks the page table, fills the TLB, and retries — tens to well over a hundred cycles, depending on how much of the walk hits the page-walk cache versus DRAM.

TLBs are small — typically a few hundred to a couple thousand entries, split into instruction and data TLBs and often a two-level hierarchy (L1 dTLB/iTLB plus a shared L2 STLB). Because each 4 KiB entry covers only 4 KiB, the **TLB reach** — total memory addressable without a miss — is modest: a 1500-entry TLB reaches roughly 6 MiB with 4 KiB pages. A backend service with a multi-gigabyte working set, hopping around a hash map or a large heap, blows past that constantly and pays a stream of page-walk costs invisible in normal profiles. This is the central motivation for **huge pages** (2 MiB / 1 GiB), covered in Chapter 4: each huge-page TLB entry covers $512\times$ or $262144\times$ more memory, multiplying reach and shortening walks. The mechanics of huge pages belong to Chapter 4; the point here is that **the TLB sits on the critical path of every access, and its capacity is a real, finite resource you can exhaust**.

Context switches and the TLB

TLB entries are, in the naive model, tied to whatever page table `CR3` currently points at. So when the kernel switches to a different process's address space, the cached translations are stale and dangerous — process B must not see A's mappings. The classic x86 behavior is that **writing `CR3` flushes the entire TLB** (except pages marked Global). Chapter 1 flagged this as part of the *indirect* cost of a context switch: after a cross-address-space switch, the new process runs "cold," refilling the TLB (and caches) with a burst of page walks. This is precisely why switching between two threads of the *same* process is cheaper than switching between processes — same address space, same `CR3`, no flush.

Flushing the whole TLB on every switch is wasteful when the machine ping-pongs between a few processes. The fix is to **tag** TLB entries with an address-space identifier so entries for multiple processes coexist and a switch need not flush. Intel calls this the **PCID** (Process Context Identifier, a 12-bit tag); ARM calls it the **ASID**. With PCIDs, `CR3` can be reloaded without flushing, and the `INVPCID` instruction invalidates

entries for a specific context. Linux has used PCIDs since the 4.14 era, and they became especially important after **Meltdown**: the KPTI (kernel page-table isolation) mitigation switches page tables on every kernel entry/exit, which without PCIDs would mean a full TLB flush per syscall — brutal. PCIDs turn that into a much cheaper tagged switch. When you read that "syscalls got more expensive after Meltdown," the TLB is a large part of why, and PCIDs are what kept it from being catastrophic.

Demand paging and page faults

The kernel does not populate physical memory when you `malloc` a gigabyte or `mmap` a file. It records the *mapping* (a virtual address range with permissions and a backing object) in the process's memory-region metadata — the VMAs, visible in `/proc/<pid>/maps` — and leaves the PTEs marked not-present. Physical pages are allocated **on demand**, when the memory is first touched. This is **demand paging**, and it is why a process's `VSZ` can be enormous while its `RSS` is small: address space is cheap; resident pages are not.

The mechanism is the **page fault**. When the CPU translates an address whose PTE is not-present (or violates permissions), it traps into the kernel's fault handler with the faulting address and the reason. The handler consults the VMAs to decide whether the access is legal and what should back the page:

- **Illegal** (no VMA, or a permission the mapping forbids) → `SIGSEGV`. The segfault is just a page fault the kernel refused to satisfy.
- **Legal** → the handler makes the page present and returns; the faulting instruction re-executes and succeeds. What it does to make the page present is where the cost lives.

The single most important distinction in this chapter is **minor vs major**:

- A **minor fault** (soft fault) is resolved **without disk I/O**. The needed data is already in RAM, or no data is needed at all. Examples: the first touch of anonymous memory (map the shared zero page, or allocate a fresh zeroed frame), a **COW** write (copy an in-memory page), or faulting in a file page that is already resident in the **page cache** (Chapter 4). Cost: microseconds — a trap, some bookkeeping, maybe a `memcpy`.

- A **major fault** (hard fault) **requires I/O**: the page's contents must be read from a backing store — a file on disk for file-backed pages, or the **swap** device for anonymous pages that were paged out. Cost: the latency of that I/O, which per Volume 1, Chapter 5 (*Storage Hardware*) is tens of microseconds on NVMe, but historically milliseconds, and *orders of magnitude* worse than a minor fault.

	Minor fault	Major fault
Also called	Soft fault	Hard fault
Requires disk I/O	No	Yes
Typical latency	Sub-microsecond to a few μ s	Tens of μ s (NVMe) to milliseconds (rotational/loaded)
Causes	First-touch anon (zero page), COW copy, file page already in page cache, shared page	File page not in cache, swapped-out anon page returning from swap
Counter	<code>min_flt</code> , <code>ru_minflt</code>	<code>maj_flt</code> , <code>ru_majflt</code>
p99 impact	Negligible in bulk	Severe — a hidden tail-latency source

You can watch both counters per process (`ps -o pid,min_flt,maj_flt,cmd`, or `getrusage(2)`'s `ru_minflt`/`ru_majflt`, or `/proc/<pid>/stat`). **Major faults are the ones that kill latency.** A single major fault on a hot request path — a swapped-out page in your cache, an `mmap`'d index page not yet resident — injects a synchronous disk read into the middle of an otherwise CPU-bound operation. The request thread blocks. There is no way to see it in application-level timing except as an inexplicable multi-millisecond spike. At fleet scale, a low rate of major faults distributed across millions of requests is a persistent p99/p999 tax (Volume 11).

Anonymous vs file-backed pages

Two categories of pages, with different lifecycles:

- **File-backed** pages mirror a region of a file (program text, shared libraries, `mmap`'d data files). Their canonical home is the file on disk,

reached through the **page cache**. A clean file page can be dropped instantly under memory pressure (it can be re-read from the file); a dirty one must be written back first. Faulting one in is a minor fault if it's in the page cache, a major fault otherwise.

- **Anonymous** pages have no file behind them — heap, stacks, anonymous `mmap`. Their only backing store is **swap**. With no swap configured, anonymous pages **cannot be evicted**; they are pinned in RAM until freed. This fact drives much of the swap discussion below.

```
flowchart TB
    ACC["CPU accesses virtual page"] --> PF{"PTE present?"}
    PF -->|"yes"| OK["Access proceeds"]
    PF -->|"no / protection"| H["Page fault handler:"]
    find VMA[""]
    H --> LEGAL{"Legal access?"}
    LEGAL -->|"no"| SEG["SIGSEGV"]
    LEGAL -->|"yes"| KIND{"What backs it?"}
    KIND -->|"anon first-touch"| ZERO["Map zero page / alloc frame"]
    MINOR[""]
    KIND -->|"write to COW page"| COW["Copy the page"]
    MINOR[""]
    KIND -->|"file page in page cache"| CACHE["Map cached frame"]
    MINOR[""]
    KIND -->|"file page not cached"| DISKF["Read from file"]
    MAJOR["disk I/O"]
    KIND -->|"swapped-out anon page"| SWAPIN["Read from swap"]
    MAJOR["disk I/O"]
    ZERO --> RET["Return, re-run instruction"]
    COW --> RET
    CACHE --> RET
    DISKF --> RET
    SWAPIN --> RET
```

Copy-on-write

Chapter 1 introduced copy-on-write as the reason `fork` is cheap. Now we can be precise. `fork` must give the child a logically independent copy of the parent's entire address space. Doing that eagerly — copying every resident page — would be enormously expensive and almost entirely wasted, because the child usually `exec`s a new program immediately, or shares most data read-only.

So `fork` copies only the **page tables**, not the pages. For every writable private page, the kernel marks the PTE **read-only** in *both* parent and child and bumps the page's reference count. Both processes now share

the same physical frames, and either can read freely. The moment either one **writes**, the read-only PTE triggers a **protection fault**. The handler recognizes this as COW: it allocates a fresh frame, copies the shared page into it, makes the writer's PTE point at the private copy with write permission, and returns. If the reference count has dropped to one (the other side already copied or exited), it can simply re-grant write permission without copying.

The result: `fork` cost is proportional to the size of the page tables, not the resident memory, and pages are duplicated **lazily, one at a time, only as they are actually modified**. Pages that are never written are never copied — text, read-only data, and untouched heap stay shared.

```
flowchart TB
    subgraph BEFORE["After fork, before any write"]
        PP1["Parent PTE (RO)"] --> F1["Frame A (shared)"]
        CP1["Child PTE (RO)"] --> F1
    end
    end
    subgraph AFTER["Child writes to the page"]
        PP2["Parent PTE (RO)"] --> F2["Frame A"]
        CP2["Child PTE (RW)"] --> F3["Frame A' (private copy)"]
    end
    end
    BEFORE -->|"write faults -> copy"| AFTER
```

COW is used far beyond `fork`. Private file mappings (`MAP_PRIVATE`) are COW against the page cache. The kernel's `MADV_FREE` and same-page merging (KSM) build on it. But the operational gotcha every backend engineer should internalize is the **memory blow-up under write load after fork**. The textbook example is **Redis persistence**: to take a point-in-time snapshot (`RDB`) or rewrite the append-only file, Redis `fork`s and lets the child serialize a consistent view while the parent keeps serving. COW makes this near-instant and, in the common case, cheap. But every key the parent *writes* during the save triggers a COW copy — and if the workload is write-heavy, the parent can duplicate a large fraction of the dataset before the child finishes. In the pathological case memory usage approaches **2×** the dataset size, transiently, on the node. This is a real capacity-planning hazard: a Redis instance sized to fit comfortably in RAM can OOM *during a background save* under a write spike. Transparent huge pages make it worse, because a COW copy is then 2 MiB instead of 4 KiB — a single small write dirties a huge page — which is exactly why Redis recommends disabling THP.

mmap : memory-mapping and its discontents

`mmap(2)` maps a region of virtual address space to a backing object and lets you access it with ordinary loads and stores instead of `read / write` syscalls. It is central to backend systems: it backs shared libraries, large allocations, shared memory, and the storage engines of many databases. The behavior depends on two orthogonal choices.

What backs the mapping:

- **File-backed** (`mmap` of a file descriptor): the mapping mirrors file contents through the **page cache** (Chapter 4). Reads fault pages in from the file; the same physical page in the page cache backs every mapper of that file and the buffered `read / write` path too.
- **Anonymous** (`MAP_ANONYMOUS`): no file; zero-filled pages backed only by swap. This is how allocators (`malloc` , via `glibc`) obtain large chunks of heap, and how you get big zero-initialized regions. Anonymous `mmap` is the subject of the allocator discussion in Chapter 4.

How writes propagate:

- **MAP_SHARED** : writes go to the shared page-cache page and are visible to other mappers and eventually written back to the file (on `msync` , or by the writeback machinery). This is true shared memory and true file mutation.
- **MAP_PRIVATE** : **copy-on-write** against the backing object. Reads see the file (or zeros); the first write copies the page and your changes stay private and are never written back. Program text and initialized data segments are `MAP_PRIVATE` file mappings.

```

flowchart LR
    subgraph PROC["Process address space"]
        VP["mmap'd virtual pages"]
    end
    subgraph KERN["Kernel"]
        PC["Page cache  
(physical frames)"]
        DISK["File on disk"]
        VP -->|"page fault -> map"| PC
        PC -->|"read: fault in"| DISK
        PC -->|"MAP_SHARED write -> writeback / msync"| DISK
        VP -->|"same frames shared with read/write path  
and other mappers"| PC
    end

```

Two controls matter for performance. **MAP_POPULATE** prefaults the whole mapping at **mmap** time, trading a slow **mmap** for no later fault storm — useful when you know you'll touch everything and want the cost up front instead of scattered through request handling. **madvise(2)** hints access patterns: **MADV_WILLNEED** kicks off readahead, **MADV_SEQUENTIAL** tells the kernel to read ahead aggressively and drop behind, **MADV_RANDOM** suppresses readahead, **MADV_DONTNEED** drops resident pages (forcing them to be re-faulted), **MADV_FREE** lazily marks anonymous pages reclaimable, and **MADV_HUGEPAGE** requests THP backing (Chapter 4).

The "should a DBMS use **mmap** ?" debate

mmap is seductive for a database storage engine: map the data file, access records as if they were in memory, and let the kernel's page cache and reclaim be your buffer pool for free. Several real systems took this route (LMDB is built on it; MongoDB's original MMAPv1 engine did, and was later replaced by WiredTiger). But the consensus among serious database engineers — crystallized in the 2022 paper "*Are You Sure You Want to Use MMAP in Your DBMS?*" by Crotty, Leis, and Pavlo — is that **mmap** is a poor foundation for a high-performance transactional DBMS. The arguments, stated at the level of confidence they deserve:

Concern	read / write (buffer pool)	mmap
Eviction control	DBMS decides what stays resident and what evicts	Kernel decides; you cannot pin your working set or control eviction order without fighting reclaim

Concern	read / write (buffer pool)	mmap
I/O latency visibility	I/O is explicit; can be async, prefetched, scheduled	A page fault is a synchronous, invisible stall inside any instruction; a query thread blocks unpredictably on a major fault
Transactional safety	DBMS controls exactly when dirty pages hit disk (WAL ordering, fsync)	Kernel can write back a dirty MAP_SHARED page at any time , breaking write-ordering guarantees and enabling torn/partial writes
Error handling	I/O errors returned from a syscall you can handle	An I/O error surfaces as a SIGBUS mid-access — hard to handle correctly
Scalability	Buffer-pool structures tuned by the DBMS	Page-table contention and TLB shootdowns on eviction across many cores; historically a single mmap-lock bottleneck

The counterpoint is real: `mmap` is dramatically simpler to build, avoids double-buffering (no copy from page cache into a private buffer pool), and performs well for read-mostly workloads whose working set fits in RAM — LMDB's sweet spot. The honest summary is not "`mmap` is always wrong" but "`mmap` **hands the kernel control over eviction, durability, and fault latency — three things a serious transactional DBMS wants to own.**" The transferable lesson is broader than databases: **any `mmap`-backed data store inherits page-fault latency and eviction unpredictability from the OS**, a property you must budget for, not a free lunch.

Swap, reclaim, and the working set

Physical memory is finite, and Linux aggressively uses all of it — for anonymous pages, for the page cache, for kernel structures. When free memory runs low, the kernel must **reclaim** pages to make room for new allocations. This is the machinery of swap and page eviction, and it is where memory pressure turns into latency.

The reclaim algorithm

Linux approximates LRU with per-cgroup, per-node **active/inactive lists**, split into anonymous and file lists (four lists total). Newly faulted pages start on the **inactive** list; a second reference promotes them to the **active** list. Reclaim scans the **tail of the inactive lists** for eviction candidates, using the PTE **Accessed** bit to detect recent use and demote or rotate pages accordingly. The split lets the kernel bias reclaim toward the kind of page that is cheaper to drop. (Kernels 6.1+ can use **MGLRU**, a multi-generation LRU that replaces the two-list scheme with aging generations; the principle — evict cold pages, protect the working set — is the same.)

Reclaiming a page differs by type:

- **Clean file page**: just drop it. It can be re-read from the file. Cheap.
- **Dirty file page**: must be **written back** to the file first, then dropped.
- **Anonymous page**: must be **written to swap** first (there is nowhere else to put it), then dropped. If there is no swap device, it *cannot be reclaimed at all*.

Two actors do reclaim. **kswapd** is a per-node kernel thread that reclaims in the background when free memory falls below the low watermark, working back up to the high watermark — asynchronous, off the allocating thread's critical path. But if an allocation needs a page *now* and **kswapd** hasn't kept up, the allocating thread itself enters **direct reclaim**: it synchronously scans and evicts pages before its allocation can proceed. Direct reclaim is a stall injected into whatever code allocated memory — a hidden, hard-to-attribute latency source, and a sign the machine is under real pressure.

vm.swappiness (0-200 in recent kernels; default 60) biases reclaim between evicting anonymous pages (swapping) and evicting file pages (dropping page cache). Low values push the kernel to reclaim file cache before swapping application memory; **0** avoids swapping anonymous memory until it is nearly unavoidable (it does **not** fully disable swap — that requires `swapoff`).

Why swap is a latency problem

Here is the chain. Swapping out an anonymous page is fine — it happens in the background. The problem is swapping it **back in**: when the process

next touches that page, it takes a **major fault** and blocks on a disk read. If the working set genuinely exceeds RAM, the system enters **thrashing**: pages are evicted, immediately re-referenced, faulted back in, and evicted again, and the machine spends nearly all its time waiting on swap I/O making no forward progress. Throughput collapses; latency goes to the moon. **Pressure Stall Information** (`/proc/pressure/memory`, the `PSI` interface) is the modern way to quantify this: it reports the fraction of time tasks stall waiting on memory.

This is why **latency-sensitive backend deployments frequently disable swap entirely**. The reasoning: for a service with a strict tail-latency SLO, a swap-in is never an acceptable outcome — you would rather fail fast (OOM) than serve a request that stalled 10 ms on a swapped page. Kubernetes historically *required* swap to be off on nodes (the kubelet refused to start otherwise) precisely so that the scheduler's memory accounting and the cgroup limits would map cleanly to physical RAM, without the confounding variable of disk-backed memory; swap support for Kubernetes has since been added as an opt-in alpha/beta feature, but the default posture in latency-sensitive fleets remains swap-off. The trade-off is stark and worth naming: **swap-off converts "slow" into "dead."** Without swap, a memory spike that swap would have absorbed instead trips the OOM killer. That is usually the *right* trade for a service behind a load balancer with retries, and the *wrong* trade for a workstation.

Keeping the working set resident

The **working set** (Volume 1, Chapter 3) is the set of pages a process actively uses over a window of time. Good behavior is a working set that **fits in RAM** and stays resident — the node-level analog of the cache-locality argument from Volume 1. When it fits, faults are rare and mostly minor; when it doesn't, you thrash. This is the governing constraint for sizing in-memory data stores: a Redis or an in-memory cache should be sized so its working set (plus COW headroom, plus overhead) is comfortably smaller than node RAM.

When you must guarantee specific pages never get evicted — a latency-critical index, crypto key material you don't want written to swap, a low-latency trading path — use `mlock(2)` or `mlockall(2)` to pin pages into RAM, exempting them from reclaim (subject to the `RLIMIT_MEMLOCK` limit). `mlock` is the explicit "this is working set, keep it resident" instruction,

and it is the standard tool for shielding hot data from the reclaim machinery.

Overcommit and the OOM killer

Linux, by default, will let processes map **more memory than the machine can actually back** with RAM plus swap. This is **overcommit**, and it is a deliberate bet: processes routinely reserve far more address space than they touch. `malloc` grabs big anonymous regions that stay mostly untouched; `fork` COW-shares pages that mostly never diverge; sparse data structures map ranges they never fill. Refusing to allocate address space until physical backing is guaranteed would waste enormous memory and break the `fork`/COW model. So Linux promises freely and allocates physical frames only on **first touch** (demand paging). The gamble pays off almost always — because the promises are rarely all called in.

The behavior is tunable via `vm.overcommit_memory`:

- **0 (heuristic, default)**: the kernel allows most overcommit but rejects allocations it judges wildly unreasonable via a heuristic. Works well in practice; the mode almost everyone runs.
- **1 (always)**: never refuse; every `mmap` / `brk` succeeds regardless. Used by workloads that legitimately map huge sparse regions (some sparse-array numeric code, some language runtimes) and know they won't touch them.
- **2 (strict / never)**: the kernel enforces a `CommitLimit = swap + physical × overcommit_ratio` (default 50%), and refuses allocations beyond it. Allocations fail honestly at `malloc` / `mmap` time rather than succeeding and detonating later. Predictable, but requires headroom tuning and can surprise software that assumes overcommit.

The gamble's downside: when the promises **are** called in — many processes touch their mapped pages at once and demand exceeds RAM plus swap — the kernel is out of memory with no page to give and nothing left to reclaim. It cannot fail the allocation (the memory was already "granted"), so it invokes the **OOM killer**: it selects a victim process and sends it `SIGKILL`. The victim is chosen by `oom_score` (a "badness" heuristic dominated by the process's memory footprint, so the biggest hog is the usual target), adjustable per process via `oom_score_adj` (−1000 to +1000; −1000 makes a process effectively

unkillable, useful for critical daemons). The kill is abrupt and unappealable — no cleanup, no chance to flush — and it is one of the more feared events in production because it appears as an unexplained process death in the logs. Chapter 4 dissects OOM selection and tuning in depth; the point here is that **overcommit and the OOM killer are two ends of one policy** — promise generously, reconcile violently.

```
flowchart TB
    A["Processes map memory  
(mmap / brk / fork COW)"] --> B["Kernel grants address space  
(overcommit: no physical backing yet)"]
    B --> C["First touch -> demand-page  
allocate physical frame"]
    C --> D{"Free memory low?"}
    D -->|"no"| C
    D -->|"yes"| E["Reclaim: drop clean file pages,  
write back dirty, swap anon"]
    E --> F{"Reclaim recovered enough?"}
    F -->|"yes"| C
    F -->|"no, nothing left"| G["OOM killer:  
pick victim by oom_score -> SIGKILL"]
```

Overcommit in containers

In a container world the failure is more localized and far more common. A container runs in a **cgroup** with a memory limit (`memory.max` in cgroup v2; Chapter 9 covers the mechanism). When the processes in that cgroup exceed their limit and reclaim within the cgroup can't recover enough, the kernel triggers a **cgroup-scoped OOM kill** — it kills a process *inside that cgroup*, not somewhere else on the node. To the orchestrator this surfaces as the container terminating with exit code **137** (128 + SIGKILL's signal 9) and a reason of `OOMKilled`. This is one of the single most common causes of container restart loops in production Kubernetes: a memory limit set too low, a workload whose RSS grows past it, and a `CrashLoopBackOff` that looks like an application bug but is a sizing problem. Getting limits right — and understanding *what* counts against them (RSS and, in cgroup v2, much of the page cache attributed to the cgroup) — is core operational competence, developed further in Chapter 9 and Book 6 (*Cloud Native*).

Practical VM behavior for backend engineers

The abstractions above produce a handful of everyday observations that trip up engineers who haven't internalized virtual memory. Getting these right is the difference between reading your monitoring correctly and chasing ghosts.

RSS vs VSZ vs page cache

`ps` and `top` show two memory figures per process, and only one of them means anything:

Metric	What it is	Is it "real" memory?
VSZ (virtual size)	Total virtual address space mapped — every VMA, including untouched <code>mmap</code> regions, unbacked reservations, guard pages, and all mapped shared libraries	No . Overcommit makes it routinely huge and largely meaningless. A JVM or Go process can show tens of GiB of VSZ while using a fraction.
RSS (resident set size)	Physical pages currently resident for the process, including shared pages (libraries, shared mappings) counted in full per process	Mostly. It is real physical memory, but over-counts shared pages across processes.
PSS (proportional set size)	Like RSS but shared pages divided by the number of sharers	The most honest per-process figure; sum of PSS across processes $\approx$ total used.
Page cache (<code>buff/cache</code> in <code>free</code>)	File-backed pages the kernel caches; reclaimable	It's "used" but available on demand — not a leak, not pressure.

Two rules follow. First, **ignore VSZ for capacity reasoning** — it tells you how much address space is mapped, not how much memory is consumed. Reason about RSS (or better, PSS, and for containers the cgroup's memory accounting). Second, **"free memory is wasted memory."** Linux fills otherwise-idle RAM with page cache; `free` showing little "free" and lots of "buff/cache" is *healthy* — that cache is accelerating your file I/O and will be surrendered the instant anything

needs the memory. The meaningful figure is **available** (the `available` column in `free`, which accounts for reclaimable cache), not **free**. Alerting on "free memory" instead of "available memory" is a classic false-alarm generator. Chapter 4 treats the page cache in full.

Page-fault latency in hot paths

Because faults are invisible in application timing, they are a favorite hiding place for tail latency. The mitigations follow directly from the mechanism:

- **Pre-fault** memory you know you'll use: `MAP_POPULATE` at `mmap` time, or touch pages during startup/warm-up so the faults happen off the request path. A cold cache served its first thousand requests slowly because every page was a fault; warming it moves that cost to deploy time.
- **mlock** the pages that must never fault — the hot index, the low-latency path — so reclaim can't evict them out from under you.
- **Disable swap** on latency-sensitive nodes so a fault is never a swap-in.
- **Huge pages** (Chapter 4) to cut TLB misses (which aren't page faults, but are the *other* invisible translation cost) for large working sets.

The unifying idea: on a latency-critical path you want **all faults to have already happened**. Every page your request touches should be resident and translated before the request arrives.

Distributed-systems lens

Virtual memory is a per-node mechanism, but its behavior sets the economics and the tail latency of an entire fleet.

- **Major faults are a hidden p99 killer.** A service that looks CPU-bound in aggregate can have a p999 dominated by swap-ins and cold `mmap` faults — synchronous disk reads spliced into request handling, invisible to application timers. This is why latency-sensitive fleets standardize on **swap-off + mlock hot data**, converting the "slow" failure mode into a "fail fast" one they can handle with retries and load balancing (Volume 11). The node-level choice directly shapes the tail of the distributed system.

- **cgroup limits and OOM kills are a top cause of container churn.** Across a large Kubernetes fleet, `OOMKilled` / `exit-137` restart loops are a leading operational failure, and every one is a memory-sizing decision: limit too tight, RSS growth unaccounted, page cache misattributed, or a load spike. Right-sizing limits — and knowing RSS vs cache vs the cgroup's own accounting — is fleet hygiene (Chapter 9, Book 6, Volume 12).
- **mmap-backed data stores inherit OS eviction and fault latency.** Databases and indexes built on `mmap` (Volume 5) hand the kernel control over what stays resident and when faults happen. That makes their tail latency a property of node memory pressure and reclaim policy, not just their own code — a coupling you must model when you co-locate them or set limits.
- **COW fork can transiently double memory.** The Redis-snapshot pattern is a fleet-wide capacity gotcha: a node sized to hold the dataset can OOM *during a background save* under write load, because COW duplicates dirtied pages. Provisioning must include COW headroom, or the save itself becomes the incident.
- **Working-set-fits-in-RAM is the node-level cache-locality law.** The same principle that governs L1/L2/L3 in Volume 1, Chapter 3 governs DRAM vs swap here, and governs in-memory store sizing across the fleet: keep the hot set in the fast tier. Every tier of the distributed memory hierarchy — registers, caches, RAM, local SSD, remote cache, remote disk — obeys the same locality logic; virtual memory is where the RAM-vs-disk boundary of that hierarchy is enforced.
- **Overcommit + OOM is the node's resource-exhaustion failure mode.** It mirrors distributed resource exhaustion exactly: promise capacity you statistically won't all need, and when the bet fails, shed load violently. The OOM killer is admission control by assassination. Under a coordinated demand spike — the correlated-failure pattern — many nodes call in their promises at once, and a fleet of overcommitted nodes OOMs together. Understanding the local mechanism is understanding one instance of the global pattern.

Key takeaways

- **Virtual memory interposes per-process address translation** between virtual and physical addresses. The MMU translates on

every access using the process's page table (rooted at `CR3`); the absence of a mapping is the isolation boundary. This one indirection buys isolation, contiguity, the illusion of abundance, and the substrate for demand paging, COW, `mmap`, and swap.

- **Page tables are radix trees** because the 64-bit address space is sparse. x86-64 uses **4 levels** (`PML4/PDPT/PD/PT` → Linux `PGD/PUD/PMD/PTE`) for 48-bit VAs, **5 levels** for 57-bit; a hardware page-table walk on a TLB miss costs up to four dependent memory reads. Key PTE bits: Present, R/W (enforces COW), U/S, Accessed, Dirty, NX.
- **The TLB caches translations** and sits on every access. Misses trigger page walks; **TLB reach** is small with 4 KiB pages, which is why huge pages matter. `CR3` reload flushes the TLB unless **PCID/ASID** tags let address spaces coexist — critical after Meltdown/KPTI.
- **Demand paging** populates memory on first touch via page faults. **Minor faults** need no I/O (first-touch, COW, page-cache hit); **major faults require disk/swap I/O** and are the hidden tail-latency source. Anonymous pages are swap-backed; file-backed pages live in the page cache.
- **Copy-on-write** makes `fork` cheap by sharing pages read-only and copying lazily on write. It also means a `fork`-and-snapshot workload (Redis) can transiently approach **2×** memory under write load — a real capacity hazard, worsened by THP.
- `mmap` maps files (via page cache) or anonymous memory, shared or private (COW). It is powerful but hands the kernel control over **eviction, durability, and fault latency** — which is the core of the argument against `mmap` as a serious DBMS's storage manager.
- **Swap turns memory pressure into latency.** Reclaim uses active/inactive LRU lists; `kswapd` reclaims in the background, direct reclaim stalls the allocator. Swap-in is a major fault; a working set larger than RAM **thrashes**. Latency-sensitive fleets disable swap and `mlock` the hot set; keep the working set resident in RAM.
- **Linux overcommits** memory because promises are rarely all called in. When they are, the **OOM killer** picks a victim by `oom_score` and `SIGKILL`s it. In containers this is a cgroup-scoped kill surfacing as **exit 137 / OOMKilled** — a leading cause of restart loops.
- **Read the right metrics. VSZ is meaningless** (mapped address space); **RSS/PSS** is real memory; **page cache is reclaimable** ("free memory is wasted memory" — watch *available*, not *free*). On hot

paths, arrange for **all faults to have already happened**: pre-fault, `mlock`, swap-off, huge pages.

Further reading

- **The Linux Programming Interface**, Michael Kerrisk, No Starch Press, 2010. Chapters 6, 7, and 49 (`mmap`) and the memory-management material are the definitive practitioner reference for the Linux side of this chapter — `mmap`, `mlock`, `madvise`, demand paging, and the `/proc` memory interfaces.
- **Understanding the Linux Kernel**, 3rd ed., Bovet and Cesati, O'Reilly, 2005, and **Professional Linux Kernel Architecture**, Wolfgang Mauerer, Wrox, 2008 — for page tables, the fault handler, reclaim, and swap at the source level. Dated on specifics, sound on the mechanism.
- **Intel 64 and IA-32 Architectures Software Developer's Manual**, Volume 3A, "Paging" chapter — the authoritative description of 4- and 5-level paging, PTE bit definitions, the page-table walk, and PCIDs. The AMD64 Architecture Programmer's Manual, Volume 2, covers the same for AMD.
- The Linux kernel documentation under `Documentation/` — especially `admin-guide/mm/` (transparent huge pages, `concepts.rst`, overcommit accounting), `vm/` internals, and the `proc(5)` man page for `/proc/<pid>/maps`, `/proc/<pid>/smaps` (per-mapping RSS/PSS), and `/proc/<pid>/status`.
- Crotty, Leis, and Pavlo, "**Are You Sure You Want to Use MMAP in Your DBMS?**", CIDR 2022 — the careful statement of why `mmap` is a poor storage-manager foundation for a transactional DBMS (eviction control, I/O stalls, `SIGBUS`, torn writes, TLB shootdowns). Read alongside the LMDB design notes for the opposing, read-mostly case.
- man pages: `mmap(2)`, `madvise(2)`, `mlock(2)`, `mincore(2)`, `getrusage(2)` (fault counters), `proc(5)`, and the `overcommit-accounting` documentation. `vm.overcommit_memory`, `vm.overcommit_ratio`, `vm.swappiness`, and the OOM `oom_score_adj` interface are documented in `Documentation/admin-guide/sysctl/vm.rst`.
- Redis documentation, "**Redis persistence**" and the latency/THP guidance — the canonical real-world description of `fork` + COW for

snapshooting and why transparent huge pages hurt it. A concrete, load-bearing example of everything in the COW section.

- Ulrich Drepper, "**What Every Programmer Should Know About Memory**" (2007), sections on virtual memory, TLBs, and page tables — still the best single narrative on why translation costs what it does. Pairs with Volume 1, Chapter 3.
- Brendan Gregg, **Systems Performance**, 2nd ed., Addison-Wesley, 2020 — the memory chapter for the diagnostic side: reading RSS/ PSS/ `smaps`, page-fault analysis, `PSI`, swap and reclaim under load, and the `bcc` / `bpfttrace` tooling to observe faults live (Chapter 11).

Chapter 4 — Memory in Practice: Allocators, Page Cache, Huge Pages, OOM

What this chapter covers. Chapter 3 built the machinery: virtual address spaces, page tables, the TLB, demand paging, overcommit, and reclaim. This chapter turns that machinery into the operational reality a backend engineer actually manages. When you run a service at scale, you do not directly manipulate page tables. You *do* choose an allocator, watch RSS climb on a Grafana dashboard, get paged for an `OOMKilled` pod at 3 a.m., argue about whether Transparent Huge Pages should be on, and stare at `free` output trying to decide whether a box is genuinely out of memory or just doing its job. Those are the four surfaces where Chapter 3's mechanisms meet your day: the **user-space allocator** that hands out heap memory, the **page cache** that makes file I/O fast and makes memory dashboards lie, **huge pages** that relieve TLB pressure on big-memory workloads, and the **OOM killer** that ends a process when overcommit finally comes due. We treat all four with Linux specifics and an honest eye on the failure modes.

The through-line: almost every memory problem you will debug in production is one of a small set — a real leak, fragmentation, a mis-sized cgroup limit, THP-induced tail latency, or an alert firing on reclaimable page cache. Knowing which one you are looking at, from the metrics alone, is the skill.

Learning goals — after this chapter you should be able to:

- Explain how `malloc / free` work *above* the kernel: how an allocator gets memory via `brk / sbrk` and `mmap`, and how size classes, free lists, per-thread/per-CPU arenas, and thread caches reduce lock contention and syscall overhead.
- Compare glibc `malloc` (ptmalloc), jemalloc, tcmalloc, and mimalloc, and justify swapping the allocator as a real fleet-wide performance and footprint lever.
- Distinguish internal from external fragmentation, explain why long-running services grow and fragment, and recognize the "a restart fixes it" smell.
- Explain the page cache: why "free memory is wasted memory," why `buff/cache` is not `used`, how write-back and the dirty ratio work, where the `fsync` durability gap lives, and when `0_DIRECT` and read-ahead matter.
- Explain huge pages (2 MB / 1 GB), the difference between hugetlbfs and THP, and give an honest account of when THP helps versus why Redis and many databases tell you to disable it.
- Explain the OOM killer end to end: overcommit, `oom_score / oom_score_adj`, the uncatchable `SIGKILL`, and — critically — cgroup-level OOM, the Kubernetes `OOMKilled / exit-code-137` failure mode, and cgroup v2 `memory.max / memory.high / memory.min`.
- Read the metrics that matter — RSS vs VSZ vs PSS vs page cache, `/proc/meminfo`, `memory.stat`, PSI — and tell a leak from fragmentation from cache, and know what to alert on.

This chapter is Linux-specific and leans throughout on Chapter 3 (Virtual Memory and Paging), on Volume 1 Chapter 3 (The Memory Hierarchy) and Chapter 4 (Cache Coherence) for the hardware underneath, on Volume 1 Chapter 8 (Performance Anti-patterns) for hot-path allocation, on Volume 5 (Databases) for `0_DIRECT` and self-managed caches, on Volume 12 / Book 6 (Containers and Kubernetes) for cgroup OOM, and on Volume 13 (Language Runtimes) for how managed runtimes sit on top of these same allocators.

The allocator sits between your program and the kernel

The kernel does not hand out bytes. It hands out *pages* — 4 KiB regions of a process's virtual address space, mapped on demand (Chapter 3). Your program, meanwhile, calls `malloc(37)` and expects 37 bytes, fast, a billion times a second. Bridging that gap — the mismatch between page-granular, syscall-priced kernel memory and byte-granular, nanosecond-priced application allocation — is the entire job of the user-space **memory allocator**.

The allocator is a library, linked into your process, that manages a pool of virtual memory it has obtained from the kernel and sub-divides it into the small, arbitrarily-sized objects your code requests. It has exactly two ways to get memory from the kernel:

- `brk / sbrk` move the "program break" — the top of the classic contiguous heap segment. Growing the heap is cheap (one syscall bumps a pointer; pages are populated on first touch), but it is a single linear region: you can only shrink it by moving the break back down, which you cannot do if any allocation above the intended new break is still live. This is one mechanical reason heaps grow but rarely shrink.
- `mmap` creates an independent anonymous memory region anywhere in the address space. Allocators use `mmap` for large allocations (glibc's default threshold is 128 KiB, tunable via `M_MMAP_THRESHOLD`) and for whole arenas. An `mmap` region can be returned to the kernel independently with `munmap`, so large allocations *can* release physical memory on free — unlike the `brk` heap.

Every design decision in a modern allocator exists to avoid two costs: the **syscall** (going to the kernel is expensive relative to a pointer bump — Chapter 5) and the **lock** (multiple threads hitting one shared free list serialize, and on a 64-core box that is death — Volume 1 Chapter 4). The result is a fairly standard architecture:

```

flowchart TD
    APP["Application threads: malloc / free"]
    subgraph ALLOC["User-space allocator (in-process library)"]
        TC1["Thread cache / per-CPU cache  
(lock-free fast path, size-classed)"]
        TC2["Thread cache / per-CPU cache"]
        A1["Arena 1"]
        A2["Arena 2"]
        CENTRAL["Central heap / spans"]
    end
    KERNEL["Kernel: brk / sbrk (main heap)"]
    PHYS["Physical pages (populated on first touch, Ch3 demand paging)"]

    APP --> TC1
    APP --> TC2
    TC1 -->|"miss / batch refill"| A1
    TC2 -->|"miss / batch refill"| A2
    A1 --> CENTRAL
    A2 --> CENTRAL
    CENTRAL -->|"grow"| KERNEL
    KERNEL --> PHYS

```

Size classes. Rather than track every possible request size, allocators round each request up to one of a fixed set of *size classes* (e.g., 8, 16, 32, 48, 64, 80, ... bytes, then coarser spacing for larger objects). All objects of a class come from the same pool and are interchangeable on `free`, which turns allocation into "pop a block off this class's free list" and free into "push it back." The rounding-up is the source of **internal fragmentation** (below), traded for speed and simplicity.

Free lists. Each size class keeps a list of freed-but-not-retained blocks. `malloc` pops; `free` pushes. This is the hot path, and it must be $O(1)$ and, ideally, lock-free.

Arenas. To avoid a single global lock, allocators partition their memory into multiple independent **arenas**, each with its own lock and its own free lists. Threads are distributed across arenas, so contention drops roughly by the number of arenas. glibc calls these arenas; jemalloc calls them arenas too. The natural extreme is **per-CPU** rather than per-thread partitioning, which tcmalloc and modern jemalloc favor: a per-CPU cache never contends because only one thread runs on a CPU at a time (given a stable CPU, aided by restartable sequences).

Thread caches. On top of arenas sits a small **per-thread cache** (tcmalloc's "thread- caching malloc" is named for it; jemalloc has `tcache`). The common case — allocate and free small objects on the same thread — never touches a lock at all: it pops and pushes a thread-local list, refilling from the arena in batches only when the cache empties or overflows. This is why a well-designed allocator's fast path is a handful of instructions.

The major allocators, and why the choice is a real lever

Swapping the allocator under a high-throughput service is one of the highest-leverage, lowest-effort performance changes available, precisely *because* the allocator sits on the hot path of every request that touches the heap — and in a managed runtime, of the runtime itself. It is usually a one-line change: `LD_PRELOAD=/usr/lib/libjemalloc.so` or linking `-ltcmalloc`, no code changes. The gains show up as higher throughput, lower tail latency (fewer lock stalls), less fragmentation, and a smaller resident footprint.

Allocator	Origin	Concurrency model	Notable traits	Typical use
<code>glibc malloc</code> (ptmalloc2)	Default on Linux	Multiple arenas (up to $\sim 8 \times \text{cores}$), per-thread arena assignment; <code>tcache</code> per thread since glibc 2.26	Ubiquitous, decent, but prone to higher fragmentation and RSS bloat on some long-running multi-thread workloads	Whatever you get if you do nothing
jemalloc	FreeBSD / Facebook	Arenas + per-thread <code>tcache</code> ; size-class design tuned for low fragmentation	Excellent fragmentation control, strong profiling (<code>prof</code>), predictable RSS; can proactively return memory (<code>background_thread</code> , <code>dirty_decay_ms</code>)	Redis (bundled default), Rust historically, many high-perf C/C++ services

Allocator	Origin	Concurrency model	Notable traits	Typical use
tcmalloc	Google	Per-thread and now per-CPU caches (restartable sequences)	Very fast small-object path, good scalability, integrated heap profiler; the Google-fleet allocator	Google-scale C++ services, gRPC-heavy workloads
mimalloc	Microsoft Research	Free-list sharding, per-thread heaps	Small, fast, good locality; sharded free lists reduce contention; simple to embed	Newer services, .NET, embedded use

The differences are real but workload-dependent. jemalloc's reputation for low fragmentation is why Redis bundles and defaults to it — a long-running in-memory store is exactly the workload where glibc's fragmentation would show up as slow, unbounded RSS growth. tcmalloc's per-CPU caches shine on many-core machines with high allocation rates. The correct engineering posture is not dogma but measurement: pick two candidates, run your real workload, and compare throughput, p99, and steady-state RSS. Do not, however, ignore the lever — running the default glibc allocator under a heap-churning service and then blaming "a memory leak" for RSS growth that is actually fragmentation is a common and avoidable mistake.

A note for Volume 13: managed runtimes do not escape this. A JVM allocates its heap from the OS in big chunks and manages objects itself with a garbage collector, but the *native* side (JNI, NIO direct buffers, thread stacks, the JIT, Netty's off-heap pools) still goes through the system allocator, and swapping in jemalloc under a JVM to fix native-memory fragmentation and RSS growth is a well-worn production move. Go and Rust ship their own allocators (Go's is a tcmalloc descendant; Rust historically used jemalloc, now the system allocator by default) — the same principles, different packaging.

Fragmentation: why long-running services grow

Internal fragmentation is wasted space *inside* an allocation: you asked for 37 bytes, the allocator gave you a 48-byte size class, 11 bytes are lost.

It is bounded and predictable — the price of size classes — and usually small in aggregate.

External fragmentation is the killer for long-running services. It is free memory that exists but cannot be used because it is chopped into pieces too small, or too scattered, to satisfy requests. Picture an arena that filled up with a mix of object sizes over hours of traffic; objects free in a scattered pattern, leaving a Swiss cheese of holes. The allocator holds plenty of free bytes, but no contiguous run large enough for the next big request, so it asks the kernel for *more* — and because the `brk` heap can only shrink from the top, the freed holes below stay mapped. RSS ratchets upward and never comes back down. The application has no leak in the "lost pointer" sense; it is simply fragmenting.

This is the mechanical root of the most common false alarm in production: **"it grows for days, then a restart fixes it, so it must be a leak."** Often it is not a leak — it is fragmentation, and the fix is an allocator with better fragmentation behavior (jemalloc, tcmalloc) or better decay tuning, not a hunt for a lost `free`. The distinguishing signal is in the allocator's own stats: a true leak shows *live* allocated bytes growing without bound; fragmentation shows a growing gap between allocated bytes and RSS (mapped-but-free memory). jemalloc's `stats.allocated` versus `stats.resident`, or `malloc_info/mallctl`, expose exactly this.

The last defense is architectural, and it ties to Volume 1 Chapter 8: **do not allocate in the hot path**. Every `malloc` in a per-request code path is lock traffic, cache misses on freshly touched memory, and fragmentation pressure. High-performance services use **object pools, arena/region allocators** (allocate a big block per request, bump-pointer within it, free the whole block at once), and pre-allocated buffers precisely to keep the general allocator off the critical path. "Allocate once, reuse forever" is the pattern; the general-purpose allocator is for setup, not for steady state.

The page cache: why "free memory" is a lie you should love

Here is the single most misunderstood fact about Linux memory, and the one that generates the most wrong alerts: **Linux uses all otherwise-idle RAM as a cache for file data, and this is correct behavior, not**

a problem. A box showing "200 MB free" out of 128 GB is not about to fall over. It is doing its job.

The **page cache** is the kernel's cache of file contents in RAM. When you read a file with ordinary buffered I/O, the data does not go disk→application; it goes disk→page cache→ application, and stays in the page cache. The next read of the same bytes is served from RAM at memory speed — no disk touched. When you write, your bytes land in the page cache (marked *dirty*) and the syscall returns immediately; the kernel flushes them to disk later. The page cache is how Linux makes the memory hierarchy of Volume 1 Chapter 3 transparent for files: DRAM transparently caches disk the way the CPU's SRAM caches DRAM, except the kernel manages it in software at page granularity.

```
flowchart LR
    APP["Application"]
    subgraph RAM["RAM"]
        PC["Page cache  
(clean + dirty pages)"]
    end
    DISK["Disk / SSD"]

    APP -->|"read(): hit → memcpy"| PC
    PC -->|"read miss → fault in"| DISK
    APP -->|"write(): copy in, mark dirty, return"| PC
    PC -->|"async write-back  
(dirty ratio / expiry)"| DISK
    APP -->|"fsync(): force flush + wait"| DISK
    DISK -->|"read-ahead: prefetch next pages"| PC
```

The principle the kernel follows is **"free memory is wasted memory."** RAM sitting idle earns nothing; RAM holding cached file data saves a disk I/O the moment that data is needed again. So Linux aggressively fills unused RAM with page cache. Crucially, this cache is **reclaimable**: the instant a process needs anonymous memory and none is free, the kernel drops clean cache pages (they are just copies of on-disk data — no write needed) and hands the memory over, transparently. Cache is not a claim on memory; it is opportunistic use of memory that is available the moment anyone wants it.

Reading free correctly

This is why `free`, `top`, and every naive "used memory" dashboard mislead. Consider:

```
$ free -h
```

	total	used	free	shared	buff/cache
available					
Mem:	125Gi	18Gi	1.2Gi	0.4Gi	
106Gi	105Gi				
Swap:	8.0Gi	0B	8.0Gi		

Only 1.2 GiB "free," and a novice panics. But look at **available: 105 GiB**. That is the number that matters — the kernel's own estimate of how much memory a new workload could get *without swapping*, counting reclaimable page cache. The 106 GiB of **buff/cache** is almost entirely reclaimable file cache doing useful work. The genuine application footprint is **used**: 18 GiB. The right mental model:

Field	What it means	Alert on it?
used	Memory in active use by processes (anonymous + non-reclaimable kernel)	This is real pressure — but see cgroups and PSI below
free	Truly unused RAM, holding nothing	No . Low free is normal and healthy
buff/cache	Page cache + buffers + reclaimable slab — file data cached in RAM	No . This is memory working, and reclaimable on demand
available	Kernel estimate of memory obtainable without swapping (free + reclaimable)	Yes . This is your real headroom

Alerting on **used** or on **free** at the node level is one of the most common capacity-and- alerting mistakes in a fleet: it fires constantly on healthy nodes (because Linux always fills cache) and trains everyone to ignore memory alerts, so the one time it matters, no one looks. Alert on **available**, on swap activity, and — best of all — on **PSI memory pressure** (below).

Write-back, the dirty ratio, and the durability gap

Dirty pages (written but not yet on disk) are flushed asynchronously by the kernel's **write-back** machinery — the per-backing-device **flush**

kernel threads (the modern successor to the old `pdflush / bdflush`). Two knobs govern when:

- `vm.dirty_background_ratio / vm.dirty_background_bytes` : when dirty pages exceed this, background write-back kicks in *asynchronously* — the application is not blocked.
- `vm.dirty_ratio / vm.dirty_bytes` : the hard ceiling. When dirty pages exceed this, writing processes are **throttled synchronously** — `write()` blocks until write-back drains enough. A burst of writes that outruns your disk manifests as latency spikes here, and on a write-heavy service this is a real and non-obvious source of p99 pain. `vm.dirtytime_expire` and `vm.dirty_expire_centisecs` bound how long a page may stay dirty regardless of ratio.

Now the part that matters for correctness, and ties directly to Volume 1 Chapter 5 (Storage) and Volume 5 (Databases): **a successful `write()` does not mean your data is on disk**. It means your data is in the page cache. If the machine loses power between the `write()` and the write-back, that data is gone, even though the syscall returned success. This is the **durability gap**, and closing it is exactly what `fsync(fd)` (and `fdatasync`, `O_SYNC`, `sync_file_range`) is for: `fsync` forces the file's dirty pages to stable storage and blocks until the device confirms. Every database's write-ahead log, every "we guarantee durability" claim, rests on `fsync` being called at the right point and the storage stack honoring it (the notorious "does the drive lie about its write cache" problem — Volume 1 Chapter 5). The page cache is a performance win precisely because it decouples the fast `write()` from the slow, durable flush — and it is a crash-consistency hazard for the same reason. Know where your `fsync` boundaries are.

Read-ahead and `O_DIRECT`

Read-ahead is the read-side optimization: when the kernel detects sequential access, it prefetches pages *ahead* of the application, so the next `read()` hits warm cache instead of waiting on disk. This is the file-level analog of the CPU's hardware prefetcher (Volume 1 Chapter 3) and is why streaming a large file sequentially is dramatically faster than random access. `posix_fadvise(POSIX_FADV_SEQUENTIAL/RANDOM/WILLNEED/DONTNEED)` and the `readahead(2)` syscall let an application tune or disable it — `FADV_RANDOM` to kill useless prefetch on a random-access workload,

`FADV_DONTNEED` to evict pages a backup job just streamed so it does not blow away a database's warm cache.

`O_DIRECT` is the opt-out: open a file with `O_DIRECT` and I/O bypasses the page cache entirely, moving data straight between the application's buffers and the device (with alignment constraints on offset, length, and buffer address). Why would you *give up* the free cache? Because a serious database manages its own cache — a buffer pool sized and evicted with knowledge the kernel lacks (which pages are index roots, which are cold leaf pages, what the query plan will touch next). Running through the page cache on top of that means **double caching**: the same data in the DB's buffer pool *and* the kernel's page cache, wasting RAM, plus unpredictable eviction the DB does not control. PostgreSQL famously *relies* on the page cache (and tunes around it); Oracle, MySQL/InnoDB (`O_DIRECT` is a common `flush_method`), and many others prefer `O_DIRECT` to own their caching end to end. This is a genuine architectural fork covered in Volume 5; the point here is that the page cache is a default, not a mandate, and the "manage your own cache" camp turns it off deliberately.

Under memory pressure, page cache is the *first* thing reclaimed (Chapter 3's reclaim path): clean pages are dropped for free, dirty pages are written back then dropped, all before the kernel resorts to swapping anonymous memory or invoking the OOM killer. That ordering is why a cache-heavy box shrinks its cache gracefully as real demand rises, and why cache should never, by itself, be read as pressure.

Huge pages: buying TLB reach

Recall the TLB from Volume 1 Chapter 3: a small, fast cache of virtual→physical translations. Every memory access needs a translation; a TLB hit is nearly free, a TLB miss walks the page table (four memory accesses on x86-64) and hurts. With the standard **4 KiB** page, each TLB entry covers 4 KiB, so a typical TLB of ~1500 entries reaches only ~6 MiB of memory before thrashing. A database with a 64 GiB buffer pool, or a JVM with a 40 GiB heap, or an in-memory store touching tens of gigabytes, blows through the TLB constantly — and TLB misses become a measurable, sometimes dominant, cost.

Huge pages fix this by making each page — and thus each TLB entry — bigger. On x86-64 the hardware supports **2 MiB** and **1 GiB** pages. A

single 2 MiB TLB entry covers $512\times$ the memory of a 4 KiB entry; the same ~ 1500 -entry TLB now reaches ~ 3 GiB. For a large, stable working set the reduction in TLB misses (and in page-table memory and walk cost) is a real throughput win — commonly a few percent, sometimes much more for pointer-chasing, memory-latency-bound workloads.

```
flowchart TB
    subgraph SMALL["4 KiB pages"]
        T1["TLB entry"] --> P1["4 KiB"]
        T2["TLB entry"] --> P2["4 KiB"]
        NOTE1["~1500 entries → ~6 MiB reach"]
        large_working_set["large working set → TLB thrash"]
    end
    subgraph HUGE["2 MiB huge pages"]
        H1["TLB entry"] --> HP1["2 MiB (512× reach)"]
        H2["TLB entry"] --> HP2["2 MiB"]
        NOTE2["~1500 entries → ~3 GiB reach"]
        far_fewer_misses["far fewer TLB misses"]
    end
```

There are two very different ways to get huge pages, and conflating them is the source of most huge-page trouble.

Explicit huge pages (hugetlbfs). You reserve a pool of huge pages up front (`vm.nr_hugepages` , or per-NUMA-node), and applications request them explicitly — via `mmap(MAP_HUGETLB)` , `shmget(SHM_HUGETLB)` , or a `hugetlbfs` mount. These pages are pinned: reserved, never swapped, never split, never transparently moved. This is deterministic and is how databases that want huge pages get them — Oracle's SGA, PostgreSQL's `huge_pages=on` , and KVM guest memory all use `hugetlbfs`. The cost is operational: you must size and reserve the pool, ideally at boot before memory fragments (a running system may not be able to find enough contiguous 2 MiB regions to grow the pool). It is explicit, predictable, and a bit of work.

Transparent Huge Pages (THP). The kernel tries to give you huge pages *automatically*, with no application changes: it hands out 2 MiB pages for suitable anonymous mappings when it can, and a background kernel thread, `khugepaged` , scans memory and *collapses* runs of 4 KiB pages into 2 MiB pages after the fact. THP has three modes via `/sys/kernel/mm/transparent_hugepage/enabled` : `always` , `madvise` (only regions that opt in with `madvise(MADV_HUGEPAGE)`), and `never` .

The THP caveat: why Redis and databases tell you to turn it off

THP sounds like a free win, and for some batch and analytics workloads it is. But for **latency-sensitive** and **fork-heavy** services it is a notorious source of tail-latency spikes and memory bloat, and the guidance from Redis, MongoDB, Oracle, Couchbase, and many others to **disable THP** (specifically the `always` mode) is well-founded, not cargo-cult. The mechanisms, accurately:

- **Allocation stalls.** To hand out a 2 MiB page, the kernel needs 2 MiB of *physically contiguous* free memory. On a fragmented system it may not have it, so a huge-page fault can trigger **synchronous compaction** — the kernel shuffles pages around to manufacture a contiguous region *while your thread waits*. That stall lands as a multi-millisecond spike in an otherwise microsecond-scale operation. On a p99-sensitive service this is exactly the jitter you are trying to eliminate, and it appears unpredictably under memory pressure.
- **khugepaged overhead.** The collapsing thread does real work scanning and rewriting page tables, competing for CPU and locks, and its activity correlates with latency blips.
- **Memory bloat.** THP rounds allocations up toward 2 MiB granularity. A process that touches a few 4 KiB pages scattered across a region can end up backed by full 2 MiB pages, inflating RSS well beyond what the application actually uses. This is the reported bloat.
- **The fork amplifier — the Redis case.** Redis persists by `forking` and letting copy-on-write share pages between parent and child (Chapter 1, Chapter 3). With 4 KiB pages, a write to one key dirties one 4 KiB page and copies 4 KiB. With THP, a write to one key can dirty a whole **2 MiB** page, copying 512× as much — so during a background save, a write-heavy Redis can see its memory balloon and its latency degrade dramatically. This CoW amplification is the specific, concrete reason the Redis docs tell you to disable THP, and it generalizes to any fork-and-CoW persistence design.

Situation	Huge pages: help or hurt?	Why
Large, stable working set (DB buffer pool, big JVM heap), long-lived	Help — use explicit <code>hugetlbfs</code>	Big TLB reach win; reserved pool avoids runtime stalls

Situation	Huge pages: help or hurt?	Why
Latency-sensitive request/response service, p99 matters	Hurt (THP <code>always</code>)	Compaction stalls and khugepaged jitter spike tails
Fork-heavy CoW persistence (Redis background save)	Hurt (THP)	2 MiB CoW amplifies copy on every dirtied page
Scattered/sparse memory access	Hurt (THP)	RSS bloat from rounding up to 2 MiB
Batch / analytics, throughput over latency, dense access	Often help (THP fine)	Fewer TLB misses, stalls don't matter without tight SLOs

The honest, practical posture: for a big database or JVM with a large stable heap, use **explicit huge pages** (`hugetlbfs`, `huge_pages=on`, `-XX:+UseLargePages`) — you get the TLB win deterministically, with the pool reserved at boot. For a **latency-sensitive** service, set THP to `madvise` or `never` and follow the vendor guidance — the default `always` on many distros is the mode that bites. `madvise` is a reasonable middle ground: allocators like `jemalloc` and `tcmalloc`, and runtimes that want huge pages, opt in per-region and avoid the blanket behavior. This is a fleet p99 gotcha: one wrong default in a base image, replicated across thousands of pods, is a fleet-wide tail-latency regression.

The OOM killer: when overcommit comes due

Chapter 3 explained **overcommit**: Linux hands out more virtual memory than it has physical RAM plus swap, betting that not everyone touches all their pages. Usually the bet pays. When it does not — when actual demand for physical pages exceeds what reclaim can free — the kernel is cornered. It cannot fault in a page that a running instruction needs, and it has already reclaimed all the cache it can and swapped what it can. Something has to give, and what gives is a whole process: the **OOM (out-of-memory) killer** selects a victim and kills it to free memory and keep the system alive.


```

flowchart TD
    START["Allocation / page fault needs a physical page"]
    FREE{"Free page available?"}
    RECLAIM["Reclaim: drop clean cache,  
write back dirty, swap anon (Ch3)"]
    OK["Grant page → continue"]
    RECOK{"Reclaim freed enough?"}
    OOM["Out of memory → invoke OOM killer"]
    SCORE["Score every task:  
oom_score = f(RSS, swap, ...) adjusted by oom_score_adj"]
    VICTIM["Pick highest badness score"]
    KILL["SIGKILL the victim (uncatchable)"]
    FREED["Memory freed → system survives,  
victim is gone with no cleanup"]

    START --> FREE
    FREE -->|yes| OK
    FREE -->|no| RECLAIM
    RECLAIM --> RECOK
    RECOK -->|yes| OK
    RECOK -->|no| OOM
    OOM --> SCORE
    SCORE --> VICTIM
    VICTIM --> KILL
    KILL --> FREED

```

```

START --> FREE
FREE -->|yes| OK
FREE -->|no| RECLAIM
RECLAIM --> RECOK
RECOK -->|yes| OK
RECOK -->|no| OOM
OOM --> SCORE --> VICTIM --> KILL --> FREED

```

How the victim is chosen

The kernel computes a **badness score** for candidate processes. The core heuristic is simple and mostly about size: a process's score is roughly proportional to its memory footprint (RSS plus swap plus page-table pages), normalized so it is comparable across the relevant memory domain. The idea is that killing the biggest hog frees the most memory for the least number of kills. You can read the current effective score at `/proc/<pid>/oom_score`.

You tune it with `oom_score_adj` (`/proc/<pid>/oom_score_adj`, range `-1000` to `+1000`). It is added into the badness calculation: **-1000 makes a process effectively unkillable** (score floored to zero), positive values make it a preferred victim. This is how you protect critical processes — the node's monitoring agent, `sshd`, the control-plane daemon — by biasing the killer away from them and toward the fungible workload. Set it too aggressively unkillable on the wrong process, though, and you can make a box that cannot recover because the killer has no legal victim. (`oom_score_adj` supersedes the old `oom_adj` interface.)

SIGKILL is brutal

The victim receives **SIGKILL** — signal 9, the uncatchable one. It cannot be trapped, handled, blocked, or ignored (Chapter 8 on signals). The process does not get to flush buffers, close sockets cleanly, finish in-flight requests, release locks, or write a shutdown log line. It is simply stopped, and its memory is reclaimed. There is no graceful cleanup, by design — the whole point is to reclaim memory *right now*, and a well-behaved shutdown that itself needs to allocate would deadlock the very condition it is resolving. For your service this means an OOM kill is *abrupt data-loss-shaped*: any state not already durably persisted is gone, held locks may be left in whatever recovery the rest of the system implements, and clients see connections drop. Design for it — assume any process can vanish between two instructions.

cgroup OOM: the **OOMKilled** pod and exit code 137

This is the failure mode you will actually meet most, and the one most often misdiagnosed. Everything above described *system-wide* OOM, triggered by the whole node running out. But containers run inside **cgroups** with memory limits (Chapter 9; Volume 12 / Book 6), and a cgroup can hit *its own* limit while the node has gigabytes of free RAM. When a cgroup's memory usage reaches its hard limit and reclaim within the cgroup cannot free enough, the kernel invokes the OOM killer **scoped to that cgroup** — it kills a process *inside the container*, not somewhere else on the node.

In Kubernetes this surfaces as a pod with status **OOMKilled** and the container's exit code **137** (128 + 9, i.e., terminated by signal 9). The single most important thing to internalize: **OOMKilled almost always means the container exceeded its cgroup memory limit, not that the node is out of memory**. Engineers routinely misread it as "the node needs more RAM" and go add nodes, when the node is fine and the container's limit is simply too low for its real working set (or the app has a genuine leak, or it is fragmenting, or a load spike pushed it over). The fix is right-sizing the limit and understanding the workload — not node capacity.

cgroup **v2** gives you a more nuanced set of controls, and knowing the difference between them is core operational competence:

Control (cgroup v2)	Type	Behavior when reached
<code>memory.max</code>	Hard limit	The wall. Exceeding it triggers cgroup OOM kill inside the cgroup (→ <code>OOMKilled</code> , 137). This is what Kubernetes' <code>resources.limits.memory</code> sets.
<code>memory.high</code>	Throttle / soft	Above this, the cgroup is aggressively reclaimed and its allocating tasks are throttled (slowed via induced reclaim/scheduling delay) — but not killed . A pressure valve, not a wall.
<code>memory.min</code>	Hard reservation	Memory below this is never reclaimed from the cgroup, even under system pressure. Guarantees a working set.
<code>memory.low</code>	Soft reservation	Memory below this is reclaimed only as a last resort — best-effort protection.
<code>memory.current</code>	Observed	Current usage; watch it against <code>max</code> / <code>high</code> .

The `max`-versus-`high` distinction is the operational lever people miss. `memory.max` alone means a workload that briefly overshoots gets killed outright. Setting `memory.high` *below* `memory.max` creates a throttle zone: as the workload approaches its limit it is slowed and reclaimed hard, giving it a chance to shed memory (e.g., a GC to run, a cache to evict) *before* it hits the fatal wall — trading latency for survival. Kubernetes historically only exposed the hard limit, but this throttle-then-kill shape is exactly what you want for services that can respond to back-pressure. `memory.min` / `memory.low` go the other way: they *protect* a latency-critical container's working set from being reclaimed when a noisy neighbor spikes.

PSI: measuring pressure before the kill

The best early-warning signal is **Pressure Stall Information (PSI)**, exposed at `/proc/pressure/memory` and per-cgroup at `<cgroup>/memory.pressure`. PSI measures the fraction of time tasks are **stalled waiting on memory** — blocked on reclaim, refaulting pages, thrashing

— as `some` (at least one task stalled) and `full` (all non-idle tasks stalled) percentages over 10 s / 60 s / 300 s windows:

```
$ cat /proc/pressure/memory
some avg10=0.00 avg60=0.12 avg300=0.31 total=8420431
full avg10=0.00 avg60=0.04 avg300=0.10 total=3021994
```

This is a *far* better alerting signal than "used memory" or "percent of limit," because it measures **actual harm** — time lost to memory shortage — rather than a level that may be perfectly healthy (all that reclaimable cache again). A cgroup with rising `memory.pressure` `full` is thrashing and heading for trouble; a cgroup at 90% of `memory.max` with zero pressure is fine. `systemd-oomd` uses PSI to proactively kill workloads *before* the kernel's hard OOM kicks in, precisely because PSI sees the pain coming. Alert on PSI.

Observing memory: telling leaks from fragmentation from cache

You cannot manage what you cannot measure, and memory metrics are riddled with traps — double-counted shared pages, cache masquerading as usage, virtual size that means almost nothing. Here are the metrics that matter and what each actually tells you.

Metric	What it counts	What it's good for	Trap
VSZ (virtual size)	Total virtual address space mapped — including memory never touched, files mmap'ed, guard pages	Almost nothing operationally	Wildly overstates real usage; a Go/JVM process may reserve huge VSZ it never populates. Do not alert on VSZ.

Metric	What it counts	What it's good for	Trap
RSS (resident set size)	Physical pages currently resident for the process, including shared pages counted in full	First-order "how much RAM is this using"	Double-counts shared memory: 10 processes sharing a 100 MB library each report +100 MB RSS, so summing RSS overcounts wildly
PSS (proportional set size)	Private pages in full + shared pages divided by the number of sharers	The honest per-process footprint; PSS sums across processes to (roughly) true total	Costlier to compute; from <code>/proc/<pid>/smaps</code>
Page cache	File data cached in RAM (<code>buff/cache</code>)	Reclaimable, not a per-process cost in the usual sense	Confused with process memory; it is not "used" in the alarming sense

RSS vs VSZ vs PSS in one line: VSZ is what the process *could* touch (mostly meaningless), RSS is what it *has* touched but over-counts sharing, and **PSS** is the number to use when you want a real, additive per-process footprint on a box where processes share libraries and memory. On a fleet, PSS-based accounting (via `smem`, or Prometheus exporters that read `/proc/<pid>/smaps_rollup`) is what tells you the truth about where RAM is going.

The primary sources:

- `/proc/meminfo` — the system-wide truth: `MemTotal`, `MemFree`, `MemAvailable`, `Cached`, `Buffers`, `Dirty`, `Writeback`, `AnonPages`, `Slab` (`SReclaimable` / `SUnreclaim`), `KReclaimable`, `HugePages_*`, `AnonHugePages`, `SwapFree`. `MemAvailable` is the field to trust for headroom.
- `/proc/<pid>/status` — `VmRSS`, `VmSize`, `RssAnon` (heap/stack), `RssFile` (mapped files), `RssShmem`. `RssAnon` is the closest quick proxy for "the app's own heap."

- `/proc/<pid>/smaps_rollup` — per-process `Pss`, `Private_Clean/Dirty`, `Shared_*`, `Swap`. This is where PSS lives.
- `smem` — a userspace tool that reports PSS/USS/RSS per process, saving you the smaps arithmetic.
- `cgroup memory.stat` — inside a container, *the* source: `anon` (anonymous memory — your real app footprint), `file` (page cache attributed to the cgroup), `kernel_stack`, `slab`, `sock`, `shmem`, `file_dirty`, `file_writeback`, and reclaim/refault counters. Crucially, `memory.current` includes page cache, so a container near its `memory.max` may be near it *because of reclaimable cache*, not app memory — check `anon` vs `file` before concluding the app is bloated.

Which problem is it?

The diagnostic skill is separating four look-alike symptoms, all of which present as "memory going up" or "OOMKilled":

- **A real leak.** `RssAnon` / `cgroup anon` grows monotonically and without bound, tracking allocation, never plateauing. The allocator's *live allocated* bytes grow. A heap profile (`jemalloc prof`, `tcmmalloc` heap profiler, `pprof`, or runtime tools like Go's `pprof` or JVM `NMT`) shows a growing allocation site. Fix: find the lost reference.
- **Fragmentation.** `anon` /RSS grows but the allocator reports lots of *free* memory inside its arenas — a growing gap between `stats.allocated` and `stats.resident` (`jemalloc`). Live bytes are stable-ish; RSS is not. The "restart fixes it" smell. Fix: better allocator or decay tuning, pooling.
- **Just cache.** `memory.current` (or `node used`) is high but it is `file / buff / cache`, not `anon`. `available / MemAvailable` is healthy, PSI is quiet. **Not a problem.** Fix: stop alerting on the wrong metric.
- **Legitimately under-provisioned.** `anon` genuinely needs more than the limit — real working set exceeds `memory.max`. PSI climbs, refaults rise, then OOMKilled. Fix: raise the limit or shrink the workload; this is a sizing decision, not a bug.

What to alert on, distilled: `node/cgroup PSI memory pressure` (`full` especially), `MemAvailable` (or `memory.max - anon` headroom), **swap-in rate** and **major page-fault / refault rate** (thrashing), and **OOM-kill events** (a kill already happened — this is a "why" alert, not a "prevent" one). Do **not** alert on raw `used`, `free`, `VSZ`, `RSS-sum`, or "percent of

`memory.max` " without separating cache from anon. Getting this right across a fleet is the difference between actionable pages and alert fatigue.

Distributed-systems lens

At fleet scale, these four surfaces are not academic — they dominate reliability and cost, and each maps to a recurring class of incident:

- **OOMKilled is a top container-restart cause.** The mechanism is *cgroup-limit* OOM, not node-memory exhaustion (exit 137, `SIGKILL`, no cleanup). The organizational failure is misdiagnosis: teams add nodes when they should right-size `memory.limits`, or set limits by copy-paste with no relation to the real working set. Fix it with PSS-based footprint data, `memory.high` throttle zones for services that can shed load, and `memory.min` protection for latency-critical pods — and by teaching everyone that 137 means "your limit, not the node."
- **The page cache misleads capacity and alerting fleet-wide.** Every healthy node looks "nearly full" because Linux fills RAM with reclaimable cache. Alerting on `used / free` generates constant false pages and trains teams to ignore memory alerts. The fix is fleet policy: alert on `MemAvailable`, swap activity, and PSI — never on `raw used/free`.
- **Allocator choice is a fleet-wide perf and footprint lever.** For high-throughput services, `LD_PRELOAD`-ing `jemalloc` or `tcmmalloc` — one line in a base image — can lift throughput, cut p99, and shrink steady-state RSS across thousands of instances, reducing both latency and the memory bill. It is among the cheapest wins available and is frequently left on the table.
- **THP is a fleet p99 gotcha.** A base image shipping THP `always` replicates compaction stalls and CoW bloat across every pod. The Redis/MongoDB/DB guidance to disable it (use `madvise` or `never`) is a fleet-wide tail-latency policy, not a per-host tweak — set it in the image or via a `tuned` profile / node bootstrap.
- **Huge pages are a fleet lever the other direction.** For big-memory data services (databases, large JVMs), explicit `hugetlbfs` reduces TLB pressure across the fleet — a throughput and efficiency gain worth the operational cost of reserving pools.

- **Fragmentation is the "restart fixes it" epidemic.** Long-running services on the default allocator grow slowly for days and get restarted on a cron "to be safe." That is fragmentation masquerading as a leak; the real fix is an allocator with better decay behavior, not a babysitting restart. At fleet scale those restarts are churn, capacity waste, and masked bugs.

Memory sizing, limits, and OOM are, in aggregate, the *daily* operational reality of running services at scale — more so than almost any single mechanism from Chapter 3. The mechanisms are the kernel's; the consequences are yours to size, alert on, and tune.

Key takeaways

- **The allocator lives in your process, between `malloc` and the kernel.** It gets pages via `brk / mmap` and sub-divides them with size classes, free lists, per-thread/per-CPU arenas, and thread caches to dodge syscalls and lock contention. glibc's `malloc` is the default; jemalloc and tcmalloc are drop-in levers for throughput, tail latency, and lower fragmentation — swapping the allocator is a real, cheap win.
- **Fragmentation, not leaks, causes much long-running memory growth.** A growing gap between live-allocated bytes and RSS is fragmentation; monotonic live-byte growth is a leak. "A restart fixes it" usually means fragmentation. Keep allocation off the hot path with pools and arenas (Volume 1 Chapter 8).
- **"Free memory is wasted memory."** Linux fills idle RAM with reclaimable page cache; low `free` is normal and healthy. Read `available / MemAvailable`, not `used / free`. Never alert on raw used/free at the node level.
- **A successful `write()` is not durability.** Data sits dirty in the page cache until write-back (governed by the dirty ratio) or `fsync`. The gap between them is where power-loss data loss lives — every database's durability rests on `fsync` (Volume 1 Chapter 5, Volume 5). `O_DIRECT` exists so self-caching databases can bypass the page cache and avoid double caching.
- **Huge pages buy TLB reach** (2 MiB / 1 GiB entries cover far more memory), a real win for large stable working sets. Use **explicit hugetlbfs** for databases and big JVMs. But **THP** (always) causes

compaction stalls, khugepaged jitter, RSS bloat, and — for fork-and-CoW designs like Redis — copy amplification; disable it (`madvise / never`) for latency-sensitive and fork-heavy services.

- **OOM kill is SIGKILL : uncatchable, no cleanup.** Victim selection is size-driven badness, tunable with `oom_score_adj` (`-1000` = protected). Assume any process can vanish between two instructions.
- **OOMKilled / exit 137 is almost always a cgroup limit, not the node.** In cgroup v2, `memory.max` is the hard wall (→ kill), `memory.high` throttles-and-reclaims without killing, and `memory.min / low` reserve a protected working set. Right-size limits; don't add nodes to fix a limit.
- **Measure with the right metric.** VSZ is noise; RSS over-counts sharing; **PSS** is the honest per-process footprint. Inside a container, split cgroup `memory.stat` anon (app) from file (cache). **Alert on PSI memory pressure**, `MemAvailable`, swap-in, and refault rate — the signals that measure actual harm.

Further reading

- **The Linux Programming Interface**, Michael Kerrisk, No Starch Press, 2010. Chapters 7 (memory allocation), 49 (memory mappings), and 15/13 (file I/O, buffering, `fsync`) are the accurate practitioner reference for `brk / sbrk` , `mmap` , and the page-cache/durability boundary.
- **jemalloc** documentation and Jason Evans, "A Scalable Concurrent malloc(3) Implementation for FreeBSD" (2006) — primary source on arenas, thread caches, and the fragmentation-control design. See also the jemalloc `TUNING.md` and `mallctl` reference for `dirty_decay_ms / background_thread` .
- **TCMalloc** design doc (`github.com/google/tcmalloc` , `docs/design.md`) and **mimalloc** (Microsoft Research technical report, `github.com/microsoft/mimalloc`) — the per-CPU-cache and sharded-free-list designs, respectively.
- **Linux kernel documentation:** `Documentation/admin-guide/mm/transhuge.rst` (THP semantics and tuning), `Documentation/admin-guide/mm/hugetlbpage.rst` (explicit huge pages), `Documentation/admin-guide/cgroup-v2.rst` (`memory.max / high / min /`

`low`, `memory.stat`), and `Documentation/accounting/psi.rst` (Pressure Stall Information).

- **Redis documentation, "Latency and CPU" / "THP" and MongoDB / Oracle THP-disable guidance** — the primary, accurate statements of the fork-CoW amplification and compaction-stall problems, from the vendors that hit them.
- **man pages:** `malloc(3)`, `mallopt(3)`, `proc(5)` (for `/proc/meminfo`, `/proc/<pid>/status`, `smaps`, `smaps_rollup`, `oom_score`, `oom_score_adj`), `madvise(2)`, `posix_fadvise(2)`, `mmap(2)` (`MAP_HUGETLB`), `open(2)` (`O_DIRECT`), `cgroups(7)`.
- **"What Every Programmer Should Know About Memory"**, Ulrich Drepper (2007) — the TLB, huge pages, and memory-hierarchy background underpinning the huge-page discussion (Volume 1 Chapter 3).
- **Brendan Gregg, *Systems Performance*, 2nd ed.** (Addison-Wesley, 2020), memory chapter, and Gregg's blog posts on PSI, `MemAvailable`, and memory saturation — for what to measure and alert on in production. The `free(1)`, `vmstat(8)`, and `smem(8)` man pages complete the observability toolkit.
- **Facebook Engineering / systemd-oomd** writeups on PSI-driven OOM avoidance — how pressure-based, userspace OOM management improves on the kernel's last-resort killer.

Chapter 5 — System Calls and the Kernel Boundary

What this chapter covers. A running program is not allowed to do anything interesting on its own. It cannot read a file, send a packet, map memory, create a process, or read the clock from a device — every one of those touches hardware or shared kernel state, and the hardware forbids user code from touching them directly. So a program *asks*. The system call is the mechanism of asking: a controlled, hardware-enforced doorway from your code in ring 3 into the kernel in ring 0, the one and only sanctioned way across the privilege boundary Volume 1, Chapter 10 (Hardware Virtualization) established. The set of syscalls is the API of the operating system — everything a libc function, a language runtime, a

database, or a web server ultimately does to the outside world bottoms out in a syscall.

This chapter is about that boundary: what crossing it costs, why the cost matters at fleet scale, and how high-performance systems are organized to cross it as rarely as possible. We walk the `SYSCALL` instruction end to end on x86-64, distinguish the *mode switch* it causes from the *context switch* it does not, account honestly for its price (worse since the 2018 speculative-execution mitigations), and then survey the entire toolbox of syscall-reduction: buffered and vectored I/O, `sendfile/splice`, `io_uring`, and the vDSO trick that services some "syscalls" without entering the kernel at all. Then we turn the boundary around and look at it as the security perimeter — the place `seccomp`, gVisor, and the LSMs enforce isolation by controlling *which* syscalls a process may make. We close on the ABI-stability rule that makes this interface a contract you can build a decade of software on.

Learning goals — after this chapter you should be able to:

- Explain why user code cannot perform privileged operations directly, and why the syscall is the *only* controlled entry into ring 0.
- Walk a syscall through the x86-64 mechanism: the register convention (`rax` = number; `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` = args), the `SYSCALL` / `SYSRET` fast path versus the legacy `int 0x80`, kernel-stack switch, syscall-table dispatch, and the `-errno` return.
- Distinguish a **mode switch** (same task, ring 3 → ring 0) from a **context switch** (different task) and account for the real cost of each, including the KPTI/Spectre overhead.
- Justify syscall reduction as a first-class performance lever and choose the right technique — buffering, `readv/writev`, `sendfile/splice`, `io_uring`, vDSO — for a given hot path.
- Explain what libc wrappers add (`errno`, portability), how `syscall()` bypasses them, and how the vDSO services `clock_gettime` / `gettimeofday` with no mode switch.
- Observe syscalls in anger with `strace`, `perf trace`, and eBPF, and know why the first is a debugging tool and the last two are production tools.
- Explain the syscall boundary as the isolation enforcement point: `seccomp-bpf`, Docker's default profile, gVisor's user-space kernel, and LSM hooks.

This chapter is Linux/x86-64-specific and builds directly on Volume 1, Chapter 10 (rings and the isolation spectrum) and Volume 2, Chapter 1 (the process/task model). It sets up Chapter 7 (Linux I/O Models — `io_uring` and zero-copy get their full treatment there), Chapter 11 (perf and eBPF), and Book 6 (Containers and Kubernetes — seccomp profiles and sandboxing).

The boundary and why it exists

Recall the hardware fact from Volume 1, Chapter 10: an x86-64 CPU runs in one of two privilege levels that matter in practice — **ring 3** (user mode) and **ring 0** (kernel/supervisor mode). A class of **privileged instructions** — loading the page-table base register `CR3`, executing `HLT`, reading and writing model-specific registers via `RDMSR / WRMSR`, doing port I/O with `IN / OUT`, masking interrupts — will simply fault if attempted in ring 3. That is the whole basis of process isolation. The application literally *cannot* reprogram the MMU to see another process's memory, cannot talk to the disk controller, cannot mask the timer interrupt that lets the scheduler preempt it, because the instructions that would do those things trap the moment it tries.

But a real program needs those effects constantly. Opening a socket reprograms a device. Allocating heap changes page tables (Chapter 3). `fork` creates a task (Chapter 1). Reading a file pulls bytes off a block device through the page cache. All of it requires ring-0 power the process does not have. The resolution is not to grant the power — it is to let the process *ask a trusted intermediary that already has it*. That intermediary is the kernel, and the ask is a **system call**.

The crucial design property is that the transition into ring 0 does not let the user jump to an arbitrary kernel address. If it did, the boundary would be worthless — you would just `SYSCALL` into the middle of a routine that skips the permission check. Instead, the hardware entry is fixed: the kernel, at boot, writes the address of *its* single entry point into a model-specific register, and the `SYSCALL` instruction transfers control only there. The user side chooses *which* service it wants by putting a number in a register; it does not choose *where* in the kernel execution lands. Everything the kernel does after that — validating arguments, checking credentials and namespaces, enforcing seccomp filters — happens on the kernel's terms, in the kernel's code, on the kernel's stack.

```

flowchart LR
    subgraph U["Ring 3 – user mode"]
        APP["Application code"]
        LIBC["libc wrapper  
(errno, portability)"]
        APP --> LIBC
    end
    GATE["SYSCALL instruction  
(the only controlled gate)"]
    subgraph K["Ring 0 – kernel mode"]
        ENTRY["Fixed entry point  
(MSR_LSTAR)"]
        DISP["syscall table dispatch  
sys_call_table[rax]"]
        HANDLER["Handler: validate args,  
check creds/namespaces/seccomp,  
do the privileged work"]
        ENTRY --> DISP --> HANDLER
    end
    LIBC --> GATE --> ENTRY
    HANDLER -->|"SYSRET, result in rax"| APP

```

This is the mental model to hold for the whole chapter: **one gate, chosen by the kernel, that the user can only request to pass through**. Performance work is about passing through it less often. Security work is about controlling what happens once you are through.

How a syscall works, mechanically (x86-64)

Let us take the simplest possible example and follow every step. A program calls `write(fd, buf, len)`. The libc wrapper does almost nothing interesting; the interesting part is the raw syscall it emits. On x86-64 Linux the convention is fixed and worth memorizing:

Register	Role
<code>rax</code>	System call number (e.g., <code>write</code> = 1, <code>read</code> = 0, <code>openat</code> = 257)
<code>rdi</code>	Argument 1
<code>rsi</code>	Argument 2
<code>rdx</code>	Argument 3
<code>r10</code>	Argument 4

Register	Role
r8	Argument 5
r9	Argument 6
rax	Return value (on the way back): result, or <code>-errno</code> in <code>[-4095, -1]</code>

Two details trip people up. First, syscalls take arguments in `r10`, not `rcx`, even though `rcx` is the fourth argument in the normal System V C calling convention. That is because the `SYSCALL` instruction itself clobbers `rcx` (it stashes the return address, `RIP`, there) and `r11` (it stashes `RFLAGS` there). The kernel ABI works around the hardware by using `r10` for the fourth argument. Second, there is no separate "error" return — success and failure both come back in `rax`, and the convention is that a return value in the range `-4095` to `-1` is interpreted as a negative `errno`. `libc` is what turns `-2` into "return `-1`, set `errno = ENOENT`."

Here is the raw sequence for `write(1, buf, len)` in assembly, deliberately unwrapped:

```

mov    rax, 1          ; syscall number for write
mov    rdi, 1          ; fd = 1 (stdout)
mov    rsi, buf        ; pointer to buffer
mov    rdx, len        ; byte count
syscall                ; enter the kernel
; on return: rax = bytes written, or negative errno

```

The `SYSCALL` instruction is the fast path, introduced with AMD64 and standard on every x86-64 CPU. The legacy mechanism was the software interrupt `int 0x80`, which went through the interrupt descriptor table — a much heavier path involving a memory read of the IDT and full interrupt-gate semantics. `SYSCALL` is deliberately lean: the CPU has the kernel's entry address pre-loaded in the `LSTAR` model-specific register, so entry is a register-to-`RIP` transfer with no table lookup. (The 32-bit-compatible fast entry is `sysenter`; the modern 64-bit world is `SYSCALL / SYSRET`.)

What `SYSCALL` does in hardware, precisely:

1. Saves the user return address into `rcx` and the user flags into `r11`.

2. Loads `RIP` from the `LSTAR` MSR — the kernel's fixed entry point.
3. Switches the CPU to ring 0 and applies the flag mask from the `SFMASK` MSR (this is how interrupts and the trap flag get disabled on entry).
4. Loads the kernel code segment from `STAR`. Notably, `SYSCALL` does **not** switch the stack pointer — `rsp` still points at the *user* stack the instant the kernel gets control.

That last point is why the very first thing the kernel entry code does is switch to the kernel stack for that task. It uses the `swapgs` instruction to get at per-CPU kernel data (the kernel stashes the kernel `GS` base so it can find its own structures without trusting any user register), loads the task's kernel stack pointer, and only then saves the full user register set onto that kernel stack. Now the kernel has a safe, trusted execution context. It reads the syscall number out of the saved `rax`, bounds-checks it against the size of the table, and dispatches:

```
/* Conceptually, the dispatch is an indexed jump through a function-  
pointer table. */  
if (nr < NR_syscalls)  
    regs->ax = sys_call_table[nr](regs);    /* run the handler */  
else  
    regs->ax = -ENOSYS;                      /* unknown syscall  
number */
```

`sys_call_table` is exactly what it sounds like: an array indexed by syscall number whose entries are pointers to handler functions (`sys_write`, `sys_openat`, and so on, generated from `syscall_64.tbl` in the kernel tree). Dispatch is an array index and an indirect call — cheap. The handler does the real work: it copies arguments in from user space with the `copy_from_user` family (which validates that the pointers actually point into the caller's address space, never trusting a user-supplied pointer), performs permission and namespace checks, and carries out the privileged operation. When it finishes, it leaves the return value in the saved `rax`, restores the user register set, executes `swapgs` again to hand `GS` back, and issues `SYSRET`, which restores `RIP` from `rcx` and `RFLAGS` from `r11`, drops back to ring 3, and resumes the user program on the instruction after its `SYSCALL`.

```
sequenceDiagram
    participant U as User code (ring 3)
    participant CPU as CPU hardware
    participant K as Kernel entry (ring 0)
    participant H as Syscall handler
    U->>U: load rax=nr, rdi/rsi/rdx/r10/r8/r9 = args
    U->>CPU: SYSCALL
    CPU->>CPU: save RIP=rcx, RFLAGS=r11
    CPU->>CPU: RIP=LSTAR MSR, switch to ring 0
    CPU->>K: enter fixed kernel entry
    K->>K: swapgs, switch to kernel stack, save user regs
    K->>H: dispatch via sys_call_table[rax]
    H->>H: copy_from_user, check creds/seccomp, do work
    H-->>K: result in rax
    K->>K: restore user regs, swapgs
    K->>CPU: SYSRET
    CPU->>U: RIP=rcx, RFLAGS=r11, back to ring 3
    U->>U: read result / -errno from rax
```

Mode switch, not context switch

A syscall crosses privilege levels but stays inside the *same task*. Nothing about the process identity changes: the same `task_struct`, the same address space, the same open file table (Chapter 1). This is a **mode switch** (also called a privilege transition or a domain crossing), and it is categorically cheaper than a **context switch**, in which the scheduler tears down one task's CPU state and installs another's. Conflating the two is one of the most common performance misunderstandings, so pin the distinction down:

Property	Mode switch (syscall)	Context switch
Task changes?	No — same process/thread	Yes — different task
Address space (<code>CR3</code>) changes?	No, in classic design (see KPTI below)	Often yes (different process)
Scheduler runs?	No	Yes — pick next task
Registers saved/restored	User GPRs onto kernel stack	Full task CPU + FPU state
TLB impact	Minimal classically; KPTI added a <code>CR3</code> reload	Flush or PCID reload on address-space change

Property	Mode switch (syscall)	Context switch
Rough cost	Tens to a few hundred ns	~1-several μ s (plus cold-cache aftershock)
Triggered by	<code>SYSCALL</code> , faults, interrupts	<code>schedule()</code> : preemption, blocking, yield

A syscall *can* lead to a context switch — if `read` finds no data ready, the task blocks, the scheduler runs, and some other task gets the CPU — but the boundary crossing itself is not one. For a syscall that does not block (`getpid`, a `write` that lands in a buffer, a `clock_gettime` that falls through to the kernel), you pay only the mode switch: save/restore of the user register set, the ring transition, the `swaps` pair, and the second-order costs — the pipeline serialization the transition forces, and the cache and TLB pollution from running kernel code in the middle of your user working set. Those second-order effects are why even a "cheap" syscall is not free: you resume in ring 3 with some of your data and translations evicted by the kernel's own footprint.

What the mitigations cost

Before 2018 the raw `getpid` round trip on a modern x86-64 core was on the order of a few tens of nanoseconds. Then Meltdown and Spectre changed the arithmetic, and the change is directly relevant to anyone counting syscalls in a hot path.

- **KPTI (Kernel Page-Table Isolation)**, the Meltdown fix, stopped mapping the full kernel into the user page tables. Now a syscall must switch `CR3` from the user page tables to the kernel page tables on entry and switch back on exit. On CPUs without PCID that means TLB flushes on every crossing; with PCID the cost is smaller but real. Either way, syscalls got a new per-crossing tax they never had before.
- **Spectre v2** mitigations (retpolines, and IBRS/IBPB/STIBP where microcode provides them) slowed indirect branches and added barrier work around privilege transitions.

The net is workload-dependent, but on affected hardware the boundary crossing got meaningfully more expensive — on some microbenchmarks the bare syscall cost roughly doubled or worse. Treat any pre-2018 "syscalls are ~50 ns" figure with suspicion, hedge your own numbers,

and — the operative lesson — recognize that the mitigations *raised the price of the boundary*, which is precisely what pushed the industry toward architectures that cross it far less often.

The cost of syscalls, and why you reduce them

So a syscall is cheap-ish but not free: on the order of hundreds of nanoseconds for the crossing alone on mitigated hardware, plus whatever the handler does, plus the cache/TLB aftershock. In isolation that is nothing. In a hot path it is everything. A service doing four syscalls per request at a million requests per second is doing four million boundary crossings a second, and at a few hundred nanoseconds each that is a meaningful slice of a core spent doing nothing but crossing — before any actual work. This is the recurring theme of the chapter and of high-performance Linux backend engineering generally: **the number of syscalls per unit of work is a first-class performance metric, and most of the I/O toolbox exists to drive it down.**

The techniques form a ladder, from "make each syscall carry more work" to "stop synchronously crossing at all":

Technique	What it reduces	Mechanism	Chapter
Buffered I/O (<code>stdio</code> , <code>bufio</code>)	Number of <code>write / read</code> calls	Accumulate in a user buffer; one syscall per buffer flush, not per record	this chapter
Vectored I/O (<code>readv / writev</code> , <code>preadv / pwritev</code>)	Calls for scattered buffers	One syscall moves many non-contiguous buffers (scatter/gather)	this chapter
<code>sendfile / splice</code>	Copies <i>and</i> calls	Move bytes fd→fd inside the kernel; no user-space bounce buffer (zero-copy)	Ch 7
<code>io_uring</code>	Synchronous crossings	Submit many ops via a shared ring; batch-submit, reap completions; near syscall-free steady state	Ch 7

Technique	What it reduces	Mechanism	Chapter
vDSO	The crossing itself	Service <code>clock_gettime / gettimeofday / getcpu</code> in user space from a kernel-updated page	this chapter
Kernel bypass (DPDK, AF_XDP)	The kernel path entirely	Poll the NIC from user space; no per-packet syscall	Ch 10

Buffering is the oldest and simplest. `fprintf` to a stream does not call `write` per line; it fills an in-process buffer and flushes when full (or on newline for a terminal). One thousand log lines become a handful of `write` syscalls instead of a thousand. The cost is a durability/visibility gap — data sits in the user buffer until flush — which is exactly why you `fflush/fsync` at points that matter and why crash-consistency discussions (Chapter 6) care about where the bytes actually are.

Vectored I/O attacks a different waste. Suppose you are writing an HTTP response: a status line, several header buffers, and a body, each a separate allocation. Naively that is many `write` calls, or an extra copy to concatenate them into one buffer first. `writew` takes an array of `iovec` (pointer, length) pairs and writes them all in one syscall, in order, with no concatenation copy:

```
struct iovec iov[3] = {
    { status_line, status_len },
    { headers,     hdr_len    },
    { body,        body_len   },
};
ssize_t n = writew(fd, iov, 3); /* one syscall, three buffers,
gathered */
```

One boundary crossing, no user-space copy to stitch the pieces together. This is why HTTP servers and RPC frameworks build responses as buffer lists and emit them with a single `writew`. `sendmsg / recvmmsg` generalize the same idea for sockets (and carry ancillary data like `SCM_RIGHTS` fd-passing, Chapter 8).

sendfile and splice go further: they eliminate the user-space copy *and* the extra crossings. Serving a static file the naive way is `read` into a user buffer then `write` it to the socket — two syscalls and two copies

(disk→page cache→user buffer, then user buffer→socket) per chunk. `sendfile(out_sock, in_file, ...)` tells the kernel to move the bytes from the file's page cache to the socket *without ever copying them into user space*, in one syscall. `splice` does the same through a pipe as a general plumbing primitive. This is the "zero-copy" family, and it is Chapter 7's subject; here the point is that it collapses both the copies and the syscall count.

`io_uring` is the endgame and the reason the industry rethought Linux I/O after the mitigation tax. Instead of one syscall per operation, the application and kernel share two ring buffers in memory — a submission queue and a completion queue. The application writes many operation descriptors into the submission ring and, in the best case, the kernel picks them up and posts completions without the application issuing a syscall per op at all (with `IORING_SETUP_SQPOLL`, a kernel thread polls the submission ring, so the steady state can be *zero* syscalls for submission). A server that once did read/write/accept as three syscalls per connection event can drive thousands of operations across a handful of `io_uring_enter` calls — or none. `io_uring` gets its full treatment in Chapter 7; note here only that its entire reason for existing is to amortize and then eliminate the boundary crossing that this chapter has been pricing.

The distributed-systems reading of all this: these are not micro-optimizations, they are fleet-level efficiency levers. A syscall reduction that saves 300 ns per request, multiplied across a service handling millions of requests per second across thousands of machines, is cores — real, budgeted, dollar-denominated capacity. The teams that run the largest I/O-bound fleets (CDNs, proxies, databases, message brokers) obsess over syscalls-per-request precisely because at their scale the boundary crossing is a line item.

libc wrappers, `syscall()`, and the vDSO

Almost no application issues raw `SYSCALL` instructions. It calls `libc` — `glibc` or `musl` — and `libc` wraps the syscall. The wrapper is thin but does three genuinely useful things: it exposes a normal C function signature, it marshals arguments into the register convention and executes `SYSCALL`, and it translates the `-errno` return into the POSIX contract of "return `-1` and set the thread-local `errno`." That is the entire reason `errno` exists: the kernel has no `errno`, it just returns negative numbers in `rax`, and

libc is the layer that turns `-ENOENT` into `errno = ENOENT`. musl and glibc differ in size and behavior (musl is smaller and simpler, glibc has more compatibility machinery and caching), but both play this role.

When you need to invoke a syscall libc does not wrap — a very new one, or one libc deliberately hides — you can reach the raw interface with the `syscall()` function:

```
#include <sys/syscall.h>
#include <unistd.h>

/* gettid had no glibc wrapper for years; you called it raw. */
pid_t tid = syscall(SYS_gettid);
```

`syscall()` still sets `errno` and still goes through the kernel; it just skips the purpose-built wrapper. It is the escape hatch for new kernel features that predate their libc support (io_uring's `io_uring_setup` / `io_uring_enter` were reached this way before liburing).

The vDSO: a "syscall" with no mode switch

Here is the clever part. Some things the kernel exposes as syscalls do not actually require kernel privilege to *compute* — they require only kernel *data*. Reading the wall-clock time is the canonical example. `clock_gettime(CLOCK_MONOTONIC, ...)` needs the current time, which the kernel derives from a hardware clock source and some scaling factors it updates on every tick. But the *reading* of that time is pure arithmetic: take the clock-source counter, apply the kernel's current multiplier and offset, done. None of that needs ring 0. It only needs the kernel's up-to-date scaling values.

So the kernel publishes those values into a page it maps read-only into every process's address space, and it maps alongside them a small shared library — the **vDSO** (virtual dynamic shared object) — containing user-space implementations of `clock_gettime`, `gettimeofday`, `time`, and `getcpu`. When your program calls `clock_gettime`, glibc first tries the vDSO version, which runs entirely in ring 3: it reads the counter (often via a `rdtsc`-based path), reads the kernel's scaling data from the shared page, does the multiply-add, and returns — **no SYSCALL, no mode switch, no boundary crossing at all**. Only if the vDSO path cannot serve the request (an exotic clock, or a clock source that genuinely requires kernel work) does it fall through to a real syscall.

```

flowchart TD
    APP["Application: clock_gettime(CLOCK_MONOTONIC)"]
    APP --> V{"vDSO path in ring 3  
__vdso_clock_gettime"}
    V -->|"fast, common case"| CALC["Read TSC + kernel scaling data  
from vvar page, compute time  
(no ring transition)"]
    CALC --> RET1["Return to caller"]
    V -->|"unsupported clock / fallback"| SC["Real SYSCALL → ring 0"]
    SC --> KH["kernel clock_gettime handler"]
    KH --> RET2["RET"]
    KUP["Kernel updates vvar page  
every tick"] -.->|"read-only mapping"| CALC

```

Why it matters: time is read *constantly*. Every log line with a timestamp, every latency measurement, every timeout check, every span in a distributed trace calls a clock function. In an observability-heavy service that can be one of the hottest calls in the process. Turning it from a ~hundreds-of-nanoseconds boundary crossing into a ~tens-of-nanoseconds arithmetic routine, at that frequency, is a large win — and it is invisible, which is the point. The vDSO is the one place the boundary is optimized by *not crossing it*, using the observation that the privilege wall exists to protect state, and reading a value that is safe to expose does not need protecting. (The vDSO superseded the older, fixed-address `vsyscall` mechanism, which had its own security problems and is now emulated or disabled; the vDSO is ASLR-friendly and is the mechanism you should think about.)

Observing syscalls

Because the syscall boundary is where a process meets the world, watching syscalls tells you exactly what a program is actually asking the kernel to do — which files it opens, which it fails to find, which network calls block, how many `futex` calls a lock-contended run makes. The tools differ enormously in cost, and the difference is instructive.

strace is the classic. It shows every syscall a process makes, with decoded arguments and return values:

```
$ strace -tt -T curl -s https://example.com -o /dev/null
openat(AT_FDCWD, "/etc/ssl/certs/ca-certificates.crt", 0_RDONLY) = 4
<0.000018>
socket(AF_INET, SOCK_STREAM|SOCK_CLOEXEC, IPPROTO_TCP) = 5 <0.000021>
connect(5, {sa_family=AF_INET, sin_port=htons(443), ...}, 16) = -1 EINP
ROGRESS <0.000040>
...
```

`strace` is built on `ptrace`, the same debugging facility a debugger uses. It arranges (via `Ptrace_Syscall`) for the traced process to *stop* on both syscall entry and syscall exit, at which point the kernel wakes the tracer, which reads the stopped process's registers, decodes the call, and lets it continue. That is the mechanism, and it explains the enormous cost: every single syscall in the tracee turns into two stops, two context switches to the tracer and back, and register reads across process boundaries. A program that makes heavy syscall use can run an order of magnitude or more slower under `strace`. This is fine — even ideal — for debugging a single misbehaving process ("why can't it find its config file?", "is it hanging in `connect` or `read`?"), and disastrous if pointed at a busy production process, whose latency it will wreck and whose real behavior it may perturb (a Heisenbug generator).

For production, use the low-overhead tools:

- **perf trace** does the same job as `strace` but is built on the kernel's tracing infrastructure (tracepoints and the perf ring buffer) rather than stop-the-world `ptrace`. It samples/streams syscall events through an in-kernel buffer without stopping the target twice per call, so its overhead is a fraction of `strace`'s. It is the right first reach on a system you cannot afford to slow down.
- **eBPF** tools (`bpftool`, and the `bcc` scripts like `syscount`, `opensnoop`, `execsnoop`) are the production standard. You attach a small verified program to the syscall tracepoints or to specific kernel functions, and it aggregates *in the kernel* — counting syscalls per process, histogramming latencies, snooping every `openat` fleet-wide — emitting only the summary to user space. Because the filtering and aggregation happen in-kernel and there is no per-call stop/copy, overhead is low enough to run continuously in production. eBPF is the whole subject of Chapter 11; for now, the ladder to remember is: **strace to debug one process, perf trace for a lighter look, eBPF to observe the fleet without perturbing it.**

A quick `strace -c` (which counts and times syscalls rather than printing each) is often the fastest way to answer "what is this process spending its kernel time on?":

```
$ strace -c -f ./server
% time      seconds  usecs/call     calls   errors syscall
-----
 41.2      0.512340         5     102400         read
 33.8      0.420110         4     102400         write
 18.1      0.225300        11      20480         futex
...
```

That table — dominated by `read/write`, or by `futex` (lock contention), or by `clock_gettime` (a hint the vDSO is not being used, or a genuinely time-obsessed loop) — is frequently the fastest route to "you are making too many syscalls, and here is which ones."

The syscall as the isolation boundary

Everything so far has treated the boundary as a performance surface. It is equally a *security* surface — arguably *the* security surface of the operating system. A process's entire ability to affect the world outside its own address space is mediated by syscalls: to touch a file, a socket, another process, the clock's *setting*, a device, it must call the kernel. Therefore, if you can control which syscalls a process may make, you can bound what damage it can do. A process that cannot call `ptrace` cannot inspect other processes; one that cannot call the mount or network-configuration syscalls cannot reconfigure the host; one that cannot call `execve` a second time cannot pivot to a new program. **Restricting the syscall set shrinks the attack surface**, and this is the foundation of every serious sandbox on Linux.

seccomp and seccomp-bpf

seccomp (secure computing mode) is the kernel's in-process syscall firewall. Its modern form, **seccomp-bpf** (mode 2), lets a process install a **classic BPF** program that the kernel runs on *every* syscall the process subsequently makes. The filter sees the syscall number and the raw argument registers (and the architecture, which you must check to avoid multiplexing attacks), and returns a verdict:

seccomp return action	Effect
SECCOMP_RET_ALLOW	Let the syscall proceed normally
SECCOMP_RET_ERRNO	Block it; return a chosen errno (e.g., EPERM) without running it
SECCOMP_RET_KILL_PROCESS / KILL_THREAD	Terminate — the process asked for something forbidden
SECCOMP_RET_TRAP	Deliver SIGSYS so the process can handle the denial
SECCOMP_RET_TRACE	Hand off to a ptrace supervisor to decide
SECCOMP_RET_LOG	Allow, but log — used to build profiles by observation

The filter is set once and is **irrevocable and inherited** across `fork` and `execve` (a process cannot loosen its own sandbox, and children stay confined), which is exactly what you want for a security boundary. Crucially, seccomp filters operate on the syscall number and register-level arguments only — they cannot dereference a user pointer to inspect, say, a path string (that would be a TOCTOU hazard, since the memory could change between check and use), so seccomp is good at "may this process call `mount` at all?" and not the right tool for "may it open *this specific* path?" (that is the LSMs' job, below).

```

flowchart TD
    APP["Process issues syscall  
(number in rax, args in registers)"]
    ENTRY["Kernel syscall entry"]
    SF["seccomp-bpf filter  
installed?"]
    DISP["Dispatch to handler"]
    EVAL["Run BPF program on  
syscall nr + args + arch"]
    V["verdict"]
    ALLOW["ALLOW"]
    DENY["Return -EPERM,  
never run handler"]
    KILL["Terminate process"]
    SIG["Deliver SIGSYS"]
    WORK["Privileged work"]

    APP --> ENTRY
    ENTRY --> SF
    SF -->|no| DISP
    SF -->|yes| EVAL
    EVAL --> V
    V -->|ALLOW| DISP
    V -->|ERRNO| DENY
    V -->|KILL| KILL
    V -->|TRAP| SIG
    DISP --> WORK

```

This is not a niche feature. **Docker applies a default seccomp profile to every container it runs** unless you explicitly disable it. That profile allows the ~300-plus syscalls a normal workload needs and *blocks* a few dozen dangerous or rarely-legitimate ones — the ones that manipulate kernel modules, reconfigure the machine, or have been sources of container escapes: `kexec_load`, `init_module/fini_module`, `reboot`, `mount/umount2`, `ptrace` (by default), `bpf`, the old obscure or namespace-manipulation calls, and so on. Blocking them costs a normal containerized service nothing (it never calls them) and removes whole categories of escape and privilege-escalation primitives from a compromised container's reach. That trade — near-zero cost to legitimate workloads, large reduction in what an attacker can reach — is why seccomp is on by default in the container runtimes and why Book 6 (Containers and Kubernetes) treats a tightened seccomp profile as baseline hardening. Kubernetes exposes it through the `seccompProfile` field (with `RuntimeDefault` as the recommended setting, and `localhost` profiles for custom ones).

gVisor: intercept the syscalls in user space

seccomp *filters* the real syscall interface; **gVisor** replaces it. As Volume 1, Chapter 10 described, Google's `runc` runtime interposes a **user-space kernel** (the *Sentry*) between the container and the host. Guest application syscalls are trapped — via `ptrace` or a KVM-based mechanism — and redirected into the Sentry, a re-implementation of a large part of the Linux system-call surface written in Go, running in user space. The Sentry services the guest's syscalls itself and makes only a small, tightly restricted set of *real* syscalls to the host kernel (itself further confined by seccomp). The security argument is defense in depth: the enormous, historically bug-prone Linux syscall surface is no longer directly reachable by untrusted guest code; a guest exploit has to get through the Sentry first, and even then the Sentry's own access to the host is minimal. The cost is compatibility (not every syscall or edge case is implemented) and performance (that interposition is not free — syscall-heavy workloads pay for it). gVisor is the clearest illustration of the chapter's thesis from the security side: **the syscall boundary is the enforcement point, and you can harden a system by controlling, filtering, or wholesale re-implementing what crosses it.**

LSMs: policy on the objects, not just the numbers

seccomp answers "which syscall numbers?" The **Linux Security Modules** framework — SELinux, AppArmor, and others — answers the richer question "may *this* subject perform *this* operation on *this* object?" LSMs work by placing **hooks** at security-relevant points deep inside syscall handlers — after the arguments have been safely copied in and resolved to kernel objects (an inode, a socket, a task). At each hook the active LSM consults its policy (SELinux's type enforcement, AppArmor's per-program path rules) and returns allow or deny. Because the hook fires *inside* the handler with fully resolved kernel objects, an LSM can enforce "this web-server process may read `/etc/nginx` but may not read `/etc/shadow` and may not connect outbound except to the app tier" — the object-level, context-sensitive policy seccomp structurally cannot express. The two compose: seccomp coarsely removes whole syscalls a workload never needs; the LSM finely governs what the permitted syscalls may touch. Together, plus namespaces and cgroups (Chapter 9), they are how the syscall boundary is turned into a real multi-tenant isolation perimeter.

ABI stability: the boundary as a contract

There is one more property of the syscall interface that makes all of the above worth building on: it does not change out from under you. Linus Torvalds' cardinal rule for the Linux kernel is **"we do not break userspace."** A syscall's number, its argument layout, and its documented behavior are a stable contract; a kernel upgrade will not renumber `write` or change what its arguments mean or repurpose an existing flag bit to break existing binaries. A program compiled against the syscall ABI in 2008 still runs on a 2026 kernel. This is why the syscall table only ever *grows* — numbers are assigned and never reused — and why a program can depend on the interface for a decade.

The corollary shapes how the interface evolves. Because you cannot change an existing syscall's meaning, new capabilities arrive as **new syscalls** or as previously-reserved **flag bits**. Where a syscall needs a new behavior, the kernel adds a successor rather than mutating the original: `dup` → `dup2` → `dup3`, `pipe` → `pipe2`, `accept` → `accept4`, `open` → `openat` → `openat2`, `clone` → `clone3`, `epoll_create` → `epoll_create1`. The pattern is almost always "same thing, plus a `flags` argument," and the reserved-must-be-zero flag fields are how *future* extensions get room without breaking today's callers: a program that passes `0` today keeps

working when a new flag is defined tomorrow, and a program that passes the new flag on an old kernel gets a clean `EINVAL` rather than silent misbehavior. That discipline — extend, never mutate; grow the table, never renumber; reserve flag space for the future — is what lets the whole ecosystem (every libc, every language runtime, every container runtime, every static binary in a scratch image) treat the syscall boundary as bedrock.

Distributed-systems lens

Pull the threads together at fleet scale, and the syscall boundary shows up as two levers and one diagnostic.

Efficiency lever. The syscall is the per-request unit of kernel interaction, and for I/O-bound services it is the cost that multiplies. A proxy, an API gateway, a database, a cache, a message broker — their throughput ceiling and tail latency are shaped substantially by how many times they cross into ring 0 per request and how much each crossing carries. Reducing syscalls-per-request via buffering, `writew`, `sendfile/splice`, and above all `io_uring` (Chapter 7) is not micro-optimization at this scale; a few hundred nanoseconds saved per crossing, across millions of requests per second across thousands of hosts, is measured in cores and in the capacity budget. The 2018 mitigation tax, which raised the price of every crossing, is a large part of *why* `io_uring` and kernel-bypass frameworks (DPDK, AF_XDP — Chapter 10) moved from exotic to mainstream: when the boundary got more expensive, the value of not crossing it went up. And the vDSO is the quiet everyday version — timestamping is so frequent in an observable service that servicing the clock in user space, invisibly, is a real fleet-wide win.

Isolation lever. For multi-tenant infrastructure — many workloads from many teams packed onto shared hosts — the syscall boundary is *the* enforcement point. seccomp profiles shrink each tenant's reachable syscall set (Docker's default profile fleet-wide, tightened further per workload); LSMs govern what the permitted syscalls may touch; gVisor re-implements the surface in user space for untrusted code. Every one of these is a statement about the syscall boundary: control what crosses it, and you bound the blast radius of a compromised workload across the whole fleet (Book 6). The container escapes that matter are, almost by definition, syscalls that should have been filtered and were not.

Diagnostic. When a service misbehaves in ways that do not show up in application metrics — it is slow, or chatty, or blocking somewhere unexpected — the syscall stream is ground truth for what it is actually doing to the kernel. `strace -c` on one instance to see the distribution; `perf trace` for a lighter look; and eBPF (`syscount` , `bpfftrace` , Chapter 11) to answer the same question continuously across the fleet without perturbing anything. "Which service is hammering the kernel, with what, and how much does each call cost?" is a question you answer at the syscall boundary, and at scale you answer it with eBPF.

Key takeaways

- User code cannot perform privileged operations; the **syscall is the single controlled gate** into ring 0, and the kernel — not the caller — chooses where execution lands and what checks run.
- On x86-64 the convention is fixed: **number in `rax` ; args in `rdi` , `rsi` , `rdx` , `r10` , `r8` , `r9` ; result (or `-errno`) in `rax` .** `SYSCALL / SYSRET` is the fast path (`LSTAR` entry, no stack switch by the instruction), superseding legacy `int 0x80` ; the kernel switches to the kernel stack, dispatches through `sys_call_table[nr]` , and returns.
- A syscall is a **mode switch, not a context switch** — same task, no scheduler, far cheaper — but it is not free: register save/restore, the ring transition, cache/TLB aftershock, and since 2018 the **KPTI/ Spectre mitigation tax** that roughly doubled the bare crossing on affected CPUs.
- **Syscalls-per-request is a first-class performance metric.** Drive it down with buffering, vectored I/O (`readv / writev`), zero-copy (`sendfile / splice` , Chapter 7), and batched async (`io_uring` , Chapter 7); the **vDSO** optimizes time-reads by not crossing at all.
- **libc wraps syscalls** (argument marshaling, `errno` from the `-errno` return, portability); `syscall()` is the raw escape hatch for calls libc does not wrap.
- **Observe** with `strace` (ptrace-based, two stops per call — great for one process, ruinous in production), `perf trace` (lighter, tracepoint-based), and **eBPF** (in-kernel aggregation, the fleet-scale tool — Chapter 11).
- The syscall boundary is **the isolation enforcement point:** **seccomp-bpf** filters which syscalls a process may make (Docker's

default profile, Kubernetes `seccompProfile`), **gVisor** re-implements the surface in user space, and **LSMs** (SELinux/AppArmor) hook inside handlers to govern which objects the permitted syscalls may touch (Book 6, Volume 1 Chapter 10).

- The interface is a **stable contract** — "don't break userspace." The table only grows; new behavior arrives as new syscalls (`openat2` , `clone3` , `accept4`) or reserved flag bits, never as mutation of an existing call.

Further reading

- **Linux man-pages:** `syscall(2)` (the per-architecture calling conventions, including the x86-64 register table), `syscalls(2)` (the full list), `vdso(7)` (what the vDSO exports and how it is mapped), `seccomp(2)` and `seccomp_unotify(2)` , `ptrace(2)` , and `writew(2)` / `sendfile(2)` . The canonical, version-tracked reference for everything in this chapter.
- **The Linux kernel source:** `arch/x86/entry/entry_64.S` (the `SYSCALL` entry path, `swaps` , stack switch), `arch/x86/entry/syscalls/syscall_64.tbl` (the number-to-handler table), and `kernel/seccomp.c` . Reading the entry assembly once demystifies the whole crossing.
- **Intel 64 and IA-32 Architectures Software Developer's Manual**, Volume 2 — the `SYSCALL` and `SYSRET` instruction descriptions and the `LSTAR` / `STAR` / `SFMASK` MSR semantics. The AMD64 Architecture Programmer's Manual, Volume 2, covers the same (AMD introduced `SYSCALL`).
- Jonathan Corbet et al., **LWN.io** — the running record of syscall and mitigation evolution: the KPTI and Spectre/Meltdown coverage (early 2018) for the mitigation-cost story, and the `io_uring` series for the batched-async design. Primary-source-quality kernel journalism.
- **gVisor documentation** (`gvisor.dev`), "What is gVisor?" and the security model / platform pages — the authoritative description of the Sentry, syscall interception, and the host restriction argument. Pairs with Volume 1, Chapter 10.
- **Docker default seccomp profile** (`default.json` in `moby/moby`) and the Docker "Seccomp security profiles" documentation — the concrete list of syscalls blocked by default and the rationale, plus the

Kubernetes "Restrict a Container's Syscalls with seccomp" task guide.

- **KPTI / Kernel Page-Table Isolation** documentation under `Documentation/x86/` and the original Meltdown (Lipp et al., 2018) and Spectre (Kocher et al., 2018) papers — for the mechanism behind the syscall mitigation tax.
- Brendan Gregg, **Systems Performance**, 2nd ed., Addison-Wesley, 2020, and his BPF work — the practical syscall-observation toolkit (`strace` vs `perf trace` vs `bpfttrace/bcc`) that Chapter 11 builds on.
- The `liburing` project and Jens Axboe's "**Efficient IO with io_uring**" design document — the primary source on the shared-ring, batch-submit model that is Chapter 7's centerpiece and the ultimate answer to syscall overhead.

Chapter 6 — File Systems and the VFS

What this chapter covers. Chapter 4 explained the page cache — the RAM layer that makes file I/O fast and makes your memory dashboards lie. Volume 1 Chapter 5 explained the media underneath — platters, NAND, NVMe, and the physics of what it takes to make a bit *stick*. This chapter fills the gap between them: the **file system**, the kernel subsystem that turns a linear array of disk blocks into named files, directories, and metadata, and — crucially — that decides what "your write is safe" actually means. Every stateful thing a backend engineer ships lives on a file system: the Postgres data directory, the Kafka segment, the RocksDB SST, the etcd WAL, the container image layer. When one of those systems claims a transaction is committed, the claim ultimately rests on a chain of assumptions about the file system beneath it — assumptions that, when wrong, produce silent data loss that survives replication and breaks consensus.

We proceed in three movements. First, the **VFS abstraction**: the object model (superblock, inode, dentry, file) that lets `open / read / write` work identically across ext4, XFS, Btrfs, and NFS, plus path lookup and the dentry cache. Second, the **on-disk reality**: how ext4, XFS, Btrfs/ZFS, and tmpfs lay out blocks, and how each solves — or sidesteps — the **crash-**

consistency problem via journaling or copy-on-write. Third, and most important for backend correctness, the **durability write path**: the layers a `write()` traverses before it is truly safe, why `fsync` is the only thing that closes the gap, the subtle directory-`fsync` bug, and the 2018 Postgres *fsyncgate* incident that taught the database industry that even `fsync` can lie.

This chapter is Linux-specific throughout, and the durability material is load-bearing for Volume 5 (Databases) and Volume 6 (Consensus) — a distributed system's "committed" is only as durable as the local `fsync` underneath it.

Learning goals — after this chapter you should be able to:

- Explain the **VFS object model** — superblock, inode, dentry, file — and how the layered indirection from a file descriptor down to on-disk blocks enables one syscall API over many file systems.
- Describe **path resolution** and the **dentry cache**, and why name lookup is cached separately from inode data.
- State precisely the **inode-versus-name model**: why a file's name lives in its directory, not its inode; how hardlinks, link counts, `unlink`, and open file descriptors interact; and why "I deleted the file but the disk is still full" happens.
- Compare **ext4**, **XFS**, **Btrfs/ZFS**, and **tmpfs** on-disk designs — extents, allocation groups, copy-on-write, checksums — and pick the right one for a database or log workload.
- Explain the **crash-consistency problem** and the two families of solutions: **journaling** (ext4 `data=ordered / journal / writeback`) and **copy-on-write** (Btrfs/ZFS), plus what `fsck` is for and why journaling mostly retired it.
- Trace the **durability write path** end to end and state exactly where data becomes safe: `write()` → page cache → writeback → block layer → device cache → media, with `fsync` / `fdatsync` / FUA as the barriers.
- Explain **fsyncgate (Postgres, 2018)** accurately, the directory-`fsync` requirement, and `O_DIRECT`, and connect all of it to why different file systems give different crash guarantees ("All File Systems Are Not Created Equal").
- Reason about the **breadth topics** — file locking, extended attributes, FD/open-file-table semantics, `/proc` and `/sys`, FUSE, and NFS

close-to-open consistency — well enough to avoid their classic footguns.

The VFS: one API, many file systems

A backend service opens a config file on ext4, reads a log segment on XFS, and stats a value in `/proc`, all with the same four syscalls: `open`, `read`, `write`, `close`. It never learns, and never needs to learn, which file system backs each path. That uniformity is not an accident of convention — it is enforced by a kernel layer called the **Virtual File System** (VFS), sometimes the Virtual Filesystem Switch.

The VFS is indirection in the classic sense: it defines *abstract objects* and *operation tables* (structs of function pointers), and each concrete file system fills in those pointers. When your process calls `read()`, the VFS resolves the descriptor to a generic `struct file` and calls `file->f_op->read_iter()` — a pointer ext4 set to `ext4_file_read_iter`, XFS set to something else, and a FUSE mount set to a stub that forwards to a userspace daemon. The syscall layer, the page cache, and your application see one interface; the diversity lives below the function-pointer boundary. This is the *pluggable storage engine* pattern you know from databases (InnoDB vs MyISAM behind one query API) — the same discipline, one level down.

Four object types carry the model.

VFS object	Represents	Lifetime / scope	Key contents
superblock	A <i>mounted</i> file system instance	One per mount	Block size, total/free blocks, the FS type's op tables, dirty inode list, root dentry
inode	A <i>file</i> (its metadata + data location) — not its name	One per file, shared across all names/opens	Size, mode/permissions, owner, timestamps, link count, pointers to data blocks/extents
dentry	A <i>directory entry</i> : one name→inode mapping	Cached in RAM (the dcache); the on-disk directory is authoritative	Name component, pointer to its inode, parent dentry

VFS object	Represents	Lifetime / scope	Key contents
file	An <i>open instance</i> of a file	One per <code>open()</code> (roughly)	Current offset, open flags/mode, a pointer to the inode (via dentry), the op table

The separation is the whole point, so state each boundary precisely.

- A **superblock** is a *mounted file system*, not a file. `mount /dev/nvme0n1p2 /data` creates one; it holds everything global to that mount — block size, free-space accounting, and the FS type's operation tables.
- An **inode** is a *file's identity and metadata*: size, permissions, ownership, the three timestamps (atime/mtime/ctime), the **link count**, and the map from file offsets to physical blocks (a block list, or extents in modern file systems). It is identified by an inode number, unique within its file system. **The inode does not contain the file's name.**
- A **dentry** (directory entry) is the name — one path component (`"passwd"`) mapped to an inode. Directories are, on disk, essentially tables of dentries. The RAM dentry object caches that mapping and points at its parent dentry, which is how the kernel reconstructs a path.
- A **file** object is one *open instance*: it holds the **offset** (so two independent `open()` s have independent positions) and the open flags, and points through its dentry at the inode.

```

flowchart TB
    subgraph proc["Process A"]
        FD0["fd 3"]
        FD1["fd 4"]
    end
    subgraph procB["Process B"]
        FDB["fd 5"]
    end

    FD0 --> F1["struct file  
offset=1024, 0_RDONLY"]
    FD1 --> F2["struct file  
offset=0, 0_WRONLY"]
    FDB --> F3["struct file  
offset=4096, 0_RDWR"]

    F1 --> D1["dentry '/etc/passwd'"]
    F2 --> D1
    F3 --> D1

    D1 --> IN["inode #8231  
size, perms, link_count=1  
extents -> blocks"]
    IN --> SB["superblock (ext4 on /dev/nvme0n1p2)"]
    IN --> BLK["data blocks / extents on media"]

```

Read the diagram from the top: a **file descriptor** is a small integer into the process's private FD table, pointing to a `struct file`. Multiple descriptors — same process or different — can point at the *same* file object (what `dup()` and `fork()` produce; more below) or at *different* file objects resolving to the *same inode* (two independent `open()` s). Every path funnels into one inode per file, and every inode belongs to one superblock. This layering — **fd** → **file** → **dentry** → **inode** → **superblock** — is the mechanical answer to how Linux mounts a dozen file systems at once and lets one program walk across all of them: each layer is generic, only the leaf operations are FS-specific.

Path resolution and the dentry cache

Turning `/var/lib/postgresql/16/main/base` into an inode is *path resolution* (the kernel calls it *pathwalk*, implemented in `fs/namei.c`). It is a component-by-component walk: start at the root dentry (or the process's current-directory dentry for a relative path), look up `var` in the root directory to get its inode and dentry, look up `lib` in *that* directory, and so on. Each step involves a permission check (execute bit on the

directory) and, on a naive implementation, a disk read of the directory's contents.

Doing that from disk on every `open()` would be ruinous, so Linux caches the results in the **dentry cache** (dcache): a hash table of recently used name→inode mappings, keyed by (parent dentry, name). A cache hit turns a directory lookup into a hash lookup with no I/O. The dcache is why re-opening a hot path is nearly free, and why a `find` over a cold tree is slow the first time and fast the second. Negative dentries — cached *non-existence* of a name — are also kept, so repeatedly `stat`-ing a file that does not exist (a real pattern: language runtimes probing library search paths) does not repeatedly hit disk.

Two consequences matter operationally. First, the dcache and the **inode cache** are distinct caches: a dentry can be evicted while its inode stays hot, or vice versa; they are reclaimed under memory pressure as "slab" memory, which is why `slabtop` shows `dentry` and `inode_cache` as large consumers on a box that has walked many paths. Second, symlink resolution happens *during* pathwalk: when the walk hits a symlink dentry, the kernel reads the link target and restarts the walk from there, with a recursion limit (40 links) to bound loops. That mid-walk restart is why a symlink can point across file systems and why a symlink loop returns `ELOOP` rather than hanging.

Files and inodes: the name is not the file

The single most consequential fact about Unix file systems — the one that explains a whole cluster of production surprises — is that **a file and its name are different objects with different lifetimes**. The inode *is* the file: its metadata and its data blocks. The name is a directory entry pointing at that inode. The link between them is counted.

Hardlinks and the link count

Every inode carries a **link count** (`st_nlink`): the number of directory entries pointing at it. A freshly created regular file has link count 1 (one name). A **hardlink** — `ln existing newname` — creates a *second* directory entry pointing at the *same inode* and bumps the count to 2. There is no "original" and "copy"; both names are equal, first-class references to one inode with one set of data blocks. Editing through one name changes the bytes seen through the other, because there is only one file.

```

flowchart LR
    N1["dir entry '/data/a.log'"] --> IN["inode #90210  
link_count = 2  
open_count = 1  
data blocks"]
    N2["dir entry '/backup/a.log'"] --> IN
    OF["open fd (a process)"] -->|holds a reference| IN
    IN --> DATA["data blocks (freed only when  
link_count==0 AND open_count==0)"]

```

`unlink()` — the syscall behind `rm` — does **not** delete a file. It removes one *directory entry* and decrements the link count. The inode and its data blocks are freed only when the link count reaches **zero** *and* no process still holds the file open. Concretely, the kernel frees a file's storage only when **link count == 0 AND open count == 0**.

The "deleted file, disk still full" gotcha

This is where the model bites in production. Suppose a long-running service has `/var/log/app.log` open for writing, and an operator runs `rm /var/log/app.log` (or a broken `logrotate` does). The directory entry is gone — `ls` shows nothing, the name is free to reuse — but the *inode's link count is now 0 while the open count is still 1*. The file therefore still exists, invisibly, and the process keeps writing to it. Its blocks are **not** reclaimed. `du` on the directory tree shows nothing; `df` shows the disk filling up; the two disagree and juniors lose an afternoon. The disk frees only when the process closes the descriptor (or exits). The forensic move is `ls -l | grep deleted` (or `ls -l /proc/<pid>/fd`), which lists the still-open, already-unlinked inodes and points at the offending process. The fix is to restart or signal the process, or truncate through `/proc/<pid>/fd/<n>` — not to hunt for a file name that no longer exists.

This mechanism is a *feature* almost as often as a footgun. Creating a temp file and immediately `unlink`-ing it while keeping the FD gives you private scratch storage that vanishes even on crash — the anonymous-temp-file idiom, the spirit of `O_TMPFILE`. Atomic config replacement uses the same invariants: write new content to a temp file, `fsync` it, then `rename()` it over the old name. `rename` is atomic within a file system, and any reader holding the old inode open keeps reading consistent bytes until it reopens; there is never a moment where the name points at a half-written file.

Symlinks versus hardlinks

A **symbolic link** is a different animal: it is its own inode whose data is a *path string*. Resolving it means re-running pathwalk on that string. Because it stores a path, not an inode reference, a symlink can point across file systems and can point at something that does not exist (a dangling link) — neither of which a hardlink can do. A hardlink shares an inode, so it is confined to a single file system (inode numbers are only unique within one), cannot target a directory (that would create cycles the tree model forbids), and keeps the data alive by the link count. Symlinks are fragile references to a *name*; hardlinks are robust references to a *file*.

On-disk structures: how the major file systems lay out bytes

Above the VFS boundary the API is uniform; below it, file systems make very different bets about how to arrange blocks on media, and those bets show up as real performance and durability differences. The four you will actually meet on backend infrastructure:

File system	Allocation model	Crash consistency	Standout traits	Typical backend use
ext4	Extents (block-mapped legacy)	Metadata journaling (JBD2); optional data journaling	Mature, predictable, default on many distros	General-purpose root/data; safe default
XFS	Extents + B+ trees, allocation groups	Metadata journaling	High parallelism, huge files/FS, excellent large sequential I/O	Databases, big storage, RHEL default
Btrfs / ZFS	Copy-on-write , extents	CoW (no overwrite in place) + checksums	Snapshots, data+metadata checksums, integrated volume management	Snapshotting fleets, integrity-critical storage

File system	Allocation model	Crash consistency	Standout traits	Typical backend use
tmpfs	Page cache pages (RAM/swap)	N/A (volatile)	No persistence, no fsync cost	/dev/shm , scratch, secrets in memory

ext4: block groups, extents, journaling

ext4 divides the device into **block groups**, each a self-contained region with its own inode table, block/inode bitmaps, and data blocks. Keeping a file's inode and data in the same block group improves locality (short seeks on HDD; better locality of reference generally). Inodes are pre-allocated in fixed tables at format time — which is why an ext4 file system can run out of *inodes* while it still has free *blocks* (`df -i` vs `df`), a real failure mode on volumes holding millions of tiny files.

The important modern change is **extents**. Classic ext2/ext3 mapped a file's logical blocks through a tree of indirect block pointers — fine for small files, wasteful for large ones (a 1 GiB file needed hundreds of thousands of pointer entries). An **extent** is a single record saying "logical blocks 0..32767 live at physical block X, contiguously." A large contiguous file needs a handful of extents instead of a huge pointer list, shrinking metadata and making sequential I/O map cleanly to sequential media access. This is the same insight Volume 1 Chapter 5 drove home: contiguous, sequential access is what storage hardware — spinning or flash — rewards.

ext4 defends contiguity with **delayed allocation**: dirtied data sits in the page cache without a physical block assignment until writeback, so the allocator can see the whole write and place it as one large extent rather than dribbling out blocks per `write()`. This reduces fragmentation but has a durability subtlety we return to: unallocated data has more latitude to be lost or reordered around a crash unless you `fsync`.

XFS: allocation groups and scalability

XFS, originally from SGI and now the RHEL default, is built for parallelism and size. It partitions the device into several **allocation groups (AGs)**, each with its own free-space and inode B+ trees, and each independently lockable. Two threads allocating in two different AGs

do not contend, so XFS scales allocation throughput with cores in a way a single-locked allocator cannot — which is exactly why it is favored for busy databases and large storage arrays. Everything internal is a **B+tree** (free space indexed by both offset and size, inodes, directories), giving logarithmic operations at very large scale. XFS allocates inodes dynamically rather than in fixed tables, so it does not hit ext4's fixed-inode-count wall. Its journal covers metadata only; XFS never offered full data journaling, betting instead on delayed allocation plus a metadata-only journal for speed. For large-file, high-concurrency, sequential-heavy workloads — logs, database data files — XFS is frequently the right default.

Btrfs and ZFS: copy-on-write, checksums, snapshots

Btrfs (Linux) and ZFS (from Solaris, widely used via OpenZFS) take a fundamentally different stance: they are **copy-on-write (CoW)** file systems that **never overwrite live data in place**. Modifying a block writes a *new* block elsewhere, then updates the pointer to it, then updates that pointer's parent, up a tree of block pointers to a single root ("uberblock" in ZFS, the tree root in Btrfs). The old blocks remain untouched until the new tree is fully written and the root pointer is atomically switched. Two enormous properties fall out of this:

- **Cheap snapshots.** A snapshot is just a retained old root pointer; the shared blocks are not copied. Snapshot-heavy workflows (backups, dev/test clones, container base layers) become nearly free.
- **Integrity checksums.** Because every block is written fresh with a checksum stored in its parent pointer, ZFS and Btrfs can detect *silent data corruption* (bit rot, misdirected writes, a lying drive) on read, and with redundancy can self-heal from a good copy. This is end-to-end integrity the overwrite-in-place file systems cannot offer.

The trade-offs are real. CoW causes **fragmentation** under random-overwrite workloads — a database rewriting B-tree pages in place turns each overwrite into a new-block allocation elsewhere, scattering a once-contiguous file. This is why Postgres/MySQL on Btrfs often means disabling CoW for the data directory (`chattr +C`), and why ZFS DB tuning centers on matching `recordsize` to the DB page size. CoW also amplifies writes (one block change dirties the whole pointer chain to the root). ZFS is a mature integrated storage-plus-volume-manager; Btrfs's single-device and RAID-1 modes are solid while its parity-RAID modes

have carried historical caveats. Choose CoW when snapshots and integrity matter more than in-place random-write speed.

tmpfs: a file system backed by RAM

tmpfs is a file system whose "storage" is the page cache itself, backed by RAM and swap. Files on a tmpfs mount (`/dev/shm` , and often `/tmp` and `/run`) live in memory; they are fast, they never touch a disk, and they vanish on reboot. `fsync` on tmpfs is a no-op because there is no durable medium to flush to — which is the point when you want a scratch area or an IPC shared-memory region, and a hazard if you mistake it for durable storage. tmpfs respects a size limit and counts against memory, so a runaway writer to `/dev/shm` can pressure the OOM killer (Chapter 4) exactly as anonymous memory would.

Crash consistency: journaling and copy-on-write

Here is the problem every persistent file system must solve. A single logical operation — create a file, append a block, rename — typically requires **several** separate on-disk updates that must all happen or none happen. Appending a block means: allocate it (update the free-space bitmap), point the inode at it, and update the inode's size and mtime. Lose power *between* those writes and the file system is left **inconsistent**: a block marked allocated that no inode references (a leak), or an inode pointing at a block still marked free (which the allocator may hand to another file — silent cross-file corruption). Media and the block layer guarantee neither the *order* in which independent writes reach the platter nor atomicity across sectors, so the file system must impose that guarantee itself.

Historically the answer was **fsck** (file system check): after an unclean shutdown, scan the *entire* file system at boot, cross-check bitmaps against inodes, and repair discrepancies. This works but does not scale — a full `fsck` of a multi-terabyte volume can take many minutes to hours, an unacceptable recovery time for a server. Two modern approaches make the common-case crash recover in seconds instead.

Journaling (ext4, XFS)

A **journal** (a write-ahead log for the file system's own metadata) turns a multi-write update into an atomic one. Before touching the real on-disk

structures, the file system writes a description of the intended change to a dedicated journal area and marks it committed. Only then does it apply the change to the actual inode/bitmap/directory locations (the "checkpoint"). The atomicity comes from a **commit record**: the change is durable if and only if its commit record made it to the journal.

```
sequenceDiagram
    participant FS as File system
    participant J as Journal (on disk)
    participant M as Main FS structures
    FS->>J: 1. write intended changes (transaction)
    FS->>J: 2. write COMMIT record (barrier before/after)
    Note over J: transaction now durable & atomic
    FS->>M: 3. checkpoint: apply changes in place
    Note over FS,M: --- CRASH before step 3 completes ---
    FS->>J: On reboot: scan journal
    alt commit record present
        FS->>M: REPLAY the transaction (redo)
    else commit record absent
        FS->>FS: DISCARD partial transaction (as if it never happened)
    end
```

On recovery the file system reads only the journal, not the whole volume: replay every committed transaction (idempotently — hence *redo*), discard any transaction lacking a commit record. Recovery is proportional to the journal size, not the file system size — seconds, not hours. That is why a modern Linux box boots cleanly after a power cut without a long `fsck` (though `fsck` still exists as a last-resort repair for corruption the journal cannot cover).

ext4 (via the JBD2 journaling layer) offers three modes, and the distinction is a genuine durability/performance lever:

- **data=ordered** (the default): journals **metadata only**, but *forces the associated data blocks to be written to their final location before the metadata transaction commits*. The ordering guarantee means you never see a metadata pointer to a block that contains stale garbage — after a crash, a file either has its old contents or its new contents at a given offset, never someone else's freed data. This is the sane default.
- **data=journal** : journals **both data and metadata** — every data block is written twice (once to the journal, once to its final location). Strongest ordering, safest, and the slowest, because of the double write. Rarely used except where its specific guarantees are required.

- **data=writeback** : journals metadata only and does **not** order data against it. Fastest, and the dangerous one: after a crash a file's metadata (new size) can be updated while its data blocks were never written, so a freshly extended file can expose **stale on-disk contents** — potentially another file's deleted data. Avoid for anything sensitive.

The crucial thing to internalize: journaling protects the **file system's own structural integrity** so it never becomes unmountable garbage. It does **not**, by itself, guarantee *your application's* data is durable at any particular instant. That is a separate promise you must extract with `fsync`, discussed next.

Copy-on-write as the alternative

CoW file systems (Btrfs, ZFS) get crash consistency almost for free, without a separate journal for data, precisely because they never overwrite live data. A transaction builds an entirely new tree of blocks off to the side; the only in-place mutation is the final, single-sector **atomic switch of the root pointer**. Before the switch, the on-disk state is entirely the old, consistent tree; after, it is entirely the new one. A crash mid-transaction simply leaves the old root in place, and the half-written new blocks are unreferenced garbage to be reclaimed. There is no window of structural inconsistency to repair, which is why ZFS has no `fsck` at all — the design makes the inconsistent state unrepresentable. The cost is the fragmentation and write-amplification discussed earlier; the benefit is a simpler, arguably stronger consistency story plus the checksums and snapshots that ride along on the same mechanism.

The durability write path: where "written" becomes "safe"

This is the section that matters most for anyone building a database, a queue, a log, or a consensus system. The central, expensive truth: **`write()` returning success does not mean your data is on stable media**. It means your bytes were copied into the kernel's page cache (Chapter 4) and the syscall returned. If the power fails one microsecond later, those bytes are gone, and the file system's journal will not save them — journaling protects *its* metadata, not *your* un-flushed data. Closing the gap is your job, and it has a specific cost.

Follow a byte from `write()` to permanence:

```
flowchart TB
    APP["app: write(fd, buf, n) returns"] --> PC["Page cache (dirty page)"]
    PC --> BL["Block layer + I/O scheduler"]
    BL --> DC["Device write cache (on the drive)"]
    DC --> MEDIA["Media: platter / NAND"]
    MEDIA --> FS["fsync(fd) / fdatasync(fd)"]
    FS --> FORCES["forces page cache to media"]
    FORCES --> PC
    style PC fill:#fdd
    style BL fill:#fdd
    style DC fill:#fdd
    style MEDIA fill:#dfd
```

Trace each hop and note exactly where durability is *not* yet achieved:

1. **write() → page cache.** Your bytes become *dirty pages* in RAM. Fast, and volatile. Nothing has been persisted. The kernel will eventually flush them via **writeback**, driven by the dirty-page thresholds (`vm.dirty_ratio` / `dirty_background_ratio`) and a periodic timer (`dirty_expire_centisecs` , ~30 s) — Chapter 4's machinery. "Eventually" is the enemy of durability.
2. **Page cache → block layer.** Writeback hands dirty pages to the block layer and I/O scheduler, which orders and merges requests. Still in transit, still not durable.
3. **Block layer → device cache.** The drive accepts the write into its own **volatile write cache** (DRAM on the controller) and, by default, acknowledges *immediately* — before the bits reach the platter or flash. Fast, and a lie about durability, which is the whole reason the next step exists.
4. **Device cache → media, via FLUSH / FUA.** To force the drive to move data from its volatile cache to non-volatile media, the kernel issues a **cache-flush** command, or tags the write **FUA** (Force Unit Access — "do not acknowledge until this specific write is on media"). These are the *write barriers* of Volume 1 Chapter 5. Only after a successful flush/FUA is the data genuinely safe against power loss

(absent power-loss-protected drive caps, which enterprise SSDs have and consumer drives usually do not).

`fsync(fd)` is the syscall that drives this whole chain to completion: it writes back all of that file's dirty pages, waits for them, and issues the cache flush, returning only when the data (and the metadata needed to find it) is on media. That is why `fsync` is *slow* — it converts an asynchronous, batched, cache-friendly operation into a synchronous round trip to physical media, and it is the single dominant cost in the commit path of every durable data system. Every design you know for amortizing it — **group commit**, batching many transactions into one `fsync`; the WAL's sequential-append shape (Volume 5) — exists to spread one expensive flush across many logical commits.

`fsync` **versus** `fdatasync`

`fsync` flushes the file's data **and** all metadata needed to retrieve it — including metadata that changed but is not strictly required for the data to be readable, like the modification timestamp. `fdatasync` is the leaner variant: it flushes the data and *only* the metadata **required to read it back** (notably the file size, if the write extended the file), and skips inessential metadata updates like timestamps. When appending to a preallocated WAL — where the file size is not changing — `fdatasync` can avoid an extra metadata write and inode flush per commit, which is a measurable win at high commit rates. Databases that care about commit latency (Postgres, MySQL) let you choose the sync method precisely for this reason.

Primitive	Data to media?	Metadata to media?	Bypasses page cache?	Typical use
<code>write()</code>	No (page cache only)	No	No	Ordinary I/O; durability deferred
<code>fsync(fd)</code>	Yes	Yes (all)	No	Full durability incl. timestamps

Primitive	Data to media?	Metadata to media?	Bypasses page cache?	Typical use
<code>fdatasync(fd)</code>	Yes	Only size/essential	No	WAL append; skip inessential metadata
<code>O_DIRECT</code> write	To device cache	—	Yes (skips page cache)	DB managing its own cache/durability
FUA / FLUSH	— (mechanism)	—	—	The barrier <code>fsync</code> uses under the hood

The directory-fsync requirement (a classic subtle bug)

Here is the footgun that has bitten nearly every storage system at least once. You create a new file, write to it, and `fsync` the file descriptor. The **file's data and inode** are now durable — but the **directory entry** that *names* the file might not be. Recall the inode/name split: the file's presence in its parent directory is a separate on-disk structure (the directory's own data blocks and dentries), owned by the *directory's* inode, not the file's. `fsync` on the file does not flush the *directory*. If the machine crashes after the file's `fsync` but before the directory's own writeback, you can reboot to find a durable inode that **no name points to** — an orphaned file, effectively lost, or a rename whose new name is gone.

The correct pattern for durably creating a file is therefore:

```
int fd = open("/data/wal/000123.log", O_WRONLY | O_CREAT, 0644);
write(fd, buf, n);
fsync(fd);                      /* file data + inode durable */
close(fd);
int dfd = open("/data/wal", O_RDONLY | O_DIRECTORY);
fsync(dfd);                     /* directory entry durable -> name now
survives crash */
close(dfd);
```

The same directory `fsync` is required after a `rename()` if you need the `rename` itself to survive a crash. The atomic-rename-for-config-replacement idiom is only *actually* crash-safe when you `fsync` the file, `rename`, then `fsync` the containing directory. This is easy to forget precisely because it works fine every time you test it — until a real power loss lands in the millisecond window.

fsyncgate: when `fsync` itself lies (Postgres, 2018)

The durability model above assumes `fsync` reliably *reports* failure. In 2018 the Postgres community discovered — the episode is remembered as "**fsyncgate**" — that on Linux this assumption was false in a way that could silently lose committed data. The mechanism, described accurately:

When writeback of a dirty page fails (a transient I/O error, a thin-provisioned volume that filled up, a disconnected network block device), the Linux kernel would mark the error on the page, but in the process of handling it, **clear the page's dirty flag** and, in versions of the era, drop the error state after reporting it *once*. Postgres's architecture separates the process that writes dirty pages from the process (the checkpointer) that later calls `fsync` to flush them. So the sequence could be:

1. A backend writes a dirty page; asynchronous writeback later *fails*, and the kernel records the error and clears the dirty bit — the page is now clean-but-not-persisted, and the error is latched to be reported to *the next* `fsync` caller.
2. Some process's `fsync` consumes that error (getting `EIO`) and the kernel then *clears* the error.
3. The checkpointer later calls `fsync` on the same file, gets **success** (the error was already consumed, the dirty bit already cleared), and concludes the data is safe.
4. Postgres advances its checkpoint, discarding the WAL records that could have recovered the lost page. The data is **gone**, and the system believes it committed.

The failure is doubly bad: the error could be delivered to a process that was not the one responsible for durability, and — critically — a failed `fsync` that is seen cannot be retried, because the kernel had already discarded the dirty page contents. Nothing was left to write again; retrying `fsync` after `EIO` returned success while the data stayed lost.

This affected not just Postgres but MySQL, MongoDB (WiredTiger), and others trusting Linux `fsync` error semantics.

Fixes came from both sides. The kernel improved error reporting so errors latch per open file description and reach every `fsync` caller since the file was opened. Postgres changed policy to **treat an `fsync` failure as unrecoverable and PANIC** (crash and recover from the WAL) rather than retry — because after such a failure the page-cache state is untrustworthy and WAL replay is the only path back to a known-good state.

The distributed-systems lesson is the load-bearing one for the rest of this suite: a database's "committed" and a consensus protocol's "durable log entry" (Volume 6) are *defined* in terms of a successful local persist. If the local persist can silently fail, the entire correctness argument built on top of it — replication that trusts each replica's durable state, a Raft leader that has "committed" an entry because a quorum reported it fsynced — rests on a false premise. `fsyncgate` was not a performance bug; it was a **distributed-correctness bug rooted in local file-system semantics**. It is why serious storage engineers treat `fsync` error handling as a first-class correctness concern, not a detail.

`O_DIRECT` : opting out of the page cache

Databases that manage their own buffer pool often do not want the kernel's page cache: it double-buffers (once by InnoDB/Postgres, again by the kernel), makes memory accounting opaque, and interposes kernel writeback heuristics between the database and the device. Opening a file with `O_DIRECT` bypasses the page cache — reads and writes DMA (almost) straight to the device from user buffers — and hands the application the obligations the page cache used to cover: block-aligned buffers, its own caching, its own read-ahead. `O_DIRECT` does **not** by itself guarantee durability: the write can still sit in the *drive's* volatile cache, so a durable `O_DIRECT` write still needs a flush/FUA (or `O_DSYNC`). What it buys is *control* — the database, not the kernel, decides what is cached and when data is flushed, exactly what a system implementing its own WAL and buffer pool wants.

Different file systems, different guarantees

A subtle and important research result underlies all of the above: **the crash-consistency guarantees an application actually gets depend**

on which file system and mount options it runs on. The OSDI 2014 study by Pillai, Chidambaram, Alagappan, Al-Kiswany, Arpaci-Dusseau, and Arpaci-Dusseau — "*All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications*" — built a tool (ALICE) to enumerate the crash states an application could observe, and found that widely deployed applications (databases, key-value stores, version-control systems) harbored **crash-vulnerabilities**: they relied on ordering or atomicity properties that *some* file systems happened to provide but the POSIX standard does **not** require. An application that appeared correct on ext4 `data=ordered` could corrupt on ext4 `data=writeback`, or on a different FS, because it depended on, say, "a `rename` is atomic and ordered after the preceding `write`" — true on one substrate, not guaranteed on another. POSIX says remarkably little about crash behavior; the concrete guarantees live in each file system's implementation and mount options.

The practical consequence: **know your substrate**. File system type, journaling mode, whether the drive's write cache and barriers are on — these are inputs to your durability argument, not sysadmin trivia. The same database binary on the same hardware gives different crash guarantees on ext4 `data=journal` vs XFS vs Btrfs vs a network block device with an intervening cache. Systems that assume "the disk is a disk" get surprised; the ones that survive production either constrain the substrate (documented, tested FS + mount options) or design recovery to tolerate the weakest guarantee it can give.

Breadth: locking, xattrs, FD semantics, pseudo-filesystems, FUSE, NFS

The VFS surfaces several more features that a backend engineer meets regularly. Each has a sharp edge worth knowing.

File locking is advisory, and does not cross machines

Linux offers two classic locking APIs. `flock()` takes a whole-file shared or exclusive lock tied to the open file *description*. `fcntl()` byte-range locks (POSIX record locks) lock a range and are tied to the *(pid, inode)* pair — with famously error-prone semantics: a POSIX lock is released when the process closes *any* descriptor to that inode, which makes them treacherous in libraries. Both are **advisory**: they coordinate only among

processes that *choose* to check. Nothing stops a process that ignores locking from writing through a "locked" file. (Mandatory locking existed but was unreliable and is effectively gone.) Linux `OFD` locks (`F_OFD_SETLK`) fix the worst POSIX semantics by tying the lock to the open file description like `flock`, and are what new range-locking code should use. The distributed caveat: **none of this coordinates across machines**. File locks are a single-kernel construct; two nodes locking "the same file" on a shared NFS mount get, at best, unreliable server-side emulation. Cross-node mutual exclusion needs a real distributed lock (etcd, ZooKeeper, a lease) — a file lock is not it.

Extended attributes

Extended attributes (xattrs) are key/value metadata attached to an inode beyond the fixed fields, namespaced as `user.*`, `system.*`, `security.*`, `trusted.*`. They carry SELinux labels (`security.selinux`), POSIX ACLs, file capabilities, and application metadata (object stores overlaid on a file system stash checksums and content-type here). They are stored by the file system (inline in the inode if small, in a separate block if large), and — a durability note consistent with everything above — an xattr change is metadata that an `fsync` covers.

The open file table, dup, shared offsets, fork inheritance

Recall the three-level structure and connect it to Chapter 1. A process's **FD table** maps small integers to entries in the kernel's **open file table** (the `struct file` objects), and each of those points to an inode. The distinctions matter:

- `dup()` / `dup2()` copy a *descriptor* so two FDs point at the **same struct file** — and therefore share the **same offset**. A write through one advances the read position of the other. This is exactly how shell redirection (`2>&1`) makes stdout and stderr share a position so their interleaved output does not overwrite itself.
- `fork()` (Chapter 1) copies the FD table, so the child's descriptors point at the **same struct file objects** as the parent's — again, **shared offsets**. Parent and child writing to an inherited log FD append in sequence rather than clobbering, *because* they share the offset. (Append safety across independent `open()` s instead needs `O_APPEND`, which makes each write atomically seek-to-end.)

- Two independent `open()` calls of the same path create **two different struct file objects** with **independent offsets**, both resolving to the one inode.

This is why "why did my two processes' writes interleave cleanly?" and "why did they clobber each other?" both have precise answers: it comes down to whether they share a `struct file` (dup/fork) or merely share an inode (independent opens), and whether `O_APPEND` is set.

/proc and /sys: the kernel as files

The VFS abstraction is powerful enough that Linux exposes much of the kernel's own state through it, as **pseudo-filesystems** with no backing storage. `/proc` presents per-process and system state as files: `/proc/<pid>/status`, `/proc/<pid>/fd/` (open descriptors — your `lsof` for deleted files), `/proc/meminfo`, `/proc/mounts`. `/sys` (sysfs) exposes the device/driver model — block-device queue parameters, scheduler settings — as a tree. `/proc/sys` (the `sysctl` tree) exposes writable tunables: `echo 3 > /proc/sys/vm/drop_caches`, or the `vm.dirty_ratio` knobs from Chapter 4. None of these are on disk; reading a file invokes a kernel function that formats the answer on the fly — the same VFS function-pointer mechanism, with "file operations" implemented as "compute and return kernel state." It is why `cat /proc/...` monitoring works uniformly, and why a container's namespaced `/proc` can honestly differ from the host's.

FUSE: file systems in userspace

FUSE (Filesystem in Userspace) lets a normal userspace program implement a file system: the kernel FUSE driver receives VFS operations and forwards them, over a device channel, to a userspace daemon that answers. This is how `sshfs`, `s3fs`, many cloud-storage mounts, and some container image tools work. The cost is round trips: every `read/getattr` crosses the kernel/user boundary and back, so FUSE trades performance for the ability to write a file system in ordinary code with ordinary libraries. For metadata-heavy workloads it can be dramatically slower than a native FS; for gluing an object store onto the POSIX API at acceptable throughput it is often exactly right.

NFS and the surprise of network file systems

The VFS lets a **network file system** sit behind the same `open / read / write` — **NFS** is the canonical example — but the *consistency model* is where local intuition breaks. NFS (v3 especially) provides **close-to-open (CTO) consistency**: changes a client makes are guaranteed visible to *other* clients only after the writer **closes** the file and the reader subsequently **opens** it. In between, each client caches aggressively and does *not* see others' concurrent changes in real time. An application assuming a POSIX-local model — "my write is immediately visible to the node reading the same file" — will observe stale data, and finds `flock` across NFS clients unreliable. NFS also introduced the "silly rename" (an unlinked-but-open file is renamed to a hidden `.nfsXXXX` name to preserve open-file semantics across a stateless protocol), and its caching means `fsync` durability now depends on the *server's* storage stack. The general lesson: **a network file system offers the file system interface but not the guarantees you assumed**. Treat "it's just a mounted directory" on NFS as a distributed system in disguise, with its own consistency model to learn — not a local disk.

Distributed-systems lens

File systems are the exact boundary where distributed data systems meet physical durability, and nearly every hard lesson in the storage half of this suite lives at that boundary.

- **A "committed" write is only as durable as the `fsync` under it.**

Volume 5's WAL and Volume 6's consensus logs define correctness in terms of durable local persistence. `fsyncgate` showed that if the local persist can silently fail, then replication trusting each replica's durable state — and a Raft/Paxos leader that "committed" because a quorum reported success — are building on sand. Local file-system semantics are a *distributed*-correctness input.

- **Different substrates give different guarantees, so distributed storage must know its ground.** "All File Systems Are Not Created Equal" is not academic: the same binary gives different crash behavior across `ext4` modes, `XFS`, `Btrfs`, and network block devices. Serious systems pin and test their substrate or design recovery for the weakest guarantee it can give.

- **The write-path cost drives the durability/latency trade-off at the heart of data systems.** Because `fsync` is a synchronous round trip to media, every commit pays for durability in latency. Group commit, batched flushes, and sequential-append WALs all exist to amortize that one expensive barrier — and you can only reason about them if you know *where* in the page-cache-to-media path durability is actually achieved.
- **Network file systems have their own consistency models.** NFS close-to-open consistency, unreliable cross-client locking, and server-dependent durability mean "just put it on the NFS mount" quietly changes your correctness model. Shared-FS storage is a distributed system, not a disk.
- **The VFS mirrors the pluggable-backend pattern you use one level up.** One API over many implementations (`ext4/XFS/Btrfs/FUSE/NFS`) is the same discipline as a database with pluggable storage engines — and only as good as the guarantees each backend actually honors.

Key takeaways

- The **VFS** provides one syscall API (`open / read / write / close / stat`) over many file systems via four objects: **superblock** (a mounted FS), **inode** (a file's metadata + block map, *not its name*), **dentry** (a name→inode entry, cached in the dcache for fast pathwalk), and **file** (an open instance, holding the offset). Descriptors resolve **fd** → **file** → **dentry** → **inode** → **superblock**.
- **The name is not the file.** The inode is the file; names are directory entries with a **link count**. `unlink` removes a name and decrements the count; storage frees only when **link count == 0 AND no open FD remains** — the source of hardlinks, the "deleted file, disk still full" gotcha, and the atomic anonymous-temp-file idiom. Symlinks store a *path*; hardlinks share an *inode*.
- **ext4** (block groups, extents, JBD2 journaling), **XFS** (allocation groups, B+ trees, high concurrency — a strong DB/large-storage default), **Btrfs/ZFS** (copy-on-write, checksums, cheap snapshots — at the cost of fragmentation under random overwrite), and **tmpfs** (RAM-backed, volatile) make different, load-bearing trade-offs.

- **Crash consistency** is solved by **journaling** (write intent + commit record to a journal, then checkpoint; replay committed / discard uncommitted on reboot — recovery in seconds, not a full `fsck`) or by **copy-on-write** (never overwrite in place; a single atomic root-pointer switch). ext4's `data=ordered / journal / writeback` modes trade safety for speed; `writeback` can expose stale data after a crash. **Journaling protects the FS's integrity, not your application's un-flushed data.**
- **A `write()` that returns is not durable** — it is a dirty page in the page cache. Durability requires driving the chain **page cache → writeback → block layer → device cache → media** to completion, which is what `fsync` does (via a FLUSH/FUA barrier). `fdatasync` skips inessential metadata (faster for WAL append). Creating a file durably also needs an **`fsync` on the containing directory** to persist the name.
- **`fsyncgate` (Postgres, 2018):** Linux could clear a dirty page's error state after reporting it once, so a failed writeback could be lost and a later `fsync` return success on data that was never persisted — and the failure could not be fixed by retry. The fix: kernel error reporting improved, and databases now **PANIC and recover from the WAL** on `fsync` failure rather than trust the page cache. `O_DIRECT` bypasses the page cache for DBs managing their own durability but still needs a flush for persistence.
- **Different file systems and mount options give genuinely different crash guarantees** ("All File Systems Are Not Created Equal") — so a distributed storage system must know and constrain its substrate. File locks are advisory and do not cross machines; `/proc` and `/sys` are the VFS exposing kernel state as files; FUSE trades performance for userspace flexibility; and NFS offers the file interface with **close-to-open** consistency, not local guarantees.

Further reading

- Robert Love, *Linux Kernel Development*, 3rd ed. — chapters on the VFS, the block I/O layer, and the page cache; the clearest treatment of the superblock/inode/dentry/file object model.
- Bovet & Cesati, *Understanding the Linux Kernel*, 3rd ed. — detailed VFS, ext, and I/O internals.

- Remzi and Andrea Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* — the "Persistence" chapters (file system implementation, FSCCK and journaling, crash consistency). Free online at <https://pages.cs.wisc.edu/~remzi/OSTEP/>.
- Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnathan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, "All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications," OSDI 2014 — <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai>.
- The PostgreSQL *fsyncgate* thread (pgsql-hackers, 2018) and the LWN.net write-up "*PostgreSQL's fsync() surprise*" (<https://lwn.net/Articles/752063/>) — the incident, the kernel error-reporting semantics, and the PANIC-on-fsync-failure resolution.
- Linux man pages: `fsync(2)`, `fdatasync(2)`, `open(2)` (`O_DIRECT`, `O_APPEND`, `O_TMPFILE`), `write(2)`, `unlink(2)`, `rename(2)`, `flock(2)`, `fcntl(2)` (OFD locks), and `path_resolution(7)`.
- The `ext4` and `xfs` documentation in the kernel tree (`Documentation/filesystems/`), and the Btrfs and OpenZFS project documentation for copy-on-write internals, snapshots, and checksums.
- Volume 1, Chapter 5 — *Storage Hardware* (the media, device caches, FUA/flush, power-loss protection); Volume 2, Chapter 4 — *Memory in Practice* (the page cache, writeback, dirty ratios); Volume 5 (Databases — WAL, group commit, `O_DIRECT`); Volume 6 (Consensus — durable log entries and why local persistence underpins distributed commit).

Chapter 7 — Linux I/O Models: Blocking, epoll, and io_uring

What this chapter covers. A backend service is, mechanically, a machine for waiting. An API gateway holding 200,000 keep-alive connections is *doing* almost nothing at any instant: the overwhelming majority of those sockets are idle, waiting for the next request byte, the next database reply, the next disk block. The single most consequential architectural decision in such a service is not the language or the

framework — it is **how it waits on I/O**. That choice determines how many connections one box can hold, how CPU-efficient each request is, and — critically — what your p99 latency does when load spikes. This chapter is the "how does one server handle 100,000+ concurrent connections" story, told as an evolution: from a thread blocked per connection, to a single thread multiplexing thousands of sockets with `epoll`, to true completion-based asynchrony with `io_uring`, and finally to the "looks-blocking" runtimes (Go, Java virtual threads, Rust/Tokio) that hide an event loop behind ordinary sequential code.

We build directly on earlier chapters. The cost of a thread — its stack, its context-switch overhead, its scheduler pressure — was quantified in Chapter 1 (The Process Model) and Chapter 2 (CPU Scheduling); that cost is exactly what pushes servers off the thread-per-connection model. The syscall boundary and its per-call cost, from Chapter 5 (System Calls and the Kernel Boundary), is what `epoll` amortizes and what `io_uring` nearly eliminates. And the page cache and `O_DIRECT` distinctions from Chapter 6 (File Systems and the VFS) are what made the old Linux AIO so disappointing and `io_uring` so overdue.

Learning goals — after this chapter you should be able to:

- Explain *why* I/O-bound concurrency is the defining problem of backend servers, and frame the C10K → C10M progression as a story about the per-connection cost of waiting.
- Describe blocking I/O with thread-per-connection precisely, and state exactly which resources (stack memory, context switches, scheduler run-queue length) make it hit a wall.
- Distinguish **readiness** models (`select` / `poll` / `epoll`) from **completion** models (POSIX/native AIO, `io_uring`), and place the reactor and proactor patterns on that axis.
- Use `epoll` correctly, including the difference between level-triggered and edge-triggered modes and the draining discipline ET demands — a classic and expensive bug source.
- Explain the shared-memory ring architecture of `io_uring` (SQ/CQ), why it removes a syscall per operation, why it delivers true async for *both* network and disk, and its security caveats.
- Explain how Go's netpoller, Java virtual threads (Loom), and `async/await` runtimes give you blocking-style ergonomics over an `epoll` /

`io_uring` event loop with user-space scheduling — and why "don't block the event loop" is a law, not a style preference.

- Match an I/O model to a workload, and reason about the tail-latency consequences of each.

The core problem: waiting is the workload

Consider a service that, for each request, reads a few kilobytes from a socket, issues one database query over another socket, waits for the reply, and writes a response. On a modern box the CPU work per request — parsing, serialization, a little business logic — might be a few tens of microseconds. The *waiting* — for the client's bytes to arrive over the network, for the database round-trip — is easily milliseconds, often tens of milliseconds. The ratio of waiting to computing is frequently 100:1 or 1000:1.

This is the essential asymmetry of backend work: **your service spends almost all of its wall time blocked on I/O, and almost none of it running instructions.** A design that dedicates an expensive resource — an OS thread — to each connection for the entire duration it is *mostly idle* is spending that resource on nothing. The whole history in this chapter is the search for a way to hold a huge number of mostly-idle connections without paying a thread's worth of cost for each.

Two numbers name the eras. **C10K** — coined by Dan Kegel in his 1999 essay "The C10K problem" — asked how a single server could handle ten thousand simultaneous clients, a number that broke the thread-per-connection assumptions of late-1990s Unix. **C10M**, popularized a decade and a half later, asks the same question at ten million: a target reachable only by ruthlessly minimizing per-connection cost, and often by pulling work out of the kernel entirely (kernel-bypass, `AF_XDP`, DPDK — beyond this chapter but see Volume 3, The Linux Network Stack). The models below are the rungs of that ladder.

```

flowchart TB
    subgraph TPC["Thread-per-connection: N threads, N stacks"]
        direction TB
        C1["conn 1"] --> T1["thread 1  
blocked in read()"]
        C2["conn 2"] --> T2["thread 2  
blocked in read()"]
        C3["conn N"] --> T3["thread N  
blocked in read()"]
        T1 --> K1["kernel run queue  
+ N stacks (~N MiB)"]
        T2 --> K1
        T3 --> K1
    end
    subgraph EL["Event loop: 1 thread, N sockets"]
        direction TB
        D1["conn 1"] --> EP["epoll_wait()  
1 thread"]
        D2["conn 2"] --> EP
        D3["conn N"] --> EP
        EP --> DISP["dispatch only  
the ready FDs"]
    end
    end

```

The left side pays $O(N)$ in memory and scheduler pressure. The right side pays $O(1)$ threads regardless of N , and CPU proportional only to the connections that are *actually ready* at a given instant. That contrast is the entire chapter in one picture.

Blocking I/O and thread-per-connection

The oldest model is the one every engineer learns first because it maps to sequential thinking. A socket is blocking by default. You call `accept()` to get a connection, hand it to a thread, and that thread calls `read()`; if no data is available, `read()` *blocks* — the kernel puts the thread to sleep (off the run queue, in state `TASK_INTERRUPTIBLE`; see Chapter 2) and wakes it when bytes arrive. The code reads top to bottom, exactly as you'd write a single-user program:

```
// one of these runs per connection, on its own thread
void handle(int fd) {
    char buf[4096];
    for (;;) {
        ssize_t n = read(fd, buf, sizeof
buf); // BLOCKS until data or EOF
        if (n <= 0)
            break; // 0 = peer closed, <0 = error
        process(buf, n);
        write(fd, buf, n); // may also block
    }
    close(fd);
}
```

The appeal is real: it is trivially correct, trivially debuggable (a stack trace tells you exactly what each connection is doing), and each connection's state lives naturally on its own call stack — no explicit state machine. For a service with tens or a few hundreds of concurrent connections, thread-per-connection is a perfectly good, and often *correct*, choice. Do not let this chapter talk you out of simplicity when your concurrency is low.

It hits a wall at scale for three concrete, kernel-level reasons, all developed in Chapters 1 and 2:

- **Stack memory.** Every thread needs a stack. The Linux default (`ulimit -s`) is commonly 8 MiB of *virtual* address space, and while only touched pages are backed by physical memory (Chapter 3), the reservation and the touched pages add up. Even at a frugal 64–256 KiB of resident stack per thread, 100,000 threads is gigabytes of RAM spent on stacks that are, per the asymmetry above, mostly parked. Thread-pool frameworks trim this, but the reservation is a real ceiling.
- **Context-switch overhead.** Waking a blocked thread, and later switching away from it, costs the direct register save/restore plus the indirect cost measured in Chapter 1 and 2: TLB and cache pollution. With thousands of runnable threads churning, the machine spends an increasing fraction of its cycles switching *between* work rather than doing work.
- **Scheduler pressure.** The CFS/EEVDF run queue (Chapter 2) is not free to manage. A run queue with thousands of runnable tasks lengthens scheduling decisions and worsens wakeup latency — and

wakeup latency is directly your tail latency (Volume 11, Reliability and Observability). The scheduler was not designed on the assumption that "number of threads" and "number of connections" are the same large number.

None of these is a hard limit at a thousand connections; all of them bite hard at a hundred thousand. C10K was, precisely, the observation that the thread-per-connection model's costs grow linearly with a quantity (connection count) that was growing far faster than per-thread cost was shrinking.

A crucial caveat, which the last section of this chapter cashes out: **the wall was a wall for 1:1 OS threads, not for the blocking-style code**. Modern runtimes revived looks-blocking programming by making the "thread" cheap — Go goroutines, Java virtual threads — while doing event-driven I/O underneath. The ergonomics of this section came back; the costs did not.

Non-blocking I/O and readiness

The way out is to stop dedicating a thread to each wait. Set the socket non-blocking with `O_NONBLOCK` (via `fcntl` or `accept4(SOCK_NONBLOCK)`), and now `read()` on an empty socket does not sleep — it returns immediately with `-1` and `errno == EWOULDBLOCK` (spelled `EAGAIN` on Linux; they are the same value). The thread is never parked in the kernel on that one FD.

By itself that only converts blocking into busy-polling, which is worse. The missing piece is a **readiness notification** mechanism: a way for *one* thread to ask the kernel "of these thousands of FDs, which ones can I read or write *right now* without blocking?" and to *sleep* until at least one can. That question — readiness multiplexing — is answered by three successive syscalls, each fixing the previous one's scaling flaw.

select and poll: O(n) every call

`select()` is the oldest. You build three bitmaps (read, write, exception), each an `fd_set`, set a bit per FD you care about, and call `select()`; it blocks until at least one FD is ready, then rewrites the bitmaps to mark which. Two flaws make it unusable at scale. First, `fd_set` is a fixed-size bitmap — `FD_SETSIZE`, almost universally **1024** — so an FD number ≥ 1024 cannot even be represented (setting it is undefined behavior, and a

classic security/corruption bug). Second, the kernel must **scan every FD** in the set on every call: cost is $O(\text{highest FD number})$, not $O(\text{interesting FDs})$. And because `select()` clobbers your bitmaps, you rebuild and recopy them across the user/kernel boundary on every single call.

`poll()` fixes the count limit: you pass an array of `struct pollfd { int fd; short events; short revents; }`, of any length, and the kernel fills in `revents`. No 1024 ceiling, and input/output are separated (`events` vs `revents`) so you don't rebuild the interest set. But the fundamental cost is unchanged: the kernel still scans the **entire array** on every call to see which FDs are ready — $O(n)$ in the number of *watched* FDs, whether or not any are active — and you still copy the whole array in and out each time. For an event loop watching 100,000 connections where 50 are ready, `poll()` does 100,000 units of work to find those 50, on every iteration.

epoll: the kernel keeps the ready list

`epoll` is Linux's answer, and it is an architectural change, not just a faster `poll()`. Instead of passing the full FD set on every wait, you register interest *once* and let the kernel maintain the state across calls. Three syscalls:

- `epoll_create1(0)` — create an epoll instance; returns an FD referring to a kernel object that holds two structures: an **interest list** (the FDs you're watching, kept in a red-black tree keyed by FD for $O(\log n)$ add/remove) and a **ready list** (a linked list of FDs that have become ready).
- `epoll_ctl(epfd, EPOLL_CTL_ADD|MOD|DEL, fd, &event)` — register, modify, or remove one FD in the interest list. You do this once per FD (and again only when its interest mask changes), *not* on every wait.
- `epoll_wait(epfd, events, maxevents, timeout)` — block until one or more watched FDs are ready, then return **only the ready ones**, filled into your `events` array.

The decisive property is how the ready list is populated. When you `epoll_ctl(ADD)` an FD, the kernel attaches a callback to that FD's wait queue. When data arrives on the socket, the network stack's wakeup path fires that callback, which appends the FD to the epoll instance's ready list. `epoll_wait()` therefore does *no scanning*: it simply returns whatever is on the ready list. Its cost is proportional to the number of **ready** FDs,

not the number of **watched** FDs. Watching 100,000 mostly-idle connections and finding the 50 that are ready now costs $\sim O(50)$, not $O(100,000)$. That is the whole game.

```

flowchart TB
    subgraph reg["Registration (once per FD)"]
        A["accept4() new conn  
set O_NONBLOCK"] --> B["epoll_ctl(ADD, fd, EPOLLIN)"]
        B --> RB["interest list  
(red-black tree)"]
    end
    end
    subgraph loop["Event loop (per iteration)"]
        W["epoll_wait()"] --> RL["ready list  
only FDs that fired"]
        RL --> DISP["for each ready fd"]
        DISP --> RD["non-blocking read() until EAGAIN"]
        RD --> H["run handler / advance state machine"]
        H --> WR["non-blocking write() (queue rest on EAGAIN)"]
        WR --> W
    end
    end
    NET["NIC IRQ / softirq:  
data arrives"] -. "wake up callback appends fd" .-> RL

```

The comparison, made concrete:

Property	select	poll	epoll
Max FDs	FD_SETSIZE (≈ 1024)	unlimited	unlimited
Cost per wait	$O(\text{max FD number})$	$O(\text{watched FDs})$	$O(\text{ready FDs})$
Interest set passed	every call (rebuilt)	every call	registered once (<code>epoll_ctl</code>)
Kernel state across calls	none	none	persistent instance
Portability	POSIX (everywhere)	POSIX (everywhere)	Linux only
Edge-triggered mode	no	no	yes (<code>EPOLLET</code>)

`epoll` is Linux-specific. The BSD family (and macOS) has **kqueue**, reached via `kqueue()` / `kevent()`, which solves the same problem with the

same "register once, kernel keeps the ready list" architecture and is, if anything, more general (it multiplexes not just socket readiness but timers, file-system events, process and signal events through one interface). Portable event-loop libraries — libev, libevent, libuv (Node.js's loop) — exist precisely to paper over `epoll` vs `kqueue` vs Windows IOCP behind one API. Solaris had `/dev/poll` and event ports in the same vein. The lesson generalized; the syscalls differ per OS.

Level-triggered vs edge-triggered — the classic bug

`epoll` supports two notification semantics, and confusing them is one of the most common and most painful bugs in event-driven code.

Level-triggered (LT) is the default and matches `poll()`'s behavior. `epoll_wait()` reports an FD as ready *whenever it is ready* — i.e., as long as there is data in the receive buffer. If you read only part of the available data and loop again, `epoll_wait()` will report the FD ready *again* because data still remains. LT is forgiving: you may read once per wakeup, do a fixed amount of work, and come back; you will be re-notified about the leftover.

Edge-triggered (ET), enabled with the `EPOLLET` flag, reports an FD only on a *transition* — roughly, only when *new* data arrives that moves the socket from not-ready to ready. After an ET notification, you will **not** be told again until *more* new data comes in, even if there is still unread data sitting in the buffer. This is more efficient (fewer wakeups, no re-scanning of FDs whose data you already know about) and is what high-performance loops often use — but it imposes a strict discipline:

With edge-triggered `epoll` you **must drain the FD completely** on each notification — `read()` in a loop until it returns `EAGAIN` / `EWOULDBLOCK`. If you stop early, the unread bytes sit in the buffer with no pending edge to re-notify you, and that connection **hangs forever** (a stuck request, a leaked connection, a mysterious partial read). The same applies to `write()` / `EPOLLOUT`: drain the send buffer until `EAGAIN`.

ET therefore only makes sense with non-blocking FDs — if the socket were blocking, the drain loop's final `read()` (with the buffer empty) would block the entire event loop. The canonical ET read handler is:

```
// edge-triggered: MUST loop until EAGAIN, FD is non-blocking
for (;;) {
    ssize_t n = read(fd, buf, sizeof buf);
    if (n > 0)          { process(buf, n); continue; }
    if (n == 0)         { close(fd); break; }           // peer closed
    if (errno == EAGAIN) { break; }                     // fully
drained — done
    if (errno == EINTR)  { continue; }                 //
interrupted, retry
    close(fd); break;                                   // real error
}
```

Aspect	Level-triggered (default)	Edge-triggered (EPOLLET)
Notifies while	data remains readable	only on new-data transition
Wakeups	more (re-notified until drained)	fewer (once per arrival)
Read discipline	may read once; re-notified for the rest	must drain until <code>EAGAIN</code>
Blocking FD safe?	tolerable	no — must be non-blocking
Failure mode	busy-ish, but correct	forget to drain → connection hangs
Typical use	simpler loops, most apps	max-throughput loops (nginx, Netty option)

A related flag, `EPOLLONESHOT`, disables an FD after one notification until you re-arm it with `epoll_ctl(MOD)`; it is the standard tool for safely handing a ready FD to a worker thread in a multi-threaded event loop without two threads racing on the same connection. Getting LT/ET right is the difference between an event loop that works and one that intermittently strands connections under load — and because ET bugs only manifest when a read is *partial* (large messages, slow clients, TLS records split across packets), they routinely escape testing and surface as tail-latency incidents in production.

The reactor: an event loop over epoll

Put the pieces together and you get the **reactor** pattern (named by Douglas Schmidt): a small number of threads — often *one* — each running a loop that calls `epoll_wait()`, and for every ready FD dispatches to a handler that does non-blocking I/O and advances that connection's state. All the connection's state that used to live implicitly on a thread's call stack now lives explicitly in a per-connection structure (a state machine, a continuation, a callback chain).

```
// the reactor, in miniature
int ep = epoll_create1(0);
add(ep, listen_fd, EPOLLIN);           // watch the listener too
struct epoll_event evs[MAX];
for (;;) {
    int n = epoll_wait(ep, evs, MAX, -1); // sleep until something
    is ready
    for (int i = 0; i < n; i++) {
        int fd = evs[i].data.fd;
        if (fd == listen_fd) {
            int c = accept4(listen_fd, ..., SOCK_NONBLOCK);
            add(ep, c, EPOLLIN | EPOLLET); // register the new
connection
        } else {
            handle_ready(fd, evs[i].events); // non-blocking read/
write, advance state
        }
    }
}
```

This scales because the number of threads is decoupled from the number of connections. One event-loop thread can hold tens or hundreds of thousands of connections, waking only for the handful ready at any instant, paying no per-connection stack and adding no per-connection entry to the scheduler run queue. This architecture powers the highest-concurrency servers in the industry:

- **nginx** runs one event-loop process per CPU core (worker processes), each an `epoll` reactor, which is why it holds enormous connection counts at low memory.
- **Redis** is famously a (mostly) single-threaded `epoll` event loop; its speed comes partly from never paying synchronization or context-switch cost on the hot path (see Chapter 4 on why its fork-based persistence interacts badly with THP).

- **Node.js** is a single-threaded JavaScript event loop over libuv (which is `epoll` on Linux, `kqueue` on BSD/macOS, IOCP on Windows).
- **Netty** (the JVM's high-performance networking framework, under gRPC-Java, Cassandra, Elasticsearch and much else) is an `epoll` / `kqueue` reactor with a small pool of event-loop threads, each owning a shard of the connections.

The reactor's virtues come with a sharp, defining constraint, which is worth stating as a law:

You must never block the event loop. Every handler runs *on* the loop thread. If one handler makes a blocking call — a synchronous DB query, a `read()` on a blocking FD, a filesystem `stat` on a slow mount — or does a long CPU-bound computation (a big JSON parse, a crypto operation, a regex on a megabyte), the *entire loop stops*. Every other connection that thread was serving is frozen until the handler returns. One slow request becomes a latency spike for thousands.

The consequences shape real service design. CPU-bound work is offloaded to a **worker thread pool**, with results posted back to the loop. Blocking dependencies must be replaced with non-blocking/async clients, or likewise pushed to a pool. Node.js's cultural aversion to synchronous APIs, and its `libuv` thread pool for filesystem and DNS work, are this law made concrete. The other cost is cognitive: because a connection's control flow is spread across callbacks fired at different times, reactor code historically degenerated into deeply nested callbacks — "callback hell" — and error handling and backpressure became genuinely hard to get right. That pain is exactly what the `async/await` and green-thread runtimes later set out to erase, without giving up the loop underneath.

From readiness to completion: the async I/O gap

Everything so far — `select`, `poll`, `epoll`, `kqueue` — is a **readiness** model. The kernel tells you *when* an FD is ready, and then **you** perform the actual `read()` / `write()` syscall. This is the reactor pattern, and it has two residual costs. First, you still make a data-moving syscall per ready FD (`epoll_wait` finds the work; the `read` does it) — two crossings per I/O. Second, and more fundamentally, **readiness only works for sockets and pipes**. A regular file on a local disk is *always* "ready" as far as `epoll` is concerned — you can always call `read()` on it — but that `read()` may

still block for milliseconds while the block layer fetches from storage. `epoll` cannot help you wait for *disk* without blocking. For disk, you need a different model entirely.

That model is **completion-based** (the **proactor** pattern, also from Schmidt): instead of being told when you *can* start an operation, you *submit* the operation and are later notified when it has **completed**, with the result. The kernel does the I/O on your behalf; you get a completion event carrying "your read of 4096 bytes finished; here they are." Readiness asks "can I go?"; completion says "it's done."

```
flowchart LR
    subgraph readiness["Readiness (epoll / reactor)"]
        R1["epoll_wait: fd is READY"] --> R2["you call read(fd)"] --> R3["you get bytes"]
        note1["1 syscall to find work  
+ 1 syscall to do it  
sockets only, not disk"]
    end
    subgraph completion["Completion (io_uring / proactor)"]
        C1["submit READ op"] --> C2["kernel performs the read"] --> C3["completion: bytes ready"]
        note2["kernel does the I/O  
works for disk AND sockets"]
    end
```

Linux's history of completion-based I/O before `io_uring` is a history of disappointment, and understanding why is what makes `io_uring` legible as a *solution* rather than just a new API:

- **POSIX AIO** (`aio_read`, `aio_write`, `aio_suspend`, ...) is the standard's answer. On Linux, glibc implements it in **user space with a hidden thread pool** — each async op is really a blocking op on a background thread. It "works" but delivers none of the efficiency promise: you're back to thread-per-operation, just concealed. It is little used for serious work.
- **Linux native AIO** (`io_submit` / `io_getevents`, the `libaio` interface) is a real in-kernel async interface — and it is notoriously disliked. Its crippling limitation: it is truly asynchronous *only* for **O_DIRECT** file I/O (Chapter 6). For ordinary page-cache-backed (buffered) file I/O, `io_submit` silently falls back to **blocking** — defeating the entire purpose. It doesn't cover network I/O usefully, and even in its intended path, operations like metadata lookups can block the

submitting thread. Databases that wanted async disk (e.g., certain storage engines) tolerated it *only* under `O_DIRECT`, and everyone agreed it was a dead end.

So on the eve of `io_uring`, Linux had: excellent scalable *readiness* for sockets (`epoll`), and no good *completion*-based async for disk. That gap — plus the per-operation syscall cost that even `epoll` couldn't remove — is what `io_uring` was built to close.

`io_uring`: shared-memory rings

`io_uring`, created by Jens Axboe and merged in **Linux 5.1 (2019)**, is the most significant advance in the Linux I/O interface in many years. Its core idea is to replace the syscall-per-operation model with **two shared-memory ring buffers** mapped into both user space and the kernel, so that submitting work and reaping results become, in the common case, *memory writes and reads with no syscall at all*.

The two rings:

- **Submission Queue (SQ)**. A ring of **SQEs** (submission queue entries). To ask for an operation, user space writes an SQE describing it — an opcode (`IORING_OP_READ`, `WRITE`, `ACCEPT`, `SEND`, `RECV`, `FSYNC`, `OPENAT`, `CLOSE`, `SENDMSG`, `POLL_ADD`, `TIMEOUT`, and dozens more), the target FD, buffer pointer, length, offset, and flags — and advances the ring's tail. It can queue *many* SQEs this way before ever entering the kernel.
- **Completion Queue (CQ)**. A ring of **CQEs** (completion queue entries). When an operation finishes, the kernel writes a CQE containing the result (bytes transferred or a negative errno) and a `user_data` field the app set on the SQE to correlate completion with request. User space reads CQEs and advances the head — again, just memory operations.

```

flowchart LR
    subgraph user["User space"]
        APP["application"]
    end
    subgraph shared["Shared memory (mmap'd, no copy)"]
        SQ["Submission Queue"]
        SQEs["read/write/accept/send/..."]
        CQ["Completion Queue"]
        CQEs["result + user_data"]
    end
    subgraph kernel["Kernel"]
        K["io_uring: performs ops  
net + disk, truly async"]
    end
    APP -- "write SQE, advance tail" --> SQ
    SQ -- "io_uring_enter() (batched)  
or SQPOLL: no syscall at all" --> K
    K -- "write CQE" --> CQ
    CQ -- "read CQE, advance head" --> APP

```

The mechanics behind that picture:

- Setup is `io_uring_setup(entries, params)`, which returns a ring FD; user space then `mmap`s the SQ, CQ, and SQE array. The **liburing** library wraps this boilerplate and is what most applications actually call (`io_uring_queue_init`, `io_uring_get_sqe`, `io_uring_prep_read`, `io_uring_submit`, `io_uring_wait_cqe`).
- To hand queued SQEs to the kernel and/or wait for completions you call `io_uring_enter()`. The batching win: you can prep **N** operations and submit them all with **one** `io_uring_enter` — N operations, one syscall, versus N syscalls with `read / epoll`. This is Chapter 5's "amortize the syscall boundary" taken to its limit.
- **SQPOLL mode** goes further: a dedicated kernel thread *polls* the submission ring, so user space submits work by writing an SQE and advancing the tail with **no syscall at all** in steady state. Combined with **IOPOLL** (busy-poll completions for low-latency NVMe), this yields a submission-to-completion path that can avoid syscalls entirely on the hot path.
- **Registered files and fixed buffers** (`io_uring_register`) let the kernel pre-pin buffers and cache file references, removing per-operation lookup and page-pinning cost — cutting the copy/refcount overhead Chapter 5 attributes to each crossing.

Why it is a genuine breakthrough, not just a faster API:

1. **True async for both network *and* disk**, buffered *and* direct — the thing native AIO never delivered. A single interface asynchronously handles socket `recv`, file `read` from the page cache, `fsync`, `accept`, and more, all through the same rings.
2. **Syscall reduction to near zero.** Batched submission collapses many operations into one `io_uring_enter`; `SQPOLL` removes even that. At high request rates the per-op syscall cost that dominated `read` / `epoll` loops largely disappears.
3. **Fewer copies and less per-op setup** via fixed buffers and registered files.
4. **Breadth of operations and chaining.** Beyond basic I/O it supports linked operations (`IOSQE_IO_LINK` — "do B after A completes"), multishot operations (one `accept` or `recv` SQE that keeps producing CQEs for each new connection/message), and timeouts — letting you express whole request pipelines with minimal kernel round-trips.

Adoption reflects this: high-performance databases and storage engines (ScyllaDB was an early, prominent adopter through its Seastar framework; PostgreSQL added `io_uring`-backed async I/O in its 18 release), proxies and web servers, and language runtimes are all moving toward it. The Rust ecosystem has `tokio-uring` and `glommio`; the JVM and Go have experimental integrations discussed in the last section.

The security caveat — hedge it

`io_uring` must be discussed with a clear caveat, because it directly affects whether you can use it in production. Its power comes from a large, complex new kernel-facing surface — many opcodes, asynchronous execution contexts, shared memory — and that surface has been a **significant source of kernel vulnerabilities**. Google's security teams reported that `io_uring` accounted for a disproportionate share of the exploitable Linux kernel bugs they saw through their kCTF/VRP program (as of early 2026; current posture may have evolved — check Google's security blog and kernel release notes), and consequently **disabled `io_uring`** across ChromeOS, Android, and their production servers. Container platforms restrict it: gVisor and many seccomp default profiles block the `io_uring` syscalls, and hardened environments turn it off via `sysctl kernel.io_uring_disabled`. The honest guidance: `io_uring` is a major performance win, but it is **relatively young and has a live**

security track record, so treat "is `io_uring` permitted in this environment?" as a real question — of kernel version, seccomp profile, and security posture (Volume 0, Book 6, Container and Cloud-Native Supply Chain Security) — not a given. Many fleets gate it behind an explicit, per-workload decision.

The "looks-blocking" comeback: user-space scheduling

The reactor solved scalability but taxed the programmer: explicit state machines, callback chains, the never-block law. The final movement in this story reclaims blocking-style *ergonomics* while keeping event-driven *scalability*, by moving the scheduler that multiplexes "connections onto threads" **out of the kernel and into user space**. You write code that *looks* like it blocks; the runtime turns each block into a park-and-resume over an `epoll` (or increasingly `io_uring`) event loop. This is exactly the M:N threading model introduced in Chapter 1 — many lightweight tasks multiplexed onto few OS threads — applied to I/O.

```
flowchart TB
    subgraph app["Application code (looks blocking)"]
        G1["goroutine / virtual thread /"]
        async["task #1: conn.Read()"]
        G2["...#2: db.Query()"]
        G3["...#N: resp.Write()"]
    end
    end
    subgraph rt["Language runtime (user-space scheduler)"]
        SCHED["M:N scheduler:"]
        park["park blocked task,"]
        run["run a ready one on the OS thread"]
    end
    end
    subgraph os["OS threads (few: ~= #cores)"]
        M1["OS thread 1"]
        M2["OS thread 2"]
    end
    end
    subgraph kern["Kernel"]
        POLL["epoll / io_uring"]
    end
    readiness["readiness & completions"]
    end
    G1 & G2 & G3 --> SCHED
    SCHED --> M1 & M2
    M1 & M2 -. "non-blocking I/O + poll" .-> POLL
    POLL -. "FD ready → mark task runnable" .-> SCHED
```

The instances differ in machinery but share the idea:

- **Go goroutines.** Go's runtime schedules many goroutines (G) onto a small set of OS threads (M), coordinated by logical processors (P) — the **G-M-P** model (Volume 4, Concurrency; Volume 13, Language Runtimes). When a goroutine does a network `Read`, the runtime sets the socket non-blocking, registers it with the **netpoller** (which is `epoll` on Linux, `kqueue` on BSD/macOS), and **parks** the goroutine, freeing its OS thread to run other goroutines. When the netpoller reports the FD ready, the runtime marks the goroutine runnable and resumes it — right where its `Read` left off. You wrote `n, err := conn.Read(buf)`, straight-line; the runtime silently ran an `epoll` event loop under it. (Blocking *file* and other syscalls Go handles differently — it hands them to a dedicated OS thread and may spin up another M so the P keeps running goroutines; native disk async via `io_uring` has been explored but network is the netpoller's domain.)
- **Java virtual threads (Project Loom).** Finalized in **Java 21 (2023, JEP 444)**, a virtual thread is a `Thread` scheduled by the JVM onto a pool of **carrier** platform threads. When virtual-thread code executes a blocking operation — `socket.read()`, a JDBC call over NIO — the JVM **unmounts** the virtual thread's continuation from its carrier and parks it, using non-blocking I/O and a poller underneath, so the carrier runs other virtual threads. The payoff is that decades of straightforward blocking Java — thread-per-request servlets, blocking clients — suddenly scale to enormous concurrency with no rewrite. The caveat is **pinning**: certain constructs (historically `synchronized` blocks holding a monitor across a blocking call, and native/`Object.wait`-style waits) pin the virtual thread to its carrier, and enough pinned carriers can starve the pool — the Loom-era version of "don't block the event loop."
- **async / await (Rust/Tokio, Python/asyncio, C#).** These are **stackless coroutines**: `async fn` compiles to a state machine (a `Future / Task`) that the compiler generates, and an executor drives it, calling `poll` and suspending at each `.await` until the awaited I/O is ready. Rust's **Tokio** runs a work-stealing multi-thread scheduler over an `epoll` reactor (via `mio`), with `io_uring` backends emerging (`tokio-uring`, `glommio`). Python's `asyncio` event loop sits on the `selectors` module — `epoll` on Linux. C#'s `Task / async` sits on the thread pool and IOCP (Windows) or `epoll` (Linux). Stackless

coroutines are cheaper in memory than a full stack but make blocking mistakes *visible in the type system*: a synchronous call in an `async` function is a compile-time-obvious "coloring" mismatch, and calling a blocking API from an `async` task still stalls that executor thread — the never-block law, re-expressed.

The unifying truth: **all of these are `epoll / io_uring` event loops with a user-space scheduler on top**. The green thread, the virtual thread, and the `async` task are three ways to represent "a paused connection waiting on I/O" that cost far less than an OS thread — kilobytes, not megabytes; a runtime queue entry, not a kernel run-queue entry. They give you the reactor's scalability with the blocking model's readability. And they inherit the reactor's cardinal rule: because they all multiplex many logical tasks onto few real threads, **a task that truly blocks or hogs a CPU stalls everything sharing that thread** — the exact failure mode from the reactor section, now wearing a friendlier API.

Zero-copy: moving less data per byte served

Orthogonal to *how you wait* is *how much you copy*, and for I/O-heavy servers the copies dominate. A naive "serve a file over a socket" does four copies and multiple crossings: disk → page cache (DMA), page cache → user buffer (`read`), user buffer → socket buffer (`write`), socket buffer → NIC (DMA), plus context switches around each syscall. The middle two go *through user space* for no reason if the application never inspects the bytes. Linux offers **zero-copy** primitives to cut them out (tying back to Chapter 5's syscall accounting and Volume 1's memory-bandwidth story):

- `sendfile(out_fd, in_fd, offset, count)` copies data between two FDs **inside the kernel**, never landing in a user buffer. Classic use: file → socket. It collapses the four copies toward two DMA copies and removes the user-space round trip entirely. This is the mechanism behind Apache Kafka's famously cheap consumer fan-out — brokers `sendfile` log segments from the page cache straight to consumer sockets, so a hot topic served to many consumers barely touches the CPU.
- `splice(fd_in, fd_out, ...)` moves data between two FDs via a kernel **pipe** buffer with no user-space copy; one end must be a pipe. It generalizes `sendfile` (socket→file, file→socket, socket→socket via

a pipe), and with `tee()` / `vmsplice()` supports duplicating and gifting user pages into the pipe.

- **MSG_ZEROCOPY** (the `SO_ZEROCOPY` socket option, for `send()`) lets the kernel transmit directly from **user pages** without copying them into the socket buffer — worthwhile for large writes. Because the pages must stay stable until the NIC has actually sent them, the kernel signals completion asynchronously via the socket's error queue (`MSG_ERRQUEUE`) — itself a small completion-model wrinkle, and one `io_uring` integrates natively (`IORING_OP_SEND_ZC`).

Zero-copy interacts with the models above: `io_uring` can issue `splice()` / `sendfile`-style and zero-copy send operations as ring ops, unifying "async" and "zero-copy" in one submission. For a CDN node, a video origin, or a Kafka broker, these primitives are the difference between being CPU/memory-bandwidth-bound and being network-bound — i.e., between wasting the box and saturating the wire.

Distributed-systems lens

The I/O model is the concurrency architecture of the service. It is not a low-level detail you can defer to a library and forget; it determines the fundamental shape — thread-per-request vs event loop vs async-runtime — and therefore the connection ceiling, the memory footprint per connection, and the CPU cost per request. Fleet-wide, that shape is dollars: the shift from thread-per-connection to `epoll` reactors to async runtimes is *why* a single modern proxy instance holds connection counts (C10K → C10M) that would have needed a rack of thread-bound boxes, at a fraction of the RAM and CPU.

Connection-heavy infrastructure lives or dies on I/O-model efficiency. The systems that sit between everything else — L7 load balancers and API gateways (Envoy, nginx), service-mesh sidecars, message brokers, database proxies (Volume 3; Volume 0 Book 6) — hold vast numbers of mostly-idle connections and forward bytes. Envoy is an `epoll/libevent` reactor precisely because a mesh sidecar riding alongside every pod must be cheap per connection; a thread-per-connection sidecar would double every service's footprint. For this whole class of software, the I/O model is the product's viability.

"Don't block the event loop" is a distributed-systems failure mode, not a coding tip. In a reactor or an async runtime, a single blocking call

— a synchronous DNS lookup, an un-pooled blocking DB driver, a CPU-heavy handler — freezes every connection sharing that thread. Under load, that converts one slow dependency into a correlated latency spike across *all* in-flight requests on that loop: your p99 and p999 detonate together (Volume 11). The mitigations are architectural: offload CPU work to a worker pool, use non-blocking clients for every downstream, bound and monitor thread-pool/carrier occupancy, and treat "a slow handler" as a system-wide availability risk. Thread-per-connection pools have the dual failure: **pool exhaustion**. When every thread is blocked on a slow downstream, new requests queue for a thread that never frees, and latency climbs as a step function — the classic thread-pool-saturation cascade that propagates backpressure the wrong way and takes services down in sympathy.

User-space scheduling mirrors the OS scheduler one level up. The Go scheduler, the Loom carrier pool, and the Tokio executor are doing, in the application, exactly what Chapter 2's CFS does in the kernel: multiplexing more logical work than there are CPUs, making fairness/preemption/starvation decisions, and turning "blocked on I/O" into "descheduled." The same reasoning transfers — a starved goroutine and a starved thread have the same shape — and so do the same pathologies: a non-preemptible hot loop (Go before asynchronous preemption; a CPU-bound async task) is a runaway task starving its peers, the app-level analog of a kernel task that won't yield.

io_uring is a fleet-level cost lever. Its syscall and copy reductions look like microseconds per operation, but multiplied across a fleet handling millions of requests per second, those microseconds are cores — real capacity and real money, plus tail-latency improvement from fewer mode transitions. That is why databases and high-throughput data-plane software are adopting it despite the operational and security overhead — and why the security caveat matters as an org-wide policy decision, not a per-service whim.

Choosing a model

The models are a ladder from simplicity to scalability; pick the lowest rung that meets your concurrency and throughput needs, because each rung up costs complexity.

Model	Scales to	Complexity	Use it when
Blocking + thread-per-conn	$\sim 10^2$ - 10^3 conns	lowest (sequential)	low concurrency; internal tools; simplicity wins; batch workers
<code>select</code> / <code>poll</code> reactor	$\sim 10^3$ (poll)	medium	portability to old/tiny systems; legacy; small FD sets
<code>epoll</code> / <code>kqueue</code> reactor	10^5 - 10^6 conns	high (state machines / callbacks)	very high connection counts; proxies, gateways, caches; you can enforce never-block
<code>io_uring</code>	10^5 - 10^6 +, incl. async disk	high + young/security-gated	max throughput; async disk <i>and</i> network; DBs, storage, data-plane — where kernel version & security policy permit
Async runtime (Go / Loom / Tokio)	10^5 - 10^6 conns	medium (blocking-style code, real gotchas)	almost all new high-concurrency services — reactor scalability with sequential ergonomics

The pragmatic reality for most engineers: **you do not choose these syscalls directly**. You pick a language and a framework — Go, a JVM stack on Netty or Loom, Rust on Tokio, Node — and it picks the model for you, usually an `epoll` / `io_uring` event loop with user-space scheduling. The reason to understand what's underneath is not to reimplement it but to **explain and predict behavior**: why a synchronous call in one handler tanked your whole service's p99, why goroutines let you write a million-connection server in blocking style, why a partial read hangs an edge-triggered loop, why disabling `io_uring` in a hardened container changed your throughput, why the Kafka broker serves fan-out for nearly free. The abstraction is excellent right up until it leaks — and when it leaks, at 3 a.m., during a load spike, this chapter is the map.

Key takeaways

- **Backend servers are I/O-bound: they spend almost all their time waiting.** The I/O model — how you wait — is the most

consequential concurrency decision in a service, setting its connection ceiling, per-connection cost, and tail latency.

- **Thread-per-connection is simple and correct for low concurrency, and hits a wall at high concurrency** on stack memory, context-switch overhead, and scheduler pressure (Chapters 1–2). This is the C10K problem; C10M is the modern bar.
- **Non-blocking sockets plus a readiness mechanism decouple thread count from connection count.** `select` ($O(n)$, ≤ 1024 FDs) and `poll` ($O(n)$, unlimited) scan every watched FD; `epoll` **returns only the ready ones** by having the kernel maintain the ready list — $O(\text{ready})$, scaling to 100k+ connections. `kqueue` is the BSD/macOS equivalent.
- **Level-triggered vs edge-triggered epoll is a real correctness distinction.** ET notifies once per arrival and **demands you drain the FD to `EAGAIN`** on non-blocking sockets; forget, and the connection hangs — a bug that hides until reads go partial under load.
- **The reactor (epoll event loop) powers nginx, Redis, Node, and Netty** and scales superbly — but *you must never block the loop*: one slow or CPU-heavy handler freezes every connection on that thread. Offload CPU work; use non-blocking downstreams.
- **Readiness (`epoll`) tells you *when* to do I/O; completion (`io_uring`) does the I/O and tells you it's *done*.** Old Linux completion I/O was broken: POSIX AIO is a hidden thread pool; native AIO (`io_submit`) is truly async only under `O_DIRECT` and disliked. Neither did async network or buffered disk.
- **`io_uring` (Linux 5.1, 2019) is the modern leap:** shared SQ/CQ rings remove the syscall-per-op (batched `io_uring_enter`, or `SQPOLL` for none), deliver true async for both network and disk, and cut copies. Weigh its **security track record** — Google disabled it in several environments; containers/seccomp often block it — as an explicit policy decision.
- **The "looks-blocking" runtimes are the synthesis.** Go goroutines (`netpoller/epoll`), Java virtual threads (Loom), and `async/await` (Tokio/asyncio) are M:N user-space schedulers over an `epoll` / `io_uring` loop: blocking-style code, event-driven scalability. They inherit the never-block law (pinning, blocking calls, CPU hogs stall their pool).

- **Zero-copy (`sendfile` , `splice` , `MSG_ZEROCOPY`) is the orthogonal win** for I/O-heavy servers — moving bytes kernel-to-kernel without a user-space round trip (why Kafka fan-out is cheap).
- **Match the model to the workload, and know what your framework chose for you** — because when the abstraction leaks under load, understanding the layer underneath is what lets you diagnose the p99 spike.

Further reading

- **Dan Kegel, "The C10K problem"** (kegel.com/c10k.html , 1999, updated through the 2000s) — the original framing of high-concurrency server I/O and a survey of the strategies (blocking threads, `select` / `poll` , edge/level readiness) that this chapter's history follows.
- **man pages** (the authoritative Linux reference): `epoll(7)` (LT vs ET semantics, `EPOLLONESHOT` , the canonical usage example), `epoll_create1(2)` , `epoll_ctl(2)` , `epoll_wait(2)` , `select(2)` , `poll(2)` , `fcntl(2)` (`O_NONBLOCK`), `accept4(2)` , `sendfile(2)` , `splice(2)` , and `socket(7)` (`SO_ZEROCOPY`).
- **The Linux Programming Interface**, Michael Kerrisk (No Starch Press, 2010), chapters 63 ("Alternative I/O Models") and 44–61 — the precise, accurate treatment of non-blocking I/O, `select` / `poll` / `epoll` , level vs edge triggering, and the readiness model.
- **Jens Axboe, "Efficient IO with io_uring"** (the `io_uring` design document / "kernel-recipes" writeup) and the `liburing` repository (github.com/axboe/liburing) with its man pages (`io_uring_setup(2)` , `io_uring_enter(2)` , `io_uring_register(2)`) — the primary sources on SQ/CQ rings, `SQPOLL` / `IOPOLL` , fixed buffers, and linked/multishot operations.
- **Lord of the io_uring** (unixism.net/loti/) — a thorough, example-driven tutorial on `io_uring` semantics and `liburing` usage.
- **Google Security Blog, "Learnings from kCTF VRP's 42 Linux kernel exploits" (2023)** and Google's subsequent guidance disabling `io_uring` on ChromeOS/Android/production (as of early 2026; verify current `io_uring` default/allowlist in your target kernel, container runtime, and seccomp profile) — the primary statement of the `io_uring` kernel-security concern this chapter hedges.

- **Douglas C. Schmidt, "Reactor: An Object Behavioral Pattern for Demultiplexing and Dispatching Handles for Synchronous Events"** and the companion **Proactor** pattern paper — the canonical descriptions of the reactor (readiness) and proactor (completion) architectures.
- **The Go runtime source and design talks** on the scheduler and **netpoller** (`src/runtime/netpoll_epoll.go`, `proc.go`) — how goroutine parking maps onto `epoll` (Volume 4, Concurrency; Volume 13, Language Runtimes).
- **JEP 444: Virtual Threads** (`openjdk.org`) and the Project Loom materials — carrier threads, continuation unmounting, and the pinning caveat, for the JVM's "looks-blocking" model.
- **The Tokio documentation** (`tokio.rs`) and **mio** — Rust's `async / await` executor over an `epoll` reactor, and the emerging `io_uring` backends (`tokio-uring`, `glommio`).
- **nginx** and **Redis** architecture documentation, and the **Netty** user guide — production reactors to read as concrete instances of this chapter's event-loop model.

Chapter 8 — Signals, Pipes, and IPC

What this chapter covers. Processes are the kernel's isolation boundary (Chapter 1 — The Process Model): separate address spaces that, by construction, cannot touch each other's memory. That isolation is the whole point — and it is also the problem. Real systems are not one process; they are a shell pipeline, a supervisor and its workers, an application and its sidecar, a container and the orchestrator that owns its lifecycle. The moment you have two processes that must *coordinate*, you need a way to punch a controlled hole through the isolation boundary. That is inter-process communication, and Linux offers a dozen mechanisms for it, from a single asynchronous bit (a signal) to a shared page of memory.

For a backend engineer, one slice of this material is disproportionately important, so we lead with it and keep returning to it: **signals are the protocol by which the container and orchestration layer controls your process's lifecycle**. When Kubernetes rolls out a new version, it does not politely ask your service to stop — it sends `SIGTERM`. Whether

your service finishes its in-flight requests and closes connections cleanly, or drops them on the floor and returns 502s to users, comes down to a signal handler you either wrote or forgot to write. A service that mishandles `SIGTERM` is a service that causes a small outage on *every single deploy*, and at high deploy frequency that is one of the most common self-inflicted availability bugs in the industry. We will make that mechanism exact.

Around that core we survey the full IPC toolbox — pipes and FIFOs, Unix domain sockets, shared memory, message queues, semaphores, and the "everything is a file descriptor" event primitives (`eventfd` / `timerfd` / `signalfd`) that fold IPC into the event loop from Chapter 7 — and we place each one on the axes that matter: throughput, latency, complexity, and whether it carries *data* or merely a *notification*.

Learning goals — after this chapter you should be able to:

- Explain what a signal is at the kernel level: an asynchronous notification with a numeric identity, a per-process *disposition* (default / ignore / handler), and delivery semantics that interrupt the target at an arbitrary instruction boundary.
- State precisely why signal handlers are dangerous — the **async-signal-safety** constraint — and use the standard escape hatches (`signalfd` , the self-pipe trick) to turn signals into pollable event-loop inputs.
- Describe the Kubernetes pod termination sequence (`SIGTERM` → grace period → `SIGKILL`) and write a correct graceful-shutdown / connection-draining path for a backend service.
- Explain the special signal semantics of **PID 1**, why they cause the "my container ignores `SIGTERM`" gotcha, and what `tini` / `dumb-init` / `--init` actually fix.
- Wire up anonymous pipes with `fork` + `dup2` (the shell `|`), reason about pipe buffering and `SIGPIPE` , and avoid the broken-pipe crash.
- Choose among Unix domain sockets, shared memory, pipes, and message queues for a local-IPC problem, and understand FD passing over `SCM_RIGHTS` .
- Frame local IPC vs network RPC as the same locality decision that governs distributed system design.

Signals: an asynchronous notification, not a channel

A signal is the smallest possible message: a small integer, delivered asynchronously to a process, that interrupts whatever it was doing. It carries no payload beyond its number (the real-time signals and `SA_SIGINFO` add a little, more below). It is closer to a hardware interrupt aimed at a process than to a message on a queue. This is the first thing to internalize: **signals are for notification and control, not for moving data**. If you find yourself trying to stream information through signals, you have chosen the wrong mechanism.

Each signal has a numeric identity and a conventional meaning. The standard signals every backend engineer should know:

Signal	Num*	Default action	Catchable?	Typical meaning
<code>SIGTERM</code>	15	Terminate	Yes	"Please shut down cleanly." The polite stop.
<code>SIGKILL</code>	9	Terminate	No	"Die now." Uncatchable, unblockable, unignorable.
<code>SIGINT</code>	2	Terminate	Yes	Interrupt from terminal (Ctrl-C).
<code>SIGQUIT</code>	3	Terminate + core	Yes	Quit from terminal (Ctrl-\\); dumps core.
<code>SIGHUP</code>	1	Terminate	Yes	Controlling terminal hung up; by convention, reload config .
<code>SIGCHLD</code>	17	Ignore	Yes	A child stopped or terminated — the reaping trigger.
<code>SIGPIPE</code>	13	Terminate	Yes	Wrote to a pipe/socket with no reader.
<code>SIGSEGV</code>	11	Terminate + core	Yes	Invalid memory access.

Signal	Num*	Default action	Catchable?	Typical meaning
SIGBUS	7	Terminate + core	Yes	Bad memory access (misalignment, mmap past EOF).
SIGABRT	6	Terminate + core	Yes	abort() ; failed assertion.
SIGSTOP	19	Stop	No	Suspend the process. Uncatchable.
SIGCONT	18	Continue	Yes	Resume a stopped process.
SIGUSR1 / SIGUSR2	10/12	Terminate	Yes	Application-defined (nginx, etc.).
SIGALRM	14	Terminate	Yes	Timer from alarm / setitimer.

* Numbers are the common values on Linux x86-64; a few (SIGUSR1, SIGCHLD, ...) differ on other architectures, which is exactly why you use the *names*, never the raw numbers, in code.

Two signals are special and worth memorizing as a pair: **SIGKILL and SIGSTOP cannot be caught, blocked, or ignored.** They are enforced by the kernel, not the process. This is a deliberate design: the system must always retain an unstoppable way to kill or freeze a process, no matter how buggy or malicious. Everything about graceful shutdown flows from this one fact — SIGTERM is a request the process may honor or delay; SIGKILL is a command it cannot refuse.

Beyond these standard signals (1-31, the historical Unix set), Linux provides **real-time signals**, SIGRTMIN through SIGRTMAX (typically 34-64). They differ in two ways that matter: multiple instances of the same RT signal are *queued* (standard signals are not — see below), and they can carry a small int /pointer payload via sigqueue. Most backend code never uses them directly, but runtimes and libraries do.

Disposition: default, ignore, or handler

Every process has, for each signal, a **disposition** — what happens when that signal is delivered:

- **Default (SIG_DFL).** The kernel's built-in action from the table above: terminate, terminate-and-core-dump, ignore, stop, or continue.
- **Ignore (SIG_IGN).** The signal is discarded. (You cannot ignore SIGKILL / SIGSTOP.)
- **Catch with a handler.** A function you register that the kernel invokes on delivery.

You set the disposition with `sigaction(2)`. You will see older code use `signal(2)`, but its semantics are historically inconsistent across Unixes (BSD vs System V behavior on handler reset and syscall restart). **Use `sigaction`**. It gives explicit control over the flags that matter:

```
#include <signal.h>
#include <string.h>

volatile sig_atomic_t shutting_down =
0; /* the ONLY safe global type here */

static void on_term(int signo) {
    shutting_down =
1; /* set a flag; do the real work in main loop
*/
}

int main(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = on_term;
    sigemptyset(&sa.sa_mask); /* which signals to block *while the
handler runs* */
    sa.sa_flags = SA_RESTART; /* auto-restart interrupted slow
syscalls */
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGINT, &sa, NULL);
    /* ... run the event loop, checking `shutting_down` ... */
}
```

Key `sigaction` flags:

- **SA_RESTART** — if a signal interrupts a slow syscall (`read`, `accept`, ...), the kernel restarts it automatically instead of failing with `EINTR`. Without this, every blocking call in your program must be prepared

to see `EINTR` and retry. (Even *with* it, a few calls are never restarted — `poll`, `epoll_wait`, `select` — so event loops must still handle `EINTR`.)

- **SA_SIGINFO** — use the three-argument handler (`sa_sigaction`) that receives a `siginfo_t`: the sending PID/UID, the faulting address for `SIGSEGV`, the `sigqueue` payload, etc.
- **SA_NOCLDWAIT** / **SA_NOCLDSTOP** — for `SIGCHLD`: don't create zombies, and don't signal on mere *stop* of a child (Chapter 1).

The async-signal-safety problem

Here is the single most important — and most misunderstood — fact about signal handlers, and a classic subtle bug.

A handler does not run "between iterations" of your program. It runs **asynchronously**: the kernel can deliver a signal, and thus invoke your handler, at *any* machine instruction boundary in the interrupted thread. If the signal arrives while your program is halfway through `malloc`, the handler runs with `malloc`'s internal data structures in an inconsistent, mid-mutation state, and with `malloc`'s internal lock held.

Now suppose your handler calls `printf`. `printf` calls `malloc`. That `malloc` tries to take the lock the interrupted `malloc` already holds — in the *same thread* — and, depending on the allocator, either deadlocks or corrupts the heap. This is not a rare race; it is a structural consequence of asynchronous delivery. Most of the C library is **not reentrant** and therefore **not safe to call from a signal handler**.

The rule: **inside a signal handler you may call only async-signal-safe functions**. POSIX defines the exact list (`signal-safety(7)` on Linux). It is short and deliberately boring: `write`, `read`, `_exit`, `kill`, `signalfd`, `sem_post`, most raw syscalls. Conspicuously *absent*: `malloc` / `free`, `printf` and all of stdio, most of the string-formatting family, anything that takes a lock, anything that touches non-atomic global state. Also: **save and restore `errno`** if your handler makes any syscall, because it runs "inside" the interrupted code which may be about to read `errno`.

```

/* SAFE: write a fixed string to stderr, no allocation, no stdio */
static void on_segv(int signo) {
    const char msg[] = "caught SIGSEGV, aborting\n";
    write(STDERR_FILENO, msg, sizeof msg - 1); /* async-signal-safe */
    _exit(1); /* not exit(); exit
runs atexit handlers */
}

```

The only data a handler should touch in shared memory is a variable of type `volatile sig_atomic_t` — the one type POSIX guarantees you can read/write atomically with respect to signal delivery. Everything else the handler communicates must go through an *async-signal-safe channel*, which is exactly what the next section is about.

The practical discipline that falls out of this: **do essentially nothing in the handler**. Set a flag, or write one byte to a pipe/eventfd, and return. Do the real work — the logging, the cleanup, the shutdown — back in your normal code, where every function is fair game.

```

sequenceDiagram
    participant Main as Main thread (normal code)
    participant K as Kernel
    participant H as Signal handler
    Note over Main: executing malloc()
    (holds allocator lock)
    K-->>Main: deliver SIGTERM (async)
    Main->>H: invoke handler at arbitrary PC
    Note over H: SAFE: write(pipe, &b, 1)
    or set volatile sig_atomic_t flag
    Note over H: UNSAFE: printf/malloc
    → re-enter locked allocator → deadlock
    H-->>Main: return; resume malloc()
    Note over Main: event loop later observes
    flag / readable pipe → shuts down

```

Blocking and pending signals: `sigprocmask`

A thread has a **signal mask** — a set of signals it currently *blocks*. A blocked signal is not lost; it becomes **pending** and is delivered as soon as it is unblocked. You manipulate the mask with `sigprocmask(2)` in a single-threaded program, or `pthread_sigmask(3)` in a multithreaded one (the mask is per-thread).

A critical subtlety: **standard signals do not queue**. The pending set is a bitmask — one bit per signal. If two `SIGCHLD`s arrive while `SIGCHLD` is blocked, exactly one is pending, and the handler runs once. This is why a correct `SIGCHLD` reaper loops on `waitpid(-1, ..., WNOHANG)` until it returns 0: one delivery may represent several dead children. (Real-time signals *do* queue, which is one reason they exist.)

Blocking is not just defensive; it is the foundation of the modern approach to signals in event-driven servers, which we turn to now.

Making signals pollable: `signalfd` and the self-pipe trick

The async-signal-safety constraint sits awkwardly with the event-loop architecture from Chapter 7. Your whole server is one thread sitting in `epoll_wait`, and you want signal arrival to be *just another readable fd* you can react to on your own terms — in normal code, with `malloc` and logging available. Two mechanisms achieve this.

The self-pipe trick (portable, decades old): create a pipe, make both ends non-blocking, and register the read end with your `epoll` set. The signal handler does exactly one async-signal-safe thing — `write(pipe_wr, &byte, 1)` — and returns. `epoll_wait` then reports the read end as readable, and your event loop handles the signal in full, safe context.

```
flowchart LR
    SIG["SIGTERM arrives"] --> H["handler:  
write(pipe_wr, &b, 1)"]
    H --> P["P (\"self-pipe\")"]
    P --> EP["EP[\"epoll_wait()  
reports pipe_rd readable\"]"]
    EP --> APP["APP[\"event loop:  
drain requests, close, exit  
(full safe context)\"]"]
```

`signalfd(2)` (Linux-specific, cleaner): create a file descriptor that becomes readable whenever one of a specified set of signals is pending, and read `struct signalfd_siginfo` records from it. There is no handler at all. The essential setup: **block the signals first** so they are never delivered the old asynchronous way — instead they stay pending and surface through the fd.

```

sigset_t mask;
sigemptyset(&mask);
sigaddset(&mask, SIGTERM);
sigaddset(&mask, SIGINT);
sigprocmask(SIG_BLOCK, &mask, NULL); /* essential: block, so they
route to the fd */

int sfd = signalfd(-1, &mask, SFD_NONBLOCK | SFD_CLOEXEC);
/* add sfd to your epoll set; when readable, read struct
signalfd_siginfo and act */

```

This is how a modern high-concurrency service should consume lifecycle signals: no async-signal-safety minefield, signal delivery unified with all other I/O readiness in the same `epoll` loop. Go's runtime does the moral equivalent internally — it dedicates handling so that your `signal.Notify` channel receives `os.Signal` values in ordinary goroutine context, never in a raw handler. (`timerfd` and `eventfd`, later in this chapter, complete this "everything is an fd" family.)

Signals as the service lifecycle protocol

Now the payoff. Everything above exists, for a backend engineer, mostly in service of one scenario: your process is being asked to stop, and how it responds determines whether users notice.

SIGTERM vs **SIGKILL** : the graceful/forceful pair

	SIGTERM (15)	SIGKILL (9)
Meaning	"Shut down cleanly."	"Terminate immediately."
Catchable	Yes — you decide what happens	No — kernel-enforced
Cleanup runs?	Yes, if you wrote a handler	None. Process just stops.
In-flight requests	Can be finished/drained	Dropped mid-flight
Open connections	Can be closed with proper FIN	Reset (<code>RST</code>) or left hanging
Buffered data / flush	You can flush	Lost

	SIGTERM (15)	SIGKILL (9)
Use it for	Normal shutdown, deploys	Last resort, hung process, OOM killer

`SIGTERM` is the beginning of a negotiation; `SIGKILL` is the end of it. The universal pattern across supervisors (`systemd`, `Docker`, `Kubernetes`, `runit`, `supervisord`) is: **send `SIGTERM`, wait a bounded grace period, then send `SIGKILL` if the process is still alive**. Your job is to do all your cleanup *within* that grace window.

Recall from Chapter 4 that the kernel **OOM killer** uses `SIGKILL` — an out-of-memory process gets no chance to clean up, which is one more reason to keep memory bounded: you never want the kernel choosing `SIGKILL` for you.

Graceful shutdown and connection draining

A correct shutdown path for a stateless HTTP/gRPC service, triggered by `SIGTERM`:

1. **Stop accepting new work.** Close the listening socket (or stop the accept loop). New connections are refused; the load balancer/orchestrator will route elsewhere.
2. **Drain in-flight requests.** Let currently-executing handlers finish, up to a deadline.
3. **Close idle keep-alive connections** cleanly (send HTTP `Connection: close` / `GOAWAY` for HTTP/2 so clients reconnect elsewhere rather than getting a mid-request reset).
4. **Flush and release** — flush logs/metrics, commit or roll back transactions, close DB pools, deregister from service discovery.
5. **Exit 0**, well before the grace period expires.

Every major framework gives you this: Go's `http.Server.Shutdown(ctx)`, Java's Spring Boot graceful shutdown, Node's `server.close()`, Python's `uvicorn/gunicorn` handlers. What they *cannot* do for you is receive the signal — that is the line of code people forget. Here is the idiomatic Go shape:


```
func main() {
    srv := &http.Server{Addr: ":8080", Handler: mux}
    go func() { srv.ListenAndServe() }()

    ctx, stop := signal.NotifyContext(context.Background(),
        syscall.SIGTERM, syscall.SIGINT)
    defer stop()
    <-ctx.Done() // blocks until SIGTERM/SIGINT

    // Give in-flight requests up to 25s; must finish inside the grace
    period.
    shutdownCtx, cancel := context.WithTimeout(context.Background(),
        25*time.Second)
    defer cancel()
    srv.Shutdown(shutdownCtx) // stops accepting, waits for active
    handlers
}
```

The number 25 is not arbitrary — it must be **less** than the orchestrator's grace period, so your clean shutdown finishes before the **SIGKILL** fallback fires.

The Kubernetes pod termination sequence

Kubernetes turns the **SIGTERM** → grace → **SIGKILL** pattern into a documented lifecycle. When a pod is deleted (a rolling update, a scale-down, a node drain), for each container:

1. The pod is marked **Terminating**. The endpoints controller removes it from the **Endpoints** / **EndpointSlice** for its Services, so kube-proxy (and ingress controllers, and mesh sidecars) *begin* to stop routing new traffic to it.
2. If a **preStop** hook is defined, it runs to completion first.
3. The container runtime sends **SIGTERM** to **PID 1** of the container.
4. The **grace period** counts down: **terminationGracePeriodSeconds** (default **30**). The clock covers the **preStop** hook *and* the post-**SIGTERM** drain — they share the budget.
5. If the container is still running when the grace period expires, the runtime sends **SIGKILL**.

```
sequenceDiagram
    participant API as kube-apiserver
    participant EPC as Endpoints controller
    participant Klt as kubelet / runtime
    participant App as Container PID 1
    API->>EPC: Pod → Terminating
    EPC->>EPC: remove from EndpointSlice
    (new traffic stops routing – async!)
    API->>Klt: begin termination (start grace clock)
    Klt->>App: run preStop hook (optional)
    Klt->>App: SIGTERM
    Note over App: stop accepting; drain in-flight;
    close conns; flush; exit 0
    alt exits before grace period
        App-->>Klt: process gone → pod removed
    else grace period expires
        Klt->>App: SIGKILL (in-flight dropped)
    end
end
```

The race that bites everyone. Step 1 (endpoint removal) and step 3 (SIGTERM) are *not* synchronized. Endpoint removal propagates asynchronously through many components — apiserver → endpoints controller → each node's kube-proxy → conntrack. Meanwhile SIGTERM may reach your app almost immediately. So for a short window, **new requests can still be routed to a pod that has already received SIGTERM**. If your handler reacts to SIGTERM by *immediately* closing the listener, those in-flight-routed requests get connection-refused — a burst of 502s on every deploy.

The standard mitigations:

- A **preStop sleep** (sleep 5 - 15) that delays SIGTERM long enough for endpoint removal to propagate, so by the time your app starts draining, the LB has already stopped sending it new work. Crude but extremely effective and framework-agnostic.
- Or have the app **keep serving for a few seconds after SIGTERM** before it stops accepting, achieving the same overlap in code.

Either way: your service **must** install a SIGTERM handler and drain. A service that lets SIGTERM hit the default action — instant termination — resets every in-flight connection on every deploy. At one deploy per service per day across a fleet of hundreds of services, that is continuous low-grade error-rate noise and a real user-facing reliability problem. This

is a distributed-systems reliability pattern (Volume 11) whose root is a single signal handler.

PID 1: the container init gotcha

Here is where the theory turns into a notorious operational bug, and it hinges on a kernel special case.

The kernel treats PID 1 differently. For an ordinary process, a signal with default disposition performs the default action (e.g. `SIGTERM` terminates). **For PID 1, the kernel does *not* apply default actions.** A signal sent to PID 1 is delivered only if PID 1 has *explicitly installed a handler* for it; otherwise it is silently dropped. Even `SIGKILL` and `SIGSTOP` are ignored when sent to PID 1 *from within its own PID namespace* (the kernel marks `init` `SIGNAL_UNKILLABLE`). This exists so you can't accidentally kill the real `init` and panic the machine.

In a container, **your application is often PID 1** of its PID namespace. Two failure modes follow:

- **Shell-form CMD.** `CMD myapp --flag` in a Dockerfile runs as `/bin/sh -c "myapp --flag"`, so `/bin/sh` becomes PID 1 and `myapp` is its child. `sh` does not forward signals to its child. `docker stop` / K8s sends `SIGTERM` to PID 1 (`sh`), which ignores it (no handler, and default actions don't apply to PID 1). Your app never even sees the signal. The orchestrator waits the full grace period, then `SIGKILL`s — an *always-hard-kill*, drop-in-flight shutdown on every deploy. Fix: **exec form** `CMD ["myapp", "--flag"]`, which makes `myapp` PID 1 directly.
- **App as PID 1 with no handler.** Even in `exec` form, if `myapp` never installs a `SIGTERM` handler, PID 1 semantics mean `SIGTERM` is dropped rather than terminating it. Same symptom: grace period wasted, then `SIGKILL`. Fix: **handle `SIGTERM`** (which you must do for draining anyway). Note the corollary: locally, `docker kill` works because the host sends `SIGKILL` from the *parent* namespace, where the unkillable protection does not apply.

There is a second PID 1 duty: **reaping zombies** (Chapter 1). PID 1 inherits orphaned processes and must `wait()` on them, or the container accumulates zombie PIDs. Application code rarely does this correctly.

The fix for both is a **minimal init process** as PID 1 that (a) forwards signals to your app and (b) reaps zombies: `tini`, `dumb-init`, or `s6/runit`. Docker's `--init` flag injects `tini`; Kubernetes users add a

lightweight init or rely on the app being a signal-aware runtime. The one-line takeaway: **if your container's PID 1 is a shell, a signal-ignorant app, or anything that neither forwards signals nor reaps children, you have a shutdown-correctness bug — use exec form and either handle SIGTERM yourself or run a real init.**

SIGHUP for config reload, SIGCHLD for reaping

Two more conventional uses round out the lifecycle picture:

- **SIGHUP — reload configuration.** Historically "the terminal hung up," SIGHUP was repurposed by daemons to mean "re-read your config without restarting." **nginx** is the canonical example: `nginx -s reload` sends SIGHUP to the master, which re-parses `nginx.conf`, spins up new workers with the new config, and gracefully shuts down old workers after they finish in-flight requests — a zero-downtime reload built entirely on signals. (nginx also uses SIGUSR1 to reopen log files for rotation and SIGUSR2 for binary upgrades.)
- **SIGCHLD — reap children.** As covered in Chapter 1, a parent that spawns workers must reap them. SIGCHLD fires on child exit; the handler (or `signalfd` reader) loops `waitpid(-1, &st, WNOHANG)` until it returns 0, because standard signals don't queue and one SIGCHLD may cover several deaths.

Pipes and FIFOs

The oldest IPC mechanism in Unix, and the one whose philosophy — small tools composed by streaming bytes — shaped everything. A pipe is a **unidirectional, in-kernel byte stream** with a write end and a read end.

Anonymous pipes and the shell |

`pipe(int fd[2])` returns two file descriptors: `fd[0]` (read) and `fd[1]` (write). Bytes written to `fd[1]` are buffered by the kernel and read from `fd[0]` in order. Because the ends are file descriptors, they are shared across `fork` (Chapter 1's FD inheritance), which is how *related* processes get connected — a parent and its children, or two children of the same parent.

The shell `|` is the archetype. For `producer | consumer`, the shell:

1. Calls `pipe()` to get `[r, w]`.
2. `fork`s the producer; in the child, `dup2(w, STDOUT_FILENO)` redirects its stdout onto the pipe's write end, closes the now-redundant `r` and `w`, and `exec`s `producer`.
3. `fork`s the consumer; in the child, `dup2(r, STDIN_FILENO)` redirects its stdin from the pipe's read end, closes `r` and `w`, and `exec`s `consumer`.
4. The parent shell closes both ends (it is neither producer nor consumer).

Closing the unused ends is not optional. The read end returns EOF only when *all* write-end FDs are closed. If any process still holds a write end open, the reader blocks forever waiting for bytes that will never come — a classic hang. `dup2` (Chapter 5) is doing the essential work: it points a well-known FD number (0 or 1) at the pipe so the `exec'd` program, which knows nothing about pipes, just reads stdin and writes stdout.

```
flowchart LR
    subgraph Prod["producer process"]
        PSTD["stdout (fd 1)"]
    end
    end
    subgraph Cons["consumer process"]
        CSTD["stdin (fd 0)"]
    end
    end
    PSTD -->| "dup2(w, 1)" | W["pipe write end"]
    W --> KP(("kernel pipe buffer  
~64 KiB"))
    KP --> R["pipe read end"]
    R -->| "dup2(r, 0)" | CSTD
```

Buffering, blocking, and atomicity

The kernel pipe buffer is finite — **64 KiB (16 pages) by default** on Linux, adjustable per pipe with `fcntl(F_SETPIPE_SZ)`. This buffer is the flow-control mechanism:

- **Write to a full pipe blocks** (or returns `EAGAIN` in non-blocking mode) until the reader drains it. This is natural backpressure — a fast producer is throttled to the consumer's rate.

- **Read from an empty pipe blocks** until data arrives, or returns 0 (EOF) if all write ends are closed.
- **Writes up to PIPE_BUF (4096 bytes) are atomic** — they will not be interleaved with other writers' data. Larger writes may be split and interleaved, which matters when multiple processes write to the same pipe (a common logging-fan-in mistake).

SIGPIPE : the broken-pipe gotcha

Writing to a pipe or socket whose read end is closed raises

SIGPIPE, whose default action terminates the process — and simultaneously the `write` would return **EPIPE**. This is one of the most surprising ways a backend process dies. Picture an HTTP server streaming a response; the client disconnects mid-stream; the server's next `write` to that socket triggers **SIGPIPE**, and with default disposition the *entire server process* dies — taking down thousands of other connections because one client hung up.

The fix is standard practice in every network server:

- **Ignore SIGPIPE process-wide** (`signal(SIGPIPE, SIG_IGN)`) and handle the **EPIPE** return from `write` / `send` as an ordinary error (close that connection, move on), **or**
- Pass **MSG_NOSIGNAL** to `send` / `sendmsg` per call to suppress the signal for that write, **or** on Linux set **SO_NOSIGPIPE**-style behavior where available.

Managed runtimes do this for you: **Go ignores SIGPIPE** for writes to its own network/file descriptors (it only lets **SIGPIPE** kill the process for writes to the original `stdout/stderr`, matching command-line-tool expectations). If you write servers in C/C++, ignoring **SIGPIPE** at startup is close to mandatory.

Named pipes (FIFOs)

An anonymous pipe requires a common ancestor to share the FDs. A **FIFO** (named pipe) lifts that restriction: `mkfifo(path, mode)` (or the `mkfifo` command) creates a special file in the filesystem that *any* process with permission can `open`. Semantics are otherwise identical — a unidirectional byte stream. `open` for reading blocks until a writer opens the other end and vice versa (unless **O_NONBLOCK**). FIFOs are handy for simple decoupled producer/consumer setups and shell scripting, but for

anything bidirectional or structured, a **Unix domain socket** is almost always the better choice.

Unix domain sockets: the local-IPC workhorse

If pipes are the oldest IPC and signals the most misunderstood, **Unix domain sockets (UDS)** are the most *important* for modern backend systems. They are the local-IPC backbone of the entire cloud-native stack.

A UDS uses the same sockets API as TCP/UDP (`socket` , `bind` , `listen` , `accept` , `connect` , `send` , `recv`) but with address family **AF_UNIX** instead of **AF_INET** . The endpoint is a **filesystem path** (`/var/run/docker.sock`) rather than an IP:port — or, on Linux, an **abstract-namespace** name (a leading NUL byte, no filesystem object, auto-cleaned on close). Because both endpoints are on the same host, the kernel short-circuits the entire network stack: **no IP routing, no TCP handshake, no checksums, no congestion control, no loopback packet processing**. The result is meaningfully lower latency and higher throughput than `127.0.0.1` TCP, with the same familiar API.

UDS supports three socket types:

- **SOCK_STREAM** — reliable, ordered byte stream (like TCP). The default choice.
- **SOCK_DGRAM** — reliable (locally!), *message-boundary-preserving* datagrams. Unlike UDP, local datagrams are not dropped.
- **SOCK_SEQPACKET** — reliable, ordered, *and* message-boundary-preserving. The best of both for message-oriented local protocols.

Where you already rely on UDS, usually without thinking about it:

- **The Docker socket** `/var/run/docker.sock` — the Docker CLI and every tool that talks to the daemon speaks HTTP over this UDS. (Mounting it into a container is also a well-known privilege-escalation risk — Book 6.)
- **Local database connections.** PostgreSQL and MySQL default to a UDS for same-host clients (`/var/run/postgresql/.s.PGSQL.5432`); it is faster than TCP loopback and sidesteps the network entirely.
- **Service-mesh and sidecar communication.** App ↔ sidecar (Envoy) traffic, and the mesh's control-plane xDS channels, frequently ride

UDS for the latency win when co-located in a pod. This is the local half of the service mesh (Volume 3).

- **systemd socket activation**, container runtime shims (containerd ↔ runc), containerd's own API, and countless daemons (Xorg , dbus , gpg-agent).

Two superpowers: FD passing and peer credentials

UDS can carry, alongside ordinary bytes, **ancillary data** ("control messages") via `sendmsg / recvmsg` — and this unlocks two things no network socket can do.

- **SCM_RIGHTS — passing file descriptors between processes.** You can send an *open file descriptor* — a socket, a file, a pipe, a `memfd` — across a UDS to an unrelated process. The kernel does not copy the bytes of a number; it installs a *new reference to the same open file description* in the receiver's FD table, exactly as `dup` would within one process. The receiver ends up sharing the very same connection/file/offset. This is the mechanism behind privilege separation (a sandboxed worker receives an already-opened socket it could not have opened itself), zero-downtime restarts that hand live listening sockets to a new process, container runtimes and the `seccomp notify fd`, and browser sandbox architectures. It is one of the most powerful and least-known capabilities in Unix.
- **SO_PEERCRED / SCM_CREDENTIALS — peer credentials.** The kernel can tell the receiver the *authenticated* PID, UID, and GID of the process on the other end — it cannot be forged, because the kernel fills it in. This is how a daemon does local authorization ("is the client root?") without passwords: `systemd` , `polkit` , and many privileged sockets rely on it.

flowchart LR

```
subgraph P1["Sender process"]
    FD1["open fd → socket/file"]
end
subgraph P2["Receiver process"]
    FD2["new fd → SAME open file"]
end
FD1 -->|"sendmsg + SCM_RIGHTS"| K["kernel dup's the open file description"]
K -->|"recvmsg"| FD2
Note["Both now share one connection / offset"]
```


Shared memory: the fastest, and the most dangerous

The mechanisms so far all *copy* data through the kernel: a `write` to a pipe or socket moves bytes into a kernel buffer, and a `read` copies them out again. **Shared memory eliminates the copy entirely.** Two processes map the *same physical pages* into their address spaces; once set up, a write by one process is instantly visible to the other with **zero syscalls and zero copies**. It is the highest-throughput, lowest-latency IPC Linux offers.

There are two APIs:

- **POSIX shared memory** (preferred): `shm_open("/name", ...)` returns an FD backed by a `tmpfs` object under `/dev/shm`; `ftruncate` sizes it; `mmap(..., MAP_SHARED, fd, 0)` maps it. Both processes `shm_open` the same name and `mmap` it. `shm_unlink` removes it. Because it is fd-based and lives in `tmpfs`, it composes cleanly with the rest of Unix.
- **System V shared memory** (legacy): `shmget / shmat / shmdt / shmctl` keyed by `key_t`, administered with `ipcs / ipcrm`. Still seen in older databases and HPC code. Its kernel-persistent, key-namespaced model is clunky compared to POSIX.
- For **related** processes, `mmap(MAP_SHARED | MAP_ANONYMOUS)` before `fork` shares the mapping with no name at all — the simplest form.

The catch, and it is a big one: **shared memory gives you the memory but not the synchronization.** Two processes writing the same page will race exactly as two threads do; you own every concurrency hazard from Volume 4. You must place synchronization primitives *in the shared region*: a `pthread_mutex / pthread_cond` created with the `PTHREAD_PROCESS_SHARED` attribute, a POSIX semaphore, or hand-rolled atomics/futexes. You must also handle a process crashing while holding a shared lock (robust mutexes, `EOWNERDEAD`) — a hazard that simply does not exist with a pipe. This is the fundamental trade: **shared memory is the fastest IPC and the one that pushes the entire correctness burden onto you.** High-performance systems — market-data buses, some databases' buffer pools, the LMAX Disruptor pattern, single-node caches — pay that price deliberately; most services should not.

A modern, fd-friendly variant: `memfd_create` creates an anonymous memory-backed file (no filesystem path). Combined with `SCM_RIGHTS`, one process can create a `memfd`, populate it, and pass the fd to another

process, which `mmap`s it — shared memory established over a UDS with no shared namespace. Sealing (`F_SEAL_*`) lets the receiver trust the buffer won't change size — used by Wayland, `dma-buf`, and Chromium's graphics IPC.

The rest of the toolbox

POSIX and System V message queues. A message queue is a kernel-managed queue of *discrete messages* (unlike a pipe's undifferentiated byte stream), with optional priorities. POSIX (`mq_open` / `mq_send` / `mq_receive`, visible under `/dev/mqueue`) is the cleaner API and, crucially on Linux, its descriptor is **pollable** — it integrates with `epoll`. System V (`msgget` / `msgsnd` / `msgrcv`) is the older, key-based variant. Message queues give you message-boundary preservation and priority ordering for free, but in practice most backend systems reach for a UDS `SOCK_SEQPACKET` or an out-of-process broker (Volume 3) instead.

Semaphores. A pure *synchronization* primitive — a counter with atomic wait/post — not a data channel. POSIX named semaphores (`sem_open`) coordinate unrelated processes; unnamed semaphores (`sem_init` in shared memory) coordinate related ones or threads. They are the classic companion to shared memory. Full treatment is in Volume 4 (Concurrency).

The "everything is a file descriptor" event primitives. Linux exposes several kernel event sources as ordinary, `epoll`-able file descriptors, which is what lets a single event loop (Chapter 7) unify all of them:

- **eventfd** — a 64-bit counter in the kernel. `write` adds to it; `read` returns and resets (or decrements with `EFD_SEMAPHORE`) it; it is readable when non-zero. The canonical lightweight cross-thread/cross-process **wakeup**: one thread's `write` makes another thread's `epoll_wait` return. `io_uring` uses an `eventfd` to signal completions into an event loop.
- **timerfd** — a timer that becomes readable when it expires. Instead of `SIGALRM` (asynchronous, all the safety problems above), you get timer expiry as a pollable fd, folded into the same loop as your sockets.
- **signalfd** — signals as an fd, covered earlier: the clean way to consume `SIGTERM` in an event loop.
- **memfd** — anonymous memory as an fd (above).

This family is the Linux answer to "how do I get *notifications* into a readiness-based event loop without signal handlers or dedicated blocking threads?" — and it is why a modern reactor can wait on sockets, timers, signals, and cross-thread wakeups in a single `epoll_wait`.

D-Bus. A higher-level message bus, mostly for desktop and system-management IPC on Linux. A broker (`dbus-daemon`, or the in-kernel-adjacent successors) mediates a **system bus** and per-session **session buses**, offering method calls, broadcast signals, and properties with an interface/object model — all transported over UDS underneath. `systemd`, `NetworkManager`, `logind`, and `PolicyKit` speak D-Bus. Backend services on servers rarely use it directly, but you will meet it when scripting host management (`busctl`, `systemctl` talks to `systemd` over D-Bus).

Comparing the mechanisms

The whole zoo collapses onto a few axes: does it carry **data or just a notification**, how **fast** is it, how much **synchronization complexity** does it impose, and does it work **across a common ancestor / across the host / across the network**.

Mechanism	Carries	Throughput	Latency	Sync burden	Reach
Signal	Notification only (a number)	n/a	Low	You must be async-signal-safe	Any process (perm-checked)
Pipe (anon)	Byte stream	Medium	Low	None (kernel-serialized)	Related processes
FIFO	Byte stream	Medium	Low	None	Any process on host
Unix domain socket	Byte/message stream + FDs + creds	High	Low	None	Any process on host

Mechanism	Carries	Throughput	Latency	Sync burden	Reach
Message queue	Discrete messages (priority)	Medium	Low	None	Any process on host
Shared memory	Raw memory	Highest	Lowest	You own all of it	Processes mapping it
eventfd/timerfd	Notification (counter/timer)	n/a	Very low	None	Related / passed FD
Network socket (TCP)	Byte stream	Network-bound	Network RTT	None	Any host

```

quadrantChart
  title IPC by role (data vs notification) and speed
  x-axis "Notification" --> "Bulk data"
  y-axis "Slower / more copies" --> "Faster / zero-copy"
  "Shared memory": [0.9, 0.95]
  "Unix domain socket": [0.75, 0.6]
  "Pipe / FIFO": [0.6, 0.5]
  "Message queue": [0.7, 0.45]
  "eventfd/timerfd": [0.15, 0.8]
  "Signal": [0.08, 0.55]
  "Network socket": [0.75, 0.2]

```

How to choose, in practice:

- **Need to notify, not transfer?** Signal (for lifecycle/control from outside) or `eventfd / timerfd` (for in-app/event-loop wakeups). Don't stream data through signals.
- **Simple one-way stream between related processes?** A pipe. It is the least code and has automatic backpressure.
- **General local IPC between two services on a host? Unix domain socket.** It is the default answer: familiar API, fast, bidirectional, FD passing, peer credentials, and a trivial upgrade path to TCP if the peer ever moves to another host.

- **Absolute lowest latency / highest throughput, and you can afford the concurrency work?** Shared memory plus explicit synchronization.
- **The peer might be on another machine?** A network socket (Volume 3) — and now you are in distributed-systems territory with partial failure, retries, and timeouts.

Distributed-systems lens

Signals and IPC feel like single-machine minutiae, but they are load-bearing for fleet-scale reliability in three concrete ways.

Signals are the container lifecycle protocol, and `SIGTERM` handling is mandatory for zero-downtime deploys. At any real deploy frequency — many services, continuous delivery, autoscaling that constantly creates and destroys pods — termination is not an edge case; it is a routine, high-frequency event. A service that ignores `SIGTERM` (or whose PID 1 swallows it) turns *every* rollout, scale-down, and node drain into a burst of dropped in-flight requests and reset connections. That shows up as elevated error rates correlated with deploys — one of the most common and most fixable availability regressions in production. Graceful shutdown with connection draining (Volume 11 — Reliability) is a distributed-systems pattern whose root cause, when it's broken, is almost always a missing or wrong signal handler.

PID 1 semantics are a fleet-wide gotcha. The kernel's special treatment of PID 1 — no default actions, unkillable within its namespace — means the *shape* of your container's entry point (shell form vs exec form, `app-as-init` vs a real `init`) silently determines whether shutdown works. Standardizing on exec-form entry points, `tini/--init` where appropriate, and signal-aware application code is a fleet-level hygiene decision, not a per-service afterthought. It also ties back to Chapter 1: PID 1 is the zombie reaper, and a container whose `init` doesn't reap leaks PIDs until it hits the namespace limit.

Local IPC vs network RPC is the same locality decision that governs system design. The gap between a Unix domain socket (sub-microsecond, no partial failure, same fate as the peer) and a network RPC (millisecond RTT, retries, timeouts, the peer can vanish independently) is enormous — orders of magnitude in latency and a categorical difference in failure model. It is the same local-vs-remote

trade-off from Volume 1: co-located processes (an app and its sidecar in one pod, a service and its local cache) should use fast local IPC — UDS or shared memory — while genuinely distributed components pay the network cost and inherit the network's failure modes (Volume 3 — The Linux Network Stack). Recognizing which side of that boundary two components sit on, and matching the IPC mechanism to it, is a recurring architectural judgment. Putting a network hop where a UDS would do wastes latency and adds failure modes; putting shared memory where you actually needed a network boundary couples two things that should have failed independently.

Finally, the **"everything is a file descriptor"** model is what lets all of this live in one event loop. `signalfd` for lifecycle, `timerfd` for deadlines, `eventfd` for wakeups, UDS and pipes for data — every one of them is a pollable fd, so the same `epoll` reactor from Chapter 7 that drives your high-concurrency request handling also drives your signal handling, your timeouts, and your inter-thread coordination. The uniformity is not an aesthetic nicety; it is what makes a correct, single-threaded-core, high-concurrency service tractable to write.

Key takeaways

- **A signal is an asynchronous notification, not a data channel.** It carries a number and interrupts the target at an arbitrary instruction. Use signals for control and lifecycle; use pipes/sockets/shared memory for data.
- **SIGKILL and SIGSTOP cannot be caught, blocked, or ignored.** Everything about graceful shutdown is the difference between the catchable `SIGTERM` (a request) and the uncatchable `SIGKILL` (a command). The OOM killer uses `SIGKILL` — bound your memory.
- **Signal handlers may call only async-signal-safe functions.** `malloc`, `printf`, and most of `libc` are off-limits because a handler can interrupt them mid-mutation. Do nothing in the handler but set a `volatile sig_atomic_t` flag or write one byte; do the real work in normal code. `signalfd` / the self-pipe trick make signals into pollable fds and sidestep the whole minefield.
- **Handle SIGTERM and drain — every backend service, no exceptions.** Stop accepting, finish in-flight, close connections cleanly, flush, exit within the grace period. Kubernetes' `SIGTERM` →

`terminationGracePeriodSeconds` → `SIGKILL` sequence makes this the mechanism of zero-downtime rollouts; account for the async endpoint-removal race with a `preStop` delay or a post-`SIGTERM` serve window.

- **PID 1 is special: no default signal actions, unkillable in its namespace.** An app or shell as container PID 1 that doesn't handle/forward `SIGTERM` gets hard-killed on every deploy. Use `exec-form CMD`, handle `SIGTERM`, and/or run a real init (`tini / --init`) that forwards signals and reaps zombies.
- **Pipes are unidirectional byte streams wired with `fork + dup2`;** close unused ends or you hang on missing EOF. **`SIGPIPE` kills you when you write to a closed peer** — ignore it and handle `EPIPE` (managed runtimes do this for you).
- **Unix domain sockets are the local-IPC default:** sockets API without the network stack, plus FD passing (`SCM_RIGHTS`) and unforgeable peer credentials. They are the backbone of the cloud-native local stack (Docker socket, local DB, sidecars).
- **Shared memory is the fastest IPC and moves the entire concurrency burden onto you.** Reach for it only when the throughput justifies owning synchronization and crash recovery.
- **Local IPC vs network RPC is the locality decision.** Co-located → fast local IPC; distributed → pay the network and its failure model. Match the mechanism to the boundary.

Further reading

- **man pages** (the authoritative Linux reference): `signal(7)` (the signal model, standard vs real-time, default actions), `signal-safety(7)` (the async-signal-safe function list), `sigaction(2)`, `sigprocmask(2)` / `pthread_sigmask(3)`, `signalfd(2)`, `eventfd(2)`, `timerfd_create(2)`, `pipe(7)`, `fifo(7)`, `unix(7)` (`AF_UNIX`, abstract namespace, `SCM_RIGHTS`, `SO_PEERCRED`), `mq_overview(7)`, `sem_overview(7)`, `shm_overview(7)`, `memfd_create(2)`, and `credentials(7)` (PID 1 / `SIGNAL_UNKILLABLE` behavior via `pid_namespaces(7)`).
- **The Linux Programming Interface**, Michael Kerrisk (No Starch Press, 2010), chapters 20–22 (signals: fundamentals, handlers, advanced), 44 (pipes and FIFOs), 46–48 (System V IPC, shared

memory), 52–53 (POSIX message queues, semaphores), and 57 (Unix domain sockets) — the precise, authoritative treatment of everything in this chapter.

- **Kubernetes documentation — "Pod Lifecycle"** (kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/), specifically the termination section on `terminationGracePeriodSeconds` , `preStop` hooks, and the `SIGTERM` → `grace` → `SIGKILL` sequence — the primary source for the graceful-shutdown flow.
- **"Graceful shutdown in Kubernetes"** writeups from the Kubernetes blog and from Google Cloud/learnk8s — the practical treatments of the endpoint-removal race and the `preStop` -sleep mitigation.
- **tini** (github.com/krallin/tini) and **dumb-init** (github.com/Yelp/dumb-init) READMEs — clear explanations of why a container needs a signal-forwarding, zombie-reaping PID 1, and Docker's `--init` documentation.
- **W. Richard Stevens & Stephen Rago, *Advanced Programming in the UNIX Environment* (APUE), 3rd ed.** — chapters 10 (signals) and 15 (IPC: pipes, FIFOs, message queues, semaphores, shared memory), the classic reference.
- **W. Richard Stevens, *UNIX Network Programming, Volume 2: Interprocess Communications*, 2nd ed.** — the definitive deep dive on pipes, message queues, semaphores, shared memory, and FD passing over Unix domain sockets.
- **nginx documentation — "Controlling nginx"** (nginx.org/en/docs/control.html) — the canonical real-world signal protocol: `SIGHUP` reload, `SIGUSR1` log reopen, `SIGUSR2` / `SIGWINCH` binary upgrade and graceful worker shutdown.
- **Go standard library:** `os/signal` (`signal.NotifyContext`), `net/http` (`Server.Shutdown`), and the runtime's `SIGPIPE` handling notes — a

production model of correct signal + graceful- shutdown handling to read as reference.

Chapter 9 — Namespaces, cgroups, and Container Internals

What this chapter covers. You deploy in containers every day. You write a Dockerfile, push an image, and a scheduler runs it on a node you never see. This chapter dismantles that abstraction and shows you the machinery underneath, because the machinery is not exotic — it is a handful of Linux kernel features you have already met in this volume, wired together. The single most important idea in the chapter is this: **a container is not a lightweight virtual machine**. It has no separate kernel, no virtual hardware, no hypervisor. A container is an ordinary Linux *process* (Chapter 1) that the kernel has been told to lie to. It is a process running with (a) **namespaces** that restrict what it can *see*, (b) **cgroups** that limit what it can *use*, (c) **capabilities, seccomp, and LSMs** that restrict what it can *do*, and (d) a **private root filesystem** assembled from image layers. There is no `container` object in the kernel. As the saying goes, *containers are a feature of the Linux kernel, not a thing*.

Once you internalize that, container behavior stops being magic. A pod that gets `OOMKilled` is a process hitting a cgroup `memory.max` (Chapter 4). A "container" that leaks zombies is a PID-namespace init that forgot to reap (Chapter 1). `kubectl exec` dropping you into a running container is `setns` joining that process's namespaces. A "container escape" is a namespace, cgroup, or capability boundary failing. This chapter is the *mechanism* behind everything Book 6 (Cloud-Native Supply Chain Security) says about container *security*, and behind everything Volume 12 says about container *operations*.

Learning goals — after this chapter you should be able to:

- State precisely what a container is in terms of kernel primitives, and explain why "it's a lightweight VM" is wrong and why that wrongness matters for isolation strength.

- Describe each namespace type — mount, PID, network, UTS, IPC, user, cgroup, time — and the global resource it virtualizes, and explain `clone / unshare / setns` and `/proc/[pid]/ns`.
- Explain PID-namespace nesting and why PID 1 inside a container must reap zombies and handle signals, tying back to Chapter 1 and Chapter 8.
- Explain user namespaces and UID/GID mapping as the basis of rootless containers, and why they are the most security-relevant namespace (Book 6).
- Deepen your cgroup knowledge from Chapters 2 and 4: the v2 unified hierarchy, the `cpu / memory / io / pids` controllers, delegation, and why cgroup accounting is the source of every container resource metric.
- Explain how an OCI image becomes a root filesystem via overlays (lower/upper/copy-up) and `pivot_root`.
- Walk the full sequence a runtime performs on `docker run / crictl run`, and place `runc / containerd / CRI-O / the CRI / Kubernetes` in one stack.

This chapter is Linux-specific. It builds directly on Chapter 1 (processes, `clone`, reaping), Chapter 2 (cgroup CPU control), Chapter 4 (cgroup memory and OOM), Chapter 5 (the syscall boundary, seccomp, capabilities), Chapter 8 (signals and IPC), and looks ahead to Chapter 10 (the network stack). It complements Volume 1 Chapter 10 (Hardware Support for Virtualization and Isolation) for the container-vs-VM boundary, Volume 3 (Networking) for CNI and service mesh, and Book 6 (Cloud-Native) for the security posture built on these primitives.

A container is a process, decorated

Start from the process. Chapter 1 defined a Linux process as a tuple: an address space, one or more threads, a set of resources (file descriptors, mounts, etc.), and metadata (PID, credentials, namespace membership). A container adds nothing to that list. It *configures* the existing fields. Specifically, running a container means launching a process whose:

- **namespace membership** points at fresh namespaces instead of the host's, so it sees its own process tree, its own mounts, its own network;

- **cgroup** is a leaf in the cgroup hierarchy with limits written into it;
- **credentials and capability set** have been narrowed, and a **seccomp** filter installed;
- **root directory** has been switched (`pivot_root`) to an overlay-assembled image filesystem.

That is the whole trick. Four decorations on a normal `task_struct`. The kernel schedules it with the same CFS you read about in Chapter 2, faults its pages through the same VM you read about in Chapter 3, and delivers its signals through the same machinery as Chapter 8. There is one kernel, shared by the host and every container on it. This is the anatomy to hold in your head:

```

flowchart TB
    subgraph host["Host kernel (ONE shared kernel)"]
        subgraph proc["Container = a normal process (task_struct)"]
            direction TB
            see["SEE – namespaces  
mount, PID, net, UTS, IPC, user, cgroup, time"]
            use["USE – cgroup v2 leaf  
cpu.max, memory.max, io.max, pids.max"]
            do["DO – capabilities dropped,  
seccomp-BPF filter, no_new_privs, LSM"]
            fs["ROOT FS – overlayfs (image layers)  
switched in via pivot_root"]
            end
            other["Other containers = other processes  
(their own namespaces + cgroups)"]
            kthreads["Host processes, kernel threads,  
scheduler, page cache, TCP/IP stack"]
            end
        end
        proc -. "same syscalls, same scheduler,  
same page tables, same kernel" .-> host
    
```

Contrast this with a virtual machine (Volume 1, Chapter 10). A VM runs a *second kernel* on *virtual hardware* presented by a hypervisor; the guest kernel's syscalls never touch the host kernel. That is a hardware-enforced boundary (Intel VT-x / AMD-V, a second level of page tables). A container's boundary is enforced entirely by *kernel software* — namespace lookups and permission checks in the same kernel the container's syscalls run against. That difference is the whole isolation-strength story, and we return to it at the end: a container escape needs only a kernel bug; a VM escape needs a hypervisor bug, a much smaller and more scrutinized surface. It is why untrusted, multi-tenant workloads increasingly run on

Namespaces: virtualizing what a process can see

A namespace virtualizes a *global* kernel resource so that processes inside the namespace see their own isolated instance of it. "Global" is the key word. Process IDs are global — there is one PID 1 on a normal system. Mount points are global. Network interfaces are global. A namespace takes one of these globals and gives a group of processes a private copy. Two processes in different PID namespaces can both be "PID 1" and never collide, because the PID namespace is the scope in which a PID is unique.

As of modern kernels (5.6+), there are eight namespace types:

Namespace	clone / unshare flag	Global resource virtualized	Since
Mount	CLONE_NEWNS	Mount points / filesystem view	2.4.19
UTS	CLONE_NEWUTS	Hostname and NIS domain name	2.6.19
IPC	CLONE_NEWIPC	SysV IPC objects, POSIX message queues	2.6.19
PID	CLONE_NEWPID	Process IDs (own process tree)	2.6.24
Network	CLONE_NEWNET	Interfaces, routes, sockets, netfilter	2.6.29
User	CLONE_NEWUSER	UID/GID number space, capabilities	3.8
Cgroup	CLONE_NEWCGROUP	The cgroup root directory view	4.6
Time	CLONE_NEWTIME	CLOCK_MONOTONIC / CLOCK_BOOTTIME offsets	5.6

```

flowchart LR
    subgraph global["Global kernel resources (host)"]
        g1["process tree"]
        g2["mount table"]
        g3["net stack"]
        g4["hostname"]
        g5["SysV IPC / mqueue"]
        g6["UID/GID space"]
        g7["cgroup tree"]
    end
    end
    subgraph ns["A container's private views"]
        n1["PID ns: own tree, own PID 1"]
        n2["mount ns: own root FS view"]
        n3["net ns: own eth0, lo, routes"]
        n4["UTS ns: own hostname"]
        n5["IPC ns: own shm/sem/mq"]
        n6["user ns: root inside = uid 1000000 outside"]
        n7["cgroup ns: sees its cgroup as /"]
    end
    end
    g1 --> n1
    g2 --> n2
    g3 --> n3
    g4 --> n4
    g5 --> n5
    g6 --> n6
    g7 --> n7

```

How namespaces are created and joined

There are three system calls, and you already met the first in Chapter 1:

- **clone(2)** — create a new process *and* place it in new namespaces in one shot, by ORing the `CLONE_NEW*` flags into the clone flags. This is how a runtime starts a container: the child is born already inside fresh namespaces. Recall Chapter 1's unifying insight — `fork`, `pthread_create`, and container creation are all `clone` with different flag bitmasks. A container is just a particular choice of flags.
- **unshare(2)** — move the *calling* process into new namespaces without creating a child. The `unshare(1)` command exposes this: `unshare --pid --net --mount --fork --map-root-user /bin/sh` gives you a shell in new PID, network, mount, and user namespaces. Great for poking at the mechanism by hand.
- **setns(2)** — *join* an existing namespace, given a file descriptor referring to it. This is how you enter a container that is already running.

Every namespace a process belongs to is exposed as a magic symlink under `/proc/[pid]/ns/`:

```
$ ls -l /proc/self/ns/
lrwxrwxrwx ... cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx ... ipc    -> 'ipc:[4026531839]'
lrwxrwxrwx ... mnt    -> 'mnt:[4026531840]'
lrwxrwxrwx ... net    -> 'net:[4026531992]'
lrwxrwxrwx ... pid    -> 'pid:[4026531836]'
lrwxrwxrwx ... user   -> 'user:[4026531837]'
lrwxrwxrwx ... uts    -> 'uts:[4026531838]'
```

The number in brackets is the namespace's inode; two processes in the *same* namespace have the same inode there. Opening one of these symlinks yields an FD you can hand to `setns`. That is exactly what `nsenter(1)` does — and exactly what `docker exec` and `kubectl exec` do under the hood. To "exec into" a container is to `open` its `/proc/[container-pid]/ns/*`, `setns` into each namespace, and then `execve` your shell. The shell is a brand-new process on the host that has *joined* the container's views. This is not a special code path; it is the same primitive a Kubernetes sidecar or an Istio/Envoy service-mesh proxy uses to *share* a pod's network namespace (Volume 3, Chapter 10; Book 6).

PID namespace: your process tree, and why PID 1 matters

A PID namespace gives a process group its own process-ID number space and its own process tree. The first process created in a new PID namespace becomes **PID 1 inside** that namespace. Two facts make PID namespaces the ones most likely to bite a backend engineer.

First, PID namespaces are **nested/hierarchical**. A process has a PID in its own namespace *and* a (different) PID in every ancestor namespace up to the host. Your container's PID 1 might be PID 1 inside and PID 48213 on the host. A process can see and signal processes in its own and descendant namespaces, never ancestor ones. This is why, from the host, you can `kill` a container process, but from inside, the container cannot even see the host's processes.

Second, **PID 1 inherits Unix init duties** (Chapter 1, Chapter 8). Two of them:

- **Reaping.** When any process's parent dies, it is re-parented to PID 1 of its namespace. PID 1 must `wait()` on these orphans or they accumulate as zombies. A naive container whose entrypoint is an

application that never reaps — say a shell script that `exec s` a server which spawns and abandons children — leaks zombies inside the container's PID table. This is the classic "why is my container full of zombies" bug. The fix is a real init as PID 1: `tini` (Docker's `--init`), `dumb-init`, or a language runtime that reaps.

- **Signal semantics.** PID 1 is special: the kernel does *not* apply default signal actions to it. A `SIGTERM` sent to PID 1 with no installed handler is *ignored*, not fatal. This is why a container running a shell as PID 1 often refuses to stop on `docker stop` and only dies when the 10-second grace period elapses and it gets `SIGKILL`. The application never saw the `SIGTERM` because the shell, as PID 1, dropped it. Correct graceful shutdown across a fleet depends on PID 1 forwarding signals — again, a real init or an app that installs handlers.
- If PID 1 in a namespace exits, the kernel sends `SIGKILL` to every other process in that namespace. The container dies with its init.

Mount namespace: the basis of a private root filesystem

The mount namespace virtualizes the set of mount points — the filesystem hierarchy a process sees. This is the namespace that makes a container's isolated root filesystem possible: inside its own mount namespace, a process can mount, unmount, and `pivot_root` without affecting the host or any other container. We cover the overlay-plus-`pivot_root` assembly in its own section below.

One subtlety that causes real production surprises is **mount propagation**. A mount point is `shared`, `private`, `slave`, or `unbindable`, and this controls whether mount/unmount events propagate between namespaces (`mount --make-rshared` and friends). Runtimes deliberately make the container's mounts `private/slave` so a mount inside a container does not leak to the host. When bind-mounting host paths in (volumes), propagation mode determines whether a later host mount under that path becomes visible inside — the source of "my volume shows an empty directory" bugs. See `mount_namespaces(7)`.

Network namespace: an isolated network stack

A network namespace is a complete, independent copy of the network stack: its own loopback, its own set of interfaces, its own routing table, its own `iptables/nftables` rules, its own socket port space. A freshly

created netns has only a down `lo`. To give a container real connectivity, the runtime (or a CNI plugin) creates a **veth pair** — a virtual Ethernet cable with two ends — leaves one end in the host namespace (attached to a bridge, e.g. `docker0`, or wired by the CNI) and *moves the other end into the container's netns*, where it becomes the container's `eth0`.

```
graph LR
    subgraph hostsns ["Host network namespace"]
        br["bridge (docker0 / cni0)"]
        vethA["veth (host end)"]
        ext["physical NIC to network"]
        br --- vethA
        br --- ext
    end
    subgraph ctrns ["Container network namespace"]
        eth0["eth0 (veth peer)"]
        lo["lo"]
    end
    vethA <== "veth pair (one cable, two ends)" ==> eth0
```

This is the entire basis of pod networking. In Kubernetes, all containers in a **pod share one network namespace** — they reach each other over `localhost` and share one IP — precisely because they are `setns`-joined into a common netns set up for the pod's "pause"/infra container. The CNI plugin (Calico, Cilium, flannel, ...) is the component that creates the veth, moves it into the pod netns, and programs routes; Volume 3 and Book 6 cover CNI in depth. When you debug "pod can't reach a service," you are debugging routing and netfilter *inside a network namespace* — enter it with `nsenter -t <pid> -n ip route`.

UTS and IPC namespaces

The **UTS namespace** ("UNIX Time-sharing System," a historical name) virtualizes just two things: the hostname and the NIS domain name. It is why `hostname` inside a container returns the container's name, and why setting it inside does not rename the host. Small but real — logging and service discovery often key off hostname.

The **IPC namespace** virtualizes System V IPC objects (shared memory segments, semaphores, message queues) and POSIX message queues (Chapter 8). Two containers in different IPC namespaces cannot see each other's `shmget` segments even at the same key. Pods that need shared-memory IPC between containers must share an IPC namespace (`shareProcessNamespace` / `hostIPC`-style options at the pod level).

User namespace: mapping UIDs, and the road to rootless

The user namespace is the most security-consequential, and the trickiest. It virtualizes the UID and GID number space *and* the capability set. Inside a user namespace, a process can be UID 0 (root) with a full capability set, while the kernel maps that UID to an *unprivileged* UID on the host. The mapping lives in `/proc/[pid]/uid_map` and `gid_map`:

```
# inside-uid  outside-uid  length
0             1000000      65536
```

This says "UID 0 inside == UID 100000 outside, for a range of 65536 IDs." A process that is root inside the container, if it somehow touches a host resource, acts with the *host* identity 100000 — an ordinary user who owns nothing important. Capabilities held inside the user namespace (Chapter 5) are only powerful *with respect to objects owned by that namespace*; `CAP_SYS_ADMIN` inside a user namespace does not let you do namespace-external privileged operations.

This is the foundation of **rootless containers**: a *non-root* host user can create a user namespace, become root *inside* it, and from there create the other namespaces (which normally require `CAP_SYS_ADMIN`) — because within the new user namespace they now hold that capability. Rootless Docker, Podman, and rootless Kubernetes are built on exactly this. The security payoff is large: a container breakout lands the attacker as an unprivileged host user, not host root (Book 6, least privilege and blast-radius reduction). The historical cost is that user namespaces themselves widened the kernel's attack surface (much privileged code became reachable by unprivileged users), which is why some hardened distros gate them.

cgroup and time namespaces

The **cgroup namespace** virtualizes the process's view of the cgroup hierarchy: it makes the container's own cgroup appear as the root (`/`) in `/proc/self/cgroup` and under `/sys/fs/cgroup`, so a container cannot see the host's cgroup paths above it. It virtualizes the naming, not the limits — the limits are the cgroups themselves, next section.

The **time namespace** (5.6+) lets a namespace offset `CLOCK_MONOTONIC` and `CLOCK_BOOTTIME` (not wall-clock `CLOCK_REALTIME`). Its main purpose is

making checkpoint/restore (CRIU) work — a restored container can keep a consistent monotonic clock — rather than everyday isolation.

cgroups: limiting what a process can use

Namespaces control what a process *sees*; **cgroups** (control groups) control what it can *use* and *account for how much it used*. You met cgroups already: Chapter 2 for CPU (shares/weight, quota, throttling) and Chapter 4 for memory (`memory.max/high/min`, cgroup OOM). Here we tie them into the container picture and cover the pieces those chapters did not.

A cgroup is a node in a hierarchy of process groups. Each cgroup has *controllers* attached (cpu, memory, io, ...), and each controller exposes *interface files* — pseudo-files under `/sys/fs/cgroup` you read for accounting and write to set limits. Processes are assigned to a cgroup; the controller enforces limits on the group as a whole.

cgroup v1 vs v2: the unified hierarchy

There are two generations, and the difference matters operationally.

cgroup v1 had a *separate hierarchy per controller*. You mounted `cpu`, `memory`, `blkio`, etc. as independent trees, and a process could sit at different positions in each. This was flexible and a mess: no coherent way to see a group's *combined* resource picture, and awkward interactions (the memory and I/O controllers could not cooperate for writeback accounting).

cgroup v2 replaced this with a **single unified hierarchy**: one tree, and every controller attaches to the same set of cgroups. A process belongs to exactly one cgroup, full stop. Key v2 rules and files:

- `cgroup.controllers` — controllers available in this cgroup;
`cgroup.subtree_control` — which of them are enabled for *children* (you enable a controller downward with `+cpu +memory`).
- The **"no internal processes" rule**: a cgroup that has controllers enabled for its children may not itself contain processes (except the root). Processes live in leaves. This is what makes the accounting coherent.

- Pressure Stall Information (`cpu.pressure` , `memory.pressure` , `io.pressure`) — the PSI signals Chapter 4 recommended alerting on — are a v2 feature.

cgroup v2 is the modern default (systemd, all current distros, Kubernetes 1.25+ defaults to it; the in-tree cgroup-v1 support is deprecated). If you still see per-controller directories under `/sys/fs/cgroup/cpu/` , `/memory/` , etc., you are on v1 (or hybrid); a single unified tree with `cgroup.controllers` at the root means v2.

Concern	cgroup v1	cgroup v2
Hierarchy	One tree <i>per controller</i>	One <i>unified</i> tree
Process placement	Different per controller	Exactly one cgroup
CPU limit	<code>cpu.cfs_quota_us</code> / <code>cpu.cfs_period_us</code> , <code>cpu.shares</code>	<code>cpu.max</code> (quota + period), <code>cpu.weight</code>
Memory limit	<code>memory.limit_in_bytes</code>	<code>memory.max</code> , <code>memory.high</code> , <code>memory.low</code> , <code>memory.min</code>
I/O limit	<code>blkio.throttle.*</code>	<code>io.max</code> , <code>io.weight</code>
PID limit	<code>pids.max</code>	<code>pids.max</code>
Pressure (PSI)	Not available	<code>cpu/memory/io.pressure</code>
Status	Deprecated	Default

The controllers that build a container's resource envelope

```

flowchart TB
    root["/sys/fs/cgroup (v2 root)"]
    root --> ks["kubepods.slice"]
    root --> sys["system.slice (host daemons)"]
    ks --> g["kubepods-guaranteed.slice"]
    ks --> bu["kubepods-burstable.slice"]
    bu --> pod["pod.slice"]
    pod --> c1["cri-containerd-.scope"]
    pod --> c2["cri-containerd-.scope (sidecar container)"]
    subgraph "cpu.max, memory.max, io.max, pids.max"
        pod --> c2
    end

```

- **cpu** — proportional sharing via `cpu.weight` (v2; default 100, range 1-10000; maps from v1 `cpu.shares` where $1024 \approx \text{weight } 100$), and a hard ceiling via `cpu.max` = "<quota> <period>" in microseconds. `cpu.max` = "50000 100000" means 50 ms of CPU per 100 ms window = half a core; exceed it and the group is **throttled** — runnable tasks are stalled until the next period. This is Kubernetes CPU *limits*, and the source of the notorious CFS-throttling tail-latency problem Chapter 2 dissected: a bursty, latency-sensitive service pinned under a low quota gets throttled mid-request. `cpu.stat` reports `nr_throttled` and `throttled_usec` — the numbers behind your throttling dashboards. Requests, by contrast, become `cpu.weight`.
- **memory** — `memory.max` is the hard wall: exceed it and the cgroup OOM killer fires, killing a process *inside the cgroup* (the Kubernetes `OOMKilled`, exit code 137 of Chapter 4), while the rest of the node is fine. `memory.high` throttles and reclaims without killing; `memory.min` / `low` protect a working set. `memory.current` and `memory.stat` (anon vs file, etc.) are the accounting Chapter 4 taught you to read.
- **io** — `io.max` throttles block-I/O bandwidth and IOPS per device (`io.max`: "8:0 rbps=10485760 wiops=1000"), and `io.weight` does proportional sharing. This is how a noisy-neighbor container is prevented from saturating a shared disk. v2's unified hierarchy is what finally let I/O throttling account cache writeback correctly.
- **pids** — `pids.max` caps the number of tasks (processes + threads) in the cgroup. `pids.max` = 1024 is **fork-bomb protection**: a runaway `() { :|:& };;` inside the container hits the cap and can no longer `clone`, protecting the host's global PID space. Cheap, and every serious deployment sets it.

Accounting is the source of all container metrics

Every "container CPU," "container memory," "container throttling," and "container I/O" number you have ever seen on a dashboard comes from reading these cgroup interface files. cAdvisor (built into the kubelet), `docker stats`, and `node exporters` read `cpu.stat`, `memory.current`, `memory.stat`, `io.stat`, and `pids.current` for each container's cgroup and emit them as metrics (Volume 11, observability). There is no separate "container telemetry" subsystem — the cgroup is the meter. When two tools disagree on a container's memory, they are almost always disagreeing about which `memory.stat` line to sum (anon + file? include kernel? include tmpfs?). Knowing the cgroup files makes those arguments resolvable.

Delegation and systemd

On a systemd host, systemd owns the cgroup v2 root and organizes it into **lices** (groups), **scopes** (externally-created process groups), and **services**. A container runtime does not scribble directly into the root; it is **delegated** a subtree (systemd's `Delegate=yes`) and manages cgroups within it. This is why Kubernetes cgroups sit under `kubepods.slice` and why `systemd` is the recommended cgroup driver for the kubelet and containerd — two managers writing the same tree without coordination corrupt each other's bookkeeping. `systemd-cgls` and `systemctl status` show you the tree.

The container root filesystem: overlays and pivot_root

We have isolated what the process sees and capped what it uses. Now it needs a filesystem — a `/` that is the container image, not the host. Two mechanisms combine: **overlays** builds the root filesystem cheaply from image layers, and **pivot_root** switches the process onto it.

OCI images are stacks of layers

An OCI image (Book 6, Chapter 1) is an ordered stack of *layers*, each a tarball of filesystem changes relative to the one below. Layer 0 might be a minimal distro, layer 1 adds your runtime, layer 2 your app. Layers are content-addressed and shared: a hundred containers from the same base image store that base *once* on disk. But a running container needs a

writable root where it can create temp files and logs, without mutating the shared read-only layers. That is exactly the problem a **union filesystem** solves.

overlays: lower (read-only) + upper (writable) → merged

Linux's `overlayfs` unions directories into a single view:

- **lowerdir** — one or more read-only layers, stacked (the image layers). Multiple lowers are colon-separated; upper layers in the string shadow lower ones.
- **upperdir** — a single writable directory (the container's private scratch layer).
- **workdir** — an empty scratch directory `overlayfs` needs internally for atomic operations.
- **merged** — the unified mountpoint the container sees as `/`.

```
flowchart TB
    subgraph image["Read-only image layers (shared across containers)"]
        l0["lower: base distro (/bin, /lib, /etc)"]
        l1["lower: runtime + libs"]
        l2["lower: app files"]
    end
    up["upper: writable layer (this container only)"]
    merged["merged = container root '/'"]
    (what the process sees)
    l0 --> merged
    l1 --> merged
    l2 --> merged
    up --> merged
```

```
mount -t overlay overlay \
-o lowerdir=/layers/app:/layers/runtime:/layers/base,\
upperdir=/containers/abc/upper,workdir=/containers/abc/work \
/containers/abc/merged
```

Reads resolve top-down: the file is served from the highest layer that has it. **Writes** trigger **copy-up**: the first time the container modifies a file that lives in a lower (read-only) layer, `overlayfs` copies the whole file up into `upperdir`, then applies the write there. The lower layer is untouched, so other containers sharing it are unaffected. This is copy-on-write at file granularity, and it has a real cost: modifying one byte of a large file in a base layer copies the *entire* file up. Write-heavy workloads that mutate

big image files should use a **volume** (a bind mount to a real filesystem) instead of the overlay upper — which is why databases in containers put their data on volumes.

Deletions in the upper layer are recorded as *whiteouts* (a character device with major/minor 0/0) that mask the lower file; removing an entire directory that exists below uses an *opaque* directory marker. This is why `rm` inside a container does not shrink the image — it adds a whiteout to the upper layer.

pivot_root switches the process onto the new root

With the merged directory built, the runtime must make it the process's `/`. Inside the container's *mount namespace*, it uses `pivot_root(new_root, put_old)`, which moves the root mount to `new_root` and stashes the old root at `put_old` so it can be unmounted and detached. Runtimes prefer `pivot_root` over the older `chroot` because `chroot` changes only the root *directory* (not the root *mount*) and has well-known escape techniques (an attacker with an open fd on a directory outside the jail, or with `CAP_SYS_CHROOT`, can climb out). `pivot_root`, followed by unmounting the old root, leaves the container with *no reference at all* to the host filesystem — a strictly stronger boundary. After the pivot the runtime mounts the container's `/proc` (a fresh proc for the new PID namespace), `/sys`, a `tmpfs` `/dev` with a minimal device set, and any **bind-mounted volumes** the spec requested.

The full pipeline: **image (manifest + config) → layer tarballs unpacked into content-addressed directories (containerd calls these *snapshots*) → overlayfs mount stacking them as lowers plus a fresh upper → pivot_root into the merged view → mount `/proc`, `/sys`, `/dev`, volumes**. Book 6, Chapter 1 covers the image side; this is the runtime side.

Restricting what a process can do: capabilities, seccomp, LSMs

Isolation of *see* and *use* is not enough; a container process still makes syscalls against the shared kernel, and some syscalls are dangerous. The

third leg narrows what the process may *do*. Chapter 5 covered these primitives; here is how the runtime wields them.

- **Capabilities.** Root's monolithic power is split into ~40 capabilities (`capabilities(7)`). A container runtime drops nearly all of them. Docker's default keeps a small set (roughly 14 — `CAP_CHOWN` , `CAP_NET_BIND_SERVICE` , `CAP_SETUID` , etc.) and drops the dangerous ones: `CAP_SYS_ADMIN` (the "new root," which alone enables mount, and much namespace manipulation), `CAP_SYS_MODULE` (load kernel modules), `CAP_SYS_PTRACE` , `CAP_NET_ADMIN` , and more. `--privileged` restores *all* capabilities and drops most other restrictions — effectively unconfined, and a frequent escape vector (Book 6).
- `no_new_privs` . A one-way `prctl` flag: once set, `execve` can never grant more privileges (no `setuid` escalation). Runtimes set it so a container process cannot regain privilege it was denied.
- **seccomp-BPF.** A classic-BPF program the kernel runs on *every* *syscall*, deciding allow/deny/errno/kill (Chapter 5). The default Docker/containerd profile blocks ~40+ rarely needed and dangerous syscalls (`keyctl` , `add_key` , `ptrace` in some configs, obscure and legacy calls, `mount` , `reboot` , `kexec_load` , ...), drastically shrinking the kernel attack surface a container can reach. Kubernetes exposes this as `seccompProfile: RuntimeDefault` .
- **LSMs.** AppArmor (Debian/Ubuntu) or SELinux (RHEL) apply a Mandatory Access Control profile on top, confining file, capability, and network access by policy. Containers get a default profile; hardened deployments write tighter ones (Book 6).

Primitive	Restricts	Container default	If it fails / is disabled
Namespaces	What you <i>see</i>	Own PID/mnt/net/uts/ipc/user	Escape: see/affect host resources
cgroups	What you <i>use</i>	cpu/memory/io/pids capped	DoS: exhaust host CPU/RAM/PIDs
Capabilities	Privileged ops	Most dropped	Mount, load modules, ptrace host
seccomp	Syscall surface	~40+ blocked	Reach kernel bugs in blocked calls

Primitive	Restricts	Container default	If it fails / is disabled
user ns	Privilege mapping	root→unprivileged (rootless)	Breakout lands as host root
LSM (AppArmor/SELinux)	MAC policy	Default profile	Broader file/network access

Notice the pattern the table encodes: **each container primitive maps to a specific security property, and each failure mode is a specific escape or DoS**. This is the exact seam where this chapter (the mechanism) meets Book 6 (the security posture). "Container hardening" is nothing more than tightening each row: drop more capabilities, restrict seccomp, enable user namespaces, apply a strict LSM profile, set cgroup limits.

Putting it together: what a container runtime actually does

Now assemble the pieces into the sequence a runtime executes when you run a container. The industry standard is the **OCI Runtime Specification**: a *bundle* is a directory containing a root filesystem plus a `config.json` describing the process, namespaces to create, cgroup limits, mounts, capabilities, seccomp profile, and so on. **runc** is the reference OCI runtime — a small Go/C program (built on `libcontainer`) that reads `config.json` and performs the syscalls. Roughly, `runc create/start` does:

```

flowchart TB
    a["Read OCI config.json (the bundle spec)"]
    b["clone/unshare: create namespaces  
(user first, then PID/mnt/net/uts/ipc/cgroup)"]
    c["Write UID/GID maps (user ns)"]
    d["Create cgroup leaf, write limits  
cpu.max, memory.max, io.max, pids.max"]
    e["Mount overlay rootfs, then pivot_root  
mount /proc, /sys, tmpfs /dev, bind volumes"]
    f["Set hostname (UTS), apply sysctls"]
    g["Attach network: CNI/runtime moves veth into netns"]
    h["Drop capabilities, set no_new_privs,  
install seccomp filter, apply LSM label"]
    i["execve the entrypoint – becomes PID 1 inside"]
    a --> b --> c --> d --> e --> f --> g --> h --> i

```

A few precise points about that sequence:

- The **order is load-bearing and security-critical**. The user namespace is set up first so everything after runs with the mapped identity. Capabilities are dropped and the seccomp filter installed *just before* `execve`, so the setup work (which needs privilege) can run but the application starts already confined. `runc` uses a small C helper (`nsexec`) that runs before the Go runtime, because some namespace transitions (especially PID and user) must happen at a precise moment relative to `fork`.
- **`runc` creates the empty network namespace, but does not usually wire it**. Network setup is delegated: Docker's `libnetwork`, or in Kubernetes the **CNI** plugin, creates the veth pair and moves one end into the netns (Volume 3, Book 6). This separation is why CNI is pluggable.
- `runc` is **not a daemon**. It runs, does the setup, `execve`s the container process, and exits. The container process is now a normal host process (with all its decorations). Something must stay behind to hold the container's stdio, watch for its exit, and let `containerd` reconnect after a restart — that is the **shim**.

The stack: Kubernetes → CRI → containerd → runc → kernel

Real deployments layer higher-level runtimes over `runc`:

```

flowchart TB
    k["kubectl / kube-apiserver"]
    kubelet["kubelet (on each node)"]
    cri["CRI (gRPC): RunPodSandbox, CreateContainer, ..."]
    ctr["containerd (or CRI-O)"]
    image_pull["image pull, snapshots (overlay), lifecycle"]
    shim["containerd-shim-runc-v2 (one per container)"]
    runc["runc (OCI runtime): the actual syscalls"]
    kern["Linux kernel: namespaces, cgroups, overlayfs, seccomp, capabilities, veth"]

    k --> kubelet
    kubelet --> cri
    cri --> ctr
    ctr --> shim
    shim --> runc
    runc --> kern

```

- **runc** — the OCI runtime; does the low-level syscall work above. Alternatives: **crun** (C, faster startup), **gVisor's runsc** (user-space kernel), **Kata's runtime** (each container in a real microVM) — drop-in OCI runtimes with different isolation strength.
- **containerd** — a daemon that manages the full container lifecycle above runc: pulling images from registries, unpacking layers into **snapshots** (the overlay lowers), managing the shim, streaming logs, exposing metrics. **CRI-O** is a functionally similar daemon built specifically for Kubernetes.
- **containerd-shim-runc-v2** — one shim process per container. It is the container's real parent for stdio and exit-status purposes; crucially, it *survives containerd restarts*, so upgrading containerd does not kill running containers. The shim reaps the container and reports its exit code up.
- **CRI (Container Runtime Interface)** — the gRPC contract between the kubelet and the runtime (`RunPodSandbox`, `CreateContainer`, `StartContainer`, ...). It is what makes the kubelet runtime-agnostic and is why Docker's own daemon was removed from the kubelet ("dockershim" removal in Kubernetes 1.24) — Kubernetes talks CRI to containerd/CRI-O directly.
- **kubelet / Kubernetes** — turns pod specs into CRI calls: create a pod sandbox (the shared network/IPC namespaces and the "pause" container that holds them), then create each container joined into that sandbox.

So a pod is: a set of containers (processes) sharing a network and IPC namespace, each in its own PID and mount namespace, each a cgroup leaf under the pod's slice, each with dropped capabilities and a seccomp filter, each rooted in its own overlay filesystem — orchestrated by a chain

of daemons that ultimately call the same `clone`, `mount`, `pivot_root`, and `cgroup` writes you could type by hand. Every layer above the kernel is convenience; the isolation is all in the bottom box.

Distributed-systems lens

Containers are the fleet's **unit of deployment**, and almost every operational surprise at scale is one of these primitives showing through.

- **Resource limits are cgroups, and their failures are cgroup failures.** A pod `OOMKilled` (exit 137) is `memory.max` enforcement in one cgroup, not a sick node (Chapter 4). Mysterious p99 latency on a service that looks CPU-idle in aggregate is almost always **CFS throttling** from `cpu.max` (Chapter 2) — `cpu.stat`'s `nr_throttled` proves it. A node that stops scheduling new work may be out of PIDs because someone forgot `pids.max`. You cannot reason about any of these without knowing they are cgroup controllers.
- **Shared-kernel isolation sets the security ceiling.** Every container on a node trusts the *same* kernel. A kernel privilege-escalation bug (historically Dirty COW, CVE-2016-5195, or Dirty Pipe, CVE-2022-0847) or a runtime bug (runc's `/proc/self/exe` overwrite, CVE-2019-5736) can breach the boundary that namespaces and capabilities draw. That is the whole reason a fleet running *untrusted* or strongly multi-tenant workloads reaches for **microVMs** (Firecracker, Kata) or **user-space kernels** (gVisor): a second, hardware- or software- enforced boundary that a mere kernel bug does not cross (Volume 1 Chapter 10, Book 6). The container-vs-microVM decision is exactly a decision about how much you trust the shared kernel.
- **The runtime stack is the fleet's execution substrate.** Kubernetes → CRI → containerd → shim → runc → kernel is the path every workload takes to run. Knowing where each layer's responsibility begins and ends is what lets you diagnose "image pulls fail" (containerd / registry), "container won't start" (runc / OCI config / seccomp), or "pod has no network" (CNI / netns) instead of blaming "Kubernetes."
- **setns is how the mesh and sidecars work.** A service-mesh data-plane proxy (Envoy) shares the pod's *network namespace* so it can transparently intercept the app's traffic; a debugging ephemeral

container `setns` joins a running pod. Sidecars, `kubectl debug`, and mesh injection are all the same `setns /shared-namespace` primitive (Volume 3 Chapter 10, Book 6).

- **User namespaces and capability-dropping bound blast radius.** Rootless containers and aggressive capability drops mean a breakout lands an attacker as an unprivileged host user with nothing, not as host root — the least-privilege posture Book 6 argues for, implemented in the UID map and the capability bounding set.
- **cgroup accounting is your telemetry.** Every container CPU/memory/IO/throttle metric your fleet emits is a read of a cgroup interface file by cAdvisor or an exporter (Volume 11). The cgroup is the meter; the dashboard is a rendering of `memory.stat` and `cpu.stat`.

The payoff of this whole chapter is demystification. Containers are the concrete primitives under everything the cloud-native books discuss. Once you see a container as *a process with namespaces, cgroups, capabilities, and an overlay root*, its behaviors, limits, and failure modes stop being properties of a black box and become properties of Linux features you can inspect, reason about, and debug directly.

Key takeaways

- **A container is a process, not a VM.** It is an ordinary Linux `task_struct` decorated with namespaces (what it *sees*), cgroups (what it *uses*), capabilities/seccomp/LSM (what it *does*), and an overlay root filesystem. There is one shared kernel and no `container` object in it — "containers are a kernel feature, not a thing."
- **Namespaces virtualize global resources.** Eight types (mount, PID, net, UTS, IPC, user, cgroup, time) each give a process group a private view of one global resource, created by `clone / unshare` flags, joined via `setns` (the mechanism behind `docker exec`, `kubectl exec`, sidecars, and mesh proxies), and visible under `/proc/[pid]/ns/`.
- **PID 1 inside a container has init duties.** It must reap orphans (or leak zombies) and forward/handle signals (or `docker stop` hangs to `SIGKILL`); the kernel ignores default signal actions for PID 1. Use a real init (`tini`, `dumb-init`) if your app does not reap.

- **User namespaces map root inside to unprivileged outside** — the basis of rootless containers and the biggest lever for shrinking breakout blast radius (Book 6).
- **cgroup v2's unified hierarchy is the modern default.** `cpu.max / cpu.weight` (throttling and shares), `memory.max / high` (the OOMKilled wall and soft throttle), `io.max`, and `pids.max` (fork-bomb protection) are the controllers that build a container's resource envelope — and their interface files are the source of *every* container resource metric.
- **The root filesystem is overlays plus `pivot_root`.** Read-only image layers as `lowerdir`, a per-container writable `upperdir`, merged into one view; writes trigger file-granular copy-up; `pivot_root` (stronger than `chroot`) switches the process onto it and detaches the host root.
- **A runtime just calls these syscalls in order.** `runc` reads the OCI `config.json` and does clone-with-namespaces → user maps → cgroup limits → overlay + `pivot_root` → drop caps + `seccomp` → exec, with networking delegated to CNI. `containerd/CRI-O` + the shim + the CRI + the kubelet layer convenience on top; the isolation is all at the bottom.
- **Shared kernel is the isolation ceiling.** A kernel or runtime bug can breach namespace and capability boundaries, which is why untrusted multi-tenant workloads move to microVMs or user-space kernels (Volume 1 Chapter 10). Hardening a container = tightening each primitive.

Further reading

- **Linux man pages** — the primary, authoritative source: `namespaces(7)`, `pid_namespaces(7)`, `mount_namespaces(7)`, `network_namespaces(7)`, `user_namespaces(7)`, `cgroups(7)`, `clone(2)`, `unshare(2)`, `setns(2)`, `pivot_root(2)`, `capabilities(7)`, `seccomp(2)`, and `overlays` in the kernel docs. `nsenter(1)` and `unshare(1)` for hands-on exploration.
- **The Linux Programming Interface**, Michael Kerrisk, No Starch Press, 2010 — Chapters on process creation, capabilities, and (in later material) namespaces; Kerrisk's LWN namespace series

("Namespaces in operation," lwn.net) is the canonical readable walkthrough of each type.

- **Linux kernel documentation:** `Documentation/admin-guide/cgroup-v2.rst` (the definitive cgroup v2 reference — controllers, interface files, the no-internal-processes rule, delegation, PSI) and `Documentation/filesystems/overlayfs.rst` (lower/upper/work, copy-up, whiteouts).
- **OCI specifications** (opencontainers.org): the **Runtime Specification** (the `config.json` bundle format runc consumes), the **Image Specification** (layers, manifest, config), and the **Distribution Specification** — the standards the whole ecosystem implements. Pair with Book 6, Chapter 1 (Container Images: OCI Format, Layers, and Attack Surface).
- **runc**, **containerd**, and **CRI-O** project documentation and source (`github.com/opencontainers/runc`, `containerd/containerd`, `cri-o/cri-o`) — `libcontainer` and `nsexec.c` in runc are worth reading to see the exact syscall order; containerd's architecture docs explain snapshots and the shim.
- **Kubernetes documentation:** "Container Runtime Interface (CRI)," the dockershim-removal notes (1.24), and the "Configure a Security Context for a Pod or Container" and "Seccomp" guides — how pod specs become the primitives in this chapter.
- **Jérôme Petazzoni**, "Containers From Scratch" talks/writeups and **Liz Rice**, "Containers From Scratch" (**GOTO/DockerCon**) and her book *Container Security* (O'Reilly, 2020) — building a container by hand from `clone`, `pivot_root`, and cgroup writes; the best way to see there is no magic. Complements Book 6.
- **Firecracker** (`firecracker-microvm.github.io`) and **gVisor** (`gvisor.dev`) documentation, and the **Firecracker NSDI 2020 paper** — the

microVM and user-space-kernel alternatives that add a second isolation boundary over the shared kernel (Volume 1, Chapter 10).

Chapter 10 — The Linux Network Stack

What this chapter covers. Every remote procedure call your service makes, every HTTP request it answers, every gRPC stream, every database round-trip — all of it is bytes moving through the Linux network stack, twice per hop: once out through the sender's kernel, once in through the receiver's. For a backend engineer this stack is not an implementation detail to be abstracted away; it is the layer where your tail latency is actually decided, where a misconfigured accept queue turns a load spike into a connection storm, where a conntrack table fills and a whole node stops accepting traffic. This chapter is a deep, mechanical tour of that stack: how a packet travels from the wire into a `recv()` call and back out, what an `sk_buff` is, how interrupts and NAPI polling share the load, how the socket layer's queues and buffers behave, how netfilter and connection tracking sit in the fast path, how the qdisc layer paces transmission, how NIC offloads and multi-queue steering let one host saturate 100 Gbit/s links, and finally how DPDK and XDP bypass most of the stack when even a well-tuned kernel is not fast enough.

We build on the rest of Volume 2. The softirq and scheduling machinery from Chapter 2 (CPU Scheduling) is exactly what runs packet processing off the interrupt path. The syscall boundary from Chapter 5 (System Calls and the Kernel Boundary) is the `send() / recv()` cost we amortize. The readiness and completion models from Chapter 7 (Linux I/O Models: Blocking, epoll, and io_uring) are the application-facing end of the socket receive queue we study here. NUMA locality from Volume 1, Chapter 6 governs where interrupts and their data should land. Full TCP congestion-control theory belongs to Volume 3 (Networking); here we cover only the kernel-side buffers and knobs a backend engineer tunes.

Learning goals — after this chapter you should be able to:

- Trace a packet end to end on both the receive and transmit paths, naming every queue, buffer, and softirq it passes through, and explain where drops happen and how to see them.

- Explain the `sk_buff`, NAPI polling, and interrupt coalescing, and why they exist.
- Distinguish the **SYN queue** from the **accept queue**, size both correctly, and diagnose overflow with real counters — the single most common connection-scaling bug.
- Reason about socket send/receive buffers, autotuning, Nagle vs `TCP_NODELAY`, and the `SO_*` options that matter for latency and connection reuse.
- Understand connection tracking and netfilter well enough to predict their performance cost and recognize conntrack-table exhaustion before it takes a node down.
- Explain NIC offloads (checksum, TSO/GSO/GRO/LRO) and multi-queue steering (RSS/RPS/RFS) and configure IRQ and CPU affinity for network processing with NUMA in mind.
- Diagnose ephemeral-port exhaustion and `TIME_WAIT` accumulation, and apply `SO_REUSEADDR` / `SO_REUSEPORT` correctly.
- Decide when kernel bypass (DPDK, XDP, AF_XDP) is justified and how modern L4 load balancers use eBPF/XDP.

The shape of the stack

The Linux network stack is a layered pipeline with queues between the layers. Data crosses three boundaries: the wire-to-hardware boundary at the NIC, the hardware-to-kernel boundary (DMA plus interrupts), and the kernel-to-application boundary (the socket API and its syscalls). Between those boundaries sit rings, buffers, and queues whose sizes and drain rates determine throughput and latency. The mental model that pays off is this: **the stack is a series of producer/consumer queues, and every one of them can overflow**. Throughput is set by the slowest consumer; latency is set by how deep the queues are allowed to grow (bufferbloat); and drops happen at whichever queue fills first. Almost every network incident a backend team sees reduces to one of those three statements.

The layers, top to bottom on transmit and bottom to top on receive, are: the socket layer (BSD sockets, per-socket send/receive buffers), the transport layer (TCP/UDP), the network layer (IP, routing, netfilter), the traffic-control layer (qdiscs, on TX only), the device layer (the driver and

its DMA rings), and the NIC hardware itself. We will walk a packet through all of them in both directions.

The receive path: from wire to `recv()`

When a frame arrives, the NIC does not interrupt the CPU for each byte. Modern NICs are bus masters: the driver has pre-allocated a **receive ring** — a circular array of descriptors, each pointing at a DMA-mappable buffer in host memory — and handed those buffer addresses to the NIC. The NIC writes the incoming frame directly into host RAM by DMA, advances the ring, and only then signals the CPU.

```
flowchart TD
    WIRE["Ethernet frame on wire"] --> NIC["NIC: DMA frame into RX ring buffer"]
    NIC --> IRQ["Hardware IRQ (MSI-X) to a specific CPU"]
    IRQ --> TOP["IRQ handler top half: napi_schedule, mask further IRQs"]
    TOP --> SOFTIRQ["NET_RX_SOFTIRQ raised, runs in softirq / ksoftirqd"]
    SOFTIRQ --> POLL["NAPI poll: drain ring up to netdev_budget"]
    POLL --> GRO["GRO: coalesce segments of one flow"]
    GRO --> NETIF["netif_receive_skb: hand skb to protocol stack"]
    NETIF --> NF1["netfilter PREROUTING, then routing decision"]
    NF1 --> LOCAL["Destined local: netfilter LOCAL_IN"]
    LOCAL --> TCP["TCP receive: sequence, ACK, reassemble"]
    TCP --> RQ["Socket receive queue (sk_receive_queue)"]
    RQ --> WAKE["Wake process blocked in recv / epoll"]
    WAKE --> APP["Application recv() copies bytes to user space"]
```

Step by step:

1. **DMA into the RX ring.** The NIC copies the frame into a buffer the driver pre-posted. No CPU involvement yet. If the ring has no free descriptors — the driver has not refilled it fast enough — the NIC drops the frame and increments a hardware counter (`rx_no_buffer`, `rx_missed_errors`, or similar, visible in `ethtool -S`). These are *hardware* drops, before the kernel ever sees the packet.
2. **The interrupt (top half).** The NIC raises an MSI-X interrupt targeted at a specific CPU (which CPU is set by IRQ affinity — see multi-queue below). The driver's hard IRQ handler does almost nothing: it acknowledges the interrupt, **masks further receive interrupts for that queue**, and calls `napi_schedule()` to raise the

`NET_RX_SOFTIRQ`. The design goal is to spend as little time as possible in hard-IRQ context, which runs with interrupts disabled.

3. **NAPI poll (the softirq).** The real work happens in softirq context (Chapter 2), either inline after the IRQ or, under sustained load, in the per-CPU `ksoftirqd` kernel thread. The driver's `poll()` function drains the RX ring, pulling frames into `sk_buff`s, up to a **budget** — `net.core.netdev_budget` (default 300 packets) or `netdev_budget_usecs` (default 2000 μ s), whichever comes first. This is NAPI: **under low load you get interrupts; under high load the NIC stays in polled mode and interrupts are suppressed**, avoiding an interrupt storm (one IRQ per packet at line rate would melt a CPU). When the ring is drained below budget, NAPI re-enables interrupts and goes back to sleep.
4. **GRO (Generic Receive Offload).** Before packets go up the stack, GRO coalesces multiple received segments belonging to the same flow into one large `sk_buff`, so the expensive per-packet trip up the stack is paid once for, say, 45 KB instead of 30 times for 1500-byte frames. This is a major throughput win and is done in software (or assisted by hardware LRO).
5. **Into the protocol stack.** `netif_receive_skb()` hands the `sk_buff` to the registered protocol handler (IP). The packet passes the **netfilter PREROUTING** hook (where DNAT and raw/conntrack rules live), then the routing decision determines whether it is for this host (`LOCAL_IN`) or to be forwarded (`FORWARD`). Connection tracking is consulted here.
6. **TCP processing.** For a local TCP segment, the kernel finds the matching socket, validates the sequence number, updates the receive window, sends or schedules an ACK, reassembles in-order data, and appends it to the socket's **receive queue** (`sk_receive_queue`). Out-of-order segments wait in a separate out-of-order queue until the gap fills.
7. **Wake the application.** The kernel marks the socket readable and wakes any thread blocked in `recv()` or registered via `epoll` (Chapter 7). The application's `recv()` copies bytes from the kernel receive buffer into user space and frees that space, which lets TCP advance the window it advertises back to the sender.

The critical backend insight: between the socket receive queue and the application there is a syscall and a copy, and if the application is slow to drain (a stalled event loop, a GC pause, head-of-line blocking on a mutex), the receive buffer fills, TCP shrinks the advertised window to zero, and back-pressure propagates all the way to the sender. **A slow consumer is felt as network back-pressure, not as a local queue growing unbounded** — this is TCP flow control doing its job, and it is why a stuck downstream shows up as elevated latency upstream.

The transmit path: from send() to wire

Transmit is the mirror image, with one extra layer — traffic control — that has no receive equivalent.

```
flowchart TD
    APP["APP[\"Application send(): copy data into socket send buffer\"] -->"]
    SB["SB[\"Socket send buffer (sk_write_queue)\"]"]
    TCPS["SB --> TCPS[\"TCP: segment per MSS/cwnd/rwnd, build skb, add headers\"]"]
    NF["TCPS --> NF[\"netfilter OUTPUT, routing, netfilter POSTROUTING (SNAT)\"]"]
    NEIGH["NF --> NEIGH[\"Neighbor / ARP resolution, add L2 header\"]"]
    QDISC["NEIGH --> QDISC[\"qdisc: enqueue (fq_codel / fq), dequeue by policy\"]"]
    DRV["QDISC --> DRV[\"Driver ndo_start_xmit: descriptor into TX ring\"]"]
    DMA["DRV --> DMA[\"NIC DMAs frame from host memory, transmits\"]"]
    COMPL["DMA --> COMPL[\"TX completion IRQ -> NET_TX_SOFTIRQ frees skb\"]"]
```

1. **send() and the send buffer.** `send()` / `write()` copies user data into the socket's **send buffer** (bounded by `SO_SNDBUF` / autotuning). If the buffer is full and the socket is blocking, the call blocks; if non-blocking, it returns `EAGAIN` and your event loop must wait for writability (Chapter 7). Once copied, the kernel owns the data and can retransmit it — TCP keeps a copy until it is ACKed.
2. **TCP segmentation.** TCP carves the byte stream into segments no larger than the MSS, subject to the congestion window (`cwnd`) and the peer's advertised receive window (`rwnd`). Each segment becomes an `sk_buff` with TCP and IP headers. With TSO/GSO (below), this segmentation is deferred so the stack carries one giant `sk_buff` almost to the wire.

3. **IP, routing, netfilter.** The packet passes the **OUTPUT** hook, a routing lookup picks the egress interface and next hop, and the **POSTROUTING** hook (SNAT/masquerade) runs. Neighbor resolution (ARP/NDP) supplies the destination MAC and the L2 header is prepended.
4. **The qdisc (traffic control).** `dev_queue_xmit()` hands the packet to the interface's **queueing discipline**. This is where packets are buffered, prioritized, paced, or dropped before the driver. On a modern kernel the default qdisc (`net.core.default_qdisc`) is typically `fq_codel`. The qdisc dequeues by its own policy and pushes to the driver.
5. **The driver and TX ring.** The driver's `ndo_start_xmit()` writes a descriptor into the **transmit ring**, pointing the NIC at the buffer. The NIC DMA's the frame out of host memory and onto the wire.
6. **TX completion.** When the NIC finishes transmitting, it raises a completion interrupt; the `NET_TX_SOFTIRQ` reclaims the now-sent `sk_buff`s and frees their memory. Only after the peer ACKs does TCP release its retained copy.

The `sk_buff`: the packet's in-kernel identity

Everything above moves one central structure: the `sk_buff` (socket buffer, universally "skb"). It is the kernel's representation of a packet in flight, and understanding its layout explains several performance behaviors.

An `sk_buff` is metadata; the packet bytes live in a separately allocated data area. The `skb` holds pointers into that area — `head`, `data`, `tail`, `end` — carving it into **headroom** (space before `data`, so lower layers can prepend headers without reallocating), the packet payload (`data..tail`), and **tailroom**. As a packet goes down the stack, each layer moves `data` backward and writes its header into the headroom — no copy, just pointer arithmetic. That is why drivers reserve generous headroom, and why XDP (below) needs headroom to insert encapsulation.

Two more features matter. First, **cloning**: `skb_clone()` produces a second `skb` head sharing the same data (via a reference-counted `skb_shared_info`); TCP uses this to keep a copy for retransmission while the original travels down the stack. Copy-on-write kicks in only if

someone must modify the shared data. Second, **fragments**: the `skb_shared_info` at the end of the data area holds an array of page fragments, so a large segment (from TSO/GSO or `sendfile`) can reference payload pages directly without linearizing them into one contiguous buffer — essential for zero-copy TX. Allocating and freeing skbs at millions of packets per second is itself a cost the whole stack is designed to minimize, which is one reason coalescing (GRO, TSO) and kernel bypass exist.

Interrupts, NAPI, and coalescing

At 10/25/100 Gbit/s, per-packet interrupts are untenable: a 10 GbE link can deliver ~14.8 million minimum-size packets per second, and no CPU can service that many hard IRQs. Two mechanisms tame this.

NAPI (New API), described above, is the software half: switch to polling under load. The hardware half is **interrupt coalescing** — the NIC waits a short interval or a small count of packets before raising an interrupt, batching several packets per IRQ. You tune it with `ethtool -C`:

```
# Show coalescing settings
ethtool -c eth0
# Coalesce: raise an RX interrupt after 50 us OR 64 frames, whichever
first
ethtool -C eth0 rx-usecs 50 rx-frames 64
# Adaptive coalescing: let the NIC vary the interval by load
ethtool -C eth0 adaptive-rx on
```

Coalescing is a **latency/throughput/CPU trade-off**. Larger `rx-usecs` batches more, cutting CPU and raising throughput, but adds up to that many microseconds of latency to every packet. Latency-sensitive services (a trading gateway, a low-latency cache) tighten it; throughput-bound bulk movers loosen it. This is a knob you actually turn in production, and it interacts with tail latency (Volume 11) directly: a fat coalescing window is invisible in the mean and very visible at p999.

The socket layer: buffers, backlog, and the two queues

Send and receive buffers, and autotuning

Each socket has a send buffer and a receive buffer. `SO_SNDBUF` / `SO_RCVBUF` set them via `setsockopt`, but Linux normally **autotunes** them within limits so you should rarely pin them. Two gotchas: the kernel roughly **doubles** the value you set (accounting for bookkeeping overhead), and setting `SO_RCVBUF` / `SO_SNDBUF` explicitly **disables autotuning** for that socket. The autotuning limits are three-valued sysctls — min, default, max:

```
# min default max, in bytes, for the receive buffer
sysctl net.ipv4.tcp_rmem      # e.g. 4096   131072  6291456
sysctl net.ipv4.tcp_wmem      # e.g. 4096   16384   4194304
# hard ceilings that also bound explicit SO_RCVBUF/SO_SNDBUF requests
sysctl net.core.rmem_max net.core.wmem_max
# receive-buffer autotuning on/off
sysctl net.ipv4.tcp_moderate_rcvbuf # 1 = enabled
```

The receive buffer must be large enough to hold a full **bandwidth-delay product** or TCP cannot keep the pipe full: the advertised window caps in-flight data, and window = buffer. On a 10 Gbit/s path with 30 ms RTT, the BDP is ~37 MB, far above the default max — so a cross-region bulk transfer that seems mysteriously capped at a few hundred Mbit/s is usually hitting `tcp_rmem`'s ceiling, not the network. Raise `net.core.rmem_max` and the top of `tcp_rmem` for such workloads. (Full window/BDP theory is Volume 3.)

The listen backlog: SYN queue vs accept queue

This is the most misunderstood corner of the socket API and the source of a recurring class of production incidents, so it earns a diagram. A listening socket has **two** queues, not one.

```
sequenceDiagram
    participant C as Client
    participant SQ as SYN queue (incomplete)
    participant AQ as Accept queue (complete)
    participant App as Server accept()
    C->>SQ: SYN
    Note over SQ: bounded by tcp_max_syn_backlog
    if full: drop, or emit SYN cookie
    SQ-->>C: SYN-ACK
    C->>AQ: ACK (handshake complete)
    Note over AQ: move from SYN queue to accept queue
    bounded by min(listen backlog, somaxconn)
    if full: drop ACK, bump ListenOverflows
    App->>AQ: accept() dequeues a ready connection
```

- The **SYN queue** (also "request queue", incomplete connection queue) holds connections in `SYN_RECV`: a SYN arrived, a SYN-ACK went out, the final ACK is awaited. Its size is bounded by `net.ipv4.tcp_max_syn_backlog`. When it fills — classically during a SYN flood — new SYNs are dropped **unless SYN cookies are enabled**.
- The **accept queue** (completed connection queue) holds fully established connections that finished the three-way handshake and are waiting for the application to call `accept()`. Its size is `min(backlog, net.core.somaxconn)`, where `backlog` is the argument you passed to `listen(fd, backlog)`. **You cannot exceed `somaxconn` no matter what you pass to `listen`.**

Two overflow behaviors, two different symptoms:

- **Accept-queue overflow.** If the handshake completes but the accept queue is full (the app is not calling `accept()` fast enough — a busy or stalled event loop), the kernel by default **silently drops the client's final ACK** and increments `ListenOverflows` and `ListenDrops`. The client thinks the connection is established and starts sending or waiting; the server eventually retransmits the SYN-ACK, forcing a retry after an RTO. This shows up as *mysterious multi-second connection latency under load* while CPU looks fine — the give-away is a climbing `ListenOverflows`. Setting `net.ipv4.tcp_abort_on_overflow=1` makes the kernel send an RST instead (fail fast rather than hang), but the real fix is a bigger queue and a faster `accept()` loop.

- **SYN-queue overflow / SYN cookies.** When the SYN queue fills, `net.ipv4.tcp_syncookies` (default 1) lets the kernel encode the connection state into the SYN-ACK's initial sequence number and keep **no** server-side state; when the client's ACK returns, the state is reconstructed from the cookie. This defeats SYN-flood memory exhaustion, at the cost of disabling a few TCP options for cookie-established connections. Cookies engage only when the queue actually overflows, so they are a safety valve, not a steady-state mechanism.

Diagnosing this is one `ss` command: for a listening socket, `Recv-Q` is the **current** accept queue depth and `Send-Q` is its **maximum**.

```
$ ss -tlnp
State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port  Process
LISTEN 129    128    0.0.0.0:8080        0.0.0.0:*          users:
(("app",pid=42,fd=6))
```

Here `Recv-Q` (129) has exceeded `Send-Q` (128 = the effective backlog): the accept queue is overflowing right now. Confirm with the global counter:

```
$ nstat -az | grep -i listen
TcpExtListenOverflows 14823 0.0
TcpExtListenDrops     14823 0.0
```

The fix is threefold and must be applied together, because the smallest of the three wins: raise `net.core.somaxconn`, pass a matching `backlog` to `listen()` (many runtimes default to 128 or even `SOMAXCONN` frozen at compile time — check your framework), and make sure the app drains the queue. The kernel default for `somaxconn` was **128** for many years and was raised to **4096** in Linux 5.4; on older kernels or containers you must set it explicitly.

TCP in the kernel: the knobs a backend engineer touches

Full TCP — congestion control, loss recovery, the state machine — is Volume 3. Here are the kernel behaviors that change how your *application* code performs.

Nagle's algorithm and delayed ACK. Nagle (RFC 896) withholds a small segment until the previous small segment is ACKed, coalescing tiny writes to avoid flooding the network with 40-byte-header, 1-byte-payload packets. Delayed ACK (RFC 1122) withholds an ACK briefly (up to ~40 ms on Linux) hoping to piggyback it on a reply. Individually reasonable, **together they interact pathologically**: a request/response protocol that does a small write then waits can stall — Nagle holds the request waiting for an ACK, the peer's delayed-ACK timer holds the ACK waiting for data, and you eat tens of milliseconds per exchange. The fix for latency-sensitive request/response traffic is `TCP_NODELAY`, which disables Nagle:

```
int one = 1;
setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &one, sizeof(one));
```

Most RPC libraries (gRPC, most HTTP servers) set `TCP_NODELAY` by default for exactly this reason. If you write a custom protocol and see inexplicable ~40 ms latency spikes, this is the first suspect.

TCP_CORK is the inverse: it *accumulates* data and refuses to send partial segments until you uncork or the buffer fills, so you can assemble a full-MSS packet from several writes (a header plus a body) and send it as one. `sendfile()`-based static servers use it to merge headers with file data. `TCP_NODELAY` and `TCP_CORK` express opposite intents — "send now, latency matters" vs "wait, packing matters."

Keepalive (`SO_KEEPALIVE` plus `tcp_keepalive_time / _intvl / _probes`) detects dead peers on idle connections; important for long-lived connection pools behind stateful load balancers and NATs that silently drop idle flows.

Netfilter, iptables/nftables, and connection tracking

Every packet that is filtered, NATed, or load-balanced by the kernel passes through **netfilter** — a set of hooks at fixed points in the IP path (PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING). `iptables` and its successor `nftables` install rules at those hooks; Kubernetes' `kube-proxy` in `iptables` mode, Docker's port publishing, and every host firewall you run are netfilter rules.

The performance story matters because in a Kubernetes cluster these rule sets are enormous. **`iptables` evaluates rules linearly**: a packet is

tested against rules in order until one matches. With one Service that is trivial; with thousands of Services and endpoints, `kube-proxy` in iptables mode generates tens of thousands of rules, and every new connection walks a long chain — $O(n)$ per connection, with rule-set *update* cost that can spike CPU when endpoints churn. `nftables` replaces linear chains with a bytecode VM and, crucially, **maps and sets** that give $O(1)$ or $O(\log n)$ lookups (hash/interval maps), so a Service lookup is a single map dereference rather than a chain walk. This is why `kube-proxy`'s nftables mode and eBPF dataplanes (Cilium, below) exist: to escape the linear-scan cost.

Connection tracking (conntrack)

Conntrack (`nf_conntrack`) is the stateful engine underneath NAT and stateful firewalling. It records every connection's tuple and state in a hash table so that reply packets can be matched to their flow and NAT translations applied consistently. It is invisible until it isn't, and when it isn't, it is a classic fleet-wide outage.

The table has a fixed maximum, `net.netfilter.nf_conntrack_max`, and a fixed number of hash buckets. **When the table fills, the kernel drops new connections and logs:**

```
nf_conntrack: table full, dropping packet
```

To a service this looks like random connection failures and timeouts with no application-level cause, often across an entire node or cluster at once — a NAT gateway, a busy ingress node, or a node running short-lived connection-heavy workloads (think a scraper, a health-checker, or a serverless burst) can exhaust conntrack in seconds. Each connection also holds an entry for a timeout after it closes (`nf_conntrack_tcp_timeout_time_wait` and friends), so high connection *churn* fills the table even when concurrent connections are modest.

Observe and size it:

```
sysctl net.netfilter.nf_conntrack_max      # table capacity
sysctl net.netfilter.nf_conntrack_count    # current entries (also /
proc/sys/.../nf_conntrack_count)
cat /proc/sys/net/netfilter/nf_conntrack_buckets # hash buckets
conntrack -S                               # per-CPU stats incl.
'insert_failed', 'drop'
conntrack -L | wc -l                       # dump the live table
```

Rising `nf_conntrack_count` toward `nf_conntrack_max`, and non-zero `insert_failed/drop` in `conntrack -S`, are the leading indicators. Mitigations: raise `nf_conntrack_max` (and the bucket count proportionally, so the hash stays shallow), shorten timeouts for churny traffic, or exempt high-volume trusted flows from tracking with a `NOTRACK` rule in the `raw` table so they never consume an entry. The deeper fix — for clusters where `conntrack` is a bottleneck — is to move the dataplane to eBPF, which can bypass `conntrack` for known flows.

The qdisc layer and bufferbloat

On transmit, every interface has a **queueing discipline**. The historical default, `pfifo_fast`, is a simple three-band FIFO. Its problem is **bufferbloat**: large, dumb FIFO buffers (in the qdisc, the driver ring, and the NIC) fill up under load and add enormous queueing delay — a bulk transfer fills the buffer, and every latency-sensitive packet behind it waits through the whole backlog. Bufferbloat is why a large upload can wreck the latency of an interactive session sharing the link.

The modern default is `fq_codel` (Fair Queuing with Controlled Delay). CoDel is an **active queue management** algorithm: instead of only dropping when the buffer is full (tail drop), it measures the *time* packets spend in the queue and starts dropping (or ECN-marking) when the standing queue delay exceeds a target (~5 ms), signaling TCP to back off before the queue grows deep. The "fq" part hashes flows into separate sub-queues and round-robins them, so one fat flow cannot starve or delay the others. Together they keep latency low under load without hand-tuning buffer sizes.

fq (fair queue, without CoDel) adds **pacing**: it spaces a flow's packets out to match its computed rate rather than sending them in bursts. This is the recommended qdisc for the **BBR** congestion-control algorithm, which depends on pacing to estimate bandwidth. You set these globally or per-interface:

```
sysctl -w net.core.default_qdisc=fq_codel      # default for new
interfaces
tc qdisc replace dev eth0 root fq              # pacing qdisc, e.g. for
BBR
tc -s qdisc show dev eth0                      # show stats incl.
drops, overlimits
```

For backend fleets the takeaway is that `fq_codel` should be your default and that `tc -s qdisc show`'s drop and backlog counters are where egress queueing latency becomes visible.

Offloads: doing packet work in the NIC

To saturate fast links the CPU must do *less* per byte, so the NIC takes over repetitive work. These are the offloads, viewable with `ethtool -k` and toggled with `ethtool -K`:

Offload	Direction	What it does
Checksum (rx/tx)	both	NIC computes/verifies IP/TCP/UDP checksums
TSO (TCP Segmentation Offload)	TX	Kernel hands the NIC one large buffer; NIC splits into MSS-sized segments
GSO (Generic Segmentation Offload)	TX	Same idea in software: defer segmentation to just before the driver
GRO (Generic Receive Offload)	RX	Software coalesces received segments of a flow into one large skb
LRO (Large Receive Offload)	RX	Hardware version of GRO; lossy, must be off for forwarding/bridging

The unifying principle is **amortization**: the per-packet cost of traversing the stack (skb allocation, header processing, function calls) is paid once for a 64 KB super-packet instead of ~43 times for 1500-byte frames. TSO/GSO can multiply TX throughput several-fold on a CPU-bound sender; GRO does the same on RX. LRO is the exception you often *disable*: because it irreversibly merges packets, a router or bridge (and thus most Kubernetes nodes) must use GRO instead, which preserves enough information to re-segment. Inspect and adjust:

```
ethtool -k eth0 | grep -E 'segmentation|generic|large|checksum'
ethtool -K eth0 gro on tso on lro off
# typical for a forwarding node
```

One debugging trap: packet captures taken on the local host see **super-sized "packets"** (e.g., 64 KB) because `tcpdump` taps the stack *after* GRO on RX and *before* TSO on TX. Those are not real wire frames; the NIC

segments them. Do not conclude your MTU is broken from a local capture — capture at a switch mirror or disable offloads temporarily to see wire-sized frames.

Multi-queue NICs, RSS/RPS/RFS, and affinity

A single CPU cannot process 100 Gbit/s of packets. Modern NICs expose **multiple hardware queues**, and the work of spreading traffic across CPUs is done by a family of steering mechanisms.

```
flowchart TD
    NIC["NIC[\"Multi-queue NIC\"]"] --> HASH["HASH[\"RSS: hash 4-tuple to a queue\"]"]
    HASH --> Q0["Q0[\"RX queue 0 -> IRQ 40\"]"]
    HASH --> Q1["Q1[\"RX queue 1 -> IRQ 41\"]"]
    HASH --> Q2["Q2[\"RX queue 2 -> IRQ 42\"]"]
    HASH --> Q3["Q3[\"RX queue 3 -> IRQ 43\"]"]
    Q0 --> C0["C0[\"CPU0 softirq: process flow A\"]"]
    Q1 --> C1["C1[\"CPU1 softirq: process flow B\"]"]
    Q2 --> C2["C2[\"CPU2 softirq: process flow C\"]"]
    Q3 --> C3["C3[\"CPU3 softirq: process flow D\"]"]
    C0 --> APP0["APP0[\"RFS steers to CPU where the app thread runs\"]"]
    C1 --> APP0
    C2 --> APP0
    C3 --> APP0
```

- **RSS (Receive Side Scaling)** is the hardware mechanism: the NIC hashes each packet's 4-tuple (a Toeplitz hash over source/dest IP and port) to one of N receive queues, each with its own interrupt. Because the hash is over the *flow*, all packets of one TCP connection land on the same queue and thus the same CPU — preserving in-order processing and cache locality for that flow. You spread flows across cores; you do not split a single flow.
- **RPS (Receive Packet Steering)** is the software fallback when the NIC has fewer queues than CPUs (or no RSS): after the IRQ, the kernel hashes the flow and steers the packet to another CPU's backlog for protocol processing. It costs an inter-CPU interrupt but spreads load.
- **RFS (Receive Flow Steering)** goes further: it tracks *which CPU the application thread that will `recv()` this flow is running on*, and steers the packet's processing to that same CPU, so the data is hot in the right L1/L2 cache when the app reads it. Accelerated RFS pushes this hint into the NIC hardware so RSS itself targets the right queue.

- **XPS (Transmit Packet Steering)** is the TX analog: it maps CPUs to TX queues so a thread transmits on a queue local to its core.

Affinity ties this to NUMA (Volume 1, Chapter 6). Each RX queue's interrupt should be pinned to a CPU, and — critically — to a CPU **on the NUMA node local to the NIC**, so the DMA'd packet data and the descriptors are in local memory. Cross-NUMA packet processing pays a remote memory penalty on every access, and at millions of packets per second that is real throughput lost.

```
# Which CPUs is IRQ 41 allowed to run on? (bitmask)
cat /proc/irq/41/smp_affinity_list
# Pin IRQ 41 to CPU 1
echo 1 > /proc/irq/41/smp_affinity_list
# Which NUMA node is the NIC on?
cat /sys/class/net/eth0/device/numa_node
# Per-queue IRQ names
grep eth0 /proc/interrupts
```

A subtlety: the `irqbalance` daemon will happily *move* your carefully pinned IRQs around. For latency-sensitive or high-PPS workloads, teams commonly disable `irqbalance` and pin RX queue IRQs statically to NIC-local cores, sometimes isolating those cores from the scheduler (`isolcpus`, `nohz_full`) so nothing else preempts packet processing. This is the same placement discipline as pinning application threads — the packets and the threads that consume them should share a NUMA node and, ideally, not fight for the same cores.

Ephemeral ports, TIME_WAIT, and reuse

A client (or a proxy dialing upstreams — your service is a client too) needs a local port for each outbound connection. The kernel allocates these from the **ephemeral range**, `net.ipv4.ip_local_port_range` (default `32768 60999`, ~28k ports). Since a connection is identified by the full 4-tuple, the limit is 28k connections **per (destination IP, destination port)** pair, not 28k total — but a service that fans out to one backend address (a single database VIP, one upstream) hits exactly that ceiling, and new `connect()` calls fail with `EADDRNOTAVAIL`.

TIME_WAIT makes it worse. When a connection closes, the side that sent the last active close (typically the client) holds the 4-tuple in `TIME_WAIT` for a fixed **60 seconds** on Linux (`TCP_TIMEWAIT_LEN`, compiled in, not

tunable via `sysctl`) to absorb delayed duplicate segments and complete the close reliably. A connection-per-request client churning thousands of short connections per second accumulates tens of thousands of `TIME_WAIT` sockets, each occupying its ephemeral tuple — and exhausts the range.

The fixes, in order of preference:

1. **Reuse connections.** The real answer is a connection pool / HTTP keep-alive / HTTP/2 or gRPC multiplexing, so you open few long-lived connections instead of one per request. This eliminates the problem rather than working around it. `TIME_WAIT` at scale is almost always a symptom of missing connection reuse.
2. `net.ipv4.tcp_tw_reuse=1` lets the kernel reuse a `TIME_WAIT` socket for a *new outbound* connection when TCP timestamps (RFC 7323) make it safe. This is the safe, recommended `sysctl` for client-side port pressure.
3. Do **not** reach for `tcp_tw_recycle` — it was aggressive, broke clients behind NAT (it keyed on per-host timestamps), and was **removed from the kernel in 4.12**. Any tuning guide that still recommends it is outdated.

On the **server/listen** side, `SO_REUSEADDR` lets you `bind()` to a port that still has sockets in `TIME_WAIT` — this is why it is standard practice to set it before binding a server socket, so a restarted service can rebind immediately instead of waiting out `TIME_WAIT` on the listener.

SO_REUSEPORT: scaling accept across cores

`SO_REUSEADDR` and `SO_REUSEPORT` are often confused. `SO_REUSEPORT` lets *multiple sockets* — typically one per worker process or thread — bind to the **same** IP and port simultaneously. The kernel then load-balances incoming connections across the listening sockets by hashing the 4-tuple, giving each worker its **own accept queue**.

```
int one = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &one, sizeof(one));
bind(fd, ...);          // each worker binds its own fd to the same
addr:port
listen(fd, backlog);
```


This solves a real scaling problem. In the classic model, N workers share one listening socket and one accept queue; they contend on that socket's lock and suffer the "thundering herd" of many workers waking for one connection. With `SO_REUSEPORT`, the kernel distributes connections in-kernel across independent queues — no shared lock, no herd, and near-linear accept scaling across cores. NGINX, Envoy, and HAProxy all support it for exactly this. The caveats: the hash distribution is not perfectly even, and a worker that dies with connections sitting in its private accept queue *drops them* (they were already assigned to that queue) — so graceful restart needs care. Combined with multi-queue RSS (packets for a flow already land on a particular CPU) and CPU-pinned workers, `SO_REUSEPORT` lets a whole request ride one core from NIC queue to accept to processing, maximizing cache locality.

Tuning knobs, gathered

The sysctls scattered above, collected as the ones you actually set on a backend host, and what each protects against:

```

# Accept queue: cap and the SYN queue
net.core.somaxconn          = 4096          # max accept-queue length
(and check listen() backlog!)
net.ipv4.tcp_max_syn_backlog = 8192         # SYN (incomplete) queue
length
net.ipv4.tcp_syncookies     = 1             # SYN-flood safety valve

# Softirq backlog: the per-CPU queue between NAPI and the stack (RPS
uses it)
net.core.netdev_max_backlog = 16384         # drops here show as /proc/
net/softnet_stat col 2
net.core.netdev_budget      = 300           # packets per NAPI poll cycle

# Socket buffer ceilings and autotuning ranges
net.core.rmem_max           = 16777216
net.core.wmem_max           = 16777216
net.ipv4.tcp_rmem            = 4096 131072 6291456
net.ipv4.tcp_wmem            = 4096 16384 4194304

# Client-side port pressure
net.ipv4.ip_local_port_range = 10240 65535
net.ipv4.tcp_tw_reuse        = 1

# Egress queueing
net.core.default_qdisc       = fq_codel

# Stateful firewall / NAT capacity
net.netfilter.nf_conntrack_max = 1048576

```

`netdev_max_backlog` deserves a note: it bounds the per-CPU input queue *between* the NAPI poll and the rest of the stack (also used by RPS). If a CPU's softirq processing cannot keep up, packets pile there and are dropped — visible as the second column of `/proc/net/softnet_stat` (per-CPU: processed, dropped, time-squeezed). A nonzero "squeezed" (third column) means NAPI hit its budget with more work pending — a sign to raise `netdev_budget` or spread load with more queues.

Observability: seeing every drop

The stack instruments itself thoroughly; the skill is knowing which counter names which queue.

- **ss** — the modern replacement for `netstat`. `ss -tln` for listen sockets (Recv-Q/Send-Q = accept queue depth/max), `ss -ti` for per-connection TCP internals (cwnd, rtt, retransmits, send/recv buffer).

`ss -s` for a summary. This is your first tool for connection-state questions.

- `nstat / /proc/net/netstat / /proc/net/snmp` — the SNMP and extended TCP counters. `nstat -az` dumps everything; the ones that matter most: `TcpExtListenOverflows`, `TcpExtListenDrops` (accept queue overflow), `TcpExtTCPSynRetrans` (SYN retransmits — a sign of drops or SYN-queue pressure), `TcpRetransSegs`, `TcpExtTCPTimeouts`.
- `ethtool -S eth0` — driver/hardware counters: `rx_dropped`, `rx_no_buffer_count`, `rx_missed_errors` (ring overrun — the NIC dropped before the kernel), `tx_dropped`. These are the *hardware* drops; if they climb, the RX ring is too small or the CPU can't drain it.
- `/proc/net/dev` — per-interface RX/TX packets, bytes, errors, drops at the interface level.
- `/proc/net/softnet_stat` — per-CPU softirq processing: processed, dropped (backlog full), squeezed (budget exhausted). The place to see softirq-layer drops.
- `conntrack -S` — conntrack insert failures and drops; watch alongside `nf_conntrack_count` vs `_max`.
- `tc -s qdisc show dev eth0` — egress qdisc drops, backlog, and overlimits.
- `tcpdump / bpftrace` — packet-level and kernel-tracepoint visibility. Remember the GRO/ TSO super-packet caveat above when reading local captures.

The diagnostic discipline is to **walk the path and read the counter at each queue**: hardware (`ethtool -S`) → softirq backlog (`softnet_stat`) → conntrack → accept queue (`ss` , `ListenOverflows`) → application. The first counter that is climbing tells you which queue is the bottleneck, and therefore which knob to turn.

Kernel bypass: XDP, AF_XDP, and DPDK

For most services a well-tuned kernel stack is more than enough. But at the extreme edge — a Tbps-scale load balancer, a DDoS scrubber, a packet broker, an NFV dataplane — the per-packet cost of the generic stack (skb allocation, the full IP/netfilter/socket path, syscalls, copies) is the bottleneck, and teams reach for **kernel bypass**.

```

flowchart LR
    subgraph Normal["Normal kernel path"]
        N1["NIC"] --> N2["driver + skb alloc"] --> N3["netfilter /  
conntrack / routing"] --> N4["TCP/socket"] --> N5["copy to user via  
recv()"]
    end
    subgraph XDPP["XDP (in-kernel eBPF)"]
        X1["NIC"] --> X2["XDP program at driver, before skb"] --> X3["DROP / PASS / TX / REDIRECT"]
    end
    subgraph Bypass["Full bypass (DPDK / AF_XDP)"]
        B1["NIC"] --> B2["poll-mode driver, DMA straight to userspace  
ring"] --> B3["app processes packets, no kernel per-packet"]
    end

```

XDP (eXpress Data Path) runs an **eBPF program at the earliest point in the driver**, on the raw DMA'd buffer **before an `sk_buff` is even allocated**. The program returns a verdict: `XDP_DROP` (discard immediately — the fastest possible drop, used for DDoS filtering at tens of millions of pps), `XDP_PASS` (continue up the normal stack), `XDP_TX` (bounce the packet back out the same NIC, e.g. as a load-balancer redirect), or `XDP_REDIRECT` (send to another interface or to an `AF_XDP` socket). Because it runs before skb allocation and is JIT-compiled and verified, XDP delivers line-rate packet manipulation *inside* the kernel, without leaving it — the sweet spot when you need speed but also want to coexist with the normal stack.

AF_XDP is a socket family that lets a userspace application receive raw frames with **zero copy** via shared memory rings (`XDP_REDIRECT` into an `AF_XDP` socket), getting most of DPDK's performance while keeping the NIC under kernel control and reusing the driver.

DPDK (Data Plane Development Kit) is the maximal approach: it **unbinds the NIC from the kernel entirely**, replaces the driver with a userspace **poll-mode driver** (no interrupts — dedicated cores busy-poll the rings), uses hugepages for its buffer pools, and hands packets to the application with no kernel involvement per packet at all. It achieves the highest throughput and lowest latency, at the cost of burning whole CPU cores at 100% (polling), losing the kernel's TCP stack and tooling (you bring your own), and dedicating the NIC to one application. It is the right tool for a purpose-built appliance, the wrong tool for a general service.

When to use which. The vast majority of backend services should use the normal stack and tune it — the topics in the rest of this chapter get

you a very long way. Reach for XDP when you need line-rate *filtering or forwarding* while still running a normal host (DDoS mitigation, L4 load balancing); reach for AF_XDP or DPDK when you are building the dataplane itself and the kernel's per-packet overhead is provably your ceiling. These are specialist tools with real operational cost, not a default.

Distributed-systems lens

Everything in this chapter compounds because in a distributed system the stack is traversed **twice per hop, on every request**. A single user request that fans out through an API gateway, three microservices, and a database crosses the network stack roughly a dozen times end to end. That means a per-packet or per-connection inefficiency you would ignore on one machine is multiplied across the whole call graph and shows up as tail latency (Volume 11).

Accept queues and buffer sizing shape tail latency and cause connection storms. The accept-queue overflow story is the canonical example: under a load spike, the app falls a little behind on `accept()`, the queue overflows, the kernel silently drops final ACKs, clients wait out an RTO and *retransmit*, which adds load, which deepens the backlog — a feedback loop that turns a small overload into a connection storm. The same shape appears with undersized socket buffers throttling throughput, and with SYN-queue pressure. Sizing these queues is not premature optimization; it is setting the shock absorbers for load spikes.

SO_REUSEPORT plus multi-queue RSS is how you scale accept across a many-core box. A modern server has dozens of cores; a single listening socket and a single accept queue is a scalability cliff. The combination — RSS spreading flows to per-queue IRQs on NIC-local NUMA cores, RFS steering processing to the consuming thread's core, `SO_REUSEPORT` giving each worker its own accept queue — lets a request ride one core from wire to application logic, and lets accept throughput scale with cores instead of bottlenecking on one lock. This is why high-performance proxies are architected around it.

Conntrack exhaustion is a classic fleet outage. In a Kubernetes cluster with NAT-heavy networking and churny short connections, a node's conntrack table fills, and the node starts dropping *new* connections with no application-level error — just timeouts. Because the trigger is connection churn, an innocuous change (a new health-check

interval, a batch job that opens many short connections) can tip a whole fleet over at once. Knowing the counter (`nf_conntrack_count` vs `_max`, `insert_failed`) turns a baffling outage into a five-minute diagnosis.

eBPF/XDP is quietly rewriting the dataplane. The industry's largest L4 load balancers run in XDP: Meta's **Katran** is an XDP-based L4 LB, and **Cilium** (Book 6, Cloud-Native Security) replaces `kube-proxy`'s iptables rules with an eBPF dataplane that does service load balancing, network policy, and NAT with hash-map lookups instead of linear chains — escaping exactly the $O(n)$ iptables and conntrack costs described above. When you deploy Cilium or a cloud provider's eBPF networking, you are moving the mechanisms in this chapter from the generic stack into programmable, per-packet eBPF — the same stack, made fast and observable at the point where your distributed system actually lives.

The through-line: your service's behavior under load is, to a large degree, the behavior of these kernel queues under load. The engineer who can name them, size them, and read their drop counters is the one who can keep tail latency flat when traffic doubles.

Key takeaways

- The stack is a chain of producer/consumer queues — RX ring, softirq backlog, socket receive buffer on ingress; socket send buffer, qdisc, TX ring on egress. Throughput is set by the slowest consumer, latency by queue depth, and **every queue can drop**. Diagnosis is walking the path and reading each queue's counter.
- NAPI plus interrupt coalescing converts an unservable interrupt storm into batched polling; coalescing (`ethtool -C`) is a real latency/throughput/CPU knob.
- A listening socket has **two** queues: the SYN queue (`tcp_max_syn_backlog`, guarded by SYN cookies) and the accept queue (`min(listen backlog, somaxconn)`). Accept-queue overflow silently drops ACKs and inflates tail latency; `ss` Recv-Q/Send-Q and `ListenOverflows` diagnose it. `somaxconn` defaulted to 128 and became 4096 in kernel 5.4 — set it explicitly.
- Socket buffers must cover the bandwidth-delay product or TCP caps throughput; autotuning handles most cases but has ceilings (`tcp_rmem / rmem_max`) you raise for long fat pipes. Setting `SO_RCVBUF / SO_SNDBUF` disables autotuning.

- `TCP_NODELAY` disables Nagle to avoid the ~40 ms Nagle/delayed-ACK stall on request/response traffic; `TCP_CORK` does the opposite for packing.
- iptables evaluates rules linearly (O(n)); nftables and eBPF use maps for O(1) lookups. Conntrack table exhaustion (`nf_conntrack_max`) is a classic node-wide outage — watch `nf_conntrack_count` and `conntrack -S` insert failures.
- `fq_codel` (AQM + fair queuing) is the default qdisc and the cure for bufferbloat; `fq` adds pacing for BBR.
- Offloads (TSO/GSO/GRO, checksum) amortize per-packet cost to saturate fast links; LRO is usually disabled on forwarding nodes. Local captures show super-packets, not wire frames.
- RSS/RPS/RFS spread and steer packet processing across cores; pin RX-queue IRQs to NIC-local NUMA cores (Volume 1, Chapter 6) and beware `irqbalance` moving them.
- Ephemeral-port and `TIME_WAIT` exhaustion are symptoms of missing connection reuse; fix with pooling first, `tcp_tw_reuse` second; never `tcp_tw_recycle` (removed in 4.12). `SO_REUSEADDR` rebinds a `TIME_WAIT` listener; `SO_REUSEPORT` load-balances accept across per-worker queues.
- Kernel bypass (XDP in-kernel, AF_XDP zero-copy, DPDK full-userspace) is the high-PPS frontier; XDP/eBPF power modern L4 load balancers (Katan, Cilium). Tune the normal stack first — it goes a very long way.

Further reading

- Christian Benvenuti, *Understanding Linux Network Internals* (O'Reilly) — the reference on the pre-NAPI-to-NAPI stack internals and `sk_buff` handling.
- The Linux kernel documentation tree: `Documentation/networking/scaling.rst` (RSS/RPS/RFS/XPS), `Documentation/networking/napi.rst`, and `Documentation/admin-guide/sysctl/net.rst` — the authoritative descriptions of the sysctls and steering mechanisms in this chapter. <https://www.kernel.org/doc/html/latest/networking/scaling.html>

- `man 7 socket` , `man 7 tcp` , `man 7 ip` — the definitive semantics of `SO_* / TCP_*` options, the listen backlog, and the sysctls (`man 7 tcp` documents `tcp_syncookies` , `tcp_rmem` , etc.).
- Cloudflare Engineering Blog, Marek Majkowski, "SYN packet handling in the wild" (and companion posts) — a precise, well-illustrated treatment of the SYN queue / accept queue two-queue model and its overflow behavior. <https://blog.cloudflare.com/>
- Toke Høiland-Jørgensen, Jesper Dangaard Brouer, et al., "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel" (CoNEXT 2018) — the XDP design paper. <https://dl.acm.org/doi/10.1145/3281411.3281443>
- The Cilium documentation, "eBPF and XDP Reference Guide" and the BPF/XDP concepts pages — how an eBPF dataplane replaces iptables/conntrack. <https://docs.cilium.io/>
- Meta Engineering, "Open-sourcing Katran, a scalable network load balancer" — XDP in production as an L4 LB. <https://engineering.fb.com/>
- The Bufferbloat project and the CoDel/ `fq_codel` papers (Nichols & Jacobson, "Controlling Queue Delay", ACM Queue 2012). <https://www.bufferbloat.net/> and <https://queue.acm.org/detail.cfm?id=2209336>
- Brendan Gregg, *Systems Performance* (2nd ed.), the networking chapter — a practical methodology for the observability tools (`ss` , `nstat` , `ethtool` , `bpftool`) used here.

Chapter 11 — Performance Analysis: perf, ftrace, and eBPF

What this chapter covers. Every prior chapter in this volume built a mechanism: the scheduler puts your thread on a CPU (Chapter 2), the MMU translates its addresses (Chapter 3), a system call crosses into the kernel (Chapter 5), the block layer moves a page to disk (Chapter 7), the network stack ships a segment (Chapter 10). This chapter is about *seeing* all of it — in production, on a running service, at line rate, without a debugger, without a recompile, and without taking the process down. It is

the chapter that turns "the p99 latency doubled after the last deploy" from a shrug into a root cause.

The tools are the least interesting part. What matters is the *method*: how you decide where to look, how you distinguish a saturated resource from a slow one, how you tell an on-CPU problem (too much work) from an off-CPU problem (waiting for something), and how you go from a symptom to a line of code with the fewest possible guesses. We anchor everything on Brendan Gregg's USE method and the RED method for services, then descend through the three great instrumentation technologies Linux gives you — hardware performance counters exposed by **perf**, the in-kernel tracer **ftrace**, and the programmable tracing runtime **eBPF** — and finish with how all of this runs continuously across a fleet of thousands of machines.

Learning goals — after this chapter you should be able to:

- Apply the **USE method** (Utilization, Saturation, Errors) to hardware resources and the **RED method** (Rate, Errors, Duration) to services, and know when to reach for each.
- Read `perf stat` output — IPC, cache misses, branch mispredictions — and connect the numbers to the microarchitecture from Volume 1 (Chapters 2, 3, 8).
- Use `perf record / report / top` for CPU profiling, understand sampling vs counting, and read and build a **flame graph**.
- Explain what **ftrace** is, the difference between static **tracepoints** and dynamic **kprobes/uprobes**, and use `function_graph` and `trace-cmd`.
- Explain **eBPF** end to end: the verifier, the JIT, maps, program types and attach points, and why it made production-safe dynamic tracing possible.
- Write **bpfftrace** one-liners and read the **BCC** tool suite (`execsnoop` , `biolatency` , `runqlat` , `tcplife` , ...) to observe CPU, memory, disk, network, the scheduler, and syscalls.
- Distinguish **on-CPU** from **off-CPU** analysis and diagnose scheduler-latency problems.
- Explain **continuous profiling** (Parca, Pyroscope, Google-Wide Profiling) and why always-on, low-overhead profiling is now table stakes for large fleets.

This chapter is Linux-specific and deliberately practical. It is the observability counterpart to every mechanism chapter before it, and it is the technical foundation under Volume 11 (SRE and Observability) and Book 6 (Cloud-Native Security), where the same eBPF machinery that traces a slow query also enforces runtime security policy.

Method first: measure, don't guess

The single most expensive mistake in performance work is optimizing something that was never the bottleneck. Engineers are pattern-matchers; shown a slow service, we leap to the last thing we read about — "it must be GC," "it must be the database," "it must be lock contention" — and then spend a week confirming a bias. The discipline that prevents this is almost embarrassingly simple: **measure, don't guess**. Form a hypothesis, but let a metric kill it before you write code. The tools in this chapter exist so that the loop from hypothesis to evidence is minutes, not days.

But raw measurement without structure is its own trap — you can stare at a hundred graphs and learn nothing. You need a *checklist* that guarantees coverage. That is what a method provides.

The USE method

The USE method, due to Brendan Gregg, is a checklist for finding resource bottlenecks. For **every resource** — CPUs, memory, disks, network interfaces, I/O controllers, interconnects — you ask three questions:

- **Utilization** — the fraction of time the resource was busy servicing work (or, for capacity-typed resources like memory, the fraction consumed). CPU at 90% busy; a disk 70% of the time doing I/O; memory 80% allocated.
- **Saturation** — the degree to which work is *queued* because the resource cannot keep up. A run queue with waiting threads; a block device with a deep request queue; swapping because memory demand exceeds capacity. Saturation is the honest signal; utilization can read a comfortable 60% while a bursty saturation is destroying your tail latency.

- **Errors** — error counts for the resource: ECC-corrected memory errors, disk I/O errors, dropped or malformed network packets, failed allocations.

The power of USE is that it is *exhaustive and cheap*. You enumerate the resources once, wire up one metric per cell, and any bottleneck of the "a resource is maxed out" variety shows up. Crucially, it steers you toward **saturation**, which is where tail latency lives and which naive dashboards (that show only utilization) hide. High utilization with zero saturation is a well-fed system; moderate utilization with periodic saturation is a system with a queuing problem you cannot see in the average.

```

flowchart TD
    A["Pick a resource: CPU / mem / disk / net"] --> B{"Errors  
present?"}
    B -->|Yes| C["Investigate errors first"]
    B -->|No| D{"Saturation present?"}
    D -->|Yes| E["Queuing bottleneck: find waiter and its cause"]
    D -->|No| F{"Utilization high?"}
    F -->|Yes| G["Resource near capacity: scale or reduce demand"]
    F -->|No| H["This resource is not the bottleneck: next resource"]
    C --> I["Root cause"]
    E --> I
    G --> I
    H --> A

```

USE is a *hardware/resource* lens. It tells you which resource is the constraint; it does not, by itself, tell you which request is slow or which user is affected. For that you want a service-oriented view.

The RED method for services

RED (Tom Wilkie's formulation, closely related to Google's "Four Golden Signals") characterizes a *service* rather than a resource. For every service — and for every endpoint within it — track:

- **Rate** — requests per second.
- **Errors** — failed requests per second (and error ratio).
- **Duration** — the distribution of request latency, always as percentiles (p50/p90/p99/p999), never as a mean. The mean of a bimodal latency distribution is a number that describes no real request.

USE and RED are complementary, not competing. RED (or the Four Golden Signals — latency, traffic, errors, saturation) is what your on-call

dashboard shows: it answers "is the service healthy, and are users hurting?" USE is what you reach for once RED says something is wrong and you need to find the constrained resource underneath. A typical incident flows RED → USE → the tracing tools in this chapter: the service's p99 (RED Duration) spikes, USE points at disk saturation, and `biolatency` (below) shows the I/O latency histogram that confirms it.

Latency vs throughput, and workload characterization

Two framings must stay separate in your head. **Throughput** is work per unit time (requests/s, bytes/s, IOPS). **Latency** is time per unit of work (ms per request). They are not the same axis and optimizing one routinely degrades the other: batching raises throughput and raises latency; running a disk queue deep maximizes IOPS and destroys per-I/O latency. Backend services are almost always latency-governed at the tail — the p99 is what breaks SLOs and cascades through call graphs (a fan-out request is as slow as its slowest dependency) — so you optimize latency subject to a throughput floor, not the reverse.

Before you profile, **characterize the workload**: who is calling, what are they asking for, how often, and how is the load shaped in time? A cache-hit-heavy read workload and a write-heavy workload on the same service have entirely different bottlenecks. Bursty arrival (the real world; traffic is rarely Poisson) produces saturation that a steady-state average never reveals. Workload characterization is the input that makes the USE/RED output interpretable.

The observability tooling map

Linux gives you a layered set of tools, and part of expertise is knowing the cheapest tool that answers the question. Counting is cheaper than sampling; sampling is cheaper than tracing every event; and static, pre-existing instrumentation (tracepoints, `/proc`) is cheaper than dynamic instrumentation (kprobes). The map below is the mental model for the rest of the chapter: which tool observes which subsystem, and roughly at what cost.

```

flowchart LR
    subgraph Subsystems
        CPU["CPU / microarch"]
        MEM["Memory / page cache"]
        DISK["Block I/O"]
        NET["Network stack"]
        SCHED["Scheduler"]
        SYS["Syscalls"]
    end
    end
    CPU --> T1["perf stat / perf record / flame graphs"]
    MEM --> T2["/proc, perf mem, page-fault tracepoints"]
    DISK --> T3["iostat, biolateness, biosnoop"]
    NET --> T4["ss, tcplife, tcpretrans, tcpdump]
    SCHED --> T5["runqlat, offcputime, perf sched]
    SYS --> T6["strace, execsnoop, syscount, opensnoop]
    T1 & T2 & T3 & T4 & T5 & T6 --> UP["USE + RED dashboards /  
continuous profiler"]

```

Hardware performance counters and perf

Underneath every backend service is a CPU whose behavior you cannot infer from `top`. Volume 1 established that a modern core is a deeply pipelined, out-of-order, speculating machine (Vol 1 Ch2) sitting on a multi-level cache hierarchy (Vol 1 Ch3), and that the difference between fast and slow code is often microarchitectural — cache misses, branch mispredictions, false sharing (Vol 1 Ch8). The **Performance Monitoring Unit (PMU)** is the hardware that measures exactly those events: a small bank of hardware counters, per logical CPU, that increment on architectural and microarchitectural events (retired instructions, cycles, cache references and misses, branch instructions and mispredicts, and hundreds of vendor-specific events). `perf`, the tool shipped in the Linux kernel tree (`tools/perf`), is the user-space front end to the kernel's `perf_events` subsystem, which programs and reads those counters.

perf stat: counting

`perf stat` runs a command (or attaches to a PID/CPU) and *counts* events over the whole run. This is the cheapest, most accurate view of what the silicon actually did:

```
$ perf stat -d ./my_service --benchmark
```

```
Performance counter stats for './my_service --benchmark':
```

```
      8,214.63 msec task-clock                #    0.998 CPUs
utilized
      142      context-switches              #    17.286 /sec
         6      cpu-migrations                #     0.730 /sec
      1,204      page-faults                  #   146.567 /sec
28,451,220,441      cycles                    #     3.464 GHz
19,004,882,101      instructions              #     0.67  insn per
cycle
      3,982,110,220      branches              #   484.75 M/sec
      71,884,201      branch-misses           #     1.81% of all
branches
      6,120,884,001      L1-dcache-loads       #   745.0  M/sec
      402,118,229      L1-dcache-load-misses   #     6.57% of all L1-
dcache accesses
      41,229,110      LLC-loads               #     5.02  M/sec
      18,004,221      LLC-load-misses         #    43.67% of all LL-
cache accesses

      8.230115 seconds time elapsed
```

Read this the way Volume 1 taught you. **IPC** (instructions per cycle) is the headline: 0.67 is poor — a modern wide core can retire 3–4 instructions per cycle, so an IPC below ~1.0 means the core is stalled most of the time, usually waiting on memory. The **LLC-load-miss rate of ~44%** confirms it: nearly half the loads that reached the last-level cache missed and went to DRAM, each miss costing on the order of a hundred-plus cycles (Vol 1 Ch3). The **1.81% branch-miss rate** is unremarkable; a rate climbing toward 5–10% would point at the misprediction penalties of Vol 1 Ch8. This is the fingerprint of a memory-bound workload: low IPC, high LLC miss rate. No amount of algorithmic micro-tuning fixes it; you fix it by changing *data layout* — improving locality, shrinking the working set below cache size, eliminating pointer chasing.

Interpretation guardrail: absolute event counts and even the "good" IPC threshold are microarchitecture- and workload-dependent. Use `perf stat` for *comparison* — before vs after a change, hot path vs cold path — and treat single absolute numbers with suspicion. For a rigorous decomposition of *where* cycles went, use the vendor **Top-down Microarchitecture Analysis** methodology, exposed via `perf stat --`

topdown / the toplev tool, which attributes stalls to frontend-bound, backend-bound, bad-speculation, and retiring categories.

perf record/report/top: sampling

Counting tells you *what* happened aggregate; it does not tell you *where* in the code. For that you **sample**. `perf record` programs a counter (by default cycles) to fire an interrupt every N events; on each interrupt the kernel captures the instruction pointer and, if asked, the call stack. Because it samples rather than instruments every instruction, overhead is bounded and tunable via the sample rate — this is what makes it usable on production:

```
# Sample on-CPU stacks at 99 Hz for a running PID for 30s.
# -F 99 (not 100) avoids lock-stepping with periodic timers.
# -g captures call stacks; --call-graph dwarf works without frame
pointers.
$ perf record -F 99 -p $(pgrep my_service) -g --call-graph dwarf -- sle
ep 30
$ perf report --stdio           # interactive TUI without --stdio
```

`perf top` is the live equivalent — a continuously updating `top`-like view of the hottest functions system-wide or per-PID, invaluable for "what is burning CPU *right now*." A note on **stack unwinding**: capturing correct call graphs requires either frame pointers (compile with `-fno-omit-frame-pointer`; many distro binaries omit them for a small speed gain, which silently breaks stack collection), DWARF call-frame information (`--call-graph dwarf`, heavier), or Intel **Last Branch Record** (`--call-graph lbr`). Missing or broken frame pointers are the single most common reason a production profile shows truncated, useless stacks; enabling them across your fleet is one of the highest-leverage observability investments you can make.

Flame graphs

`perf report`'s tree is accurate but hard to read at scale. The **flame graph**, invented by Brendan Gregg, is the visualization that made CPU profiling legible. It is built from the same sampled stacks:

1. Collect thousands of stack samples (`perf record -g`).

2. Fold each stack into a single semicolon-delimited line (`func_a;func_b;func_c`) and count identical lines (`stackcollapse-perf.pl`).
3. Render each unique stack as a column of stacked boxes — one box per frame, parent below child — and set each box's **width proportional to how many samples contained that frame** (`flamegraph.pl`).

```
$ perf record -F 99 -p $(pgrep my_service) -g -- sleep 30
$ perf script | stackcollapse-perf.pl | flamegraph.pl > profile.svg
```

How to *read* one:

- **The x-axis is not time.** Frames are sorted alphabetically and merged; width means "fraction of samples," i.e., fraction of on-CPU time. A wide box is expensive.
- **The y-axis is stack depth.** The bottom is the root (e.g., thread entry); each box directly above is a callee.
- **You hunt for wide plateaus.** A wide box near the *top* of the graph is a leaf that itself burns CPU — the actual hot code. A wide box that is wide only because of a *tall tower* above it is a function whose cost is in its callees. The eye scans left-to-right along the top for the widest leaves; those are where the CPU actually went.

```
flowchart TB
    root["main() - 100% width"]
    root --> h["handle_request() - 70%"]
    root --> gc["gc_collect() - 30%"]
    h --> ser["serialize() - 45%"]
    h --> db["db_query() - 25%"]
    ser --> mc["memcpy() - 40% - WIDE LEAF: real hotspot"]
    db --> sys["sys_read()"]
    gc --> mark["mark_heap() - 30%"]
```

That illustration says: 40% of on-CPU time is inside `memcpy` under serialization, and 30% is garbage collection. Two clear targets, found in seconds by eye — this is why flame graphs became the default profiling output across the industry. The same technique extends to **off-CPU flame graphs** (below), **differential flame graphs** (before/after, colored by delta), and mixed kernel+user stacks.

ftrace: the kernel's built-in tracer

`perf` samples and counts. When you need to trace *specific kernel events* — every call to a function, every scheduler switch, every block I/O issue — with full fidelity rather than a statistical sample, the kernel's built-in tracer **ftrace** is the oldest and lowest-dependency option. It requires no external agent; it is driven entirely through the `tracefs` filesystem, usually mounted at `/sys/kernel/tracing`.

Two categories of instrumentation feed ftrace, and the distinction is fundamental to everything that follows (including eBPF):

- **Static tracepoints** — stable, named instrumentation points *compiled into the kernel* by its developers at meaningful events: `sched:sched_switch`, `block:block_rq_issue`, `syscalls:sys_enter_openat`, `net:netif_receive_skb`. They have a documented, versioned format and near-zero cost when disabled. Because they are a maintained ABI, they are the *preferred* attach point when one exists.
- **Dynamic instrumentation** — **kprobes** (and the return variant **kretprobes**) let you instrument *almost any kernel function* by patching a breakpoint at runtime, and **uprobes** do the same for user-space functions. No recompilation, no reboot. The cost is stability: you are attaching to internal functions whose names, arguments, and existence can change between kernel versions.

Enabling the function graph tracer to see the call flow under a syscall:

```
cd /sys/kernel/tracing
echo function_graph > current_tracer
echo do_sys_openat2 > set_graph_function      # trace only under this
function
echo 1 > tracing_on ; cat trace_pipe ; echo 0 > tracing_on
```

#	CPU	DURATION	FUNCTION CALLS
#			
2)			do_sys_openat2() {
2)			getname() {
2)	1.204 us		__check_object_size();
2)	3.881 us		}
2)			do_filp_open() {
2)			path_openat() {
2) +	18.402 us		link_path_walk.part.0();
2)	0.512 us		vfs_open();
2) !	121.760 us		}
2) !	140.113 us		}
2) !	150.402 us		}

The **+** and **!** markers flag calls exceeding latency thresholds — here `path_openat` dominates the `openat`, which ties directly to the VFS path-walk work of Chapter 6. In practice you rarely poke `tracefs` by hand; `trace-cmd` (record/report front end) and the **KernelShark** GUI wrap it, and `trace-cmd record -e sched -e block ...` captures whole subsystems by tracepoint. `ftrace` remains the right tool when you want deterministic, full-fidelity function-level tracing of the kernel with nothing installed. But for programmable, aggregating, production-grade tracing, the state of the art moved to eBPF.

eBPF: programmable, production-safe tracing

eBPF is the most important development in Linux observability in the last decade, and understanding *why* requires seeing the problem it solved. Before eBPF, dynamic tracing meant one of two bad options. Either you used a tool like SystemTap, which compiled your probe into a kernel module and loaded it — powerful, but a bug in that module panics the kernel, so nobody ran it in production. Or you exported every event to user space and analyzed it there — safe, but the data volume (millions of events per second) made it prohibitively expensive. eBPF dissolves the dilemma: you run **safe, verified programs inside the kernel**, attached to hooks, aggregating data in kernel memory, and shipping only summaries to user space.

What eBPF actually is

eBPF ("extended BPF") is a general-purpose in-kernel virtual machine. A program is a small chunk of bytecode, targeting a RISC-like 64-bit

instruction set, that the kernel runs in response to an event. The lifecycle is the key to its safety and performance:

```
flowchart LR
    SRC["Restricted C or bpftrace source"] --> CC["clang / bpftrace  
compiles to eBPF bytecode"]
    CC --> LD["bpf() syscall loads program"]
    LD --> VER["Verifier: proves safety statically"]
    VER -->|reject| ERR["Load fails: no unsafe program runs"]
    VER -->|accept| JIT["JIT compiles bytecode to native machine code"]
    JIT --> HOOK["Attach to hook: kprobe / uprobe / tracepoint / XDP /  
socket / LSM"]
    HOOK -->|event fires| RUN["Program runs in kernel context"]
    RUN --> MAP["Reads / writes eBPF maps in kernel memory"]
    MAP --> US["User-space tool reads maps: histograms, counts"]
```

- **The verifier** is the crux. Before any program runs, the kernel's verifier performs static analysis proving the program is safe: it must terminate (originally: no loops at all; modern kernels allow *bounded* loops the verifier can prove terminate), it cannot access memory out of bounds, it cannot dereference arbitrary pointers, and it has a bounded instruction/complexity budget. A program that fails verification is *rejected at load time* — it never executes. This is what makes eBPF safe to run on a production kernel: the safety is proven, not trusted.
- **The JIT** compiles the verified bytecode to native machine code, so a running probe executes at near-native speed rather than being interpreted.
- **Maps** are the data structures shared between the kernel program and user space (and between programs): hash maps, arrays, per-CPU variants (for lock-free aggregation), ring buffers, LRU maps, stack-trace maps. This is how a probe *aggregates in the kernel* — incrementing a histogram bucket per event and letting user space read the finished histogram — instead of forwarding raw events. That in-kernel aggregation is the whole efficiency story.
- **Attach points / program types** span far more than tracing: **kprobes/uprobes** and **tracepoints** (dynamic and static tracing), **XDP** (packet processing at the driver, before `sk_buff` allocation — Chapter 10), **tc** and socket filters (traffic control, `SO_ATTACH_BPF`), **cgroup** hooks, **perf_events**, and **BPF-LSM** (security policy at LSM hooks). The same runtime that traces a slow function also drops

DDoS packets in the NIC driver and enforces a security policy — which is exactly why eBPF underpins both modern observability (Volume 11) and runtime security tools like **Falco** and **Tetragon** (Book 6).

Why this "revolutionized observability": you can now attach a probe to essentially any kernel or user function, on a live production host, with overhead low enough to leave running, with a kernel-guaranteed safety proof, and no module, no patch, and no reboot. That combination did not previously exist.

bpfftrace: the one-liner language

You almost never write raw eBPF bytecode. Two front ends dominate. **BCC** (BPF Compiler Collection) lets you write probes in C with a Python/Lua harness and ships a large suite of production tools. **bpfftrace** is a high-level, awk-inspired language for ad-hoc one-liners and short scripts — the tracing equivalent of `awk`, and where you should start.

A bpfftrace program is a set of `probe /filter/ { action }` blocks. The essentials:

- **Probes:** `kprobe:vfs_read`, `kretprobe:vfs_read`, `uprobe:/bin/bash:readline`, `tracepoint:syscalls:sys_enter_openat`, `tracepoint:sched:sched_switch`, `interval:s:1` (timer), `BEGIN / END`.
- **Built-ins:** `pid`, `tid`, `comm` (process name), `nsecs`, `arg0..argN` (probe arguments), `retval`, `kstack / ustack` (kernel/user stacks), `cpu`.
- **Maps and aggregations:** `@name[key] = count()`, `sum()`, `hist()` (power-of-2 histogram), `lhist()` (linear), `avg()`, `stats()`.

Real one-liners you will actually use:

```
# 1. Count system calls by name, system-wide, until Ctrl-C.
bpfttrace -e 'tracepoint:raw_syscalls:sys_enter { @[probe] = count(); }'

# 2. Which processes are open()ing files, and which files? (execsnoop-
lite)
bpfttrace -e 'tracepoint:syscalls:sys_enter_openat {
    printf("%-16s %s\n", comm, str(args->filename)); }'

# 3. Latency histogram of vfs_read() in microseconds – the canonical
pattern.
bpfttrace -e '
    kprobe:vfs_read { @start[tid] = nsecs; }
    kretprobe:vfs_read /@start[tid]/ {
        @us = hist((nsecs - @start[tid]) / 1000);
        delete(@start[tid]); }'

# 4. Count syscalls per process (find the syscall-chatty service).
bpfttrace -e 'tracepoint:raw_syscalls:sys_enter { @[comm] = count(); }'
```

The histogram idiom in (3) is the workhorse of latency analysis: timestamp on entry into a per-thread map, subtract on return, bucket the delta. Its output is a text histogram:

```
@us:
[1]                213 |
@@@               |
[2, 4)            1918 |
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[4, 8)            942 |
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[8, 16)           221 |
@@@@               |
[16, 32)           64 |
@                  |
[32, 64)           12 |
|                  |
[64, 128)           3 |
|                  |
```

Notice the value: this histogram was computed **entirely in the kernel** (each event just bumped a per-CPU bucket) and only the finished buckets crossed to user space. Tracing millions of reads this way costs a few percent CPU, not the order-of-magnitude slowdown that forwarding every event to user space would incur. That is the eBPF efficiency thesis in one screen.

The BCC tool ecosystem

BCC ships dozens of ready-made, battle-tested tools (packaged as `bcc-tools` / `bpfcc-tools`), each targeting a subsystem. These are the tools you reach for by name in an incident; internally they are the same probe-and-aggregate patterns as above. A representative sample, organized by the subsystems of this volume:

Tool	Subsystem	What it shows
<code>execsnoop</code>	processes (Ch 1)	every <code>execve</code> — new processes as they launch
<code>opensnoop</code>	files (Ch 6)	every <code>open</code> / <code>openat</code> and the file/return code
<code>runqlat</code>	scheduler (Ch 2)	run-queue latency histogram (time runnable-but-not-running)
<code>runqlen</code>	scheduler	run-queue length over time
<code>offcputime</code>	scheduler	why threads block, aggregated by off-CPU stack
<code>biolatency</code>	block I/O (Ch 7)	block-device I/O latency as a histogram
<code>biosnoop</code>	block I/O	per-I/O trace: PID, device, sector, latency
<code>tcplife</code>	network (Ch 10)	one line per TCP session: peers, bytes, duration
<code>tcpretrans</code>	network	TCP retransmissions as they happen
<code>tcpconnect</code> / <code>tcpaccept</code>	network	active/passive connections with peers
<code>syscount</code>	syscalls (Ch 5)	syscall counts by type or by process
<code>cachestat</code>	page cache (Ch 4/7)	page-cache hit/miss ratio
<code>profile</code>	CPU	timed on-CPU stack sampler (flame-graph source)

`biolateness` output, for instance, is exactly the histogram you want when USE says "disk is saturated":

```
# biolateness 10 1
      usecs      : count      distribution
      128 -> 255      : 12      |@@|
      256 -> 511      : 208     |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
      512 -> 1023     : 191     |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
      1024 -> 2047    : 40      |@@@@@|
      2048 -> 4095    : 6       |
      8192 -> 16383   : 3       |
<- tail: contended device
```

The bimodal shape — a fast mode around 300µs (likely cache/SSD hits) and a slow tail into milliseconds — is the visual signature of a device or queue under contention, invisible in the `iostat` average of `await`.

On-CPU vs off-CPU: the two halves of latency

Here is the analytical split that reorganizes everything. A thread's wall-clock time is either **on-CPU** (running, consuming cycles) or **off-CPU** (not running: blocked on I/O, on a lock, on a condition variable, or simply runnable but waiting for a CPU). Profilers like `perf record` and flame graphs, by default, only see **on-CPU** time — they sample the instruction pointer, and a sleeping thread has no instruction executing to sample. So if your service is slow because it spends its time *waiting*, an on-CPU flame graph will be nearly empty and profoundly misleading. It will show you the 5% of time spent computing and hide the 95% spent blocked.

Off-CPU analysis fills the gap. Instead of sampling running threads, you trace the scheduler: when a thread goes off-CPU (`sched_switch` away), record its stack and a timestamp; when it comes back, add the elapsed time to that stack's total. Aggregated, that is an **off-CPU flame graph** — the map of where your program *waits*. `offcputime` (BCC) produces exactly this. Between an on-CPU and an off-CPU flame graph you have accounted for *all* of a thread's wall-clock time, which is the only honest way to explain a latency number.

Scheduler latency: runqlat

A specific and common off-CPU cause deserves its own tool. A thread can be **runnable** — it has work to do, nothing is blocking it — and still not running, because every CPU is busy and it is waiting in a run queue. This

is **scheduler latency** (run-queue latency), and it is the classic reason a service is slow while its own CPU profile looks fine and the box isn't even at 100% utilization on average. Bursty arrivals, CPU oversubscription (too many threads or a too-tight cgroup `cpu.max` — Chapter 2), and noisy neighbors all produce it. `runqlat` measures it directly as a histogram of the time between "became runnable" and "started running":

```
# runqlat 10 1
      usecs           : count      distribution
      2 -> 3          : 1401      |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
      4 -> 7           : 890       |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
      8 -> 15          : 233       |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
     16 -> 31          : 44        |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
    2048 -> 4095        : 61        |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
<- scheduling delay
     4096 -> 8191      : 18        |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
<- tail latency source
```

Most wakeups schedule in microseconds; the millisecond tail is threads that sat runnable while CPUs were saturated. If that tail correlates with your service's p99, your problem is not the code the profiler shows — it is CPU saturation and scheduling delay (USE's *saturation* cell for the CPU resource), and the fix is capacity, concurrency limits, or CPU-quota tuning, not a hot loop. This is the concrete payoff of holding on-CPU and off-CPU as separate ideas.

Putting it together: a "service is slow" walkthrough

Method and tools converge in the incident loop. Suppose the on-call graph shows a service's p99 latency doubled after a deploy (RED: Duration up; Rate and Errors flat). A disciplined path:

- 1. **Confirm the symptom precisely (RED).** Is p99 up across all endpoints or one? Correlated with the deploy timestamp? Get the latency distribution, not the mean.
- 2. **USE the resources.** On an affected host, walk the resources: CPU utilization *and* saturation (`runqlat` , run-queue length), memory (RSS growth, page faults, swap), disk (`iostat` , `biolatency`), network (`ss -ti` , `tcpretrans`). One cell usually lights up.
- 3. **Split on-CPU vs off-CPU.** If CPU utilization rose, take an on-CPU flame graph (`perf record / profile`) and look for a new wide plateau — often the literal code the deploy added. If CPU is *not* the story,

take an off-CPU flame graph (`offcputime`): the service is waiting, and the graph names what on.

4. **Follow the widest box to a subsystem tool.** Off-CPU stack bottoms out in `vfs_read` on a data file? `biolatency` to characterize the disk. In a `futex`? Lock contention — `offcputime` already grouped it by the waiting stack. In `tcp_recvmmsg`? A slow dependency — `tcplife` / `tcpretrans` and move up the call graph to the downstream service.
5. **Confirm the fix by comparison.** Re-measure with the *same* tool. A differential flame graph or a before/after `perf stat` (did IPC recover? did the LLC-miss rate drop?) is the evidence that closes the loop — and guards against the "measure, don't guess" failure of declaring victory on a change that moved nothing.

The point of the sequence is that at each step a *measurement* selects the next tool. You never jump to "it's probably the database"; you let `biolatency` or `tcplife` say so.

Distributed-systems lens: fleet-scale, always-on observability

Everything so far assumed you are logged into one box. That is how you *learn* these tools and how you handle a specific incident, but it does not scale to a fleet of thousands of hosts running hundreds of services, deploying dozens of times a day. The distributed-systems reframing is the shift from *on-demand, one-host* profiling to *continuous, fleet-wide* profiling — and it changes what performance work is *for*.

Continuous profiling

Continuous profiling means every host runs a low-overhead sampling profiler *all the time*, continuously shipping compact stack profiles to a central store where they are aggregated, retained over time, and queryable by service, version, and time range. The seminal system is Google's **Google-Wide Profiling (GWP)**, described in the 2010 paper "Google-Wide Profiling: A Continuous Profiling Infrastructure for Data Centers," which sampled a small fraction of machines across the fleet continuously and made whole-datacenter CPU attribution a routine query. The open-source successors — **Parca**, **Pyroscope** (now part of Grafana), and Polar Signals' work — use **eBPF** to sample stacks (`perf_event` -based on-CPU sampling) with no code changes to the profiled applications,

storing profiles in the **pprof** format and often correlating them via **frame-pointer**-based unwinding across the fleet.

Two properties make this transformative at scale:

- **It is retrospective.** When p99 regresses at 3 a.m., you do not need to reproduce it and attach a profiler — the profile of the exact bad window is already stored. You query "show me the CPU flame graph for `checkout-service` v347 between 03:10 and 03:20," and you get it.
- **It is comparative across the fleet.** You can build a flame graph that aggregates CPU across *every instance of every service*, and ask the question that only matters at scale: **which code is burning the most CPU across the entire fleet?** At a large company, a single 2%-wide plateau in the fleet-wide flame graph — a wasteful serialization path, an over-eager log formatter, a hot `memcpy` — can represent thousands of cores and a seven-figure annual bill. The USE method plus a fleet flame graph is, quite literally, how you find the 2% of code that costs 40% of your compute budget. Performance work becomes a *cost-optimization* function, not just a latency one.

The economics only close because the profiler is nearly free to run continuously — sub-1% overhead per host is the design target — which is precisely the `eBPF/perf_events` sampling property established earlier. Always-on observability was impossible when tracing meant a kernel module or a full event stream; eBPF is what made it a fleet-wide default.

One runtime, two disciplines

The final structural point: the same eBPF machinery is the substrate for two large parts of this curriculum. On the **operations** side, it is the backbone of modern SRE observability (Volume 11) — continuous profiling, network flow visibility (Cilium/Hubble build on eBPF at the CNI layer, Chapter 10), and per-syscall/per-I/O telemetry without agents in the application. On the **security** side, the identical attach points — kprobes, tracepoints, and especially **BPF-LSM** — let tools like **Falco** and **Cilium Tetragon** observe and enforce policy on syscalls and kernel events at runtime (Book 6). An engineer who understands the verifier, maps, and program types understands both. That convergence — one safe, verified, JITed in-kernel runtime serving performance, networking, and security — is why eBPF, not any single tool, is the through-line of this chapter.

Key takeaways

- **Method beats tools.** Apply the **USE method** (Utilization, Saturation, Errors) to resources and **RED** (Rate, Errors, Duration) to services. Let saturation, not utilization, tell you where tail latency lives. Measure to kill hypotheses; never optimize on a guess.
- **perf stat reads the silicon.** IPC, cache-miss rate, and branch-miss rate connect a slow service to the microarchitecture of Volume 1 (Ch 2, 3, 8). Low IPC with a high LLC-miss rate is a memory-bound fingerprint you fix with data layout, not micro-optimization. Compare, don't read absolute numbers.
- **Sample to find hot code; count to measure it.** `perf record / report / top` sample stacks at bounded overhead; **flame graphs** turn thousands of stacks into a picture where width equals cost and wide leaves are the hotspots. Frame pointers (or DWARF/LBR) are mandatory for correct stacks.
- **ftrace** is the built-in, dependency-free kernel tracer; the **static tracepoint vs dynamic kprobe/uprobe** distinction (stable ABI vs any function, but fragile) recurs throughout tracing.
- **eBPF** is safe (verifier-proven), fast (JITed), and aggregates **in the kernel** via maps, so you can trace live production with a few percent overhead, no module, no reboot. That is why it revolutionized observability and now also carries networking (XDP) and security (BPF-LSM).
- **bpftool** for one-liners, **BCC** for the tool suite (`execsnoop` , `runqlat` , `biolatency` , `tcplife` , `offcputime` , ...) — one tool per subsystem you met earlier in this volume.
- **On-CPU vs off-CPU** is the master split. A near-empty on-CPU profile on a slow service means it is *waiting*; use off-CPU flame graphs and `runqlat` (scheduler latency) to see where.
- **At fleet scale, profile continuously.** eBPF-based always-on profiling (Parca, Pyroscope, and the GWP lineage) is retrospective and fleet-wide; a single wide plateau in the fleet flame graph is thousands of cores of cost. This is the backbone of SRE (Vol 11) and, via the same runtime, runtime security (Book 6).

Further reading

- **Brendan Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. (Addison-Wesley, 2020)** — the definitive practitioner text for this chapter: USE method, perf, ftrace, and a full treatment of the tooling. Pair with his *BPF Performance Tools* (Addison-Wesley, 2019), the canonical reference for bpftrace, BCC, and the tool suite, and his site (brendangregg.com), especially the USE-method page, the flame-graph pages, and the "Linux Performance" tools map.
- **Linux kernel documentation:** `Documentation/trace/ftrace.rst` (ftrace and tracefs), `Documentation/trace/kprobes.rst`, `Documentation/trace/uprobes.rst`, and the perf subsystem docs; man pages `perf(1)`, `perf-stat(1)`, `perf-record(1)`, `perf-report(1)`, `perf-top(1)`. The perf tooling lives in the kernel tree under `tools/perf`.
- **The eBPF documentation and community:** **ebpf.io** (concept overview and the "What is eBPF" guide), the kernel BPF docs under `Documentation/bpf/`, and the **BCC** and **bpftrace** repositories on GitHub (`iovisor/bcc`, `bpftrace/bpftrace`), whose `tools/` directories and the bpftrace reference guide and one-liner tutorial are the best hands-on references.
- **"Google-Wide Profiling: A Continuous Profiling Infrastructure for Data Centers,"** Ren, Tune, Moseley, Shi, Rus, Hundt (IEEE Micro, 2010) — the origin of fleet-wide continuous profiling and the intellectual basis for Parca and Pyroscope. See also the **pprof** format and tool (`github.com/google/pprof`), **Parca** (`parca.dev`), and **Grafana Pyroscope** docs.
- **Tom Wilkie, "The RED Method"** (Grafana/Weaveworks talks and blog posts) and **Google, *Site Reliability Engineering*** (O'Reilly, 2016), the "Monitoring Distributed Systems" chapter for the Four Golden Signals — the service-level companions to USE.
- **Intel, *Top-down Microarchitecture Analysis Method*** (Intel 64 and IA-32 Architectures Optimization Reference Manual, and Ahmad Yasin's ISPASS 2014 paper) with the **toplev** tool (`andikleen/pmu-tools`) — the rigorous way to attribute stalls to frontend/backend/bad-speculation/ retiring, extending the `perf stat` reading in this chapter.

- **Cilium, Falco, and Tetragon** project documentation (cilium.io, falco.org, github.com/cilium/tetragon) — the networking (XDP/tc) and runtime-security (BPF-LSM) faces of the same eBPF runtime, connecting this chapter to Chapter 10, Volume 11 (SRE), and Book 6 (Cloud-Native Security).

Chapter 12 — Boot, init, and systemd

What this chapter covers. Every service you operate runs on a machine that, at some moment, was a cold pile of silicon that had to bootstrap itself into a Linux system running your process under supervision. You rarely watch that happen — a scheduler hands you a node that is already up — but the mechanism matters the moment something goes wrong: a node stuck in the `initramfs` shell, a service that will not restart, a `systemctl status` that says `activating (start) forever`, a rolling deploy that hangs because PID 1 swallowed your `SIGTERM`. This chapter traces the full path from firmware to your service being managed as a supervised unit, and then goes deep on the thing that manages it: **systemd**. The throughline is that systemd is not "the thing that starts daemons." It is a node-local control plane — service supervisor, socket multiplexer, cgroup manager, sandbox builder, and log router — and almost every property a cluster orchestrator gives you (health checks, resource limits, restart policies, graceful shutdown, log shipping) exists first, and more primitively, in systemd on each node.

We are Linux-specific and mechanism-first. When we say "systemd holds the socket" we mean a real file descriptor passed across `execve`; when we say "each service gets a cgroup" we mean a directory under `/sys/fs/cgroup` that only systemd is allowed to write.

Learning goals — after this chapter you should be able to:

- Trace the boot chain — firmware/UEFI → bootloader → kernel + `initramfs` → pivot to the real root → `init` as PID 1 — and say precisely what each stage does and why `initramfs` exists.
- Read and reason about the kernel command line (`/proc/cmdline`) and know which knobs matter in production.

- Explain what SysV init actually did, why sequential `rc` scripts and PID-file supervision do not scale, and what systemd changed — fairly, including the criticism.
- Model systemd's world: unit types (service, socket, target, timer, mount, slice, scope, device, path), unit-file structure, and the difference between *requirement* dependencies (`Wants` / `Requires`) and *ordering* (`After` / `Before`).
- Use socket activation, `Type=notify` readiness, watchdogs, and restart policies to build a service that starts lazily, reports readiness honestly, and recovers from crashes.
- Harden a unit with systemd's sandboxing directives (`NoNewPrivileges` , `ProtectSystem` , `PrivateTmp` , `SystemCallFilter` , capability bounding) and connect them to Chapter 5 (seccomp, capabilities) and Chapter 9 (namespaces, cgroups).
- Explain how systemd + cgroups + socket activation + sandboxing is the same model container runtimes implement, why containers usually do *not* run systemd, and how this all feeds the kubelet in Book 6 / Volume 12.

This chapter builds on Chapter 1 (processes, PID 1, reaping, signals), Chapter 5 (system calls, seccomp, capabilities), Chapter 8 (signals and IPC), and Chapter 9 (namespaces and cgroups). It looks ahead to Volume 11 (deployments and rollouts) and Book 6 / Volume 12 (orchestration).

The boot chain

From power-on to your service running, control passes through a fixed sequence of increasingly capable programs, each of which loads and hands off to the next. The invariant is that every stage is smaller and dumber than the environment it is trying to reach, so its whole job is to locate and start the next stage.

```

flowchart LR
    fw["Firmware  
UEFI or legacy BIOS  
POST, init hardware"]
    bl["Bootloader  
GRUB2 / systemd-boot  
or UEFI stub"]
    k["Kernel  
vmlinuz decompresses,  
drivers, mounts initramfs"]
    ir["initramfs  
/init: find + mount  
the real root fs"]
    sr["switch_root  
exec real /sbin/init"]
    sd["systemd (PID 1)  
reach default.target"]

    fw --> bl
    bl --> k
    k --> ir
    ir --> sr
    sr --> sd

```

Firmware (UEFI or legacy BIOS). On power-on the CPU begins executing firmware from a fixed address. The firmware runs POST, initializes enough hardware to find a boot device, and then loads a bootloader. On a **legacy BIOS** system it reads the first 512-byte sector (the MBR), which contains a tiny first-stage loader. On **UEFI** — which is what essentially all server-class hardware and cloud instances use now — the firmware understands a FAT filesystem called the **EFI System Partition (ESP)**, mounted at `/boot/efi`, and directly loads and runs `.efi` executables (PE binaries) named in NVRAM boot variables. UEFI **Secure Boot**, when enabled, verifies each stage's signature against keys in firmware before executing it, which is why the bootloader (`shim` , then GRUB) and increasingly the kernel image must be signed.

Bootloader (GRUB2, systemd-boot, or a UEFI stub). The bootloader's job is narrow: load the Linux kernel image (`vmlinuz`) and the `initramfs` into memory, assemble the **kernel command line**, and jump into the kernel's entry point. GRUB2 is the traditional choice and can read many filesystems, present a menu, and load kernels by path. On UEFI you can skip a general bootloader: the Linux kernel can be built with an **EFI stub** so the firmware loads `vmlinuz.efi` directly, and modern setups increasingly ship a **Unified Kernel Image (UKI)** — kernel, `initramfs`, and command line bundled into one signed PE binary — which makes the whole boot payload signable end-to-end for Secure Boot.

Whatever the mechanism, the output is the same: a kernel and an `initramfs` in RAM, plus a command-line string.

Kernel + `initramfs`. The kernel decompresses itself, sets up the CPU, memory management, and the scheduler, and enumerates hardware for which it has built-in drivers. But it now faces a bootstrapping problem: **the real root filesystem is usually not directly reachable yet.** The root might be on an NVMe device needing a module, inside an LVM logical volume, on a LUKS- encrypted partition needing a passphrase, on software RAID, or over the network (iSCSI, NFS). The kernel cannot contain every storage driver and volume-management tool as built-ins without becoming enormous. So the bootloader also handed it an **`initramfs`** ("initial RAM filesystem"): a compressed `cpio` archive that the kernel unpacks into an in-memory `tmpfs` and uses as a temporary root. The `initramfs` contains a minimal userspace — a shell, `udev`/`systemd`, kernel modules, and tools like `cryptsetup`, `lvm`, `mdadm` — and an executable `/init`.

The kernel mounts this `initramfs`, then executes `/init` in it as the first userspace process. That `/init` (generated by **`dracut`** on Fedora/RHEL, or **`initramfs-tools`** on Debian/Ubuntu) does the real work of *reaching* the root: load the storage/network modules, assemble RAID and LVM, prompt for and unlock LUKS volumes, wait for the root device to appear (`udev` settle), then mount the real root filesystem read-only at `/sysroot`.

Pivot to the real root. With the real root mounted under `/sysroot`, the `initramfs` `/init` calls `switch_root`: it moves the mount to `/`, frees the `initramfs` memory, and `exec`s the real system's `init` (`/sbin/init`, which is a symlink to `systemd` on almost all modern distros). This is close cousin to the `pivot_root` you met in Chapter 9 for containers — both replace a process's root filesystem — but `switch_root` is the boot-time variant that additionally discards the throwaway `initramfs`. After `switch_root`, PID 1 is the real `init` running from the real disk. If something goes wrong before this point — no driver for the root device, a failed LUKS unlock, a missing UUID — you land in the **`initramfs emergency shell`** (`dracut:/#`), which is one of the two or three "the node won't boot" scenarios an on-call engineer must recognize on sight.

`init` as PID 1. From here, PID 1 is the service manager and never exits for the life of the system. Because it is PID 1 it inherits the two irreducible `init` duties from Chapter 1 — it must **reap** every orphaned zombie the whole system produces, and it is the process the kernel routes

some signals to specially. `systemd` does both, and layers a control plane on top.

The kernel command line

The command line is the single string the bootloader passes to the kernel, visible at runtime in `/proc/cmdline`:

```
$ cat /proc/cmdline
BOOT_IMAGE=/vmlinuz-6.8.0 root=UUID=6f1e...c2 ro quiet splash \
systemd.unit=multi-user.target rd.luks.uuid=... console=ttyS0,115200
```

It is one of the most operationally important strings on the machine, and it splits into three audiences. **The kernel** consumes parameters like `root=` (which device or UUID is the root filesystem), `ro` (mount it read-only initially so `fsck` can run), `console=` (where early boot output goes — `ttyS0` for the serial console you rely on in the cloud and in a datacenter KVM), and hardware tunables (`nomodeset`, `intel_iommu=on`, `mitigations=off`). **The `initramfs`** consumes `rd.*` options (dracut): `rd.luks.uuid=`, `rd.lvm.vg=`, `rd.break` (drop to a shell at a chosen point — invaluable for debugging a broken boot). **`systemd`** consumes `systemd.*`: `systemd.unit=` overrides the default target (append `systemd.unit=rescue.target` or the bare word `single` to boot to single-user mode for recovery), `systemd.log_level=debug`, `systemd.mask=`. Knowing you can edit this line in the GRUB menu at boot — press `e`, append a parameter, boot once — is the difference between recovering a node and reimaging it.

Before `systemd`: `SysV` init and its limits

To see why `systemd` looks the way it does, you have to see what it replaced. The traditional Unix `init`, inherited from AT&T System V, is a small PID 1 driven by `/etc/inittab` and organized around **runlevels**: numbered system states (0 = halt, 1 = single-user, 3 = multi-user text, 5 = multi-user graphical, 6 = reboot). Entering a runlevel meant running a directory of shell scripts. Under `/etc/rc.d/rc3.d/` (or `/etc/rc3.d/`) sat symlinks like `S20nginx` and `K80postgresql` pointing at real scripts in `/etc/init.d/`. On entering a runlevel, the `rc` driver ran every `K*` (kill) script to stop things, then every `S*` (start) script, **in lexical order of the two-digit number** — which is how you encoded dependencies: give `nginx` a higher number than the network so it starts later.

That model has structural problems that get worse at scale:

- **It is sequential.** Scripts run one at a time, each blocking the next. Boot time is the sum of every service's startup, even when most are independent and could run in parallel. On a server with dozens of daemons this is dead time on every boot and every reboot in a fleet.
- **Dependencies are implicit and fragile.** The `S20 / S80` numbering is a hand-maintained total order standing in for a real dependency graph. LSB init-script headers (`# Required-Start: $network`) improved this, but it is still a topological sort a human encodes into filenames.
- **There is no supervision.** A classic daemon **double-forks** to detach from its parent and reparent to PID 1, writes a PID file, and disappears. SysV init started it and then had *no idea whether it was alive*. Crash detection meant a separate watchdog reading a possibly-stale PID file. There was no built-in restart, no readiness signal, no way to know a service was actually serving versus merely forked.
- **Cleanup is unreliable.** Because the daemon detached, its children scattered across the process tree. Stopping "the service" meant killing a PID from a file and hoping you got all the workers; strays leaked.
- **No resource control.** SysV init had no concept of bounding a service's CPU, memory, or process count. A runaway daemon took down its neighbors.

Ubuntu's **Upstart** (event-based) and Apple's **launchd** (which pioneered socket activation and on-demand launching, 2005) were the transitional answers. `systemd` absorbed the good ideas from both and added a dependency-graph engine and deep cgroup integration.

systemd: PID 1 as a service manager

`systemd`, written by Lennart Poettering and Kay Sievers and first released in 2010, became the default init on Fedora (2011), then RHEL 7, Debian 8 (2015), Ubuntu 15.04, and effectively every mainstream distribution since. It replaces the pile of shell scripts with a single declarative model. The debate around that transition was real and worth stating fairly: critics objected to **scope creep** (`systemd` grew to manage logging, DNS resolution, network config, time sync, device management, containers, and more, well beyond "start daemons"), to a **monolithic, tightly-**

coupled design that is hard to replace piecemeal, to **binary journal logs** replacing grep-able plain-text files, and to the loss of the transparent, debuggable shell-script model. The counter-argument is that the old model's transparency was mostly the transparency of a system that did not actually solve dependency ordering, supervision, or resource control — and those are exactly the problems that matter when you run hundreds of services on thousands of nodes. Both things are true. In production today, systemd is what you operate, so learn it well.

The central abstraction is the **unit**: a named, typed object systemd knows how to activate, deactivate, and track. Everything systemd manages — a daemon, a mount point, a listening socket, a timer, a slice of the resource tree — is a unit, described by a declarative **unit file**.

Unit types

Type	Suffix	What it manages
service	.service	A daemon or one-shot process — the workhorse
socket	.socket	A listening socket systemd holds; enables socket activation
target	.target	A named sync point / group of units (the runlevel successor)
timer	.timer	A time- or event-triggered activation of another unit (cron successor)
mount / automount	.mount / .automount	A filesystem mount point (from /etc/fstab or explicit)
slice	.slice	A node in the cgroup resource tree grouping services
scope	.scope	A cgroup for externally-created processes systemd did not fork
device	.device	A udev -exposed device, usable as a dependency
path	.path	Activates a unit when a filesystem path appears or changes

Type	Suffix	What it manages
swap	.swap	A swap device or file

Unit files live in a search path with clear override precedence: `/usr/lib/systemd/system/` (shipped by packages, do not edit), `/run/systemd/system/` (runtime), and `/etc/systemd/system/` (local admin, highest precedence). Drop-in fragments in `<unit>.d/*.conf` directories override individual settings without replacing the whole file — `systemctl edit <unit>` creates exactly such a drop-in, which is the correct way to customize a vendor unit.

A minimal service unit:

```
# /etc/systemd/system/orders-api.service
[Unit]
Description=Orders API
After=network-online.target
Wants=network-online.target

[Service]
ExecStart=/usr/local/bin/orders-api --config /etc/orders/config.yaml
Restart=on-failure
User=orders

[Install]
WantedBy=multi-user.target
```

`[Unit]` holds metadata and dependencies; `[Service]` holds how to run it; `[Install]` describes what happens on `systemctl enable` (it creates the symlink that pulls this unit into `multi-user.target`). After writing or changing a unit file you must run `systemctl daemon-reload` so `systemd` re-parses its configuration — a step that trips up everyone at least once.

Dependencies vs ordering — the distinction that confuses everyone

`systemd` separates two orthogonal questions that SysV's numbering conflated:

1. **Requirement** — *if A is started, should B also be started?* Expressed with `Wants=` (weak: pull B in, but A survives B failing) and `Requires=` (strong: if B fails to start or later fails, A is stopped too). Related: `Requisite=` (B must already be active), `BindsTo=` (A stops if B stops,

even unexpectedly), `PartOf=` (stop/restart of B propagates to A),
`Conflicts=` (starting A stops B).

2. **Ordering** — *in what sequence?* Expressed with `After=` and `Before=`, which are purely about sequencing and say nothing about whether the other unit is even wanted.

These are independent, and the independence is the whole point. `Requires=B` **without** `After=B` means "pull B in, but start both in parallel" — which is usually a bug, because A may race ahead of B. You almost always want both: `Requires=postgresql.service` **and** `After=postgresql.service`. Conversely `After=network.target` without any `Wants / Requires` means "if the network is coming up, wait for it, but don't force it." The default, absent any ordering directive, is **no ordering at all**: systemd starts everything in parallel and lets the dependency graph and socket activation impose only the order that is actually declared. This parallelism is the first reason systemd boots faster than SysV.

```
flowchart BT
    net["network-online.target"]
    db["postgresql.service"]
    cache["redis.service"]
    api["orders-api.service"]
    mt["multi-user.target"]
    api -->|"After + Requires"| db
    api -->|"After + Wants"| cache
    api -->|"After + Wants"| net
    mt -->|"WantedBy pulls in"| api
```

The arrows encode "started after / depends on." `multi-user.target` is the aggregation point: enabling `orders-api` added it to that target's wants, so reaching the target pulls the whole subgraph up, in parallel except where `After=` forces sequencing.

Targets replace runlevels

A **target** is a unit with no behavior of its own — it is a named synchronization point that groups other units, the direct successor to runlevels. `multi-user.target` is the rough equivalent of runlevel 3 (full text-mode multi-user), `graphical.target` of runlevel 5, `rescue.target` of single-user. `default.target` is a symlink to whichever the system should reach on boot; on a server it points at `multi-user.target`. Special targets are boot milestones you order against rather than runlevels: `basic.target`, `sysinit.target`, `network-online.target`,

`shutdown.target`. Switch targets at runtime with `systemctl isolate rescue.target`; change the default with `systemctl set-default multi-user.target`.

systemctl — the interface

`systemctl` is how you drive all of this. The commands you actually use daily:

```
systemctl status orders-api          # state, PID, cgroup, recent log
lines
systemctl start / stop / restart orders-api
systemctl reload orders-api
# send the unit's ExecReload (e.g. SIGHUP), no restart
systemctl enable --now orders-api    # create WantedBy symlink AND
start it
systemctl disable orders-api         # remove the symlink (no start at
boot)
systemctl cat orders-api             # show the effective unit file +
drop-ins
systemctl edit orders-api            # create/modify a drop-in override
systemctl list-units --type=service --state=running
systemctl list-dependencies orders-api
systemctl daemon-reload             # re-read unit files after editing
systemctl is-active / is-enabled / is-failed orders-api
# scriptable, exit-code driven
```

The `is-*` subcommands return machine-readable exit codes, which is what your configuration- management and health-check scripts should key off, never string-matching `status` output.

journald: structured logging

systemd ships its own logging daemon, `systemd-journald`, and it changes the logging model. Under SysV, a daemon's stdout/stderr went nowhere useful and real logs went through `syslog` to plain-text files. `journald` instead captures **everything a service writes to stdout/stderr** (because the service is a child of systemd, its output is simply piped to the journal), plus the kernel ring buffer, plus native structured messages, and stores them as **structured, indexed, binary records**. Each entry is a set of key-value fields — not just a text line — including trusted metadata the *service could not forge* because systemd attaches it: `_PID`, `_UID`, `_SYSTEMD_UNIT`, `_BOOT_ID`, `_COMM`, plus any `PRIORITY`, `MESSAGE_ID`, or custom fields the program emitted via the native protocol.

```

journalctl -u orders-api          # everything from this unit
journalctl -u orders-api -f       # follow (tail -f)
journalctl -u orders-api --since "10 min ago" -p err
# errors in last 10 min
journalctl -u orders-api -b       # this boot only; -b -1 =
previous boot
journalctl -u orders-api -o json  # structured output for
shipping/parsing
journalctl -k                     # kernel messages (dmesg
equivalent, persisted)
journalctl --disk-usage           # how big the journal is

```

Persistence is a choice: with `/var/log/journal/` present the journal survives reboots; with only `/run/log/journal/` (tmpfs) it is volatile and lost on reboot — a fact worth checking on a node before you go debugging a crash from yesterday. `journalctl -b -1` reading the *previous* boot's log is often how you diagnose why a node rebooted. The binary format is the most common complaint about journald; in practice you either query it with `journalctl` filters (which are far more precise than `grep` over text, because you filter on real fields) or you configure `ForwardToSyslog=` / ship the journal to your central logging system — which on a fleet you do anyway. The structured fields are exactly what a log pipeline wants.

Socket activation

Socket activation is one of systemd's best ideas and directly descends from `inetd` and Apple's `launchd`. The model: **systemd creates and holds the listening socket, and starts the actual service only when the first connection arrives** — then passes the already-open, already-bound listening file descriptor to the service across `execve`.

```
sequenceDiagram
    participant C as Client
    participant S as systemd (PID 1)
    participant Svc as Service process
    Note over S: at boot, opens and binds
orders-api.socket, LISTEN state
    C->>S: connect() to :8080
    Note over S: socket buffers the SYN;
connection queued
    S->>Svc: fork + exec service,
pass listen fd (LISTEN_FDS=1)
    Svc->>Svc: sd_listen_fds(): adopt fd 3,
accept() the queued connection
    Svc-->>C: serve response
    Note over S,Svc: socket stays owned by systemd;
service can restart without dropping it
```

The mechanics: a `.socket` unit tells `systemd` what to listen on. `systemd` binds it early (before the service exists) and watches it with its event loop. When a connection lands, `systemd` starts the paired `.service`, and passes the listening socket as file descriptor `3` (and up), setting the environment variables `LISTEN_FDS` (how many) and `LISTEN_PID` (whose they are). The service calls `sd_listen_fds()` from `libsystemd` (or reads the env vars directly) to adopt those fds instead of calling `socket()` / `bind()` / `listen()` itself.

```
# /etc/systemd/system/orders-api.socket
[Unit]
Description=Orders API socket

[Socket]
ListenStream=8080
# Accept=no (default): one service instance handles all connections
# Accept=yes: spawn a per-connection instance from a template unit
# (orders-api@.service)

[Install]
WantedBy=sockets.target
```

Three payoffs make this more than a curiosity:

- **Boot parallelization without ordering headaches.** Because `systemd` binds *all* sockets up front, before any daemon starts, services that talk to each other can start simultaneously. If service A connects to B before B is ready, the connection simply sits in B's socket buffer (the kernel queued it) until B calls `accept()`. You no

longer need to carefully order "start B, wait for B to be ready, then start A" — the socket buffer absorbs the race. This is a major reason systemd boots fast.

- **Lazy / on-demand start.** Rarely-used services (a debug endpoint, an admin tool) need not run until first use, saving memory. This is precisely the `inetd` super-server model, modernized.
- **Zero-downtime restart.** The listening socket's lifetime is decoupled from the service's. You can stop, upgrade, and restart the service while systemd keeps the socket open and buffering incoming connections; clients see a brief latency blip instead of connection refused. This is a genuinely useful primitive for restarts of a stateless daemon.

cgroups: systemd owns the resource tree

Chapter 9 explained cgroups as the kernel mechanism that limits what a process can *use*. systemd is the userspace component that *organizes and writes* that tree. This is not a coincidental pairing — it is architectural. On a cgroup v2 system there must be a **single writer** to the unified hierarchy (multiple uncoordinated writers corrupt each other's limits), and on a systemd machine **that single writer is systemd (PID 1)**. Every process systemd starts is placed in its own cgroup, and those cgroups are organized into a tree of **slices**.

```

flowchart TB
    root["-.slice (root of cgroup tree)"]
    sys["system.slice
system services"]
    usr["user.slice
per-user sessions"]
    mach["machine.slice
VMs / nspawn containers"]
    s1["orders-api.service"]
    s2["postgresql.service"]
    s3["nginx.service"]
    u1["user-1000.slice
session-3.scope"]
    root --> sys
    root --> usr
    root --> mach
    sys --> s1
    sys --> s2
    sys --> s3
    usr --> u1

```

Two unit types map the process tree onto the cgroup tree. A **slice** (`.slice`) is a branch — a grouping node with no processes of its own but with resource limits that apply to everything beneath it, so you can bound a whole class of services collectively. The default branches are `system.slice` (system daemons), `user.slice` (interactive sessions), and `machine.slice` (VMs and `systemd-nspawn` containers), all under the root `-.slice`. A **scope** (`.scope`) is a cgroup for processes `systemd` did *not* fork itself but was asked to manage — a session, or a container the runtime created — so they still get accounted and controlled.

You set resource control declaratively as unit properties, and `systemd` translates them into cgroup v2 interface-file writes:

```

[Service]
CPUWeight=200           # relative CPU share vs siblings (->
cpu.weight)
CPUQuota=150%           # hard cap: 1.5 CPUs worth of time (->
cpu.max)
MemoryHigh=1.5G         # soft throttle: reclaim pressure above this
(-> memory.high)
MemoryMax=2G            # hard wall: OOM-kill above this (->
memory.max)
TasksMax=512            # fork-bomb protection (-> pids.max)
IOWeight=100            # relative I/O share (-> io.weight)

```

You can inspect the live tree with `systemd-cgls` (the cgroup hierarchy as a process tree) and `systemd-cgtop` (live per-cgroup CPU/memory/IO, a `top` for the resource tree). This is the same machinery — the same `memory.max`, the same `cpu.max` — that a container runtime writes when it starts a pod. Which is the whole point of the next connection: **systemd's cgroup+socket+sandbox model is the model containers implement.**

Service hardening: systemd as a sandbox builder

Because `systemd` is the process that launches your service, it can set up the process's security context *before* the `execve` — dropping privileges, restricting syscalls, hiding parts of the filesystem, narrowing capabilities. These are the same primitives from Chapter 5 (`seccomp`, capabilities) and Chapter 9 (mount and other namespaces), exposed as declarative unit directives. Turning them on for a service is often a few lines and no code change, which makes `systemd` sandboxing one of the highest-leverage hardening steps available for a bare-metal or VM service. A hardened unit:

```

# /etc/systemd/system/orders-api.service
[Unit]
Description=Orders API (hardened)
After=network-online.target postgresql.service
Wants=network-online.target
Requires=postgresql.service

[Service]
Type=notify
ExecStart=/usr/local/bin/orders-api --config /etc/orders/config.yaml

# --- identity: never run as root ---
DynamicUser=yes
# allocate a transient UID/GID for this service
# (or: User=orders / Group=orders for a fixed system account)

# --- privilege reduction ---
NoNewPrivileges=yes           # setuid/setgid binaries can never re-
gain privilege
CapabilityBoundingSet=        # drop ALL capabilities (empty = none)
AmbientCapabilities=          # grant none; add CAP_NET_BIND_SERVICE
here if binding <1024

# --- filesystem isolation ---
ProtectSystem=strict          # entire fs read-only except explicit
ReadWritePaths
ProtectHome=yes               # /home, /root, /run/user made
inaccessible
PrivateTmp=yes                # private /tmp and /var/tmp (own mount
namespace)
ReadWritePaths=/var/lib/orders # the only writable location
ProtectKernelTunables=yes     # /proc/sys, /sys read-only
ProtectKernelModules=yes      # block module load/unload
ProtectControlGroups=yes      # /sys/fs/cgroup read-only
ProtectProc=invisible         # hide other processes in /proc

# --- kernel attack surface ---
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX # no raw/packet
sockets, etc.
RestrictNamespaces=yes        # cannot create new namespaces
LockPersonality=yes           # no personality() ABI switching
MemoryDenyWriteExecute=yes    # no W^X-violating mappings (blocks
common exploits)
SystemCallFilter=@system-service # seccomp allowlist of "normal
service" syscalls
SystemCallFilter=~@privileged @resources # subtract dangerous groups
SystemCallErrorNumber=EPERM

# --- resource envelope (cgroup) ---
MemoryMax=2G

```

```

TasksMax=512
LimitNOFILE=65536                # rlimit: max open file descriptors

Restart=on-failure
RestartSec=2s

[Install]
WantedBy=multi-user.target

```

A few of these deserve a second look. `SystemCallFilter=` compiles to a **seccomp-BPF** program (Chapter 5) — `@system-service` is a curated allowlist of the syscalls a well-behaved daemon needs, and the `~` form subtracts named groups like `@privileged` (mount, ptrace, reboot, kernel-module ops) and `@resources`. `ProtectSystem=strict` and `PrivateTmp=yes` are implemented by putting the service in its own **mount namespace** with read-only bind mounts and a private `/tmp` — no code change, kernel-enforced. `DynamicUser=yes` allocates a throwaway UID for the service's lifetime so there is no long-lived account to compromise. `NoNewPrivileges=yes` is the same `prctl(PR_SET_NO_NEW_PRIVS)` bit that container runtimes set.

Do not guess at this. Run `systemd-analyze security orders-api.service`, which scores each service's exposure (roughly 0 = locked down, 10 = wide open) and lists exactly which directives would tighten it. Treating that score as a lint check across a fleet's units is a cheap, concrete way to raise the security floor of every node.

Lifecycle: readiness, watchdogs, restart

Supervision is systemd's answer to SysV's biggest gap. Three mechanisms turn "we forked it" into "we know it is healthy."

Service type / readiness. `Type=` tells systemd how to know the service has finished starting — which matters because ordered dependents wait for "started," and you want that to mean "ready to serve," not "the process exists."

Type=	"Started" means	Use for
<code>simple</code>	the process was <code>exec 'd</code> (immediately)	trivial daemons that are ready at once

Type=	"Started" means	Use for
<code>exec</code>	<code>execve</code> succeeded (catches early <code>exec</code> failures)	most simple daemons, slightly safer
<code>forking</code>	the parent exited after forking a daemon child	classic double-forking daemons (<code>PIDFile=</code>)
<code>oneshot</code>	the process ran to completion (exit 0)	setup/migration steps; pairs with <code>RemainAfterExit=yes</code>
<code>notify</code>	the service sent <code>READY=1</code> via <code>sd_notify</code>	services that must signal true readiness
<code>dbus</code>	the service took its D-Bus name	D-Bus services

`Type=notify` is the honest one and the one to prefer for a real backend service. The service calls `sd_notify(0, "READY=1")` — a datagram to the `AF_UNIX` socket named in the `NOTIFY_SOCKET` environment variable — **after** it has loaded config, opened its database pool, warmed caches, and bound its port. Only then does systemd consider it started and release dependents. This is the node-local ancestor of a Kubernetes readiness probe: "don't send me traffic / don't start my dependents until I say I'm ready." A minimal Go example:

```
import "github.com/coreos/go-systemd/v22/daemon"

func main() {
    srv := setUp()           // load config, connect DB, warm caches
    listenAndServe(srv)      // bind and start accepting
    daemon.SdNotify(false, daemon.SdNotifyReady) // READY=1: now
    systemd unblocks dependents
    // ... on shutdown, before draining:
    // daemon.SdNotify(false, daemon.SdNotifyStopping)
}
```

Watchdog. With `WatchdogSec=30s`, systemd expects the service to ping it at least every 30 seconds via `sd_notify(0, "WATCHDOG=1")`. Miss the deadline and systemd concludes the process is hung (not crashed — *hung*, which a PID check would never catch) and acts on the restart policy. This is a liveness probe implemented in the init system. Pair it with

`Restart=on-watchdog` (or `on-failure`, which includes watchdog timeouts).

Restart policy. `Restart=` controls automatic recovery: `no` (default), `on-failure` (non-zero exit, signal, timeout, or watchdog), `on-abnormal`, `always`. Guard it against crash loops with rate limiting: `StartLimitIntervalSec=` and `StartLimitBurst=` (e.g. "more than 5 starts in 10s → give up and mark the unit failed rather than spin forever"), with `RestartSec=` inserting a backoff delay between attempts. Without a start limit, a service that crashes instantly on a bad config will restart in a tight loop and peg a core; the limit is what turns that into a clean `failed` state you can alert on. This is exactly the crash-loop-backoff logic Kubernetes reimplements at the pod level — systemd had it first, per node.

Timers: the cron successor

Timer units activate other units on a schedule and replace `cron` for system tasks, with several advantages: the triggered work runs as a normal supervised unit (so it gets journald logging, resource limits, and sandboxing for free), and `Persistent=true` catches up runs missed while the machine was off. A timer plus its service:

```
# /etc/systemd/system/db-backup.timer
[Unit]
Description=Nightly DB backup

[Timer]
OnCalendar=*-.* 02:30:00
# every day at 02:30 (systemd calendar syntax)
Persistent=true           # run on next boot if the machine was
off at 02:30
RandomizedDelaySec=300
# jitter up to 5 min: de-synchronize a fleet's backups

[Install]
WantedBy=timers.target
```

```
# /etc/systemd/system/db-backup.service
[Unit]
Description=DB backup job

[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup-db.sh
```

`RandomizedDelaySec=` is a fleet-scale detail: without jitter, a thousand nodes all fire the same `OnCalendar` job at exactly 02:30 and stampede your object store or backup target. Enable the timer (not the service) with `systemctl enable --now db-backup.timer`; list active timers with `systemctl list-timers`. Also useful: `OnBootSec=`/`OnUnitActiveSec=` for relative-to-boot or periodic triggers instead of wall-clock.

The relationship to containers

Now the payoff. Everything above — supervise a process, put it in a cgroup with limits, restrict its syscalls and capabilities, give it a private filesystem and namespaces, hold its socket — is *the same set of operations a container runtime performs* (Chapter 9). `systemd` and `runc` are doing the same job with the same kernel primitives; they differ in packaging. So the container picture has three distinct relationships to `systemd` that engineers routinely confuse.

The host runs `systemd`; the container usually does not. On a Kubernetes node, `systemd` (PID 1 on the host) supervises `kubelet` and `containerd`, and it owns the cgroup tree those runtimes carve pods into. Inside a container, though, **PID 1 is normally your application itself**, not an init system — a container is meant to be a single process (Chapter 1, Chapter 9). This is the correct default, but it drags in the reaping and signal-handling gotcha from Chapter 1: your app, now PID 1, must **reap orphaned zombies** and **handle `SIGTERM`** for graceful shutdown, because the kernel gives PID 1 no default signal dispositions. If your app does neither, you leak zombies and `docker stop` / pod termination hangs until the 30-second grace period elapses and the runtime sends `SIGKILL`. The fix is a *tiny* init like `tini` or `dumb-init` as PID 1 (which just reaps and forwards signals) — **not** a full `systemd` inside the container. Running `systemd` in a container is possible but wants elevated privileges and rarely earns its complexity; reach for it only when you genuinely need to run several cooperating services in one image.

`systemd`'s cgroup driver and the kubelet. When both are present, they must agree on how the cgroup tree is written. The kubelet and the container runtime each have a **cgroup driver** setting — `cgroupfs` (write cgroup files directly) or `systemd` (ask `systemd` to create the cgroups). On a `systemd` host, cgroup v2 demands a single writer, so the correct choice is the **`systemd` cgroup driver** on *both* kubelet and runtime; mixing drivers gives you two writers fighting over the tree and produces flaky

resource enforcement and node instability. This is a direct, operational consequence of "systemd owns the resource tree."

systemd-nspawn. systemd also ships its own container manager, `systemd-nspawn` — think "`chroot` on steroids," a lightweight container that boots an OS tree in its own namespaces, landing under `machine.slice` and managed via `machinectl`. It is closer to a lightweight VM / system container than to a single-process app container, and it is mostly used for OS testing, build environments, and running a full init inside a namespace — not for shipping microservices. Its existence is a reminder that the container primitives are general kernel features, and systemd is one of several tools that compose them.

Boot performance

Because boot is parallelized against a dependency graph, systemd can tell you exactly where the time went — invaluable when a fleet's nodes take too long to become schedulable after a reboot.

```
$ systemd-analyze time
Startup finished in 2.1s (firmware) + 3.4s (loader) + 1.2s (kernel)
+ 6.8s (initrd) + 12.3s (userspace) = 25.8s

$ systemd-analyze blame          # slowest units, longest first
6.212s postgresql.service
3.870s docker.service
1.402s NetworkManager-wait-online.service
...

$ systemd-analyze critical-chain # the ordered dependency path that
gated boot
multi-user.target @12.3s
└─orders-api.service @11.1s +1.2s
   └─postgresql.service @4.8s +6.2s
      └─network-online.target @4.7s
```

The distinction between `blame` and `critical-chain` matters: `blame` lists the slowest units in isolation, but a unit can be slow and *off the critical path* (nothing waited for it), so optimizing it changes nothing. `critical-chain` shows the actual sequential path that gated reaching the target — that is where boot-time optimization pays off. `systemd-analyze plot > boot.svg` renders the whole parallel startup as a timeline. A frequent real finding is a `*-wait-online.service` blocking for its full timeout because

the network never reached the state it wanted — an ordering assumption, not slow hardware.

Distributed-systems lens

Zoom out and the shape of systemd is the shape of an orchestrator, one level down. **Service lifecycle management — start, stop, restart, health, resource limits — is the node-local foundation that cluster orchestrators build on, not an alternative to them.** The correspondence is nearly one-to-one:

systemd concept (per node)	Orchestrator concept (per cluster)
Type=notify / sd_notify(READY=1)	readiness probe
WatchdogSec / WATCHDOG=1	liveness probe
Restart=on-failure + StartLimitBurst	restart policy + CrashLoopBackOff
MemoryMax / CPUQuota on a unit	pod resource limits
slices (system.slice , nesting)	namespaces / cgroup hierarchy per pod
socket activation	(the pattern service meshes and inetd -style front ends generalize)
journald structured logs	node log agent → central logging

The kubelet does not replace this machinery — it **drives** it. It asks the container runtime to create cgroups (which, on a systemd node, means systemd creates them), it supervises `containerd` as a systemd unit, and it reimplements at the cluster level the same liveness/readiness/restart loop systemd runs at the node level. Understanding systemd is therefore understanding the substrate Kubernetes stands on; a "node not ready" or "pods stuck terminating" incident very often bottoms out in systemd, cgroup-driver mismatch, or PID-1 signal handling — not in the control plane at all (Book 6, Volume 12).

Fleet uniformity is built out of these primitives. Thousands of nodes are made identical by shipping identical **unit files** (via your configuration-management or image-build pipeline), so every node

supervises services the same way; by defining **resource slices** so a class of workloads is bounded identically everywhere; and by shipping **journal** logs to a central system so `journalctl -o json` on each node becomes a queryable, structured stream. A unit file checked into a repo, `systemd-analyze security`-linted in CI, and rolled out fleet-wide is the node-level analog of a reviewed, admission-controlled pod spec.

The PID-1 and signal semantics from Chapter 1 are load-bearing for deploys (Volume 11). A graceful rolling deploy depends on a precise shutdown handshake: the orchestrator (or systemd) sends `SIGTERM`, the process must **catch it, stop accepting new work, drain in-flight requests, and exit** within the grace period, or it is `SIGKILL` ed mid-request. `Type=notify` with a `STOPPING=1` notification lets a service tell systemd "I've started draining" so load can be shed cleanly first. If PID 1 in your container ignores `SIGTERM` (because it is your app and you never installed a handler, or because a shell wrapper isn't forwarding signals), every rolling deploy silently degrades into a `SIGKILL` that severs live connections. The unglamorous node-level details — who is PID 1, does it reap, does it forward `SIGTERM`, does it report readiness honestly — are exactly what determine whether a fleet-wide rollout is invisible to users or a wave of 502s.

Key takeaways

- **Boot is a fixed hand-off chain:** firmware/UEFI → bootloader (GRUB/systemd-boot/EFI stub) → kernel + initramfs → `switch_root` to the real root → `init` (systemd) as PID 1. initramfs exists solely to contain the drivers and tools needed to *find and mount the real root* (modules, LVM, LUKS, RAID, network root) before pivoting to it.
- **The kernel command line (`/proc/cmdline`) is split three ways** — kernel (`root=` , `ro` , `console=`), initramfs (`rd.*`), and systemd (`systemd.unit=`). Editing it at the GRUB prompt (`systemd.unit=rescue.target` , `rd.break`) is how you recover a node instead of reimaging it.
- **SysV init failed at scale** on four axes: sequential (not parallel) startup, implicit filename-numbered dependencies, no real supervision (detached daemons + stale PID files), and no resource control. systemd replaces the shell-script pile with a declarative dependency graph — and the criticism of its scope creep and binary logs is fair but does not restore those four missing capabilities.

- **Units are the model.** Services, sockets, targets, timers, mounts, slices, scopes, devices, and paths, described by unit files with clear override precedence (`/etc/systemd/system` > `/run` > `/usr/lib`); remember `daemon-reload` after edits.
- **Requirement (Wants / Requires) and ordering (After / Before) are orthogonal.** The default is *parallel, unordered*; you almost always want both a requirement and an ordering edge (`Requires=` + `After=`). Targets are the runlevel successor and act as synchronization points.
- **Socket activation** — `systemd` holds the listening socket and starts the service on first connection, passing the fd across `execve` (`sd_listen_fds` , `LISTEN_FDS`) — gives boot parallelization (the socket buffer absorbs start-order races), lazy start (`inetd` 's successor), and zero-downtime restart (socket outlives the process).
- **systemd is the single writer of the cgroup v2 tree.** Services live in per-unit cgroups organized into slices; resource control is declarative (`MemoryMax` , `CPUQuota` , `TasksMax`). This is the *same* machinery container runtimes use, which is why the cgroup driver must match (`systemd` on both kubelet and runtime).
- **Sandboxing is a few declarative lines, kernel-enforced**
pre- exec : `NoNewPrivileges` , `ProtectSystem=strict` , `PrivateTmp` , `SystemCallFilter=` (`seccomp`, Chapter 5), `CapabilityBoundingSet=` , `DynamicUser` . Audit with `systemd-analyze security` .
- **Supervision means readiness, liveness, and restart.**
`Type=notify` + `sd_notify(READY=1)` is honest readiness; `WatchdogSec` + `WATCHDOG=1` catches *hangs*; `Restart=on-failure` with `StartLimitBurst` gives crash-loop-safe recovery — the node-local ancestors of Kubernetes probes and `CrashLoopBackOff`.
- **Containers usually do not run systemd** — PID 1 is your app, which must reap zombies and handle `SIGTERM` (use `tini` / `dumb-init` , not `systemd`, when it doesn't). The *host* runs `systemd`, which supervises the runtime and owns the cgroups pods land in. `systemd-nspawn` is a separate, system-container tool.
- **Node lifecycle management is the substrate orchestrators build on.** Consistent unit files, resource slices, and shipped journald logs are how a fleet is made uniform; PID-1 signal and readiness semantics are what make rolling deploys graceful instead of a wave of `SIGKILL` s and 502s (Volume 11, Book 6, Volume 12).

Further reading

- **systemd man pages** — the authoritative reference, and unusually good: `systemd(1)`, `systemd.unit(5)`, `systemd.service(5)`, `systemd.socket(5)`, `systemd.target(5)`, `systemd.timer(5)`, `systemd.slice(5)`, `systemd.resource-control(5)` (cgroup properties), `systemd.exec(5)` (the full sandboxing directive set), `systemd.kill(5)`, `sd_notify(3)`, `sd_listen_fds(3)`, `systemctl(1)`, `journalctl(1)`, `systemd-analyze(1)`, `bootup(7)` (the boot-target sequence), and `daemon(7)` (old-vs-new daemon conventions). Available at freedesktop.org/software/systemd/man and via `man`.
- **Lennart Poettering, "systemd for Administrators"** — the original blog series (0pointer.net/blog) walking through units, socket activation, cgroups, journald, and readiness protocols from the author; still the clearest narrative introduction to the design intent.
- **freedesktop.org systemd documentation** (systemd.io) — including "Writing Network Configuration Managers," the cgroup delegation notes ("Control Group APIs and Delegation"), and the `sd_notify` readiness/watchdog protocol description.
- **Linux kernel documentation:** `Documentation/admin-guide/cgroup-v2.rst` (the single-writer rule and controller interface `systemd` drives) and `Documentation/filesystems/ramfs-rootfs-initramfs.rst` (how the kernel unpacks and pivots off the `initramfs`).
- **dracut** (`man dracut`, `man dracut.cmdline`) and **initramfs-tools** documentation — how the `initramfs` is generated and which `rd.*` command-line options it honors; essential for debugging a node that hangs before `switch_root`.
- **UEFI Specification** (uefi.org) and the **GRUB manual** (gnu.org/software/grub/manual) — the firmware boot protocol and the bootloader that implements it; pair with the **Boot Loader Specification** and **Unified Kernel Image** docs on systemd.io for the modern signed-boot path.
- **The Linux Programming Interface**, Michael Kerrisk, No Starch Press, 2010 — daemons, signals, process groups/sessions, and the classic double-fork daemonization that `systemd` supersedes; background for the PID-1 and signal semantics in Chapter 1 and this chapter.

- **Kubernetes documentation:** "Configuring a cgroup driver" and "Container Runtimes" (why the kubelet and runtime must share the `systemd` cgroup driver), and the pod-lifecycle / probe docs — the cluster-level restatement of the node-level supervision in this chapter (Book 6, Volume 12).
- **tini** (github.com/krallin/tini) and **dumb-init** (github.com/Yelp/dumb-init) — minimal container inits that solve exactly the PID-1 reaping and signal-forwarding problem so you do *not* run `systemd` inside a container (Chapter 1, Chapter 9).